Universidade Estadual de Campinas

Enemy Leo used Appeal! It's super effective! Pikachu fainted!

CSES Problem Set

Pedro Assunção, Pedro Ferreira, Yvens Porto

2026-08-29

Contents

1 Introductory Problems

1.1 Apple Division

There are n apples with known weights. Your task is to divide the apples into two groups so that the difference between the weights of the groups is minimal.

Input
The first input line has an integer n: the number of apples. The next line has n integers $p_1, p_2, \ldots, p_n$: the weight of each apple.

Output
Print one integer: the minimum difference between the weights of the groups.

Constraints
• $1 \leq n \leq 20$ • $1 \leq p_i \leq 10^9$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(){
    ll n; cin >> n;
    ll p[n], pot = 1, s = 0, dif = 20LL * inf;

    for(int i=0; i<n; i++){
        cin >> p[i];
        s += p[i];
        pot *= 2;
    }

    for(int i = 0; i < pot; i++){
        ll k = i;
        ll soma = 0, num = 0;
        while(k > 0){
            if(k%2 == 1){
                soma += p[num];
            }
            k /= 2;
            num ++;
        }
        dif = min(dif, abs(2 * soma - s) );
    }

    cout << dif << '\n';
}
```

1.2 Bit Strings

Your task is to calculate the number of bit strings of length n. For example, if $n = 3$, the correct answer is 8, because the possible bit strings are 000, 001, 010, 011, 100, 101, 110, and 111.

Input
The only input line has an integer n.

Output
Print the result modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

```cpp
const ll mod = 1000000007;

ll pot(ll b, ll n){
    if(n == 0){
        return 1;
    }
    ll  k = pot(b, n/2);

    k = (k * k) % mod;

    if(n % 2 == 0){
        return k % mod;
    }

    else {
        return (b * k) % mod;
    }
}

int main(){
    ll a;
    cin >> a;
    cout << pot(2, a) << endl;
}
```

1.3 Chessboard And Queens

Your task is to place eight queens on a chessboard so that no two queens are attacking each other. As an additional challenge, each square is either free or reserved, and you can only place queens on the free squares. However, the reserved squares do not prevent queens from attacking each other. How many possible ways are there to place the queens?

Input
The input has eight lines, and each of them has eight characters. Each square is either free (.) or reserved (*).

Output
Print one integer: the number of ways you can place the queens.

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
char tab[10][10];

void damas(int n) {
    int k=0, m = 0, v[n];

    for(int i=0; i<n; i++){
        v[i] = -1;
    }

    while(1){
        v[m]++;
        if(v[m] == n){
            if(m == 0) break;
            v[m] = -1;
            m--;
            continue;
        }
        if(tab[v[m]][m] == '*') continue;

        char a = 0;
        for(int i=0; i<m; i++){
            a = v[m] == v[i];
            a = a || (v[m] == m - i + v[i]);
            a = a || (v[m] == i - m + v[i]);
            if(a){
                break;
            }
        }
        if(a) continue;
        m++;

        if(m == n){
            k++;
            m--;
        }
    }

    printf("%d\n", k);
}

void solve(){
    int n = 8;

    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            cin >> tab[i][j];
        }
    }

    damas(n);
}
```

1.4 Coin Piles

You have two coin piles containing a and b coins. On each move, you can either remove one coin from the left pile and two coins from the right pile, or two coins from the left pile and one coin from the right pile. Your task is to efficiently find out if you can empty both the piles.

Input
The first input line has an integer t: the number of tests. After this, there are t lines, each of which has two integers a and b: the numbers of coins in the piles.

Output
For each test, print "YES" if you can empty the piles and "NO" otherwise.

Constraints
• $1 \leq t \leq 10^5$ • $0 \leq a, b \leq 10^9$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> v){
```

```
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int a, b;
    cin >> a >> b;
    if((2*b - a) % 3 == 0 && 2*b - a >= 0 && 2*a - b >= 0){
        cout << "YES\n";
        return;
    }
    cout << "NO\n";
}
```

1.5 Creating Strings

Given a string, your task is to generate all different strings that can be created using its characters.

Input
The only input line has a string of length n. Each character is between a–z.

Output
First print an integer k: the number of strings. Then print k lines: the strings in alphabetical order.

Constraints
• $1 \le n \le 8$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

vector<string> palavras;

void perm(string s, string r){
    if(s == ""){
        palavras.pb(r);
        return;
    }
    int letras[26] = {0};

    for(int i = 0; i < s.size(); i++){
        if(letras[s[i] - 'a'] == 0){
            string copia = s;
            copia.erase(i, 1);
            string dig = "";
            dig += s[i];
            perm(copia, dig + r);
        }
        letras[s[i] - 'a'] ++;
    }
}

void solve(){
    string s; cin >> s;
    perm(s, "");
    cout << palavras.size() << '\n';
    sort(all(palavras));
    for(string s : palavras){
        cout << s << '\n';
    }
}
```

1.6 Digit Queries

Consider an infinite string that consists of all positive integers in increasing order: 12345678910111213141516171819202122232425... Your task is to process q queries of the form: what is the digit at position k in the string?

Input
The first input line has an integer q: the number of queries. After this, there are q lines that describe the queries. Each line has an integer k: a 1-indexed position in the string.

Output
For each query, print the corresponding digit.

Constraints
• $1 \le q \le 1000$ • $1 \le k \le 10^{18}$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
char tab[10][10];

void solve(){
    ll k, soma = 0;
    cin >> k;
    ll cont = 1, pot = 1, ant = 0;
    while(soma < k){
        ant = soma;
        soma += 9 * pot * cont;
        pot *= 10;
        cont ++;
    }
    ll copia = k;
    ll dig = cont - 1;
    pot /= 10;
    copia -= ant + 1;

    string s = to_string(copia/dig + pot);

    cout << s[copia % dig] << '\n';

}
```

1.7 Gray Code

A Gray code is a list of all 2^n bit strings of length n, where any two successive strings differ in exactly one bit (i.e., their Hamming distance is one). Your task is to create a Gray code for a given length n.

Input
The only input line has an integer n.

Output
Print 2^n lines that describe the Gray code. You can print any valid solution.

Constraints
• $1 \le n \le 16$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
int v[26];

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n; cin >> n;
    vector<int> v; v.pb(0); v.pb(1);
    int pot = 2;

    for(int i = 0; i < n - 1; i++){
        int tam = v.size();
        for(int j = tam - 1; j >= 0; j--){
            v.pb(v[j] + pot);
        }
        pot *= 2;
    }

    for(int i = 0; i < v.size(); i++){
        string s;
        for(int j = 0; j < n; j++){
            if(v[i] % 2 == 0) s.pb('0');
            if(v[i] % 2 == 1) s.pb('1');
            v[i] /= 2;
        }
        cout << s << '\n';
    }
}
```

1.8 Grid Coloring I

You are given an $n \times m$ grid where each cell contains one character A, B, C or D. For each cell, you must change the character to A, B, C or D. The new character must be different from the old one. Your task is to change the characters in every cell such that no two adjacent cells have the same character.

Input
The first line has two integers n and m: the number of rows and columns. The next n lines each have m characters: the description of the grid.

Output
Print n lines each with m characters: the description of the final grid. You may print any valid solution. If no solution exists, just print IMPOSSIBLE.

Constraints
• $1 \le n, m \le 500$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
```

```
2
3  void solve(){
4      int n, m; cin >> n >> m;
5      char M[n + 2][m + 2];
6
7      for(int i = 0; i <= n + 1; i++)
8          for(int j = 0; j <= m + 1; j++)
9              M[i][j] = 'A';
10
11     for(int i = 1; i <= n; i++)
12         for(int j = 1; j <= m; j++)
13             cin >> M[i][j];
14
15     for(int sum = 2; sum <= m + n; sum++)
16     {
17         for(int i = max(1, sum - m); i <= min(n, sum - 1); i
               ++)
18         {
19             int j = sum - i;
20             for(int k = 0; k < 4; k++)
21             {
22                 if(M[i][j] - 'A' == k || M[i - 1][j] - 'A' ==
                       k || M[i][j - 1] - 'A' == k) continue;
23                 M[i][j] = 'A' + k;
24                 break;
25             }
26         }
27     }
28
29     for(int i = 1; i <= n; i++)
30     {
31         for(int j = 1; j <= m; j++)
32         {
33             cout << M[i][j];
34         }
35         cout << '\n';
36     }
37 }
```

1.9 Grid Path Description

There are 88418 paths in a 7×7 grid from the upper-left square to the lower-left square. Each path corresponds to a 48-character description consisting of characters D (down), U (up), L (left) and R (right). For example, the path corresponds to the description DRURRRRRDDDLUULDDDLDRRURDDLLLLLURULURRUULDLLDDDD. You are given a description of a path which may also contain characters ? (any direction). Your task is to calculate the number of paths that match the description.

Input
The only input line has a 48-character string of characters ?, D, U, L and R.

Output
Print one integer: the total number of paths.

```
1  const int n = 7, n2 = 49, inf = 2e9, M = 1e9 + 7;
2  vector<string> seqs;
3  string s;
4
5  int tab[n+2][n+2], total = 0;
6
7  void completar(int x, int y, int soma, char ant){
8      if(soma != 1 && s[soma-2] != '?' && s[soma-2] != ant)
           return;
9
10     int fimh = (tab[x-1][y] == 0) + (tab[x+1][y] == 0);
11     int fimv = (tab[x][y-1] == 0) + (tab[x][y+1] == 0);
12
13     if(fimh == 0 && fimv == 2 || fimv == 0 && fimh == 2)
           return;
14
15     if(x == 1 && y == n && soma != n2) return;
16     if(soma == n2){
17         total ++;
18         return;
19     }
20
21     if(fimv + fimh == 0) return;
22
23     if(tab[x-1][y] == 0){
24         tab[x-1][y] = 1;
25         completar(x-1, y, soma+1, 'L');
26         tab[x-1][y] = 0;
27     }
28     if(tab[x+1][y] == 0){
29         tab[x+1][y] = 1;
30         completar(x+1, y, soma+1, 'R');
31         tab[x+1][y] = 0;
32     }
33     if(tab[x][y-1] == 0){
34         tab[x][y-1] = 1;
35         completar(x, y-1, soma+1, 'U');
36         tab[x][y-1] = 0;
37     }
38     if(tab[x][y+1] == 0){
39         tab[x][y+1] = 1;
40         completar(x, y+1, soma+1, 'D');
41         tab[x][y+1] = 0;
42     }
43 }
44
45 void solve(){
46     tab[1][1] = 1;
47     for(int i = 0; i <= n + 1; i ++){
48         tab[i][0] = 1;
49         tab[i][n+1] = 1;
50         tab[0][i] = 1;
51         tab[n+1][i] = 1;
52     }
53
54     cin >> s;
55     completar(1, 1, 1, 'S');
56     cout << total << '\n';
57 }
```

1.10 Increasing Array

You are given an array of n integers. You want to modify the array so that it is increasing, i.e., every element is at least as large as the previous element. On each move, you may increase the value of any element by one. What is the minimum number of moves required?

Input
The first input line contains an integer n: the size of the array. Then, the second line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the minimum number of moves.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```
1  const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
2
3  int main() {
4      ll n; cin >> n;
5      ll a = 0, total = 0;
6
7      for(int i = 0; i < n; i++){
8          ll x; cin >> x;
9          total += max(0LL, a - x);
10         a = max(a, x);
11     }
12
13     cout << total << '\n';
14
15     return 0;
16 }
```

1.11 Knight Moves Grid

There is a knight on an $n \times n$ chessboard. For each square, print the minimum number of moves the knight needs to do to reach the top-left corner.

Input
The only line has an integer n.

Output
Print the number of moves for each square.

Constraints
• $4 \le n \le 1000$

```
1  const int MAX = 1000;
2
3  int n;
4  int M[MAX][MAX];
5  vector<pair<int, int>> horse = {{+1, +2}, {-1, +2}, {+1, -2},
       {-1, -2},
6                                  {+2, +1}, {-2, +1}, {+2, -1},
                                       {-2, -1}};
7
8  bool check(int x) {
9      return (x >= 0 && x < n);
10 }
11
12 void bfs(int a, int b)
13 {
14     M[a][b] = 0;
15     queue<pair<int, int>> q;
16     q.push({a, b});
17     while(!q.empty())
18     {
19         pair<int, int> p = q.front(); q.pop();
20         for(auto x : horse)
21         {
```

```
            int x1 = p.first + x.first, y1 = p.second + x.
                 second;
            if(check(x1) && check(y1) && M[x1][y1] == -1)
            {
                q.push({x1, y1});
                M[x1][y1] = M[p.first][p.second] + 1;
            }
        }
    }
}

void solve(){
    cin >> n;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
           M[i][j] = -1;

    bfs(0,0);

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) cout << M[i][j] << ' ';
        cout << '\n';
    }
}
```

1.12 Mex Grid Construction

Your task is to construct an $n \times n$ grid where each square has the smallest nonnegative integer that does not appear to the left on the same row or above on the same column.

Input
The only line has an integer n.

Output
Print the grid according to the example.

Constraints
- $1 \le n \le 100$

```
void solve(){
    int n; cin >> n;
    int M[n][n];

    vector<vector<bool>> col(n);
    for(int i = 0; i < n; i++)  {
        M[i][0] = M[0][i] = i;
        col[i].resize(2 * n, 0);
        col[i][i] = 1;
    }

    for(int i = 1; i < n; i++) {
        vector<bool> linha(2 * n, 0);
        linha[i] = 1;
        for(int j = 1; j < n; j++) {
            for(int k = 0; k < 2 * n; k++) {
                if(linha[k] == 0 && col[j][k] == 0)  {
                    M[i][j] = k;
                    break;
                }
            }
            col[j][M[i][j]] = true;
            linha[M[i][j]] = true;
        }
    }
```

```

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) cout << M[i][j] << ' ';
        cout << '\n';
    }
}
```

1.13 Missing Number

You are given all numbers between $1, 2, \ldots, n$ except one. Your task is to find the missing number.

Input
The first input line contains an integer n. The second line contains $n - 1$ numbers. Each number is distinct and between 1 and n (inclusive).

Output
Print the missing number.

Constraints
- $2 \le n \le 2 \cdot 10^5$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int v[maxn];

int main() {
    int n; cin >> n;
    v[0] = 1;
    for(int i = 0; i < n; i++){
        int x; cin >> x;
        v[x] = 1;
    }

    for(int i = 0; i <= n; i++){
        if(v[i] != 1) cout << i << '\n';
    }

    return 0;
}
```

1.14 Number Spiral

A number spiral is an infinite grid whose upper-left square has number 1. Here are the first five layers of the spiral:
Your task is to find out the number in row y and column x.

Input
The first input line contains an integer t: the number of tests. After this, there are t lines, each containing integers y and x.

Output
For each test, print the number in row y and column x.

Constraints

- $1 \le t \le 10^5$ • $1 \le y, x \le 10^9$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(){
    ll x, y; cin >> y >> x;
    ll a = max(x, y);
    ll resp = (a-1)*(a-1);

    if(a % 2 == 0) resp += y + a - x;
    else resp += x + a - y;

    cout << resp << '\n';
}
```

1.15 Palindrome Reorder

Given a string, your task is to reorder its letters in such a way that it becomes a palindrome (i.e., it reads the same forwards and backwards).

Input
The only input line has a string of length n consisting of characters A–Z.

Output
Print a palindrome consisting of the characters of the original string. You may print any valid solution. If there are no solutions, print "NO SOLUTION".

Constraints
- $1 \le n \le 10^6$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
int v[26];

void printv(vector<int> &v) {
    for(int i=0; i < v.size(); i++) cout << v[i] << ' ';
    cout << '\n';
}

void solve(){
    string s; cin >> s;
    int a = 0, meio = -1;

    for(char c : s) v[c - 'A']++;

    for(int i=0; i<26; i++) {
        if(v[i] % 2 == 1) {
            a++;
            meio = i;
        }
    }

    s = "";

    if(a >= 2){
        cout << "NO SOLUTION\n";
        return;
    }
```

```
    for(int i=0; i<26; i++)
        for(int j = 0; j < v[i]/2; j++)
            s.pb((char) ('A' + i));

    cout << s;
    if(meio != -1) cout << (char) ('A' + meio);
    reverse(s.begin(), s.end());
    cout << s << '\n';
}
```

1.16 Permutations

A permutation of integers $1, 2, \ldots, n$ is called beautiful if there are no adjacent elements whose difference is 1. Given n, construct a beautiful permutation if such a permutation exists.

Input
The only input line contains an integer n.

Output
Print a beautiful permutation of integers $1, 2, \ldots, n$. If there are several solutions, you may print any of them. If there are no solutions, print "NO SOLUTION".

Constraints
• $1 \leq n \leq 10^6$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int main() {
    ll n; cin >> n;
    if(n == 1) {
        cout << "1\n";
        return 0;
    }
    if(n < 4) {
        cout << "NO␣SOLUTION\n";
        return 0;
    }
    if(n == 4) {
        cout << "2␣4␣1␣3\n";
        return 0;
    }

    vector<int> v1, v2;

    for(int i = 1; i <= n/2; i++) v1.pb(i);
    for(int i = n/2 + 1; i <= n; i++) v2.pb(i);

    for(int i = 0; i < n; i++) {
        if(i % 2 == 1) {
            cout << v1.back() << '␣';
            v1.pop_back();
        }
        else {
            cout << v2.back() << '␣';
            v2.pop_back();
        }
    }

    cout << '\n';

    return 0;
}
```

1.17 Raab Game I

Consider a two player game where each player has n cards numbered $1, 2, \ldots, n$. On each turn both players place one of their cards on the table. The player who placed the higher card gets one point. If the cards are equal, neither player gets a point. The game continues until all cards have been played. You are given the number of cards n and the players' scores at the end of the game, a and b. Your task is to give an example of how the game could have played out.

Input
The first line contains one integer t: the number of tests. Then there are t lines, each with three integers n, a and b.

Output
For each test case print YES if there is a game with the given outcome and NO otherwise. If the answer is YES, print an example of one possible game. Print two lines representing the order in which the players place their cards. You can give any valid example.

Constraints
• $1 \leq t \leq 1000$ • $1 \leq n \leq 100$ • $0 \leq a, b \leq n$

```
bool comparator(int a, int b) {
    cout << "?␣" << a << '␣' << b << endl;
    string ans; cin >> ans;
    if(ans == "YES") return true;
    else return false;
}

void solve() {
    int n; cin >> n;
    int a, b; cin >> a >> b;

    if((a == 0 && b != 0) || (b == 0 && a != 0)){
        cout << "NO\n";
        return;
    }
    if(a == 0 && b == 0){
        cout << "YES\n";
        for(int i = 1; i <= n; i++) cout << i << '␣';
        cout << '\n';
        for(int i = 1; i <= n; i++) cout << i << '␣';
        cout << '\n';
        return;
    }
    if(a + b > n){
        cout << "NO\n";
        return;
    }
    if(b == 1){
        cout << "YES\n";
        for(int i = 1; i <= n; i++) cout << i << '␣';
        cout << '\n';
        cout << a + 1 << '␣';
        for(int i = 1; i <= a; i++) cout << i << '␣';
        for(int i = a + 2; i <= n; i++) cout << i << '␣';
        cout << '\n';
        return;
    }
    if(a == 1){
        swap(a, b);
        cout << "YES\n";
        cout << a + 1 << '␣';
        for(int i = 1; i <= a; i++) cout << i << '␣';
        for(int i = a + 2; i <= n; i++) cout << i << '␣';
        cout << '\n';
        for(int i = 1; i <= n; i++) cout << i << '␣';
        cout << '\n';
        return;
    }

    cout << "YES\n";
    for(int i = 1; i <= a; i++) cout << i << '␣';
    cout << a + b << '␣';
    for(int i = 1; i <= b - 1; i++) cout << i + a << '␣';
    for(int i = a + b + 1; i <= n; i++) cout << i << '␣';
    cout << '\n';

    cout << a << '␣';
    for(int i = 1; i <= a - 1; i++) cout << i << '␣';
    for(int i = 1; i <= b; i++) cout << i + a << '␣';
    for(int i = a + b + 1; i <= n; i++) cout << i << '␣';
    cout << '\n';
}
```

1.18 Repetitions

You are given a DNA sequence: a string consisting of characters A, C, G, and T. Your task is to find the longest repetition in the sequence. This is a maximum-length substring containing only one type of character.

Input
The only input line contains a string of n characters.

Output
Print one integer: the length of the longest repetition.

Constraints
• $1 \leq n \leq 10^6$

```
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int v[maxn];

int main() {
    string s; cin >> s;
    s.pb('Z');
    int atual = 1, maxi = 0;
    char ant = 'z';

    for(char c : s) {
        if(c == ant) {
            atual ++;
        }
        else {
            ant = c;
            maxi = max(maxi, atual);
            atual = 1;
        }
    }

    cout << maxi << '\n';

    return 0;
}
```

1.19 String Reorder

Your task is to reorder the characters of a string so that no two adjacent characters are the same. What is the lexicographically minimal such string?

Input
The only line has a string of length n consisting of characters A–Z.

Output
Print the lexicographically minimal reordered string where no two adjacent characters are the same. If it is not possible to create such a string, print -1.

Constraints
- $1 \leq n \leq 10^6$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

bool check(vector<int> &v) {
    int sum = 0;
    for(int i = 0; i < 26; i++) {
        sum += v[i];
    }
    for(int i = 0; i < 26; i++) {
        if(v[i] > (sum + 1) / 2) {
            return false;
        }
    }
    return true;
}

int find_best(char pst, vector<int> &v) {
    int best = -1;
    for(int i = 0; i < 26; i++) {
        if(pst == 'A' + i || v[i] == 0) continue;
        v[i]--;
        if(check(v)) {
            best = i;
            break;
        }
        v[i]++;
    }
    return best;
}

void solve(){
    string s; cin >> s;
    vector<int> v(26);
    for(int i = 0; i < s.size(); i++){
        v[s[i] - 'A']++;
    }
    string ans = "";
    char pst = '$';

    while(ans.size() < s.size()) {
        int novo = find_best(pst, v);
        if(novo == -1) {
            cout << -1 << '\n';
            return;
        }
        ans += 'A' + novo;
        pst = 'A' + novo;
    }

    cout << ans << '\n';
}
```

1.20 Tower Of Hanoi

The Tower of Hanoi game consists of three stacks (left, middle and right) and n round disks of different sizes. Initially, the left stack has all the disks, in increasing order of size from top to bottom. The goal is to move all the disks to the right stack using the middle stack. On each move you can move the uppermost disk from a stack to another stack. In addition, it is not allowed to place a larger disk on a smaller disk. Your task is to find a solution that minimizes the number of moves.

Input
The only input line has an integer n: the number of disks.

Output
First print an integer k: the minimum number of moves. After this, print k lines that describe the moves. Each line has two integers a and b: you move a disk from stack a to stack b.

Constraints
- $1 \leq n \leq 16$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void hanoi(int n, int ini, int fim){
    if(n == 0) return;
    int mid = 6 - ini - fim;

    hanoi(n - 1, ini, mid);
    cout << ini << ' ' << fim << '\n';
    hanoi(n - 1, mid, fim);
}

void solve(){
    int n; cin >> n;
    int total = 1;
    for(int i = 0; i < n; i++) total *= 2;
    total--;
    cout << total << '\n';
    hanoi(n, 1, 3);
}
```

1.21 Trailing Zeros

Your task is to calculate the number of trailing zeros in the factorial $n!$. For example, $20! = 2432902008176640000$ and it has 4 trailing zeros.

Input
The only input line has an integer n.

Output
Print the number of trailing zeros in $n!$.

Constraints
- $1 \leq n \leq 10^9$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> &v) {
    for(int i=0; i < v.size(); i++) {
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n; cin >> n;
    int total = 0;
    for(int i = 5; i < M; i *= 5) {
        total += n / i;
    }
    cout << total << '\n';
}
```

1.22 Two Knights

Your task is to count for $k = 1, 2, \ldots, n$ the number of ways two knights can be placed on a $k \times k$ chessboard so that they do not attack each other.

Input
The only input line contains an integer n.

Output
Print n integers: the results.

Constraints
- $1 \leq n \leq 10000$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(ll n){
    ll total = (n * n) * (n * n - 1);

    total -= 4 * 2;
    total -= 8 * 3;
    total -= (4 * (n - 3)) * 4;
    total -= (4 * (n - 4)) * 6;
    total -= ((n - 4) * (n - 4)) * 8;

    cout << total / 2 << '\n';
}

int main() {
    int t; cin >> t; for(int i = 1; i <= t; i++)
    solve(i);
    return 0;
}
```

1.23 Two Sets

Your task is to divide the numbers $1, 2, \ldots, n$ into two sets of equal sum.

Input
The only input line contains an integer n.

Output
Print "YES", if the division is possible, and "NO" otherwise. After this, if the division is possible, print an example of how to create the sets. First, print the number of elements in the first set followed by the elements themselves in a separate line, and then, print the second set in a similar way.

Constraints
- $1 \leq n \leq 10^6$

```cpp
const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n;
    cin >> n;
    if(n % 4 == 2 || n % 4 == 1){
        cout << "NO\n";
        return;
    }
    cout << "YES\n";
    vector<int> v1;
    vector<int> v2;
    if(n % 4 == 3){
        v1.pb(1); v1.pb(n-1); v2.pb(n);
        for(int i = 2; i <= (n - 1) / 2; i++){
            if(i%2 == 1){
                v1.pb(i); v1.pb(n-i);
            }
            else{
                v2.pb(i); v2.pb(n-i);
            }
        }
    }

    if(n % 4 == 0){
        for(int i = 1; i <= (n) / 2; i++){
            if(i%2 == 1){
                v1.pb(i); v1.pb(n+1-i);
            }
            else{
                v2.pb(i); v2.pb(n+1-i);
            }
        }
    }

    cout << v1.size() << '\n'; printv(v1);
    cout << v2.size() << '\n'; printv(v2);
}
```

1.24 Weird Algorithm

Consider an algorithm that takes as input a positive integer n. If n is even, the algorithm divides it by two, and if n is odd, the algorithm multiplies it by three and adds one. The algorithm repeats this, until n is one. For example, the sequence for $n = 3$ is as follows: $3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$ Your task is to simulate the execution of the algorithm for a given value of n.

Input
The only input line contains an integer n.

Output
Print a line that contains all values of n during the algorithm.

Constraints
- $1 \leq n \leq 10^6$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

int main() {
    ll n; cin >> n;
    while(n != 1){
        cout << n << ' ';
        if(n % 2 == 0) n = n/2;
        else n = 3*n + 1;
    }
    cout << "1\n";
    return 0;
}
```

2 Sorting And Searching

2.1 Apartments

There are n applicants and m free apartments. Your task is to distribute the apartments so that as many applicants as possible will get an apartment. Each applicant has a desired apartment size, and they will accept any apartment whose size is close enough to the desired size.

Input
The first input line has three integers n, m, and k: the number of applicants, the number of apartments, and the maximum allowed difference. The next line contains n integers $a_1, a_2, \ldots, a_n$: the desired apartment size of each applicant. If the desired size of an applicant is x, they will accept any apartment whose size is between $x-k$ and $x+k$. The last line contains m integers $b_1, b_2, \ldots, b_m$: the size of each apartment.

Output
Print one integer: the number of applicants who will get an apartment.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $0 \leq k \leq 10^9$ • $1 \leq a_i, b_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, m, k;
    cin >> n >> m >> k;
    int a[n], b[m];

    for(int i=0; i<n; i++) cin >> a[i];
    for(int i=0; i<m; i++) cin >> b[i];

    sort(a, a+n);
    sort(b, b+m);

    int atual = 0, total = 0;

    for(int i=0; i<m; i++){
        if(atual >= n) break;

        if(abs(b[i] - a[atual]) <= k){
            atual ++;
            total ++;
        }
        else if(b[i] - a[atual] > k){
            atual ++;
            i --;
        }
    }

    cout << total << '\n';
}
```

2.2 Array Division

You are given an array containing n positive integers. Your task is to divide the array into k subarrays so that the maximum sum in a subarray is as small as possible.

Input
The first input line contains two integers n and k: the size of the array and the number of subarrays in the division. The next line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum sum in a subarray in the optimal division.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k \leq n$ • $1 \leq x_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

bool ans(vector<int> v, ll m, int k){
    int n = v.size();
    ll at = 0;
    int cont = 0;

    for(int i = 0; i < n; i++){
        if(cont > k) break;
        if(at + v[i] > m){
            at = v[i];
            if(at > m){
                cont = k + 1;
                break;
            }
            cont++;
        }
        else{
            at += v[i];
        }
    }
    if(at > 0) cont++;

    return cont <= k;
}

void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    ll l = 1, r = 1;
    for(int i = 0; i < n; i++){
        cin >> v[i];
        r   += v[i];
    }

    while(r - l > 1){
        ll m = (l + r) / 2;

        if(ans(v, m, k)) r = m;
        else l = m;
    }

    cout << r << '\n';
}
```

2.3 Collecting Numbers

You are given an array that contains each number between $1 \dots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order. On each round, you go through the array from left to right and collect as many numbers as possible. What will be the total number of rounds?

Input
The first line has an integer n: the array size. The next line has n integers $x_1, x_2, \dots, x_n$: the numbers in the array.

Output
Print one integer: the number of rounds.

Constraints
- $1 \le n \le 2 \cdot 10^5$

```
const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;

int32_t main(){
    int n; cin >> n;
    pair<int, int> p[n + 1];
    for(int i = 0; i < n; i++){
        cin >> p[i].first;
        p[i].second = i;
    }
    sort(p, p + n);
    p[n] = {-1, -1};

    int at = 0;

    for(int i = 0; i < n; i++){
        if(p[i].second > p[i+1].second) at++;
    }

    cout << at << '\n';
}
```

2.4 Collecting Numbers II

You are given an array that contains each number between $1 \dots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order. On each round, you go through the array from left to right and collect as many numbers as possible. Given m operations that swap two numbers in the array, your task is to report the number of rounds after each operation.

Input
The first line has two integers n and m: the array size and the number of operations. The next line has n integers $x_1, x_2, \dots, x_n$: the numbers in the array. Finally, there are m lines that describe the operations. Each line has two integers a and b: the numbers at positions a and b are swapped.

Output
Print m integers: the number of rounds after each swap.

Constraints
- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

```
const int M = 1e9 + 7;

void printv (vector<int> v){
    for(int x : v) cout << x << '␣';
    cout << '\n';
}
void printvpii (vector<pii> v){
    for(pii x : v) cout << x.second << "␣";
    cout << '\n';
}

int main () { _
    int n, m; cin >> n >> m;
    vector<int> v1(n);
    vector<pii> v2(n);
    for(int i = 0; i < n; i++){
        cin >> v1[i];
        v2[i] = {v1[i], i+1};
    }
    sort(v2.begin(), v2.end());
    v2.pb({-1, -1});
    int total = 0;
    for(int i = 0; i < n; i++)
        if(v2[i+1].S < v2[i].S) total++;

    for(int i = 0; i < m; i++){
        int a, b; cin >> a >> b; a--; b--;
        int x = v1[a]-1, y = v1[b]-1;
        swap(v1[a], v1[b]);
        if(x > y) swap(x, y);
        total -= (v2[x].second > v2[x+1].second) + (v2[y].
              second > v2[y+1].second) + (v2[y-1].second > v2
              [y].second);
        if(x > 0) total -= (v2[x-1].second > v2[x].second);
        if(y == x + 1) total += v2[x].second > v2[y].second;
        swap(v2[x].second, v2[y].second);
        total += (v2[x].second > v2[x+1].second) + (v2[y].
              second > v2[y+1].second) + (v2[y-1].second > v2
              [y].second);
        if(x > 0) total += (v2[x-1].second > v2[x].second);
        if(y == x + 1) total -= v2[x].second > v2[y].second;
        cout << total << '\n';
    }

    return 0;
}
```

2.5 Concert Tickets

There are n concert tickets available, each with a certain price. Then, m customers arrive, one after another. Each customer announces the maximum price they are willing to pay for a ticket, and after this, they will get a ticket with the nearest possible price such that it does not exceed the maximum price.

Input
The first input line contains integers n and m: the number of tickets and the number of customers. The next line contains n integers $h_1, h_2, \dots, h_n$: the price of each ticket. The last line contains m integers $t_1, t_2, \dots, t_m$: the maximum price for each customer in the order they arrive.

Output
Print, for each customer, the price that they will pay for their ticket. After this, the ticket cannot be purchased again. If a customer cannot get any ticket, print -1.

Constraints
- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le h_i, t_i \le 10^9$

```
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, m; cin >> n >> m;
    multiset<int> h;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        h.insert(-a);
    }

    for(int i=0; i<m; i++){
        int t; cin >> t;
        auto lwb = h.lower_bound(-t);

        if(lwb == h.end()) cout << "-1\n";
        else{
            cout << -*lwb << '\n';
            h.erase(lwb);
        }
    }
}
```

2.6 Distinct Numbers

You are given a list of n integers, and your task is to calculate the number of distinct values in the list.

Input
The first input line has an integer n: the number of values. The second line has n integers $x_1, x_2, \dots, x_n$.

Output
Print one integers: the number of distinct values.

Constraints
- $1 \le n \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$

```
int main(){
    int n, x;
    set<int> s;
    scanf("%d", &n);
    for(int i=0; i<n; i++){
        scanf("%d", &x);
        s.insert(x);
    }
    printf("%d\n", s.size());
}
```

2.7 Distinct Values Subarrays

Given an array of n integers, count the number of subarrays where each element is distinct.

Input
The first line has an integer n: the array size. The second line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print the number of subarrays with distinct elements.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n; cin >> n;
    vector<int> x(n);

    for(int i = 0; i < n; i++) cin >> x[i];

    set<int> at;
    ll p1 = 0, p2 = 0;
    ll ans = 0;

    for(p1 = 0; p1 < n; p1++) {
        while(p2 < n && at.find(x[p2]) == at.end()) {
            at.insert(x[p2]);
            p2++;
        }

        ans += (p2 - p1);
        at.erase(x[p1]);
    }

    cout << ans << '\n';
}
```

2.8 Distinct Values Subarrays II

Given an array of n integers, your task is to calculate the number of subarrays that have at most k distinct values.

Input
The first input line has two integers n and k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of subarrays.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, k; cin >> n >> k;
    vector<int> v;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        v.pb(a);
    }

    map<int, int> rep;

    int p1 = 0, p2 = 0;
    ll cont = 0;
    int at = 0;
    int dis = 0;

    while(p1 < n || p2 < n){
        if(p2 == n || (rep[v[p2]] == 0 && dis >= k)){
            rep[v[p1]] --;
            if(rep[v[p1]] == 0) dis --;
            cont += at;
            at --;
            p1 ++;
        }
        else if(rep[v[p2]] == 0 && dis < k){
            rep[v[p2]] = 1;
            dis ++;
            at ++;
            p2 ++;
        }
        else{
            rep[v[p2]] ++;
            at ++;
            p2 ++;
        }
    }

    cout << cont << '\n';
}
```

2.9 Distinct Values Subsequences

Given an array of n integers, count the number of subsequences where each element is distinct. A subsequence is a sequence of array elements from left to right that may have gaps.

Input
The first line has an integer n: the array size. The second line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print the number of subsequences with distinct elements. The answer can be large, so print it modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n; cin >> n;
    map<int, int> mapa;

    for(int i = 0; i < n; i++) {
        int a;
        cin >> a;
        mapa[a]++;
    }

    ll ans = 1;
    for(auto x : mapa)
        ans = (ans * (x.second + 1)) % M;

    cout << ans - 1 << '\n';
}
```

2.10 Factory Machines

A factory has n machines which can be used to make products. Your goal is to make a total of t products. For each machine, you know the number of seconds it needs to make a single product. The machines can work simultaneously, and you can freely decide their schedule. What is the shortest time needed to make t products?

Input
The first input line has two integers n and t: the number of machines and products. The next line has n integers $k_1, k_2, \ldots, k_n$: the time needed to make a product using each machine.

Output
Print one integer: the minimum time needed to make t products.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq t \leq 10^9$ • $1 \leq k_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

__int128_t ans(vector<ll> k, ll m){
    __int128_t sum = 0;
    for(auto x : k) sum += m / x;
    return sum;
}

void solve(){
    ll n, t; cin >> n >> t;
    vector<ll> k(n);

    for(int i = 0; i < n; i++){
        cin >> k[i];
    }

    ll l = 0, r = linf, m;

    while(r - l > 1){
        m = (l + r) / 2;
        __int128_t sum = ans(k, m);
        if(sum >= t) r = m;
        else l = m;
    }

    cout << r << '\n';
}
```

2.11 Ferris Wheel

There are n children who want to go to a Ferris wheel, and your task is to find a gondola for each child. Each gondola may have one or two children in it, and in addition, the total weight in a gondola may not exceed x. You know the weight of every child. What is the minimum number of gondolas needed for the children?

Input
The first input line contains two integers n and x: the number of children and the maximum allowed weight. The next line contains n integers $p_1, p_2, \ldots, p_n$: the weight of each child.

Output
Print one integer: the minimum number of gondolas.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x \le 10^9$ • $1 \le p_i \le x$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, x;
    cin >> n >> x;
    int p[n];

    for(int i=0; i<n; i++) cin >> p[i];
    sort(p, p+n);

    int fim = n-1, inicio = 0, total = n;

    while(fim > inicio){
        if(p[fim] + p[inicio] <= x){
            fim --;
            inicio ++;
            total --;
        }
        else{
            fim --;
        }
    }

    cout << total << '\n';
}
```

2.12 Josephus Problem I

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, every other child is removed from the circle until there are no children left. In which order will the children be removed?

Input
The only input line has an integer n.

Output
Print n integers: the removal order.

Constraints
• $1 \le n \le 2 \cdot 10^5$

```cpp
int main(){
    int n;
    vector<int> a, b, resp;
    cin >> n;

    for(int i=0; i<n; i++){
        a.push_back(i+1);
    }

    int i=1;
    while(a.size()>0){
        if(a.size() == 1){
            resp.push_back(a[0]);
            break;
        }
        for(; i<a.size(); i+=2){
            if(i != 0) b.push_back(a[i-1]);
            resp.push_back(a[i]);
        }

        if(i == a.size()){
            b.push_back(a[a.size()-1]);
            i = 0;
        }else{
            i = 1;
        }
        a = b;
        b.clear();
    }

    for(int i=0; i<n; i++){
        cout << resp[i] << ' ';
    }
    cout << '\n';

    return 0;
}
```

2.13 Josephus Problem II

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, repeatedly k children are skipped and one child is removed from the circle. In which order will the children be removed?

Input
The only input line has two integers n and k.

Output
Print n integers: the removal order.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le k \le 10^9$

```cpp
ordered_set s;

int main() {
  int n, k;
  cin >> n >> k;
  for(int i = 1; i <= n; i++) s.insert(i);
  int at = 0;
  vector<int> v;

  while(s.size() != 0){
    at = (at + k) % s.size();
    auto it = s.find_by_order(at);
    v.push_back(*it);
    s.erase(it);
  }

  for(auto x : v) cout << x << ' ';
  cout << '\n';

}
```

2.14 Maximum Subarray Sum

Given an array of n integers, your task is to find the maximum sum of values in a contiguous, nonempty subarray.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print one integer: the maximum subarray sum.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le x_i \le 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    ll prefix = 0;
    ll minimo = 0;
    ll resp = -inf;

    for(int i = 0; i < n; i ++){
        int a; cin >> a;
        prefix += a;
        resp = max(resp, prefix - minimo);
        minimo = min(minimo, prefix);
    }

    cout << resp << '\n';
}
```

2.15 Maximum Subarray Sum II

Given an array of n integers, your task is to find the maximum sum of values in a contiguous subarray with length between a and b.

Input
The first input line has three integers n, a and b: the size of the array and the minimum and maximum subarray length. The

second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print one integer: the maximum subarray sum.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a \leq b \leq n$ • $-10^9 \leq x_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
const ll linf = 4e18;

void solve() {
    int n, a, b; cin >> n >> a >> b;
    vector<ll> prefix(n + 1); prefix[0] = 0;

    for(int i = 1; i <= n; i++) {
        cin >> prefix[i];
        prefix[i] += prefix[i - 1];
    }

    multiset<ll> minimo;
    ll resp = -linf;

    for(int i = a; i <= n; i++){
        ll at = prefix[i];

        minimo.insert(prefix[i - a]);
        if(i > b) minimo.erase(minimo.find(prefix[i - b - 1])
            );

        resp = max(resp, at - *minimo.begin());
    }

    cout << resp << '\n';
}
```

2.16 Missing Coin Sum

You have n coins with positive integer values. What is the smallest sum you cannot create using a subset of the coins?

Input
The first line has an integer n: the number of coins. The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin.

Output
Print one integer: the smallest coin sum.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;

int32_t main(){
    int n; cin >> n;
    int p[n];
    for(int i = 0; i < n; i++){
        cin >> p[i];
    }
    sort(p, p + n);

    int at = 1;

    for(int i = 0; i < n; i++){
        if(at >= p[i]) at += p[i];
        else break;
    }

    cout << at << '\n';
}
```

2.17 Movie Festival

In a movie festival n movies will be shown. You know the starting and ending time of each movie. What is the maximum number of movies you can watch entirely?

Input
The first input line has an integer n: the number of movies. After this, there are n lines that describe the movies. Each line has two integers a and b: the starting and ending times of a movie.

Output
Print one integer: the maximum number of movies.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a < b \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a, b;
        cin >> a >> b;

        v.pb({b, a});
    }

    sort(v.begin(), v.end());

    int total = 0, atual = 0;

    for(auto x : v){
        int ini = x.second, ed = x.first;
        if(ini >= atual){
            atual = ed;
            total++;
        }
    }

    cout << total << '\n';
}
```

2.18 Movie Festival II

In a movie festival, n movies will be shown. Syrjälä's movie club consists of k members, who will be all attending the festival. You know the starting and ending time of each movie. What is the maximum total number of movies the club members can watch entirely if they act optimally?

Input
The first input line has two integers n and k: the number of movies and club members. After this, there are n lines that describe the movies. Each line has two integers a and b: the starting and ending time of a movie.

Output
Print one integer: the maximum total number of movies.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq a < b \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, k; cin >> n >> k;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a, b;
        cin >> a >> b;

        v.pb({b, a});
    }

    sort(v.begin(), v.end());

    int total = 0;
    multiset<int> atual;

    for(auto x : v){
        int ini = x.second, ed = x.first;

        if(atual.empty()){
            atual.insert(ed);
            total ++;
            continue;
        }

        if(ini >= *atual.begin()){
            auto it = atual.lower_bound(ini);
            if(it == atual.end() || *it > ini) it = prev(it);
            atual.erase(it);
            atual.insert(ed);
            total++;
        }
        else if(atual.size() < k) {
            atual.insert(ed);
            total++;
        }
    }

    cout << total << '\n';
}
```

2.19 Nearest Smaller Values

Given an array of n integers, your task is to find for each array position the nearest position to its left having a smaller value.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print n integers: for each array position the nearest position with a smaller value. If there is no such position, print 0.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

bool cmp(pair<int, int> a, pair<int, int> b){
    if(a.first < b.first) return true;
    else if(a.first == b.first){
        return a.second > b.second;
    }
    return false;
}

void solve(){
    int n; cin >> n;
    vector<pair<int, int>> a(n + 1);
    a[0] = {0, 0};
    for(int i = 1; i <= n; i++){
        cin >> a[i].first;
        a[i].second = i;
    }

    sort(a.begin(), a.end(), cmp);

    set<int> s;
    s.insert(0);

    vector<int> res(n + 1);

    for(int i = 1; i <= n; i++){
        auto it = s.lower_bound(a[i].second);
        it--;
        res[a[i].second] = *it;
        s.insert(a[i].second);
    }

    for(int i = 1; i <= n; i++) cout << res[i] << ' ';
    cout << '\n';
}
```

2.20 Nested Ranges Check

Given n ranges, your task is to determine for each range if it contains some other range and if some other range contains it. Range $[a, b]$ contains range $[c, d]$ if $a \leq c$ and $d \leq b$.

Input
The first input line has an integer n: the number of ranges. After this, there are n lines that describe the ranges. Each line

has two integers x and y: the range is $[x, y]$. You may assume that no range appears more than once in the input.

Output
First print a line that describes for each range (in the input order) if it contains some other range (1) or not (0). Then print a line that describes for each range (in the input order) if some other range contains it (1) or not (0).

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x<y \leq 10^9$

```cpp
bool comp(pii a, pii b){
    if(a.F < b.F) return 1;
    else if(a.F == b.F){
        return a.S > b.S;
    }
    return 0;
}

void solve() {
    int n; cin >> n;
    pair<int, int> v[n];
    map<pii, int> mapa;
    for(int i = 0; i < n; i++){
        int a, b; cin >> a >> b;
        v[i] = {a, b};
        mapa[v[i]] = i;
    }
    sort(v, v + n, comp);

    ordered_set s1;
    int contido[n];

    for(int i = 0; i < n; i++){
        contido[i] =  i - s1.order_of_key(v[i].second);
        s1.insert(v[i].second);
    }

    ordered_set s2;
    int contem[n];

    for(int i = n - 1; i >= 0; i--){
        contem[i] =  s2.order_of_key(v[i].second + 1);
        s2.insert(v[i].second);
    }

    int transf[n];

    for(int i = 0; i < n; i++){
        transf[mapa[v[i]]] = i;
    }

    for(int i = 0; i < n; i++) contem[transf[i]] == 0 ?  cout
        << 0 << '␣' : cout <<  1 << '␣';
    cout << '\n';
    for(int i = 0; i < n; i++) contido[transf[i]] == 0 ? cout
        << 0 << '␣' : cout << 1 << '␣';
    cout << '\n';

}
```

2.21 Nested Ranges Count

Given n ranges, your task is to count for each range how many other ranges it contains and how many other ranges contain it. Range $[a, b]$ contains range $[c, d]$ if $a \leq c$ and $d \leq b$.

Input
The first input line has an integer n: the number of ranges. After this, there are n lines that describe the ranges. Each line has two integers x and y: the range is $[x, y]$. You may assume that no range appears more than once in the input.

Output
First print a line that describes for each range (in the input order) how many other ranges it contains. Then print a line that describes for each range (in the input order) how many other ranges contain it.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x<y \leq 10^9$

```cpp
bool comp(pii a, pii b){
    if(a.F < b.F) return 1;
    else if(a.F == b.F){
        return a.S > b.S;
    }
    return 0;
}

void solve() {
    int n; cin >> n;
    pair<int, int> v[n];
    map<pii, int> mapa;
    for(int i = 0; i < n; i++){
        int a, b; cin >> a >> b;
        v[i] = {a, b};
        mapa[v[i]] = i;
    }
    sort(v, v + n, comp);

    ordered_set s1;
    int contido[n];

    for(int i = 0; i < n; i++){
        contido[i] =  i - s1.order_of_key(v[i].second);
        s1.insert(v[i].second);
    }

    ordered_set s2;
    int contem[n];

    for(int i = n - 1; i >= 0; i--){
        contem[i] =  s2.order_of_key(v[i].second + 1);
        s2.insert(v[i].second);
    }

    int transf[n];

    for(int i = 0; i < n; i++){
        transf[mapa[v[i]]] = i;
    }

    for(int i = 0; i < n; i++) cout << contem[transf[i]] << '
        ␣';
    cout << '\n';
    for(int i = 0; i < n; i++) cout << contido[transf[i]] <<
        '␣';
    cout << '\n';
```

```
46
47 }
```

2.22 Playlist

You are given a playlist of a radio station since its establishment. The playlist has a total of n songs. What is the longest sequence of successive songs where each song is unique?

Input
The first input line contains an integer n: the number of songs. The next line has n integers $k_1, k_2, \ldots, k_n$: the id number of each song.

Output
Print the length of the longest sequence of unique songs.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k_i \leq 10^9$

```cpp
const int M = 1e9 + 7;

int main () { _
    int n; cin >> n;
    vector<int> k(n);

    int cnt = 0;
    map<int, int> mp;

    for(int i = 0; i < n; i++){
        int x; cin >> x;
        if(mp.find(x) == mp.end()){
            cnt++;
            mp[x] = cnt;
        }
        k[i] = mp[x];
    }

    vector<int> f(cnt+1);
    for(int i=0; i<=cnt; i++){
        f[i] = -1;
    }

    int ini = 0, fim = 0;
    int M = 0;

    while(ini <= n-1 && fim <= n-1){
        if(ini <= f[k[fim]] && f[k[fim]] <= fim){
            ini = f[k[fim]] + 1;
        }
        f[k[fim]] = fim;
        M = max(M, fim - ini + 1);
        fim++;
    }

    cout << M << '\n';

    return 0;
}
```

2.23 Reading Books

There are n books, and Kotivalo and Justiina are going to read them all. For each book, you know the time it takes to read it. They both read each book from beginning to end, and they cannot read a book at the same time. What is the minimum total time required?

Input
The first input line has an integer n: the number of books. The second line has n integers $t_1, t_2, \ldots, t_n$: the time required to read each book.

Output
Print one integer: the minimum total time.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq t_i \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

void solve(){
    int n; cin >> n;
    vector<int> a(n);

    for(int i = 0; i < n; i++){
        cin >> a[i];
    }

    sort(a.begin(), a.end());
    reverse(a.begin(), a.end());

    ll soma = 0;

    // se a soma > 2 * maior, tem como reorganizar para todo
        mundo ler o mais rapido possivel

    for(int i = 0; i < n; i++){
        soma += a[i];
    }

    if(soma >= 2 * a[0]) cout << soma << '\n';
    else cout << 2 * a[0] << '\n';
}
```

2.24 Restaurant Customers

You are given the arrival and leaving times of n customers in a restaurant. What was the maximum number of customers in the restaurant at any time?

Input
The first input line has an integer n: the number of customers. After this, there are n lines that describe the customers. Each line has two integers a and b: the arrival and leaving times of a customer. You may assume that all arrival and leaving times are distinct.

Output

Print one integer: the maximum number of customers.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a<b \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a, b;
        cin >> a >> b;

        v.pb({a, 1});
        v.pb({b, -1});
    }

    sort(v.begin(), v.end());

    int maxi = 0;
    int total = 0;

    for(auto p : v){
        total += p.second;
        maxi = max(maxi, total);
    }

    cout << maxi << '\n';
}
```

2.25 Room Allocation

There is a large hotel, and n customers will arrive soon. Each customer wants to have a single room. You know each customer's arrival and departure day. Two customers can stay in the same room if the departure day of the first customer is earlier than the arrival day of the second customer. What is the minimum number of rooms that are needed to accommodate all customers? And how can the rooms be allocated?

Input
The first input line contains an integer n: the number of customers. Then there are n lines, each of which describes one customer. Each line has two integers a and b: the arrival and departure day.

Output
Print first an integer k: the minimum number of rooms required. After that, print a line that contains the room number of each customer in the same order as in the input. The rooms are numbered $1, 2, \ldots, k$. You can print any valid solution.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a \leq b \leq 10^9$

```cpp
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, a, b; cin >> n;
    vector<tuple<int, int, int>> v;
    for(int i = 1; i <= n; i++){
        cin >> a >> b;
        v.push_back({a, 0, i});
        v.push_back({b, 1, i});
    }
    sort(all(v));
    queue<int> fila;
    vector<int> result(n + 1);
    int cont = 0;

    for(auto x : v){
        if(get<1>(x) == 0){
            if(fila.empty()){
                cont++;
                result[get<2>(x)] = cont;
            }
            else{
                result[get<2>(x)] = fila.front();
                fila.pop();
            }
        }
        else{
            fila.push(result[get<2>(x)]);
        }
    }

    cout << cont << '\n';
    for(int i = 1; i <=n; i++) cout << result[i] << ' ';
    cout << '\n';

}
```

2.26 Stick Lengths

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length. You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length. What is the minimum total cost?

Input
The first input line contains an integer n: the number of sticks. Then there are n integers: $p_1, p_2, \ldots, p_n$: the lengths of the sticks.

Output
Print one integer: the minimum total cost.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq p_i \leq 10^9$

```cpp
const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;

int32_t main(){
    int n; cin >> n;
    if(n == 1){
        cout << "0\n";
        return 0;
    }
    int p[n], soma = 0;
    for(int i = 0; i < n; i++){
        cin >> p[i];
        soma += p[i];
    }
    sort(p, p+n);
    int minimo = soma;
    for(int i = 0; i <= n/2; i++){
        soma -= 2*p[i];
        minimo = min(soma - (n - 2*(i + 1)) * p[i + 1],
              minimo);
    }

    for(int i = n/2 + 1; i < n - 1; i++){
        soma -= 2*p[i];
        minimo = min(soma - (n - 2*(i + 1)) * p[i], minimo);
    }

    cout << minimo << '\n';
}
```

2.27 Subarray Divisibility

Given an array of n integers, your task is to count the number of subarrays where the sum of values is divisible by n.

Input
The first input line has an integer n: the size of the array. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $-10^9 \leq a_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    ll sum = 0;
    vector<ll> v;
    v.pb(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        a = (a % n + n) % n;
        sum = (sum + a) % n;
        v.pb(sum);
    }

    ll cont = 0;

    map<ll, ll> rep;

    for(auto a : v){
        ll k = rep[a];
        rep[a] ++;
        cont += k;
    }

    cout << cont << '\n';
}
```

2.28 Subarray Sums I

Given an array of n positive integers, your task is to count the number of subarrays having sum x.

Input
The first input line has two integers n and x: the size of the array and the target sum x. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x, a_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, x; cin >> n >> x;
    ll sum = 0;
    set<ll> v;
    v.insert(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        sum += a;
        v.insert(sum);
    }

    int cont = 0;

    for(auto a : v){
        auto it = v.find(x + a);
        if(it != v.end())
            cont++;
    }

    cout << cont << '\n';
}
```

2.29 Subarray Sums II

Given an array of n integers, your task is to count the number of subarrays having sum x.

Input
The first input line has two integers n and x: the size of the array and the target sum x. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $-10^9 \leq x, a_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, x; cin >> n >> x;
    ll sum = 0;
    vector<ll> v;
    v.pb(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        sum += a;
        v.pb(sum);
    }

    ll cont = 0;

    map<ll, ll> rep;

    for(auto a : v){
        ll k = rep[a - x];
        rep[a] ++;
        cont += k;
    }

    cout << cont << '\n';
}
```

2.30 Sum Of Four Values

You are given an array of n integers, and your task is to find four values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print four integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 1000$ • $1 \leq x, a_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int, int>> v;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }

    if(n < 4){
        cout << "IMPOSSIBLE\n";
        return;
    }

    vector<pair<int, pair<int, int>>> pares;

    for(int i = 0; i < n; i++){
        for(int j = i + 1; j < n; j++){
            pares.push_back({v[i].first + v[j].first, {i, j
                }});
        }
    }

    sort(pares.begin(), pares.end());

    int fim = pares.size() - 1, ini = 0;
    int soma = 0;

    while(soma != x && fim != ini){
        soma = pares[fim].first + pares[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
        else{
            set<int> s;
            s.insert(pares[ini].second.first);
            s.insert(pares[ini].second.second);
            s.insert(pares[fim].second.first);
            s.insert(pares[fim].second.second);
            if(s.size() != 4){
                if(pares[ini].first == pares[ini + 1].first){
                    ini ++;
                }
                else{
                    fim --;
                }
                soma = -1;
            }
        }
    }

    if(soma == x) cout << pares[ini].second.first + 1 << ' '
        << pares[ini].second.second + 1 << ' ' <<
                pares[fim].second.first + 1 << ' '
                    << pares[fim].second.second +
                     1 << '\n';
    else cout << "IMPOSSIBLE\n";
}
```

2.31 Sum Of Three Values

You are given an array of n integers, and your task is to find three values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print three integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 5000$ • $1 \leq x, a_i \leq 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
vector<pair<int, int>> v;

pair<int, int> two_values(int ini, int fim, int x) {
```

```
    int soma = 0;

    do{
        soma = v[fim].first + v[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
    } while(soma != x && fim != ini);

    if(soma == x){
        return {v[ini].second + 1, v[fim].second + 1};
    }

    else return {-1, -1};
}

void solve() {
    int n, x; cin >> n >> x;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }

    if(n < 3){
        cout << "IMPOSSIBLE\n";
        return;
    }

    sort(v.begin(), v.end());

    for(int i = 0; i < n - 2; i++){
        pair<int, int> p = two_values(i + 1, n - 1, x - v[i].
            first);
        if(p.first != -1){
            cout << v[i].second + 1 << ' ' << p.first << ' '
                << p.second << '\n';
            return;
        }
    }

    cout << "IMPOSSIBLE\n";
}
```

2.32 Sum Of Two Values

You are given an array of n integers, and your task is to find two values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print two integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x, a_i \leq 10^9$

```
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }

    sort(v.begin(), v.end());

    int fim = n-1, ini = 0;
    int soma = 0;

    while(soma != x && fim != ini){
        soma = v[fim].first + v[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
    }

    if(soma == x) cout << v[ini].second + 1 << ' ' << v[fim].
        second + 1 << '\n';
    else cout << "IMPOSSIBLE\n";
}
```

2.33 Tasksand Deadlines

You have to process n tasks. Each task has a duration and a deadline, and you will process the tasks in some order one after another. Your reward for a task is $d - f$ where d is its deadline and f is your finishing time. (The starting time is 0, and you have to process all tasks even if a task would yield negative reward.) What is your maximum reward if you act optimally?

Input
The first input line has an integer n: the number of tasks. After this, there are n lines that describe the tasks. Each line has two integers a and d: the duration and deadline of the task.

Output
Print one integer: the maximum reward.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, d \leq 10^6$

```
const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

void solve(){
    int n; cin >> n;
    int d;
    ll soma = 0;
    vector<int> a(n + 1);
    for(int i = 1; i <= n; i++){
        cin >> a[i] >> d;
        soma += d;
    }

    sort(a.begin() + 1, a.end());

    for(int i = 1; i <= n; i++){
        soma -= (ll)a[i] * (n - i + 1);
    }

    cout << soma << '\n';
}
```

2.34 Towers

You are given n cubes in a certain order, and your task is to build towers using them. Whenever two cubes are one on top of the other, the upper cube must be smaller than the lower cube. You must process the cubes in the given order. You can always either place the cube on top of an existing tower, or begin a new tower. What is the minimum possible number of towers?

Input
The first input line contains an integer n: the number of cubes. The next line contains n integers $k_1, k_2, \ldots, k_n$: the sizes of the cubes.

Output
Print one integer: the minimum number of towers.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k_i \leq 10^9$

```
const int M = 1e9 + 7;

int main () { _
    int n; cin >> n;
    multiset<int> s;

    for(int i = 0; i < n; i++){
        int x; cin >> x;
        if(s.upper_bound(x) == s.end()) s.insert(x);
        else{
            s.erase(s.upper_bound(x));
            s.insert(x);
        }
    }
    cout << s.size() << '\n';

    return 0;
}
```

2.35 Traffic Lights

There is a street of length x whose positions are numbered $0, 1, \ldots, x$. Initially there are no traffic lights, but n sets of traffic lights are added to the street one after another. Your task is to calculate the length of the longest passage without traffic lights after each addition.

Input
The first input line contains two integers x and n: the length of the street and the number of sets of traffic lights. Then, the next line contains n integers $p_1, p_2, \ldots, p_n$: the position of each set of traffic lights. Each position is distinct.

Output
Print the length of the longest passage without traffic lights after each addition.

Constraints
• $1 \leq x \leq 10^9$ • $1 \leq n \leq 2 \cdot 10^5$ • $0<p_i<x$

```cpp
const int M = 1e9 + 7;

int main () { _
    int x, n; cin >> x >> n;

    set<int> p; p.insert(0); p.insert(x);
    multiset<int> dist; dist.insert(x);

    for(int i = 0; i < n; i++){
        int y; cin >> y;
        auto it = p.upper_bound(y);
        int M = *it;
        it --;
        int m = *it;

        dist.erase(dist.lower_bound(M-m));

        dist.insert(M-y);
        dist.insert(y-m);

        p.insert(y);
        cout << *dist.crbegin() << '\n';
    }

    return 0;
}
```

3 Dynamic Programming

3.1 Array Description

You know that an array has n integers between 1 and m, and the absolute difference between two adjacent values is at most 1. Given a description of the array where some values may be unknown, your task is to count the number of arrays that match the description.

Input
The first input line has two integers n and m: the array size and the upper bound for each value. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Value 0 denotes an unknown value.

Output
Print one integer: the number of arrays modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 100$ • $0 \leq x_i \leq m$

```cpp
const int maxn = 1e5 + 5, maxm = 1e2 + 5, inf = 2e9, M = 1e9 + 7;

// dp[i][j] = numero de maneiras ate a posicao i e terminando em j
int dp[maxn][maxm];
vector<int> x(maxn);

int calc(int n, int m){
    if(x[0] != 0) dp[0][x[0]] = 1;
    else{
        for(int j = 1; j <= m; j++) dp[0][j] = 1;
    }

    for(int i = 1; i < n; i++){
        if(x[i] == 0){
            for(int j = 1; j <= m; j++){
                dp[i][j] = ( (ll) dp[i - 1][j - 1] + dp[i - 1][j] + dp[i - 1][j + 1]) % M;
            }
        }
        else{
            dp[i][x[i]] = ( (ll) dp[i - 1][x[i] - 1] + dp[i - 1][x[i]] + dp[i - 1][x[i] + 1]) % M;
        }
    }

    int tot = 0;
    for(int j = 1; j <= m; j++) tot = (tot + dp[n - 1][j]) % M;

    return tot;
}

void solve() {
    int n, m; cin >> n >> m;

    for(int i = 0; i < n; i++) cin >> x[i];

    cout << calc(n, m) << '\n';
}
```

3.2 Book Shop

You are in a book shop which sells n different books. You know the price and number of pages of each book. You have decided that the total price of your purchases will be at most x. What is the maximum number of pages you can buy? You can buy each book at most once.

Input
The first input line contains two integers n and x: the number of books and the maximum total price. The next line contains n integers $h_1, h_2, \ldots, h_n$: the price of each book. The last line contains n integers $s_1, s_2, \ldots, s_n$: the number of pages of each book.

Output
Print one integer: the maximum number of pages.

Constraints
• $1 \leq n \leq 1000$ • $1 \leq x \leq 10^5$ • $1 \leq h_i, s_i \leq 1000$

```cpp
const int maxn = 1e3 + 5, maxx = 1e5 + 5, inf = 2e9, M = 1e9 + 7;

// dp[j] = melhor que consigo usando j dinheiros
int dp[maxx];
vector<int> h(maxn), s(maxn);

int calc(int n, int x){
    // vai usando um livro por vez
    for(int i = 1; i <= n; i++){
        for(int j = x; j >= h[i]; j--){
            dp[j] = max(dp[j - h[i]] + s[i], dp[j]);
        }
    }

    return dp[x];
}

void solve() {
    int n, x; cin >> n >> x;

    for(int i = 1; i <= n; i++) cin >> h[i];
    for(int i = 1; i <= n; i++) cin >> s[i];

    cout << calc(n, x) << '\n';
}
```

3.3 Coin Combinations I

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ways you can produce a money sum x using the available coins. For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 8 ways:
• $2+2+5$ • $2+5+2$ • $5+2+2$ • $3+3+3$ • $2+2+2+3$ • $2+2+3+2$ • $2+3+2+2$ • $3+2+2+2$

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 1;

    for(int i = 0; i < maxn; i++){
        if(dp[i] == 0) continue;

        for(int coin : coins){
            if(i + coin >= maxn) continue;
            dp[i + coin] = (dp[i + coin] + dp[i]) % M;
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = 0;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    cout << dp[x] << '\n';
}
```

3.4 Coin Combinations II

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ordered ways you can produce a money sum x using the available coins. For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 3 ways:
• $2+2+5$ • $3+3+3$ • $2+2+2+3$

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output

Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 1;

    for(int coin : coins){
        for(int i = 0; i < maxn; i++){
            if(i + coin >= maxn) continue;
            dp[i + coin] = (dp[i + coin] + dp[i]) % M;
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = 0;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    cout << dp[x] << '\n';
}
```

3.5 Counting Numbers

Your task is to count the number of integers between a and b where no two adjacent digits are the same.

Input
The only input line has two integers a and b.

Output
Print one integer: the answer to the problem.

Constraints
• $0 \leq a \leq b \leq 10^{18}$

```cpp
const int maxbse = 9, maxexp = 18, maxm = 1005, inf = 2e9, M
    = 1e9 + 7;

// dp[a][b] = quantidade < a * 10 ^ b sem dig adjacentes
    iguais.
ll dp[maxbse + 1][maxexp + 1];

void calcdp(){
    for(int b = 1; b <= maxbse; b++) dp[b][0] = b - 1;
    dp[1][1] = 9;

    for(int e = 1; e <= maxexp; e++){
        // [1, 99999] = [1, 89999] + [90000, 99999]
        //                         + [0001, 8999]
        if(e == 2) dp[1][e] = dp[9][e - 1] + dp[9][e - 2] +
            1; // adiciona o "90"
        if(e > 2)  dp[1][e] = dp[9][e - 1] + dp[9][e - 2] -
            dp[1][e - 3]; // remove "[90000, 90099]"

        for(int b = 2; b <= maxbse; b++){
            // [1, 69999] = [1, 59999] + [60000, 69999]
            //                         [0001, 9999] - [1, 6999]
                + [1, 5999]
            if(e == 1)
                dp[b][e] = dp[b - 1][e] + dp[1][e] - dp[b][e
                    - 1] + dp[b - 1][e - 1] + 1; //
                    adiciona o "60"
            else
                dp[b][e] = dp[b - 1][e] + dp[1][e] - dp[b][e
                    - 1] + dp[b - 1][e - 1] - dp[1][e - 2];
                     // remove "[60000, 60099]"
        }
    }
}

// quantidade no range [0, x - 1]
ll ans(ll x){
    if(x == 0) return 0;
    // guarda todos os digitos
    vector<int> dig;
    // guarda o expoente
    int e = -1;

    while(x > 0){
        dig.pb(x % 10);
        x /= 10;
        e++;
    }
    reverse(all(dig));

    // pega os numeros em [0, d00000 - 1]
    ll tot = dp[dig[0]][e] + 1;

    for(int i = 1; i < dig.size(); i++){
        e--;
        // pega os numeros em [0, d0000 - 1]
        tot += dp[dig[i]][e];

        // remove os digitos que iniciam em 00
        if(e >= 2 && dig[i] > 0 && dig[i-1] > 0) tot -= dp
            [1][e - 1];
        // adiciona o 10
        if(e == 0 && dig[i] > 0 && dig[i-1] > 0) tot++;

        // se ha dois digitos iguais consecutivos, podemos
            parar
        if(dig[i] == dig[i-1]) break;

        // se o digito cresce, precisamos tirar os 'dd'
            contados
        if(dig[i] > dig[i-1]){
            tot += -dp[dig[i-1] + 1][e] + dp[dig[i-1]][e];
        }
    }

    return tot;
}

void solve() {
    calcdp();
    ll a, b; cin >> a >> b;
    cout << ans(b + 1) - ans(a) << '\n';
}
```

3.6 Counting Tilings

Your task is to count the number of ways you can fill an $n \times m$ grid using 1×2 and 2×1 tiles.

Input
The only input line has two integers n and m.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10$ • $1 \leq m \leq 1000$

```cpp
const int maxn = 10, maxpotn = 1030, maxm = 1005, inf = 2e9,
    M = 1e9 + 7;

/*
complete[n] sao todos os possiveis bitsets de "proximos
    andares" dado que um andar tem n casas vazias
exemplo: se n = 2, podemos completar de 3 possibilidades, {1
    1 1}, {1 0 0}, {0 0 1}:
| i | i | i | i |   | i |   |   |   | i |
| i | i | i | i |   | i | - | - |   | - | - | i |
*/
vector<int> complete[maxn + 1];

// dp[n][m] quantidade de formas de fazer m-1 andares
    completos e o do topo representado pelo bitset n
ll dp[maxpotn][maxm];

// a transicao[n] sao todos os possiveis andares depois de
    receber um topo com o bitset n
vector<int> transicao[maxpotn];
int n, m;

// calcula complete
void calcomplete(){
    complete[0].pb(0);
    complete[1].pb(1);
    for(int i = 2; i <= maxn; i++){
        // completa o com i-1 e coloca um domino em pe
        for(auto x : complete[i - 1]){
            complete[i].pb(x * 2 + 1);
        }
        // completa o com i-2 e coloca um domino deitado
        for(auto x : complete[i - 2]){
            complete[i].pb(x * 4 + 0);
        }
    }
}

// calcula transicao
void calctransicao(){
    for(int i = 0; i < (1 << n); i++){
        vector<int> v;
        int aux = i;
        int tot = 0;
        // salva seq de 0 e 1 de i
        while(aux != 0){
            int zero = 0;
            while(aux % 2 == 0){
                zero ++;
                aux /= 2;
            }
            v.pb(zero);

            int um = 0;
```

```
            while(aux % 2 == 1){
                um ++;
                aux /= 2;
            }
            v.pb(um);
            tot += zero + um;
        }
        // garante n bits
        if(tot != n){
            v.pb(n - tot);
            v.pb(0);
        }

        // fila que guarda as transicoes
        queue<int> fila;
        fila.push(0);

        while(!v.empty()){
            // se estamos em uma seq de 1's o proximo eh
                 sempre 0
            int fsz = fila.size();
            for(int j = 0; j < fsz; j++){
                int at = fila.front();
                at = at << v.back();
                fila.push(at);
                fila.pop();
            }
            v.pop_back();

            // em sequencia de 0's, precisamos adicionar os
                 complete
            fsz = fila.size();
            for(int j = 0; j < fsz; j++){
                int at = fila.front();
                at = at << v.back();
                for(auto x : complete[v.back()]){
                    fila.push(at + x);
                }
                fila.pop();
            }
            v.pop_back();
        }

        // salva a fila no vetor transicao
        while(!fila.empty()){
            transicao[i].pb(fila.front());
            fila.pop();
        }
    }
}

void solve() {
    cin >> n >> m;
    calcomplete();
    calctransicao();
    // uma forma do primeiro andar ter 0 dominos
    dp[0][1] = 1;

    // calcula dp
    for(int j = 1; j <= m; j++){
        for(int i = 0; i < (1 << n); i++){
            for(auto x : transicao[i]){
                dp[x][j + 1] = (dp[x][j + 1] + dp[i][j]) % M;
            }
        }
    }

    // quantidade de formas do andar m + 1 ter 0 pecas e o
         resto estar completo
    cout << dp[0][m + 1] << '\n';
}
```

3.7 Counting Towers

Your task is to build a tower whose width is 2 and height is n. You have an unlimited supply of blocks whose width and height are integers. For example, here are some possible solutions for $n = 6$:
Given n, how many different towers can you build? Mirrored and rotated towers are counted separately if they look different.

Input
The first input line contains an integer t: the number of tests. After this, there are t lines, and each line contains an integer n: the height of the tower.

Output
For each test, print the number of towers modulo $10^9 + 7$.

Constraints
• $1 \leq t \leq 100$ • $1 \leq n \leq 10^6$

```
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

int f[maxn];

void calc(){
    f[0] = 1; f[1] = 2;
    int s = 3;
    for(int i = 2; i < maxn; i++){
        f[i] = (7LL * f[i - 1] + (11)(M - 2) * s) % M;
        s = (s + f[i]) % M;
    }
}

void solve() {
    int n; cin >> n;
    cout << f[n] << '\n';
}

int main() {
    calc();
    int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

3.8 Dice Combinations

Your task is to count the number of ways to construct sum n by throwing a dice one or more times. Each throw produces an outcome between 1 and 6. For example, if $n = 3$, there are 4 ways:
• $1+1+1$ • $1+2$ • $2+1$ • 3

Input
The only input line has an integer n.

Output
Print the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

```
const int maxn = 1000005, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];

int calc(int n){
    if(n < 0) return 0;
    if(dp[n] != 0) return dp[n];

    for(int i = 1; i <= dado; i++) dp[n] = (dp[n] + calc(n -
         i)) % M;

    return dp[n] ;
}

void solve() {
    int n; cin >> n;
    dp[0] = 1;
    cout << calc(n) << '\n';
}
```

3.9 Edit Distance

The edit distance between two strings is the minimum number of operations required to transform one string into the other. The allowed operations are:
• Add one character to the string. • Remove one character from the string. • Replace one character in the string.
For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I. Your task is to calculate the edit distance between two strings.

Input
The first input line has a string that contains n characters between A–Z. The second input line has a string that contains m characters between A–Z.

Output
Print one integer: the edit distance between the strings.

Constraints
• $1 \leq n, m \leq 5000$

```
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    string s, t;
    cin >> s >> t;

    int n = s.size(), m = t.size();

    int dp[n + 1][m + 1];

    for(int i = 0; i <= n; i++) dp[i][0] = i;
    for(int j = 0; j <= m; j++) dp[0][j] = j;

```

```
    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            dp[i][j] = inf;
            if(s[i-1] == t[j-1])
                dp[i][j] = min(dp[i][j], dp[i - 1][j - 1]);
            dp[i][j] = min(dp[i][j], dp[i][j - 1] + 1);
            dp[i][j] = min(dp[i][j], dp[i - 1][j] + 1);
            dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + 1);
        }
    }

    cout << dp[n][m] << '\n';
}
```

3.10 Elevator Rides

There are n people who want to get to the top of a building which has only one elevator. You know the weight of each person and the maximum allowed weight in the elevator. What is the minimum number of elevator rides?

Input
The first input line has two integers n and x: the number of people and the maximum allowed weight in the elevator. The second line has n integers $w_1, w_2, \ldots, w_n$: the weight of each person.

Output
Print one integer: the minimum number of rides.

Constraints
• $1 \le n \le 20$ • $1 \le x \le 10^9$ • $1 \le w_i \le x$

```
const int maxn = 1050000, maxm = 25, inf = 2e9, M = 1e9 + 7;

// dp[X] = {k, W} -> com o bitset X, k eh o menor numero de
    viagens, W eh o peso total da menor viagem
pair<int, int> dp[maxn];
bool tested[maxn];
int pot[maxm];

int cmp(int a, int b)
{
    int count1 = pc(a);
    int count2 = pc(b);

    if (count1 <= count2)
        return false;
    return true;
}

void solve() {
    int n, x; cin >> n >> x;
    vector<int> w(n);
    for(int i = 0; i < n; i++) cin >> w[i];
    pot[0] = 1;
    for(int i = 1; i < maxm; i++) pot[i] = pot[i - 1] * 2;
    for(int i = 0; i < maxn; i++) dp[i] = {inf, inf};
    dp[0] = {1, 0};

    queue<int> fila;
    fila.push(0);
    tested[0] = true;

    while(!fila.empty()){
        int X = fila.front();
        fila.pop();
        for(int i = 0; i < n; i++){
            int Y = (X | pot[i]);
            if(Y == X) continue;
            if(dp[X].S + w[i] <= x)
                dp[Y] = min(dp[Y], {dp[X].F, dp[X].S + w[i]})
                    ;
            else
                dp[Y] = min(dp[Y], {dp[X].F + 1, w[i]});

            if(!tested[Y]){
                fila.push(Y);
                tested[Y] = true;
            }
        }
    }

    cout << dp[pot[n] - 1].F << '\n';
}
```

3.11 Grid Paths I

Consider an $n \times n$ grid whose squares may have traps. It is not allowed to move to a square with a trap. Your task is to calculate the number of paths from the upper-left square to the lower-right square. You can only move right or down.

Input
The first input line has an integer n: the size of the grid. After this, there are n lines that describe the grid. Each line has n characters: . denotes an empty cell, and * denotes a trap.

Output
Print the number of paths modulo $10^9 + 7$.

Constraints
• $1 \le n \le 1000$

```
const int maxn = 1e3 + 5, inf = 2e9, M = 1e9 + 7;

int dp[maxn][maxn];
char grid[maxn][maxn];
int n;

int calc(int a, int b){
    if(a < 0 || b < 0 || a >= n || b >= n) return 0;
    if(dp[a][b] >= 0) return dp[a][b];
    if(grid[a][b] == '*') return dp[a][b] = 0;

    return dp[a][b] = (calc(a + 1, b) + calc(a, b + 1)) % M;
}

void solve() {
    cin >> n;

    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            dp[i][j] = -1;
            cin >> grid[i][j];
        }
    }

    dp[n - 1][n - 1] = grid[n - 1][n - 1] == '*' ? 0 : 1;
    calc(0, 0);

    cout << dp[0][0] << '\n';
}
```

3.12 Increasing Subsequence

You are given an array containing n integers. Your task is to determine the longest increasing subsequence in the array, i.e., the longest subsequence where every element is larger than the previous one. A subsequence is a sequence that can be derived from the array by deleting some elements without changing the order of the remaining elements.

Input
The first line contains an integer n: the size of the array. After this there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the length of the longest increasing subsequence.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```
const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

template<typename T> int lis(vector<T> &v){
    vector<T> ans;
    for (T t : v){
        auto it = lower_bound(ans.begin(), ans.end(), t);
        if (it == ans.end()) ans.push_back(t);
        else *it = t;
    }
    return ans.size();
}

void solve() {
    int n; cin >> n;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    cout << lis<int> (v) << '\n';
}
```

3.13 Increasing Subsequence II

Given an array of n integers, your task is to calculate the number of increasing subsequences it contains. If two subsequences have the same values but in different positions in the array, they are counted separately.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of increasing subsequences modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
const int M = 1e9 + 7;

struct BIT {
    vector<int> bit;
    int n;

    BIT(int n) {
        this->n = n;
        bit.assign(n, 0);
    }

    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret = (ret + bit[r]) % M;
        return ret;
    }

    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] = (bit[idx] + delta) % M;
    }
};

void solve() {
    int n; cin >> n;
    int ans = 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<int> aux = a;
    sort(aux.begin(), aux.end());
    aux.erase(unique(aux.begin(), aux.end()), aux.end());
    for (int i = 0; i < n; i++) {
        a[i] = lower_bound(aux.begin(), aux.end(), a[i]) -
            aux.begin() + 1;
    }

    BIT bit(n + 1); bit.add(0, 1);

    for(int i = 0; i < n; i++) {
        int soma = bit.sum(a[i] - 1);
        ans += soma; ans %= M;
        bit.add(a[i], soma);
    }

    cout << ans << '\n';
}
```

3.14 Longest Common Subsequence

Given two arrays of integers, find their longest common subsequence. A subsequence is a sequence of array elements from left to right that can contain gaps. A common subsequence is a subsequence that appears in both arrays.

Input
The first line has two integers n and m: the sizes of the arrays. The second line has n integers $a_1, a_2, \ldots, a_n$: the contents of the first array. The third line has m integers $b_1, b_2, \ldots, b_m$: the contents of the second array.

Output
First print the length of the longest common subsequence. After that, print an example of such a sequence. If there are several solutions, you can print any of them.

Constraints
• $1 \leq n, m \leq 1000$ • $1 \leq a_i, b_i \leq 10^9$

```
vector<vector<int>> dp;
vector<int> a, b;

int lcs(int n, int m) {
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

    return dp[n][m];
}

void solve() {
    int n, m; cin >> n >> m;
    a.resize(n);
    b.resize(m);

    for(int i = 0; i < n; i++) cin >> a[i];
    for(int i = 0; i < m; i++) cin >> b[i];

    dp.resize(n + 1, vector<int>(m + 1, 0));

    int i = n, j = m, at = lcs(n, m);
    vector<int> ans;
    cout << at << '\n';

    while(i > 0 && j > 0)
    {
        if (a[i - 1] == b[j - 1]) {
            ans.push_back(a[i - 1]);
            i--; j--;
        }
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }

    reverse(ans.begin(), ans.end());
    for(auto x : ans) cout << x << '␣';
    cout << '\n';
}
```

3.15 Minimal Grid Path

You are given an $n \times n$ grid whose each square contains a letter. You should move from the upper-left square to the lower-right square. You can only move right or down. What is the lexicographically minimal string you can construct?

Input
The first line has an integer n: the size of the grid. After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output
Print the lexicographically minimal string.

Constraints
• $1 \leq n \leq 3000$

```
vector<vector<char>> M;
int n;

void solve() {
    cin >> n;
    M.resize(n + 1, vector<char>(n + 1));

    for(int i = 0; i <= n; i++) M[i][0] = '$';
    for(int i = 0; i <= n; i++) M[0][i] = '$';

    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            cin >> M[i][j];

    string s; s.push_back(M[1][1]);
    for(int sum = 3; sum <= 2 * n; sum++)
    {
        char best = 'Z';
        for(int i = max(1, sum - n); i <= min(n, sum - 1); i
            ++)
        {
            int j = sum - i;
            if(M[i - 1][j] != '$') best = min(best, M[i][j]);
            if(M[i][j - 1] != '$') best = min(best, M[i][j]);
        }

        s.push_back(best);
        for(int i = max(1, sum - n); i <= min(n, sum - 1); i
            ++)
        {
            int j = sum - i;
            if(M[i - 1][j] == '$' && M[i][j - 1] == '$' || M[
                i][j] != best) M[i][j] = '$';
        }
    }

    cout << s << '\n';
}
```

3.16 Minimizing Coins

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to produce a sum of money x using the available coins in such a way that the number of coins is minimal. For example, if the coins are $\{1, 5, 7\}$ and the desired sum is 11, an optimal solution is $5 + 5 + 1$ which requires 3 coins.

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output
Print one integer: the minimum number of coins. If it is not possible to produce the desired sum, print -1.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 0;

    for(int i = 0; i < maxn; i++){
        if(dp[i] == inf) continue;

        for(int coin : coins){
            if(i + coin >= maxn) continue;
            dp[i + coin] = min(dp[i + coin], dp[i] + 1);
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = inf;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    if(dp[x] == inf) dp[x] = -1;
    cout << dp[x] << '\n';
}
```

3.17 Money Sums

You have n coins with certain values. Your task is to find all money sums you can create using these coins.

Input
The first input line has an integer n: the number of coins. The next line has n integers $x_1, x_2, \ldots, x_n$: the values of the coins.

Output
First print an integer k: the number of distinct money sums. After this, print all possible sums in increasing order.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x_i \leq 1000$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    bitset<maxn> mark;
    mark[0] = 1;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        mark = (mark | mark << a);
    }

    cout << mark.count() - 1 << '\n';
    for(int i = 1; i < maxn; i++)
        if(mark[i] == 1)
            cout << i << ' ';
    cout << '\n';
}
```

3.18 Mountain Range

There are n mountains in a row, each with a specific height. You begin your hang gliding route from some mountain. You can glide from mountain a to mountain b if mountain a is taller than mountain b and all mountains between a and b. What is the maximum number of mountains you can visit on your route?

Input
The first line has an integer n: the number of mountains. The next line has n integers $h_1, h_2, \ldots, h_n$: the heights of the mountains.

Output:
Print one integer: the maximum number of mountains.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq h_i \leq 10^9$

```cpp
const int inf = 2e9;
vector<int> lef, rig, a;
const int MAX = 2e5 + 5;

struct seg {
    int n, t[4*MAX];

    int query(int v, int tl, int tr, int l, int r) {
        if (l > r)
            return 0;
        if (l == tl && r == tr)
            return t[v];
        int tm = (tl + tr) / 2;

        int left = query(v*2, tl, tm, l, min(r, tm));
        int right = query(v*2+1, tm+1, tr, max(l, tm+1), r);

        return max(left, right);
    }

    void update(int v, int tl, int tr, int pos, int new_val) {
        if (tl == tr) {
            t[v] = new_val;
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm)
                update(v*2, tl, tm, pos, new_val);
            else
                update(v*2+1, tm+1, tr, pos, new_val);
            t[v] = max(t[v*2], t[v*2+1]);
        }
    }
};

void calc() {
    int n = a.size();
    lef.resize(n), rig.resize(n);
    stack<int> st;
    for(int i = 0; i < n; i++) {
        while(!st.empty() && a[st.top()] < a[i]) st.pop();
        lef[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
    while(!st.empty()) st.pop();
    for(int i = n - 1; i >= 0; i--) {
        while(!st.empty() && a[st.top()] < a[i]) st.pop();
        rig[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
}

void solve() {
    int n; cin >> n; n += 2;
    a.resize(n); a[0] = inf;
    for(int i = 1; i <= n - 2; i++) cin >> a[i];
    a[n - 1] = inf;

    calc();

    vector<pair<int, int>> seq(n);
    for(int i = 0; i < n; i++) seq[i] = {a[i], i};
    sort(seq.begin(), seq.end());

    seg tree; tree.n = n;

    for(int i = 0; i < n - 2; i++) {
        int idx = seq[i].second;
        int L = lef[idx], R = rig[idx];

        int ans_left = tree.query(1, 0, n - 1, L + 1, idx);
        int ans_right = tree.query(1, 0, n - 1, idx, R - 1);

        tree.update(1, 0, n - 1, idx, max(ans_left, ans_right) + 1);
    }

    cout << tree.query(1, 0, n - 1, 0, n - 1) << "\n";
}
```

3.19 Projects

There are n projects you can attend. For each project, you know its starting and ending days and the amount of money you would get as reward. You can only attend one project during a day. What is the maximum amount of money you can earn?

Input
The first input line contains an integer n: the number of projects. After this, there are n lines. Each such line has three integers a_i, b_i, and p_i: the starting day, the ending day, and the reward.

Output
Print one integer: the maximum amount of money you can earn.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq b_i \leq 10^9$ • $1 \leq p_i \leq 10^9$

```cpp
const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<pair<pii, int>> v(n);

    for(int i = 0; i < n; i++) cin >> v[i].f.f >> v[i].f.s >>
          v[i].s;

    set<int> checkpoints;
    for(int i = 0; i < n; i++) {
        checkpoints.insert(v[i].f.f);
        checkpoints.insert(v[i].f.s);
    }

    unordered_map<int, int> comprime;
    int tot = 1;
    for(auto x : checkpoints){
        comprime[x] = tot;
        tot++;
    }

    sort(all(v));

    vector<ll> dp(tot + 1);
    int at = 0;

    for(int i = 1; i <= tot; i++){
        dp[i] = max(dp[i], dp[i - 1]);
        while(i == comprime[v[at].f.f]){
            dp[comprime[v[at].f.s]] = max(dp[comprime[v[at].f
                .s]], dp[i - 1] + v[at].s);
            at ++;
        }
    }

    cout << dp[tot] << '\n';
}
```

3.20 Rectangle Cutting

Given an $a \times b$ rectangle, your task is to cut it into squares. On each move you can select a rectangle and cut it into two rectangles in such a way that all side lengths remain integers. What is the minimum possible number of moves?

Input
The only input line has two integers a and b.

Output
Print one integer: the minimum number of moves.

Constraints
• $1 \leq a, b \leq 500$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int a, b;
    cin >> a >> b;

    int dp[a + 1][b + 1];

    for(int i = 1; i <= min(a, b); i++) dp[i][i] = 0;

    for(int i = 1; i <= a; i++){
        for(int j = 1; j <= b; j++){
            if(i == j) continue;
            dp[i][j] = inf;
            for(int k = 1; k < i; k++)
                dp[i][j] = min(dp[i][j], 1 + dp[i - k][j] +
                      dp[k][j]);
            for(int k = 1; k < j; k++)
                dp[i][j] = min(dp[i][j], 1 + dp[i][j - k] +
                      dp[i][k]);
        }
    }

    cout << dp[a][b] << '\n';
}
```

3.21 Removal Game

There is a list of n numbers and two players who move alternately. On each move, a player removes either the first or last number from the list, and their score increases by that number. Both players try to maximize their scores. What is the maximum possible score for the first player when both players play optimally?

Input
The first input line contains an integer n: the size of the list. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list.

Output
Print the maximum possible score for the first player.

Constraints

• $1 \leq n \leq 5000$ • $-10^9 \leq x_i \leq 10^9$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<int>   v(n + 1);
    vector<ll > acc(n + 1);

    for(int i = 1; i <= n; i++){
        cin >> v[i];
        acc[i] = acc[i-1] + v[i];
    }

    ll dp[n + 1][n + 1];

    for(int i = 0; i <= n; i++) dp[i][i] = v[i];

    for(int k = 1; k < n; k++){
        for(int i = 1; i <= n - k; i++){
            int j = i + k;
            dp[i][j] = (acc[j] - acc[i-1]) - min(dp[i + 1][j
                ], dp[i][j - 1]);
        }
    }

    cout << dp[1][n] << '\n';
}
```

3.22 Removing Digits

You are given an integer n. On each step, you may subtract one of the digits from the number. How many steps are required to make the number equal to 0?

Input
The only input line has an integer n.

Output
Print one integer: the minimum number of steps.

Constraints
• $1 \leq n \leq 10^6$

```cpp
const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];

vector<int> digits(int x){
    vector<int> v;
    while(x > 0){
        if(x % 10 > 0)
            v.pb(x % 10);
        x /= 10;
    }
    return v;
}

void calc(int n){
    dp[0] = 0;
    for(int i = 1; i <= n; i++){
        vector<int> v = digits(i);
        for(auto x : v)
            dp[i] = min(dp[i], dp[i - x] + 1);
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = inf;

    int n;
    cin >> n;
    calc(n);

    cout << dp[n] << '\n';
}
```

3.23 Two Sets II

Your task is to count the number of ways numbers $1, 2, \ldots, n$ can be divided into two sets of equal sum. For example, if $n = 7$, there are four solutions:
• $\{1, 3, 4, 6\}$ and $\{2, 5, 7\}$ • $\{1, 2, 5, 6\}$ and $\{3, 4, 7\}$ • $\{1, 2, 4, 7\}$ and $\{3, 5, 6\}$ • $\{1, 6, 7\}$ and $\{2, 3, 4, 5\}$

Input
The only input line contains an integer n.

Output
Print the answer modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 500$

```cpp
const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

// dp[i][j] eh o numero de sets de tamanho ate i cuja soma eh
    j
ll dp[maxn][(ll)maxn * (maxn + 1) / 2];

void solve() {
    int n; cin >> n;

    for(int i = 0; i <= n; i++) dp[i][0] = 0;
    for(int i = 0; i <= n * (n + 1) / 2; i++) dp[0][i] = 0;
    dp[0][0] = 1;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n * (n + 1) / 2; j++){
            dp[i][j] = (dp[i][j] + dp[i - 1][j] + dp[i - 1][
                max(j - i, 0)]) % M;
        }
    }

    if(n * (n + 1) % 4 != 0){
        cout << "0\n";
    }
    else{
        cout << dp[n][n * (n + 1) / 4] << '\n';
    }

}
```

4 Graph Algorithms

4.1 Building Roads

Byteland has n cities, and m roads between them. The goal is to construct new roads so that there is a route between any two cities. Your task is to find out the minimum number of roads required, and also determine which roads should be built.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After that, there are m lines describing the roads. Each line has two integers a and b: there is a road between those cities. A road always connects two different cities, and there is at most one road between any two cities.

Output
First print an integer k: the number of required roads. Then, print k lines that describe the new roads. You can print any valid solution.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;

void dfs(int i){
    check[i] = 1;
    for(auto x : G[i]){
        if(check[x] == 0) dfs(x);
    }
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            dfs(i);
            ans.pb(i);
        }
    }

    cout << ans.size() - 1 << '\n';
    for(int i = 1; i < ans.size(); i++){
        cout << ans[i - 1] << ' ' << ans[i] << '\n';
    }
}
```

4.2 Building Teams

There are n pupils in Uolevi's class, and m friendships between them. Your task is to divide the pupils into two teams in such a way that no two pupils in a team are friends. You can freely choose the sizes of the teams.

Input
The first input line has two integers n and m: the number of pupils and friendships. The pupils are numbered $1, 2, \ldots, n$. Then, there are m lines describing the friendships. Each line has two integers a and b: pupils a and b are friends. Every friendship is between two different pupils. You can assume that there is at most one friendship between any two pupils.

Output
Print an example of how to build the teams. For each pupil, print "1" or "2" depending on to which team the pupil will be assigned. You can print any valid team. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;
bool flag = true;

void dfs(int i){
    for(auto x : G[i]){
        if(check[x] == 0){
            check[x] = 3 - check[i];
            dfs(x);
        }
        else if(check[x] == check[i]) flag = false;
    }
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            check[i] = 1;
            dfs(i);
        }
    }

    if(!flag){
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 1; i <= n; i++){
        cout << check[i] << ' ';
    }
    cout << '\n';
}
```

4.3 Coin Collector

A game has n rooms and m tunnels between them. Each room has a certain number of coins. What is the maximum number of coins you can collect while moving through the tunnels when you can freely choose your starting and ending room?

Input
The first input line has two integers n and m: the number of rooms and tunnels. The rooms are numbered $1, 2, \ldots, n$. Then, there are n integers $k_1, k_2, \ldots, k_n$: the number of coins in each room. Finally, there are m lines describing the tunnels. Each line has two integers a and b: there is a tunnel from room a to room b. Each tunnel is a one-way tunnel.

Output
Print one integer: the maximum number of coins you can collect.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le k_i \le 10^9$ • $1 \le a, b \le n$

```cpp
const int MAX = 1e5 + 5;

int n, m;
vector<int> g[MAX];
vector<int> gi[MAX]; // grafo invertido
int vis[MAX];
stack<int> S;
int comp[MAX]; // componente conexo de cada vertice

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}
```

```

vector<int> new_g[MAX];
int new_k[MAX];
int dp[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : new_g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

void solve(){
    cin >> n >> m;

    vector<int> k(n);
    for(int i = 0; i < n; i++) cin >> k[i];

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        for(auto x : g[i]) {
            if(comp[x] != comp[i]) new_g[comp[i]].push_back(
                comp[x]);
        }
        new_k[comp[i]] += k[i];
    }

    vi ret = topo_sort(n);

    int ans = 0;

    for(int i = ret.size() - 1; i >= 0; i--) {
        dp[ret[i]] = new_k[ret[i]];
        for(auto x : new_g[ret[i]]) {
            dp[ret[i]] = max(dp[x] + new_k[ret[i]], dp[ret[i
                ]]);
        }
        ans = max(ans, dp[ret[i]]);
    }

    cout << ans << '\n';

}
```

4.4 Counting Rooms

You are given a map of a building, and your task is to count the number of its rooms. The size of the map is $n \times m$ squares, and each square is either floor or wall. You can walk left, right, up, and down through the floor squares.

Input
The first input line has two integers n and m: the height and width of the map. Then there are n lines of m characters describing the map. Each character is either . (floor) or # (wall).

Output
Print one integer: the number of rooms.

Constraints
• $1 \le n, m \le 1000$

```
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

char tab[maxn][maxn];
int check[maxn][maxn];
int n, m;

void dfs(int i, int j){
    if(check[i][j] == 0) return;
    check[i][j] = 0;
    dfs(i + 1, j);
    dfs(i, j + 1);
    dfs(i - 1, j);
    dfs(i, j - 1);
}

void solve(){
    cin >> n >> m;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] == '.') check[i][j] = 1;
        }
    }

    int total = 0;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            if(check[i][j] == 1){
                total++;
                dfs(i, j);
            }
        }
    }

    cout << total << '\n';
}
```

4.5 Course Schedule

You have to complete n courses. There are m requirements of the form "course a has to be completed before course b". Your task is to find an order in which you can complete the courses.

Input
The first input line has two integers n and m: the number of courses and requirements. The courses are numbered $1, 2, \ldots, n$. After this, there are m lines describing the requirements. Each line has two integers a and b: course a has to be completed before course b.

Output
Print an order in which you can complete the courses. You can print any valid order that includes all the courses. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int INF = 1000000000000000000;
const int MAX = 100010;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);

    if(ans.size() == 0)
        cout << "IMPOSSIBLE\n";
    else {
        for(auto x : ans) cout << x + 1 << ' ';
        cout << '\n';
    }

    return 0;
}
```

4.6 Cycle Finding

You are given a directed graph, and your task is to find out if it contains a negative cycle, and also give an example of such a cycle.

Input
The first input line has two integers n and m: the number of

nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, the input has m lines describing the edges. Each line has three integers a, b, and c: there is an edge from node a to node b whose length is c.

Output
If the graph contains a negative cycle, print first "YES", and then the nodes in the cycle in their correct order. If there are several negative cycles, you can print any of them. If there are no negative cycles, print "NO".

Constraints
• $1 \leq n \leq 2500$ • $1 \leq m \leq 5000$ • $1 \leq a, b \leq n$ • $-10^9 \leq c \leq 10^9$

```cpp
const int MAX = 2505;
const int INF = 1000000000000000000;

struct Edge {
    int a, b, cost;
};

int n, m;
vector<int> g[MAX], gi[MAX];
vector<Edge> edges;
int vis[MAX];
stack<int> S;
int comp[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

void solve()
{
    cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c;
        a--; b--;
        edges.push_back({a, b, c});
        if(a == b && c < 0) {
            cout << "YES\n" << a + 1 << ' ' << b + 1 << '\n';
            return;
        }
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    int tot = 0;
    for(int i = 0; i < n; i++)
        tot = max(tot, comp[i]);

    vector<vector<int>> C(tot + 1);

    for(int i = 0; i < n; i++)
        C[comp[i]].push_back(i);

    vector<int> d(n, INF), p(n, -1);
    for(int i = 0; i <= tot; i++) {
        if(C[i].size() <= 1) continue;

        int v = C[i][0]; n = C[i].size();
        d[v] = 0;

        int x;
        for (int i = 0; i < n; ++i) {
            x = -1;
            for (Edge e : edges) {
                if (comp[e.a] != comp[v] || comp[e.b] != comp[v]) continue;
                if (d[e.a] < INF)
                    if (d[e.b] > d[e.a] + e.cost) {
                        d[e.b] = max(-INF, d[e.a] + e.cost);
                        p[e.b] = e.a;
                        x = e.b;
                    }
            }
        }

        if(x != -1) {
            int y = x;
            for (int i = 0; i < n; ++i)
                y = p[y];

            vector<int> path;
            for (int cur = y;; cur = p[cur]) {
                path.push_back(cur);
                if (cur == y && path.size() > 1)
                break;
            }
            reverse(path.begin(), path.end());

            cout << "YES\n";
            for (int u : path)
               cout << u + 1 << ' ';
            return;
        }

    }

    cout << "NO\n";
}
```

4.7 De Bruijn Sequence

Your task is to construct a minimum-length bit string that contains all possible substrings of length n. For example, when $n = 2$, the string 00110 is a valid solution, because its substrings of length 2 are 00, 01, 10 and 11.

Input
The only input line has an integer n.

Output
Print a minimum-length bit string that contains all substrings of length n. You can print any valid solution.

Constraints
• $1 \leq n \leq 15$

```cpp
bool mark[1 << 15];
string s;
int mod;

void dfs(int u) {
    s += '0' + (u % 2);
    mark[u] = 1; int v = (u << 1) % mod;
    if(!mark[v + 1]) dfs(v + 1);
    else if(!mark[v]) dfs(v);
}

void solve(){
    int n; cin >> n;
    mod = 1 << n;
    s.resize(n - 1, '0');
    dfs(0);
    cout << s << '\n';
}
```

4.8 Distinct Routes

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n. You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Input
The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b. There are no two teleporters whose starting and ending room are the same.

Output
First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Constraints
• $2 \leq n \leq 500$ • $1 \leq m \leq 1000$ • $1 \leq a, b \leq n$

```cpp
int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
```

```
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.addEdge(a, b, 1);
    }

    cout << G.calc(0, n - 1) << '\n';
    int ini = 0;
    for(auto& e : G.adj[ini]) {
        if(e.flow() == 0) continue;
        e.c = e.oc;
        vector<int> pth; pth.push_back(0); pth.push_back(e.to);
        int at = e.to;
        while(at != n - 1) {
            for(auto& e1 : G.adj[at]) {
                if(e1.flow() == 0) continue;
                e1.c = e1.oc;
                at = e1.to;
                pth.push_back(at);
                break;
            }
        }
        cout << pth.size() << '\n';
        for(auto x : pth) cout << x + 1 << ' ';
        cout << '\n';
    }
}
```

4.9 Download Speed

Consider a network consisting of n computers and m connections. Each connection specifies how fast a computer can send data to another computer. Kotivalo wants to download some data from a server. What is the maximum speed he can do this, using the connections in the network?

Input
The first input line has two integers n and m: the number of computers and connections. The computers are numbered $1, 2, \ldots, n$. Computer 1 is the server and computer n is Kotivalo's computer. After this, there are m lines describing the connections. Each line has three integers a, b and c: computer a can send data to computer b at speed c.

Output
Print one integer: the maximum speed Kotivalo can download data.

Constraints
• $1 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
const int MAX = 1e5 + 5;
int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        int c; cin >> c;
        G.addEdge(a, b, c);
    }

    cout << G.calc(0, n - 1) << '\n';
}
```

4.10 Flight Discount

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have one discount coupon, using which you can halve the price of any single flight during the route. However, you can only use the coupon once. When you use the discount coupon for a flight whose price is x, its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Input
The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä. After this there are m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. Each flight is unidirectional. You can assume that it is always possible to get from Syrjälä to Metsälä.

Output
Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

const int INF = 1000000000000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d) {
    int n = adj.size();
    d.assign(2 * n, INF);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        if(v >= n) {
```

```
            for (auto edge : adj[v - n]) {
                int to = edge.first + n;
                int len = edge.second;

                if (d[v] + len < d[to]) {
                    q.erase({d[to], to});
                    d[to] = d[v] + len;
                    q.insert({d[to], to});
                }
            }
        }
        else {
            for (auto edge : adj[v]) {
                int to = edge.first;
                int len = edge.second;

                if (d[v] + len < d[to]) {
                    q.erase({d[to], to});
                    d[to] = d[v] + len;
                    q.insert({d[to], to});
                }
                to = to + n;
                len /= 2;
                if (d[v] + len < d[to]) {
                    q.erase({d[to], to});
                    d[to] = d[v] + len;
                    q.insert({d[to], to});
                }
            }
        }
    }
}

signed main() {
    int n, m; cin >> n >> m;
    adj.resize(n);

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        adj[a - 1].push_back({b - 1, c});
    }

    vector<int> d;
    dijkstra(0, d);

    cout << d[2 * n - 1] << '\n';

    return 0;
}
```

4.11 Flight Routes

Your task is to find the k shortest flight routes from Syrjälä to Metsälä. A route can visit the same city several times. Note that there can be several routes with the same price and each of them should be considered (see the example).

Input
The first input line has three integers n, m, and k: the number of cities, the number of flights, and the parameter k. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä. After this, the input has m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. All flights are one-way flights. You may assume that there are at least k distinct routes from Syrjälä to Metsälä.

Output
Print k integers: the prices of the k cheapest routes sorted according to their prices.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$
• $1 \le k \le 10$

```
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

const int INF = 1000000000000000000;

vector<vector<pair<int, int>>> adj;
vector<priority_queue<int>> d;

void dijkstra(int s) {
    int n = adj.size();

    d.assign(n, priority_queue<int>());
    d[s].push(0);

    priority_queue<pair<int, int>, vector<pair<int, int>>,
        greater<pair<int, int>>> q;
    q.push({0, s});

    while (!q.empty()) {
        int tot = q.top().first;
        int v = q.top().second;
        q.pop();

        if (d[v].size() == 10 && tot > d[v].top()) continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
            int new_dist = tot + len;

            if (d[to].size() < 10) {
                d[to].push(new_dist);
                q.push({new_dist, to});
            }
            else if(d[to].top() > new_dist) {
                d[to].pop();
                d[to].push(new_dist);
                q.push({new_dist, to});
            }
        }
    }
}

signed main() {
    int n, m, k; cin >> n >> m >> k;
    adj.resize(n);

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        a--; b--;
        adj[a].push_back({b, c});
    }
    dijkstra(0);

    while(d[n - 1].size() > k) d[n - 1].pop();
    vector<int> ans;
    for(int i = 0; i < k; i++){
        ans.push_back(d[n - 1].top());
        d[n - 1].pop();
    }
    reverse(ans.begin(), ans.end());
    for(auto x : ans) cout << x << ' ';
    cout << '\n';

    return 0;
}
```

4.12 Flight Routes Check

There are n cities and m flight connections. Your task is to check if you can travel from any city to any other city using the available flights.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output
Print "YES" if all routes are possible, and "NO" otherwise. In the latter case also print two cities a and b such that you cannot travel from city a to city b. If there are several possible solutions, you can print any of them.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 1e5+5;

int n, m;

vector<int> g[MAX];
vector<int> gi[MAX]; // grafo invertido
int vis[MAX];
stack<int> S;
int comp[MAX]; // componente conexo de cada vertice

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}
```

```

int vischeck[MAX];
void dfscheck(int k) {
    vischeck[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vischeck[g[k][i]]) dfscheck(g[k][i]);
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    bool flag = false;
    for(int i = 0; i < n; i++) {
        if(comp[i] != comp[0]) {
            cout << "NO\n";
            dfscheck(0);
            if(vischeck[i]) cout << i + 1 << ' ' << 1 << '\n'
                ;
            else cout << 1 << ' ' << i + 1 << '\n';

            flag = true;
            break;
        }
    }

    if(!flag) cout << "YES\n";

}
```

4.13 Game Routes

A game has n levels, connected by m teleporters, and your task is to get from level 1 to level n. The game has been designed so that there are no directed cycles in the underlying graph. In how many ways can you complete the game?

Input
The first input line has two integers n and m: the number of levels and teleporters. The levels are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from level a to level b.

Output
Print one integer: the number of ways you can complete the game. Since the result may be large, print it modulo $10^9 + 7$.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```

const int INF = 1000000000000000000;
const int MAX = 100010;
const int M  = 1e9 + 7;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);
    vector<int> tot(n, 0);

    int ini = 0; tot[0] = 1;

    int at = 0;
    for(int i = 0; i < ans.size(); i++)
        if(ans[i] == ini) at = i;

    for(int i = at; i < n; i++) {
        int at = ans[i];
        for(auto x : g[at]) {
            tot[x] = (tot[x] + tot[at]) % M;
        }
    }

    cout << tot[n - 1] << '\n';

    return 0;
}
```

4.14 Giant Pizza

Uolevi's family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings. Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input
The first input line has two integers n and m: the number of family members and toppings. The toppings are numbered $1, 2, \ldots, m$. After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x" (topping x is good) or "- x" (topping x is bad).

Output
Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution. If there are no valid solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n, m \le 10^5$ • $1 \le x \le m$

```

struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }
    void setValue(int x) { either(x, x); }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ?: dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        rep(i,0,2*N) if (!comp[i]) dfs(i);
        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};

void solve(){
    int n, m; cin >> n >> m;

    TwoSat ts(m);

    for(int i = 0; i < n; i++) {
        char c; int a; cin >> c >> a; a--;
        char d; int b; cin >> d >> b; b--;
        if(c == '-') a = ~a;
        if(d == '-') b = ~b;
        ts.either(a, b);
    }

    if(!ts.solve()) {
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 0; i < m; i++) {
        if(ts.values[i]) cout << "+ ";
        else cout << "- ";
```

```
    }
    cout << '\n';
}
```

4.15 Hamiltonian Flights

There are n cities and m flight connections between them. You want to travel from Syrjälä to Lehmälä so that you visit each city exactly once. How many possible routes are there?

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. Then, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output
Print one integer: the number of routes modulo $10^9 + 7$.

Constraints
• $2 \le n \le 20$ • $1 \le m \le n^2$ • $1 \le a, b \le n$

```
const int MAX = 20;
const int MOD = 1e9 + 7;

int dp[MAX][1 << MAX];
int adj[MAX][MAX];

void calc(int n) {
    dp[0][1] = 1;
    for(int b = 0; b < (1 << (n - 1)); b++) {
        for(int at = 0; at < n - 1; at++) {
            int pot = 1 << at;
            if(!(pot & b)) continue;
            for(int nxt = 0; nxt < n - 1; nxt++) {
                int pot2 = 1 << nxt;
                if(pot2 & b || !adj[at][nxt]) continue;
                dp[nxt][b | pot2] = ((ll)dp[nxt][b | pot2] +
                     (ll) dp[at][b] * adj[at][nxt]) % MOD;
            }
        }
    }
}

void solve() {
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        adj[a - 1][b - 1]++;
    }

    calc(n);

    int ans = 0;

    for(int at = 0; at < n - 1; at++) {
        if(!adj[at][n - 1]) continue;
        ans = ((ll)ans + (ll) dp[at][(1 << (n - 1)) - 1] *
              adj[at][n - 1]) % MOD;
    }

    cout << ans << '\n';
}
```

4.16 High Score

You play a game consisting of n rooms and m tunnels. Your initial score is 0, and each tunnel increases your score by x where x may be both positive or negative. You may go through a tunnel several times. Your task is to walk from room 1 to room n. What is the maximum score you can get?

Input
The first input line has two integers n and m: the number of rooms and tunnels. The rooms are numbered $1, 2, \ldots, n$. Then, there are m lines describing the tunnels. Each line has three integers a, b and x: the tunnel starts at room a, ends at room b, and it increases your score by x. All tunnels are one-way tunnels. You can assume that it is possible to get from room 1 to room n.

Output
Print one integer: the maximum score you can get. However, if you can get an arbitrarily large score, print -1.

Constraints
• $1 \le n \le 2500$ • $1 \le m \le 5000$ • $1 \le a, b \le n$ • $-10^9 \le x \le 10^9$

```
const ll inf=LLONG_MAX;
struct Ed{int a,b,w,s(){return a<b?a:-a;}};
struct Node{ll dist=inf; int prev=-1;};

void bellmanFord(vector<Node>&nodes,vector<Ed>&eds, int s){
    nodes[s].dist=0;
    sort(all(eds),[](Ed a,Ed b){return a.s()<b.s();});

    int lim=sz(nodes)/2+2;
    rep(i,0,lim) for (Ed ed:eds){
        Node cur=nodes[ed.a],&dest=nodes[ed.b];
        if (abs(cur.dist)==inf) continue;
        ll d=cur.dist+ed.w;
        if (d<dest.dist){
            dest.prev=ed.a;
            dest.dist=(i<lim-1?d:-inf);
        }
    }
    rep(i,0,lim) for(Ed e:eds){
        if (nodes[e.a].dist==-inf)
            nodes[e.b].dist=-inf;
    }
}

void solve() {
    int n, m; cin >> n >> m;
    vector<Node> nodes(n);
    vector<Ed> edges(m);
    for(int i = 0; i < m; i++) {
        int a, b, w; cin >> a >> b >> w;
        edges[i] = {a - 1, b - 1, -w};
    }
    bellmanFord(nodes, edges, 0);
    if(nodes[n - 1].dist == -inf) cout << -1 << '\n';
    else cout << -nodes[n - 1].dist << '\n';
}
```

4.17 Investigation

You are going to travel from Syrjälä to Lehmälä by plane. You would like to find answers to the following questions:
• what is the minimum price of such a route? • how many minimum-price routes are there? (modulo $10^9 + 7$) • what is the minimum number of flights in a minimum-price route? • what is the maximum number of flights in a minimum-price route?

Input
The first input line contains two integers n and m: the number of cities and the number of flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. After this, there are m lines describing the flights. Each line has three integers a, b, and c: there is a flight from city a to city b with price c. All flights are one-way flights. You may assume that there is a route from Syrjälä to Lehmälä.

Output
Print four integers according to the problem statement.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
const int INF = 1000000000000000000;
const int M = 1e9 + 7;

struct Node {
    int mn = INF;
    int cnt = 1;
    int mnsz = 1;
    int mxsz = 1;
};

vector<vector<pair<int, int>>> adj;
vector<Node> d;

void dijkstra(int s) {
    int n = adj.size();

    d.resize(n);
    d[s] = {0, 1, 1, 1};

    priority_queue<pair<int, int>, vector<pair<int, int>>,
         greater<pair<int, int>>> q;
    q.push({0, s});

    while (!q.empty()) {
        int len = q.top().first;
        int v = q.top().second;
        q.pop();

        if(len > d[v].mn) continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
            int new_dist = d[v].mn + len;

            if(d[to].mn == new_dist) {
                d[to].cnt = (d[to].cnt + d[v].cnt) % M;
                d[to].mnsz = min(d[to].mnsz, d[v].mnsz + 1);
                d[to].mxsz = max(d[to].mxsz, d[v].mxsz + 1);
```

```
            }
            else if(d[to].mn > new_dist) {
                d[to].mn = new_dist;
                d[to].cnt = d[v].cnt;
                d[to].mnsz = d[v].mnsz + 1;
                d[to].mxsz = d[v].mxsz + 1;
                q.push({new_dist, to});
            }
        }
    }
}

signed main() {
    int n, m; cin >> n >> m;
    adj.resize(n);

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        a--; b--;
        adj[a].push_back({b, c});
    }

    dijkstra(0);

    cout << d[n-1].mn << ' ' << d[n-1].cnt << ' ' << d[n-1].
         mnsz - 1 << ' ' << d[n-1].mxsz - 1 << '\n';

    return 0;
}
```

4.18 Knights Tour

Given a starting position of a knight on an 8×8 chessboard, your task is to find a sequence of moves such that it visits every square exactly once. On each move, the knight may either move two steps horizontally and one step vertically, or one step horizontally and two steps vertically.

Input
The only line has two integers x and y: the knight's starting position.

Output
Print a grid that shows how the knight moves (according to the example). You can print any valid solution.

Constraints
- $1 \leq x, y \leq 8$

```
const int MAX = 8;

vector<vector<int>> ans = { {1, 16, 63, 22, 3, 18, 55, 46},
                            {40, 23, 2, 17, 58, 47, 4, 19},
                            {15, 64, 41, 62, 21, 54, 45, 56},
                            {24, 39, 32, 59, 48, 57, 20, 5},
                            {33, 14, 61, 42, 53, 30, 49, 44},
                            {38, 25, 36, 31, 60, 43, 6, 9},
                            {13, 34, 27, 52, 11, 8, 29, 50},
                            {26, 37, 12, 35, 28, 51, 10, 7}};

void solve() {
    int x, y; cin >> y >> x; int at = ans[x - 1][y - 1];

    for(int i = 0; i < MAX; i++) {
        for(int j = 0; j < MAX; j++) cout << (ans[i][j] - at
             + MAX * MAX) % (MAX * MAX) + 1 << ' ';
        cout << '\n';
    }

}
```

4.19 Labyrinth

You are given a map of a labyrinth, and your task is to find a path from start to end. You can walk left, right, up and down.

Input
The first input line has two integers n and m: the height and width of the map. Then there are n lines of m characters describing the labyrinth. Each character is . (floor), # (wall), A (start), or B (end). There is exactly one A and one B in the input.

Output
First print "YES", if there is a path, and "NO" otherwise. If there is a path, print the length of the shortest such path and its description as a string consisting of characters L (left), R (right), U (up), and D (down). You can print any valid solution.

Constraints
- $1 \leq n, m \leq 1000$

```
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

vector<pii> macaco = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
vector<char> pos = {'D', 'U', 'R', 'L'};
vector<char> ans;
char tab[maxn][maxn];
int check[maxn][maxn];
pii pai[maxn][maxn];
int n, m;

template <typename T,typename U>
pair<T,U> operator+(const pair<T,U> & l,const pair<T,U> & r)
     {
    return {l.first+r.first,l.second+r.second};
}
template <typename T,typename U>
pair<T,U> operator-(const pair<T,U> & l,const pair<T,U> & r)
     {
    return {l.first-r.first,l.second-r.second};
}

bool bfs(pii A, pii B){
    queue<pii> fila;

    fila.push(A);

    while(!fila.empty()){
        pii X = fila.front();
        fila.pop();

        if(X == B) {
            break;
        }

        for(int i = 0; i < 4; i++){
            pii N = X + macaco[i];
            if(check[N.F][N.S] != 0){
                fila.push(N);
                check[N.F][N.S] = 0;
                pai[N.F][N.S] = X;
            }
        }
    }

    if(check[B.F][B.S] != 0) return false;

    pii X = B;
    while(X != A){
        for(int i = 0; i < 4; i++){
            if(macaco[i] == X - pai[X.F][X.S])
                ans.pb(pos[i]);
        }
        X = {pai[X.F][X.S]};
    }
    reverse(all(ans));
    return true;
}

void solve(){
    cin >> n >> m;
    pii A, B;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] != '#') check[i][j] = 1;
            if(tab[i][j] == 'A') A = {i, j};
            if(tab[i][j] == 'B') B = {i, j};
        }
    }

    if(!bfs(A, B)) cout << "NO\n";
    else{
        cout << "YES\n";
        cout << ans.size() << '\n';
        for(char x : ans) cout << x;
        cout << '\n';
    }
}
```

4.20 Longest Flight Route

Uolevi has won a contest, and the prize is a free flight trip that can consist of one or more flights through cities. Of course, Uolevi wants to choose a trip that has as many cities as possible. Uolevi wants to fly from Syrjälä to Lehmälä so that he visits the maximum number of cities. You are given the list of possible flights, and you know that there are no directed cycles in the flight network.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. Each flight is a one-way flight.

Output

First print the maximum number of cities on the route. After this, print the cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $2 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```cpp
const int INF = 1000000000000000000;
const int MAX = 100010;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);
    vector<int> dist(n, -1), pai(n, -1);

    int ini = 0;
    dist[0] = 1;

    int at = 0;
    for(int i = 0; i < ans.size(); i++)
        if(ans[i] == ini) at = i;

    for(int i = at; i < n; i++) {
        int at = ans[i];
        for(auto x : g[at]) {
            if(dist[x] < dist[at] + 1) {
                dist[x] = dist[at] + 1;
                pai[x] = at;
            }
        }
    }

    vector<int> res;

    if(dist[n - 1] == -1) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }

    for(int i = n - 1; i != 0; i = pai[i]) {
        res.push_back(i + 1);
    }
    res.push_back(1); reverse(res.begin(), res.end());

    cout << res.size() << '\n';
    for(auto x : res) cout << x << ' ';
    cout << '\n';

    return 0;
}
```

4.21 Mail Delivery

Your task is to deliver mail to the inhabitants of a city. For this reason, you want to find a route whose starting and ending point are the post office, and that goes through every street exactly once.

Input
The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered $1, 2, \ldots, n$, and the post office is located at crossing 1. After that, there are m lines describing the streets. Each line has two integers a and b: there is a street between crossings a and b. All streets are two-way streets. Every street is between two different crossings, and there is at most one street between two crossings.

Output
Print all the crossings on the route in the order you will visit them. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
$2 \leq n \leq 10^5$ $1 \leq m \leq 2 \cdot 10^5$ $1 \leq a, b \leq n$

```cpp
const int MAX = 1e5 + 5;

template<bool directed=false> struct euler {
    int n;
    vector<vector<pair<int, int>>> g;
    vector<int> used;

    euler(int n_) : n(n_), g(n) {}
    void add(int a, int b) {
        int at = used.size();
        used.push_back(0);
        g[a].emplace_back(b, at);
        if (!directed) g[b].emplace_back(a, at);
    }
    pair<bool, vector<pair<int, int>>> get_path(int src) {
        if (!used.size()) return {true, {}};
        vector<int> beg(n, 0);
        for (int& i : used) i = 0;
        // {{vertice, anterior}, label}
        vector<pair<pair<int, int>, int>> ret, st = {{{src, -1}, -1}};
        while (st.size()) {
            int at = st.back().first.first;
            int& it = beg[at];
            while (it < g[at].size() and used[g[at][it].second]) it++;
            if (it == g[at].size()) {
                if (ret.size() and ret.back().first.second != at)
                    return {false, {}};
                ret.push_back(st.back()), st.pop_back();
            } else {
                st.push_back({{g[at][it].first, at}, g[at][it].second});
                used[g[at][it].second] = 1;
            }
        }
        if (ret.size() != used.size()+1) return {false, {}};
        vector<pair<int, int>> ans;
        for (auto i : ret) ans.emplace_back(i.first.first, i.second);
        reverse(ans.begin(), ans.end());
        return {true, ans};
    }
    pair<bool, vector<pair<int, int>>> get_cycle() {
        if (!used.size()) return {true, {}};
        int src = 0;
        while (!g[src].size()) src++;
        auto ans = get_path(src);
        if (!ans.first or ans.second[0].first != ans.second.back().first)
            return {false, {}};
        ans.second[0].second = ans.second.back().second;
        ans.second.pop_back();
        return ans;
    }
};

void solve(){
    int n, m;
    cin >> n >> m;

    euler g(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g.add(a, b);
    }

    auto [flag, ans] = g.get_path(0);

    if(!flag || ans[0].first != ans.back().first) cout << "IMPOSSIBLE\n";
    else {
        for(auto x : ans) cout << x.first + 1 << ' ';
    }
    cout << '\n';
}
```

4.22 Message Route

Syrjälä's network has n computers and m connections. Your task is to find out if Uolevi can send a message to Maija, and if it is possible, what is the minimum number of computers on such a route.

Input
The first input line has two integers n and m: the number of computers and connections. The computers are numbered $1, 2, \ldots, n$. Uolevi's computer is 1 and Maija's computer is n. Then, there are m lines describing the connections. Each line has two integers a and b: there is a connection between those computers. Every connection is between two different computers, and there is at most one connection between any two computers.

Output
If it is possible to send a message, first print k: the minimum number of computers on a valid route. After this, print an example of such a route. You can print any valid solution. If there are no routes, print "IMPOSSIBLE".

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;
vi pai;

bool bfs(int a, int b){
    queue<int> fila;
    fila.push(a);
    check[a] = 1;
    pai[a] = 0;

    while(!fila.empty()){
        int x = fila.front();
        fila.pop();

        if(x == b) {
            break;
        }

        for(auto y : G[x]){
            if(check[y] != 1){
                fila.push(y);
                check[y] = 1;
                pai[y] = x;
            }
        }
    }

    if(check[b] != 1) return false;
    return true;
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);
    pai.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    if(!bfs(1, n)){
        cout << "IMPOSSIBLE\n";
        return;
    }

    int x = n;
    ans.pb(n);

    while(pai[x] != 0){
        ans.pb(pai[x]);
        x = pai[x];
    }
    reverse(all(ans));

    cout << ans.size() << '\n';
    for(int i = 0; i < ans.size(); i++){
        cout << ans[i] << ' ';
    }
    cout << '\n';
}
```

4.23 Monsters

You and some monsters are in a labyrinth. When taking a step to some direction in the labyrinth, each monster may simultaneously take one as well. Your goal is to reach one of the boundary squares without ever sharing a square with a monster. Your task is to find out if your goal is possible, and if it is, print a path that you can follow. Your plan has to work in any situation; even if the monsters know your path beforehand.

Input
The first input line has two integers n and m: the height and width of the map. After this there are n lines of m characters describing the map. Each character is . (floor), # (wall), A (start), or M (monster). There is exactly one A in the input.

Output
First print "YES" if your goal is possible, and "NO" otherwise. If your goal is possible, also print an example of a valid path (the length of the path and its description using characters D, U, L, and R). You can print any path, as long as its length is at most $n \cdot m$ steps.

Constraints
• $1 \le n, m \le 1000$

```cpp
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

vector<pii> macaco = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
vector<char> pos = {'D', 'U', 'R', 'L'};
vector<pii> ans;
char tab[maxn][maxn];
pii pai[maxn][maxn];
int n, m;
pii A;
vector<pii> Monster;

template <typename T,typename U>
pair<T,U> operator+(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first+r.first,l.second+r.second};
}
template <typename T,typename U>
pair<T,U> operator-(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first-r.first,l.second-r.second};
}

pii bfs(){
    queue<pii> fila;

    for(auto x : Monster){
        fila.push(x);
        pai[x.F][x.S] = {0, 0};
    }
    fila.push(A);
    pai[A.F][A.S] = {0, 0};

    while(!fila.empty()){
        pii X = fila.front();
        fila.pop();

        for(int i = 0; i < 4; i++){
            pii N = X + macaco[i];
            if(tab[N.F][N.S] == '.'){
                fila.push(N);
                tab[N.F][N.S] = tab[X.F][X.S];
                pai[N.F][N.S] = X;
            }
        }
    }

    pii ans = {0, 0};

    for(int i = 1; i <= n; i++){
        if(tab[i][1] == 'A') ans = {i, 1};
        if(tab[i][m] == 'A') ans = {i, m};
    }
    for(int i = 1; i <= m; i++){
        if(tab[1][i] == 'A') ans = {1, i};
        if(tab[n][i] == 'A') ans = {n, i};
    }

    return ans;
}

void solve(){
    cin >> n >> m;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] == 'A') A = {i, j};
            if(tab[i][j] == 'M') Monster.pb({i, j});
        }
    }

    pii res = bfs();

    if(res.F == 0){
        cout << "NO\n";
        return;
    }

    while(pai[res.F][res.S].F != 0){
        ans.pb(res - pai[res.F][res.S]);
```

```
        res = pai[res.F][res.S];
    }

    reverse(all(ans));

    cout << "YES\n";
    cout << ans.size() << '\n';
    for(pii x : ans){
        for(int i = 0; i < 4; i++){
            if(x == macaco[i]) cout << pos[i];
        }
    }
    cout << '\n';
}
```

4.24 Planets And Kingdoms

A game has n planets, connected by m teleporters. Two planets a and b belong to the same kingdom exactly when there is a route both from a to b and from b to a. Your task is to determine for each planet its kingdom.

Input
The first input line has two integers n and m: the number of planets and teleporters. The planets are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: you can travel from planet a to planet b through a teleporter.

Output
First print an integer k: the number of kingdoms. After this, print for each planet a kingdom label between 1 and k. You can print any valid solution.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 1e5+5;

int n, m;

vector<int> g[MAX];
vector<int> gi[MAX]; // grafo invertido
int vis[MAX];
stack<int> S;
int comp[MAX]; // componente conexo de cada vertice

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

int vischeck[MAX];
void dfscheck(int k) {
    vischeck[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vischeck[g[k][i]]) dfscheck(g[k][i]);
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    vector<int> transf(n);
    int at = 1;

    for(int i = 0; i < n; i++) {
        if(transf[comp[i]] == 0) {
            transf[comp[i]] = at;
            at++;
        }
    }

    cout << at - 1 << '\n';
    for(int i = 0; i < n; i++) cout << transf[comp[i]] << ' '
        ;
    cout << '\n';

}
```

4.25 Planets Cycles

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). You start on a planet and then travel through teleporters until you reach a planet that you have already visited before. Your task is to calculate for each planet the number of teleportations there would be if you started on that planet.

Input
The first input line has an integer n: the number of planets. The planets are numbered $1, 2, \ldots, n$. The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$.

Output
Print n integers according to the problem statement.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le t_i \le n$

```
const int MAX = 2e5 + 5, LOG = 30;

int n, q;
vector<int> g[MAX], gi[MAX], comp_elem[MAX], comp_graph_inv[
    MAX];
// root, dist_root
tuple<int, int> dist[MAX];
stack<int> S;
int vis[MAX], comp[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

void dfs_calc(int s, int p) {
    dist[s] = {get<0>(dist[p]), get<1>(dist[p]) + 1};
    if(get<1>(dist[s]) == 1) get<0>(dist[s]) = g[s][0];

    for(auto x : comp_graph_inv[s]) dfs_calc(x, s);
}

signed main() {
    cin >> n;

    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        g[i].push_back(a - 1);
        gi[a - 1].push_back(i);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        if (comp[i] == i) {
            int at = i, pos = 0;
            do {
                comp_elem[i].push_back(at);
                at = g[at][0];
            } while (at != i && comp[at] == i);
        }
    }

    for(int i = 0; i < n; i++)
        for(auto x : comp_elem[i])
            for(auto u : g[x])
                if(comp[u] != i) comp_graph_inv[comp[u]].
```

```
                    push_back(i);

    for(int i = 0; i < n; i++) dist[i] = {comp[i], 0};
    for(int i = 0; i < n; i++) {
        if((g[i].size() == 1 && g[i][0] == i) || (comp_elem[
             comp[i]].size() > 1 && comp[i] == i)) {
            dist[i] = {i, -1};
            dfs_calc(i, i);
        }
    }

    for(int i = 0; i < n; i++) {
        auto d = dist[i];
        cout << get<1>(d) + comp_elem[comp[get<0>(d)]].size()
              << ' ';
    }
    cout << '\n';

    return 0;
}
```

4.26 Planets Queries I

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). Your task is to process q queries of the form: when you begin on planet x and travel through k teleporters, which planet will you reach?

Input
The first input line has two integers n and q: the number of planets and queries. The planets are numbered $1, 2, \ldots, n$. The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$. Finally, there are q lines describing the queries. Each line has two integers x and k: you start on planet x and travel through k teleporters.

Output
Print the answer to each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le t_i \le n$ • $1 \le x \le n$ • $0 \le k \le 10^9$

```
const int MAX = 2e5 + 5, LOG = 30;

int n, q;
int bin_lift[MAX][LOG];

signed main() {
    cin >> n >> q;

    for(int i = 1; i <= n; i++) cin >> bin_lift[i][0];

    for(int i = 1; i < LOG; i++)
        for(int s = 1; s <= n; s++)
            bin_lift[s][i] = bin_lift[bin_lift[s][i-1]][i-1];

    while(q--) {
        int u, k; cin >> u >> k;
        int at = 0, dest = u;
        while(k > 0) {
            if(k % 2) dest = bin_lift[dest][at];
            at++; k/=2;
        }

        cout << dest << '\n';
    }

    return 0;
}
```

4.27 Planets Queries II

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). You have to process q queries of the form: You are now on planet a and want to reach planet b. What is the minimum number of teleportations?

Input
The first input line contains two integers n and q: the number of planets and queries. The planets are numbered $1, 2, \ldots, n$. The second line contains n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. Finally, there are q lines describing the queries. Each line has two integers a and b: you are now on planet a and want to reach planet b.

Output
For each query, print the minimum number of teleportations. If it is not possible to reach the destination, print -1.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 2e5 + 5, LOG = 30;

int n, q;
vector<int> g[MAX], gi[MAX], comp_elem[MAX], comp_graph[MAX],
      comp_graph_inv[MAX];
// root, dist_root, tin, tout
tuple<int, int, int> dist[MAX];
stack<int> S;
int vis[MAX], comp[MAX], comp_place[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

int t = 0;
void dfs_calc(int s, int p) {
    dist[s] = {get<0>(dist[p]), get<1>(dist[p]) + 1, t,
          t++};
    if(get<1>(dist[s]) == 1) get<0>(dist[s]) = g[s][0];

    for(auto x : comp_graph_inv[s]) dfs_calc(x, s);

    get<3>(dist[s]) = t; t++;
}

signed main() {
    cin >> n >> q;

    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        g[i].push_back(a - 1);
        gi[a - 1].push_back(i);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        if (comp[i] == i) {
            int at = i, pos = 0;
            do {
                comp_elem[i].push_back(at);
                comp_place[at] = pos++;
                at = g[at][0];
            } while (at != i && comp[at] == i);
        }
    }

    for(int i = 0; i < n; i++)
        for(auto x : comp_elem[i])
            for(auto u : g[x])
                if(comp[u] != i) {
                    comp_graph[i].push_back(comp[u]);
                    comp_graph_inv[comp[u]].push_back(i);
                }

    for(int i = 0; i < n; i++) {
        if((g[i].size() == 1 && g[i][0] == i) || (comp_elem[
             comp[i]].size() > 1 && comp[i] == i)) {
            dist[i] = {i, -1, 0, 0};
            dfs_calc(i, i);
        }
    }

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b; a--; b--;
        auto da = dist[a], db = dist[b];
        if(comp[a] == comp[b]) {
            int ans = comp_place[b] - comp_place[a];
            if(ans < 0) ans += comp_elem[comp[a]].size();
            cout << ans << '\n';
        }
        else if(comp[get<0>(da)] == comp[b]) {
            int ans = comp_place[b] - comp_place[get<0>(da)];
            if(ans < 0) ans += comp_elem[comp[get<0>(da)]].
                  size();
            ans += get<1>(da);
            cout << ans << '\n';
        }
        else if(get<0>(da) == get<0>(db) && get<2>(da) > get
              <2>(db) && get<3>(da) < get<3>(db)) {
```

```
            cout << get<1>(da) - get<1>(db) << '\n';
        }
        else {
            cout << -1 << '\n';
        }
    }

    return 0;
}
```

4.28 Police Chase

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city. What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Input
The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered $1, 2, \ldots, n$. The bank is located at crossing 1, and the harbor is located at crossing n. After this, there are m lines that describing the streets. Each line has two integers a and b: there is a street between crossings a and b. All streets are two-way streets, and there is at most one street between two crossings.

Output
First print an integer k: the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le a, b \le n$

```
int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need
            flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for
            random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] +
                            1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.addEdge(a, b, 1);
        G.addEdge(b, a, 1);
    }

    cout << G.calc(0, n - 1) << '\n';
    for(int i = 0; i < n; i++)
        for(int j = 0; j < sz(G.adj[i]); j+=2) {
            auto e = G.adj[i][j];
            if(i < e.to && G.leftOfMinCut(i) ^ G.leftOfMinCut
                (e.to))
                cout << i + 1 << ' ' << e.to + 1 << '\n';
        }
}
```

4.29 Road Construction

There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads. A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the new roads. Each line has two integers a and b: a new road is constructed between cities a and b. You may assume that every road will be constructed between two different cities.

Output
Print m lines: the required information after each day.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
int n, m;

struct dsu {
    vector<int> id, sz;

    dsu(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(),
        0); }

    int find(int a) { return a == id[a] ? a : id[a] = find(id
        [a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a;
    }
};

void solve(){
    cin >> n >> m;
    dsu D(n);

    int tot = n, maxi = 1;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        if(D.find(a) != D.find(b)) tot--;
        D.unite(a, b);
        maxi = max(maxi, D.sz[D.find(a)]);
        cout << tot << ' ' << maxi << '\n';
    }
}
```

4.30 Road Reparation

There are n cities and m roads between them. Unfortunately, the condition of the roads is so poor that they cannot be used. Your task is to repair some of the roads so that there will be a decent route between any two cities. For each road, you know its reparation cost, and you should find a solution where the total cost is as small as possible.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the roads. Each line has three integers a, b and c: there is a road between cities a and b, and its reparation cost is c. All roads are two-way roads. Every road is between two different cities, and there is at most one road between two cities.

Output
Print one integer: the minimum total reparation cost. However, if there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
int n, m;
vector<tuple<int, int, int>> edg; // {peso,[x,y]}

```

```
struct dsu {
    vector<int> id, sz;

    dsu(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(),
          0); }

    int find(int a) { return a == id[a] ? a : id[a] = find(id
         [a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a;
    }
};

pair<int, vector<tuple<int, int, int>>> kruskal(int n) {
    dsu D(n);
    sort(edg.begin(), edg.end());

    int cost = 0;
    vector<tuple<int, int, int>> mst;
    for (auto [w,x,y] : edg) if (D.find(x) != D.find(y)) {
        mst.emplace_back(w, x, y);
        cost += w;
        D.unite(x,y);
    }

    int root = D.find(0);
    for(int i = 0; i < n; i++)
        if (D.find(i) != root) return{-1, {}};

    return {cost, mst};
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c;
        edg.push_back({c, a - 1, b - 1});
    }

    auto ans = kruskal(n);
    if(ans.first == -1) cout << "IMPOSSIBLE\n";
    else cout << ans.first << '\n';
}
```

4.31 Round Trip

Byteland has n cities and m roads between them. Your task is to design a round trip that begins in a city, goes through two or more other cities, and finally returns to the starting city. Every intermediate city on the route has to be distinct.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the roads. Each line has two integers a and b: there is a road between those cities. Every road is between two different cities, and there is at most one road between any two cities.

Output

First print an integer k: the number of cities on the route. Then print k cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
graph G;
vi check;
vi pai;

pii bfs(int a){
    queue<int> fila;
    fila.push(a);
    check[a] = 1;
    pai[a] = 0;

    int p1 = 0, p2 = 0;

    while(!fila.empty()){
        int x = fila.front();
        fila.pop();

        for(auto y : G[x]){
            if(check[y] == 1 && y != pai[x]){
                p1 = x;
                p2 = y;
                break;
            }
            else if(check[y] != 1){
                fila.push(y);
                check[y] = 1;
                pai[y] = x;
            }
        }

        if(p1 != 0) break;
    }

    return {p1, p2};
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);
    pai.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    pii p;

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            p = bfs(i);
            if(p.F != 0) break;
        }
    }

    if(p.F == 0){
        cout << "IMPOSSIBLE\n";
        return;
    }

    vi ans1, ans2, ans;

    int x = p.F;
    ans1.pb(x);
    while(pai[x] != 0){
        ans1.pb(pai[x]);
        x = pai[x];
    }

    x = p.S;
    ans2.pb(x);
    while(pai[x] != 0){
        ans2.pb(pai[x]);
        x = pai[x];
    }

    int bck = 0;
    while(ans1.back() == ans2.back()){
        bck = ans1.back();
        ans1.pop_back(); ans2.pop_back();
    }

    cout << ans1.size() + ans2.size() + 2 << '\n';
    for(int i = 0; i < ans1.size(); i++){
        cout << ans1[i] << ' ';
    }
    cout << bck << ' ';
    for(int i = ans2.size() - 1; i >= 0; i--){
        cout << ans2[i] << ' ';
    }
    cout << ans1[0] << '\n';
}
```

4.32 Round Trip II

Byteland has n cities and m flight connections. Your task is to design a round trip that begins in a city, goes through one or more other cities, and finally returns to the starting city. Every intermediate city on the route has to be distinct.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the flights. Each line has two integers a and b: there is a flight connection from city a to city b. All connections are one-way flights from a city to another city.

Output

First print an integer k: the number of cities on the route. Then print k cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int INF = 1'000'000'000'000'000'000;

vector<vector<int>> adj;
vector<int> vis, pai;
```

```cpp
vector<int> ans;
int cnt = 0;

bool dfs(int v) {
    vis[v] = 1;
    for(auto x : adj[v]) {
        if(vis[x] == 1) {
            ans.push_back(x);
            int at = v;
            while(at != x) {
                ans.push_back(at);
                at = pai[at];
            }
            ans.push_back(x);

            return 1;
        }
        else if (vis[x] == 0) {
            pai[x] = v;
            if(dfs(x)) return 1;
        }
    }
    vis[v] = 2;
    return 0;
}

signed main() {
    int n, m; cin >> n >> m;
    adj.resize(n); vis.resize(n); pai.resize(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        adj[a].push_back(b);
    }

    for(int i = 0; i < n; i++) {
        if(vis[i] == 0) {
            if(dfs(i)) {
                reverse(ans.begin(), ans.end());
                cout << ans.size() << '\n';
                for(auto x : ans) cout << x + 1 << ' ';
                cout << '\n';
                return 0;
            }
        }
    }

    cout << "IMPOSSIBLE\n";

    return 0;
}
```

4.33 School Dance

There are n boys and m girls in a school. Next week a school dance will be organized. A dance pair consists of a boy and a girl, and there are k potential pairs. Your task is to find out the maximum number of dance pairs and show how this number can be achieved.

Input
The first input line has three integers n, m and k: the number of boys, girls, and potential pairs. The boys are numbered $1, 2, \ldots, n$, and the girls are numbered $1, 2, \ldots, m$. After this, there are k lines describing the potential pairs. Each line has two integers a and b: boy a and girl b are willing to dance together.

Output
First print one integer r: the maximum number of dance pairs. After this, print r lines describing the pairs. You can print any valid solution.

Constraints
• $1 \le n, m \le 500$ • $1 \le k \le 1000$ • $1 \le a \le n$ • $1 \le b \le m$

```cpp
mt19937 rng((int) chrono::steady_clock::now().
      time_since_epoch().count());

struct kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> vis, ma, mb;

    kuhn(int n_, int m_) : n(n_), m(m_), g(n),
        vis(n+m), ma(n, -1), mb(m, -1) {}

    void add(int a, int b) { g[a].push_back(b); }

    bool dfs(int i) {
        vis[i] = 1;
        for (int j : g[i]) if (!vis[n+j]) {
            vis[n+j] = 1;
            if (mb[j] == -1 or dfs(mb[j])) {
                ma[i] = j, mb[j] = i;
                return true;
            }
        }
        return false;
    }
    int matching() {
        int ret = 0, aum = 1;
        for (auto& i : g) shuffle(i.begin(), i.end(), rng);
        while (aum) {
            for (int j = 0; j < m; j++) vis[n+j] = 0;
            aum = 0;
            for (int i = 0; i < n; i++)
                if (ma[i] == -1 and dfs(i)) ret++, aum = 1;
        }
        return ret;
    }
};

pair<vector<int>, vector<int>> recover(kuhn& K) {
    K.matching();
    int n = K.n, m = K.m;
    for (int i = 0; i < n+m; i++) K.vis[i] = 0;
    for (int i = 0; i < n; i++) if (K.ma[i] == -1) K.dfs(i);
    vector<int> ca, cb;
    for (int i = 0; i < n; i++) if (!K.vis[i]) ca.push_back(i
        );
    for (int i = 0; i < m; i++) if (K.vis[n+i]) cb.push_back(
        i);
    return {ca, cb};
}

void solve() {
    int n, m, k; cin >> n >> m >> k;
    kuhn G(n, m);
    for(int i = 0; i < k; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.add(a, b);
    }

    int t = G.matching();
    cout << t << '\n';
    for(int i = 0; i < n; i++) {
        if(G.ma[i] != -1) cout << i + 1 << ' ' << G.ma[i] + 1
            << '\n';
    }
}
```

4.34 Shortest Routes I

There are n cities and m flight connections between them. Your task is to determine the length of the shortest route from Syrjälä to every city.

Input
The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$, and city 1 is Syrjälä. After that, there are m lines describing the flight connections. Each line has three integers a, b and c: a flight begins at city a, ends at city b, and its length is c. Each flight is a one-way flight. You can assume that it is possible to travel from Syrjälä to all other cities.

Output
Print n integers: the shortest route lengths from Syrjälä to cities $1, 2, \ldots, n$.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

const int INF = 1'000'000'000'000'000'000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                q.erase({d[to], to});
                d[to] = d[v] + len;
                p[to] = v;
                q.insert({d[to], to});
            }
        }
    }
}

signed main() {
    int n, m; cin >> n >> m;
    adj.resize(n+1);
```

```
35
36      for(int i = 0; i < m; i++){
37          int a, b, c; cin >> a >> b >> c;
38          adj[a].push_back({b, c});
39      }
40
41      vector<int> d, p;
42      dijkstra(1, d, p);
43
44      for(int i = 1; i <= n; i++){
45          cout << d[i] << ' ';
46      }
47      cout << '\n';
48
49      return 0;
50  }
```

4.35 Shortest Routes II

There are n cities and m roads between them. Your task is to process q queries where you have to determine the length of the shortest route between two given cities.

Input
The first input line has three integers n, m and q: the number of cities, roads, and queries. Then, there are m lines describing the roads. Each line has three integers a, b and c: there is a road between cities a and b whose length is c. All roads are two-way roads. Finally, there are q lines describing the queries. Each line has two integers a and b: determine the length of the shortest route between cities a and b.

Output
Print the length of the shortest route for each query. If there is no route, print -1 instead.

Constraints
• $1 \le n \le 500$ • $1 \le m \le n^2$ • $1 \le q \le 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
1  const int inf = LLONG_MAX;
2
3  void floydWarshall(vector<vector<int>> &dist) {
4      int n = dist.size();
5      for(int k = 0; k < n; k++)
6      for(int j = 0; j < n; j++)
7      for(int i = 0; i < n; i++)
8      if(dist[i][k] != inf && dist[k][j] != inf) {
9          dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
               ;
10     }
11 }
12
13 signed main() {
14     int n, m, q; cin >> n >> m >> q;
15     vii dist(n, vector<int>(n, inf));
16
17     for(int i = 0; i < m; i++){
18         int a, b, c; cin >> a >> b >> c;
19         dist[a-1][b-1] = min(dist[a-1][b-1], c);
20         dist[b-1][a-1] = min(dist[b-1][a-1], c);
21     }
22     for(int i = 0; i < n; i++) dist[i][i] = 0;
23
24     floydWarshall(dist);
25
26     while(q--) {
27         int a, b; cin >> a >> b;
28         cout << (dist[a-1][b-1] == inf ? -1 : dist[a-1][b-1])
               << '\n';
29     }
30
31     return 0;
32 }
```

4.36 Teleporters Path

A game has n levels and m teleporters between them. You win the game if you move from level 1 to level n using every teleporter exactly once. Can you win the game, and what is a possible way to do it?

Input
The first input line has two integers n and m: the number of levels and teleporters. The levels are numbered $1, 2, \ldots, n$. Then, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from level a to level b. You can assume that each pair (a, b) in the input is distinct.

Output
Print $m+1$ integers: the sequence in which you visit the levels during the game. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
1  const int MAX = 1e5 + 5;
2  int n, m;
3  vector<int> G[MAX];
4
5  template<bool directed=false> struct euler {
6      int n;
7      vector<vector<pair<int, int>>> g;
8      vector<int> used;
9
10     euler(int n_) : n(n_), g(n) {}
11     void add(int a, int b) {
12         int at = used.size();
13         used.push_back(0);
14         g[a].emplace_back(b, at);
15         if (!directed) g[b].emplace_back(a, at);
16     }
17
18     pair<bool, vector<pair<int, int>>> get_path(int src) {
19         if (!used.size()) return {true, {}};
20         vector<int> beg(n, 0);
21         for (int& i : used) i = 0;
22         vector<pair<pair<int, int>, int>> ret, st = {{{src,
               -1}, -1}};
23         while (st.size()) {
24             int at = st.back().first.first;
25             int& it = beg[at];
26             while (it < g[at].size() and used[g[at][it].
                   second]) it++;
27             if (it == g[at].size()) {
28                 if (ret.size() and ret.back().first.second !=
                       at)
29                     return {false, {}};
30                 ret.push_back(st.back()), st.pop_back();
31             } else {
32                 st.push_back({{g[at][it].first, at}, g[at][it
                       ].second});
33                 used[g[at][it].second] = 1;
34             }
35         }
36         if (ret.size() != used.size()+1) return {false, {}};
37         vector<pair<int, int>> ans;
38         for (auto i : ret) ans.emplace_back(i.first.first, i.
               second);
39         reverse(ans.begin(), ans.end());
40         return {true, ans};
41     }
42 };
43
44 void solve() {
45     cin >> n >> m;
46     euler<true> E(n);
47     for(int i = 0; i < m; i++) {
48         int a, b; cin >> a >> b; a--; b--;
49         E.add(a, b);
50     }
51
52     auto x = E.get_path(0);
53     if(!x.first || x.second.back().first != n - 1) cout << "
           IMPOSSIBLE\n";
54     else {
55         for(auto k : x.second) cout << k.first + 1 << ' ';
56         cout << '\n';
57     }
58 }
```

5 Range Queries

5.1 Distinct Values Queries

You are given an array of n integers and q queries of the form: how many distinct values are there in a range $[a, b]$?

Input
The first input line has two integers n and q: the array size and number of queries. The next line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b.

Output
For each query, print the number of distinct values in the range.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```cpp
struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n), is_first(n), next_eq(n);
    map<int, int> M;
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = n - 1; i >= 0; i--) {
        auto it = M.find(v[i]);
        if(it == M.end()) {
            is_first[i] = 1;
            next_eq[i] = n;
        }
        else {
            is_first[i] = 1;
            is_first[it -> second] = 0;
            next_eq[i] = it -> second;
        }
        M[v[i]] = i;
    }

    vector<pair<pair<int, int>, int>> query(q);
    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        query[i] = {{a - 1, b - 1}, i};
    }

    sort(all(query));
    vector<int> ans(q);

    FT bit(n);
    for(int i = 0; i < n; i++) if(is_first[i]) bit.update(i, 1);

    int at = 0;
    for(int i = 0; i < q; i++) {
        int a = query[i].first.first, b = query[i].first.second;
        for(; at < a; at++) if(next_eq[at] <= n - 1) bit.update(next_eq[at], 1);
        ans[query[i].second] = bit.query(b + 1) - bit.query(a);
    }

    for(auto x : ans) cout << x << '\n';
}
```

5.2 Distinct Values Queries II

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u check if every value in range $[a, b]$ is distinct

Input
The first line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
For each query of type 2, print YES if every value in the range is distinct and NO otherwise.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

```cpp
const int MAX = 2e5;

int n, q;
vector<int> v, next_eq, prev_eq;
set<int> seq[2 * MAX];

namespace seg {
    // o proximo que eh igual a alguem do node
    int seg[4*MAX];

    int build(int p=1, int l=0, int r=n-1) {
        if (l == r) {
            auto it = seq[v[l]].lower_bound(l + 1);
            if(it != seq[v[l]].end()) return seg[p] = *it;
            return seg[p] = MAX + 1;
        }
        int m = (l+r)/2;
        return seg[p] = min(build(2*p, l, m), build(2*p+1, m+1, r));
    }
    int query(int a, int b, int p=1, int l=0, int r=n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return MAX + 1;
        int m = (l+r)/2;
        return min(query(a, b, 2*p, l, m), query(a, b, 2*p+1, m+1, r));
    }
    void update(int a, int p=1, int l=0, int r=n-1) {
        if (l == r) {
            auto it = seq[v[l]].lower_bound(l + 1);
            if(it != seq[v[l]].end()) seg[p] = *it;
            else seg[p] = MAX + 1;
        }
        else {
            int m = (l+r)/2;
            if (a <= m)
                update(a, p*2, l, m);
            else
                update(a, p*2+1, m+1, r);
            seg[p] = min(seg[p*2], seg[p*2 + 1]);
        }
    }
};

void solve(){
    cin >> n >> q; v.resize(n);
    vector<int> all_numbers;

    for(int i = 0; i < n; i++) {
        cin >> v[i];
        all_numbers.push_back(v[i]);
    }

    vector<pair<int, pair<int, int>>> query(q);
    for(int i = 0; i < q; i++) {
        int a, b, c; cin >> a >> b >> c;
        if(a == 1) {
            query[i] = {a, {b - 1, c}};
            all_numbers.push_back(c);
        }
        else {
            query[i] = {a, {b - 1, c - 1}};
        }
    }

    sort(all(all_numbers));
    all_numbers.erase(unique(all(all_numbers)), all_numbers.end());

    for(int i = 0; i < n; i++) v[i] = lower_bound(all(all_numbers), v[i]) - all_numbers.begin();

    for(int i = 0; i < n; i++) seq[v[i]].insert(i);

    seg::build();

    for(int i = 0; i < q; i++) {
        if(query[i].F == 1) {
            query[i].S.S = lower_bound(all(all_numbers), query[i].S.S) - all_numbers.begin();
            auto [u, k] = query[i].S;

            vector<int> alts;
            alts.push_back(u);

            auto it = seq[v[u]].find(u);
            if(it != seq[v[u]].begin()) alts.push_back(*prev(it));
            seq[v[u]].erase(it);

            v[u] = k;
            seq[k].insert(u);

            it = seq[v[u]].find(u);
            if(it != seq[v[u]].begin()) alts.push_back(*prev(it));

            for(auto x : alts) seg::update(x);
        }
        else {
            auto [a, b] = query[i].S;
            cout << ((seg::query(a, b) <= b) ? "NO\n" : "YES\n");
        }
    }
```

```
    }
}
```

5.3 Dynamic Range Minimum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the minimum value in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

```
const int MAX = 2e5 + 5;
const int INF = 2e9;
int v[MAX];

namespace seg {
    ll seg[4*MAX];
    int n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = min(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    void build(int n2, int* v2) {
        n = n2, v = v2;
        build();
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return INF;
        int m = (l+r)/2;
        return min(query(a, b, 2*p, l, m), query(a, b, 2*p+1,
            m+1, r));
    }
    void update(int a, int x, int p=1, int l=0, int r=n-1) {
        if (l == r) {
            seg[p] = x;
        }
        else {
            int m = (l+r)/2;
            if (a <= m)
                update(a, x, p*2, l, m);
            else
                update(a, x, p*2+1, m+1, r);
            seg[p] = min(seg[p*2], seg[p*2+1]);
        }
    }
};
```

```

void solve(){
    int n, q; cin >> n >> q;

    for(int i = 0; i < n; i++) {
        cin >> v[i];
    }

    seg::build(n, v);

    for(int i = 0; i < q; i++) {
        int qi; cin >> qi;
        if(qi == 1) {
            int k, u; cin >> k >> u; k--;
            seg::update(k, u);
        }
        else {
            int a, b; cin >> a >> b; a--; b--;
            cout << seg::query(a, b) << '\n';
        }
    }
}
```

5.4 Dynamic Range Sum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n);
    FT bit(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
        bit.update(i, v[i]);
    }

    for(int i = 0; i < q; i++) {
        int a, b, c; cin >> a >> b >> c; b--;
        if(a == 1) {
            bit.update(b, c - v[b]);
            v[b] = c;
        }
        else {
            c--;
            cout << bit.query(c + 1) - bit.query(b) << '\n';
        }
    }
}
```

5.5 Forest Queries

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n). Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Input
The first input line has two integers n and q: the size of the forest and the number of queries. Then, there are n lines describing the forest. Each line has n characters: . is an empty square and * is a tree. Finally, there are q lines describing the queries. Each line has four integers y_1, x_1, y_2, x_2 corresponding to the corners of a rectangle.

Output
Print the number of trees inside each rectangle.

Constraints
• $1 \leq n \leq 1000$ • $1 \leq q \leq 2 \cdot 10^5$ • $1 \leq y_1 \leq y_2 \leq n$ • $1 \leq x_1 \leq x_2 \leq n$

```
const int MAX = 1005;

int pref[MAX][MAX];

void solve(){
    int n, q; cin >> n >> q;

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            char c; cin >> c;
            pref[i][j] = (c == '*' ? 1 : 0);
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            pref[i][j] += pref[i][j - 1];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            pref[i][j] += pref[i - 1][j];
```

```
        }
    }

    while(q--) {
        int x1, x2, y1, y2;
        cin >> y1 >> x1 >> y2 >> x2;

        int ans = pref[y2][x2] + pref[y1 - 1][x1 - 1] - pref[
            y2][x1 - 1] - pref[y1 - 1][x2];
        cout << ans << '\n';
    }
}
```

5.6 Forest Queries II

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or has a tree. Your task is to process q queries of the following types:
Change the state (empty/tree) of a square. How many trees are inside a rectangle in the forest?

Input
The first input line has two integers n and q: the size of the forest and the number of queries. Then, there are n lines describing the forest. Each line has n characters: . is an empty square and * is a tree. Finally, there are q lines describing the queries. The format of each line is either "1 y x" or "2 y_1 x_1 y_2 x_2".

Output
Print the answer to each query of the second type.

Constraints
• $1 \le n \le 1000$ • $1 \le q \le 2 \cdot 10^5$ • $1 \le y, x \le n$ • $1 \le y_1 \le y_2 \le n$ • $1 \le x_1 \le x_2 \le n$

```
const int maxn = 1e3 + 10;
int v[maxn][maxn], bit[maxn][maxn];

void update( int i, int j, int val ){
    for( int a = i; a < maxn; a += a&-a ){
        for( int b = j; b < maxn; b += b&-b ) bit[a][b] +=
            val;
    }
}

int query( int i, int j ){
    int resp = 0;
    for( int a = i; a > 0; a -= a&-a ){
        for( int b = j; b > 0; b -= b&-b ) resp += bit[a][b];
    }
    return resp;
}

int solve( int a, int b, int c, int d ){
    return query( c, d ) - query( a - 1, d ) - query( c, b -
        1 ) + query( a - 1, b - 1 );
}

int main(){
    int n, q; cin >> n >> q;
    for( int i = 1; i <= n; i++ ){
        for( int j = 1; j <= n; j++ ){
            char c; cin >> c;
            v[i][j] = ((c == '*') ? 1 : 0 );
            if( v[i][j] ) update( i, j, v[i][j] );
        }
    }
    while( q-- ){
        int t; cin >> t;
        if( t == 1 ){
            int a, b; cin >> a >> b;
            if( v[a][b] ) update( a, b, -1 );
            else update( a, b, 1 );
            v[a][b] ^= 1;
        }
        else{
            int a, b, c, d; cin >> a >> b >> c >> d;
            cout << solve( a, b, c, d ) << endl;
        }
    }
}
```

5.7 Hotel Queries

There are n hotels on a street. For each hotel you know the number of free rooms. Your task is to assign hotel rooms for groups of tourists. All members of a group want to stay in the same hotel. The groups will come to you one after another, and you know for each group the number of rooms it requires. You always assign a group to the first hotel having enough rooms. After this, the number of free rooms in the hotel decreases.

Input
The first input line contains two integers n and m: the number of hotels and the number of groups. The hotels are numbered $1, 2, \ldots, n$. The next line contains n integers $h_1, h_2, \ldots, h_n$: the number of free rooms in each hotel. The last line contains m integers $r_1, r_2, \ldots, r_m$: the number of rooms each group requires.

Output
Print the assigned hotel for each group. If a group cannot be assigned a hotel, print 0 instead.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le h_i \le 10^9$ • $1 \le r_i \le 10^9$

```
const int MAX = 2e5+20;
const ll LINF = 1e18;

namespace seg {
    ll seg[4*MAX], lazy[4*MAX];
    int n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = 0;
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = max(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    void build(int n2, int* v2) {
        n = n2, v = v2;
        build();
    }
    ll query(int a, int p=1, int l=0, int r=n-1) {
        if(seg[p] < a) return -1;
        if(l == r) return l;
        int m = (l+r)/2;
        int pos = query(a, 2*p, l, m);
        if(pos != -1) return pos;
        return query(a, 2*p+1, m+1, r);
    }
    ll update(int a, int x, int p=1, int l=0, int r=n-1) {
        if (a < l || r < a) {
            return seg[p];
        }
        if(l == r) {
            return seg[p] = seg[p] + x;
        }
        int m = (l+r)/2;
        return seg[p] = max(update(a, x, 2*p, l, m), update(a
            , x, 2*p+1, m+1, r));
    }
};

void solve() {
    int n, m; cin >> n >> m;

    int h[n];
    for(int i = 0; i < n; i++) cin >> h[i];

    seg::build(n, h);

    for(int i = 0; i < m; i++) {
        int s; cin >> s;
        int ans = seg::query(s);
        cout << ans + 1 << ' ';
        if(ans != -1)
            seg::update(ans, -s);
    }
    cout << '\n';
}
```

5.8 Increasing Array Queries

You are given an array that consists of n integers. The array elements are indexed $1, 2, \ldots, n$. You can modify the array using the following operation: choose an array element and increase its value by one. Your task is to process q queries of the form: when we consider a subarray from position a to position b, what is the minimum number of operations after which the subarray is increasing? An array is increasing if each element is greater than or equal with the previous element.

Input
The first input line has two integers n and q: the size of the array and the number of queries. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Finally, there are q lines that describe the queries. Each line has two integers a and b: the starting and ending position of a subarray.

Output
For each query, print the minimum number of operations.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$ • $1 \le a \le b \le n$

```cpp
const int maxn = 2e5 + 10;
ll v[maxn];

struct node{
    vector<ll> v, sv;
    ll cont = 0, maxi = 0;
    node(){}
    node( ll x ): cont(0), maxi(x) { v.push_back(0); v.push_back(x); sv.push_back(0); sv.push_back(x); }
    node operator + ( node n ){
        node resp;
        ll sum = 0;
        resp.cont = cont + n.cont;
        resp.v.push_back(0); resp.sv.push_back(0);
        for( int i = 1; i < v.size(); i++ ){
            sum += v[i]; resp.maxi = v[i];
            resp.v.push_back(v[i]);
            resp.sv.push_back(sum);
        }
        for( int i = 1; i < n.v.size(); i++ ){
            if( n.v[i] < resp.maxi ){ resp.cont += resp.maxi - n.v[i]; n.v[i] = resp.maxi; }
            else resp.maxi = n.v[i];
            sum += n.v[i];
            resp.v.push_back(n.v[i]); resp.sv.push_back(sum);
        }
        return resp;
    }
} seg[4*maxn];

struct aux_query{
    ll maxi, resp; aux_query( ll maxi = 0, ll resp = 0 ) : maxi(maxi), resp(resp) {}
    aux_query operator + ( aux_query a ){ return aux_query( max(maxi, a.maxi), resp + a.resp ); }
} nulo;

void build( int pos, int ini, int fim ){
    if( ini == fim ){
        seg[pos] = node(v[ini]);
        return;
    }
    int mid = ( ini + fim )/2;
    int l = 2*pos, r = 2*pos + 1;
    build( l, ini, mid ); build( r, mid + 1, fim );
    seg[pos] = seg[l] + seg[r];
}

aux_query query( int pos, int ini, int fim, int ki, int kf, ll val ){
    if( ki > fim || ini > kf ) return nulo;
    if( ki <= ini && fim <= kf ){
        ll id = upper_bound( seg[pos].v.begin(), seg[pos].v.end(), val ) - seg[pos].v.begin();
        return aux_query( max( seg[pos].v.back(), val ), (id - 1)*val - seg[pos].sv[id - 1] + seg[pos].cont );
    }
    int mid = ( ini + fim )/2;
    int l = 2*pos, r = 2*pos + 1;
    aux_query ql = query( l, ini, mid, ki, kf, val );
    aux_query qr = query( r, mid + 1, fim, ki, kf, max( ql.maxi, val ) );
    return ql + qr;
}

int main(){
    int n, q; scanf("%d %d", &n, &q );
    for( int i = 1; i <= n; i++ ) scanf("%d", &v[i] );
    build( 1, 1, n );
    while( q-- ){
        int a, b; scanf("%d %d", &a, &b );
        aux_query x = query( 1, 1, n, a, b, 0 );
        printf("%lld\n", x.resp );
    }
}
```

5.9 List Removals

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Input
The first input line has an integer n: the initial size of the list. During the process, the elements are numbered $1, 2, \ldots, k$ where k is the current size of the list. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list. The last line has n integers $p_1, p_2, \ldots, p_n$: the positions of the elements to be removed.

Output
Print the elements in the order they are removed.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$ • $1 \le p_i \le n - i + 1$

```cpp
template <class T>
    using ord_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void solve() {
    int n; cin >> n;
    vector<int> x(n);
    for(auto &a : x) cin >> a;

    ord_set<int> s;
    for(int i = 0; i < n; i++) s.insert(i);

    for(int i = 0; i < n; i++) {
        int p; cin >> p;
        auto pos = s.find_by_order(p - 1);
        cout << x[*pos] << ' ';
        s.erase(pos);
    }
    cout << '\n';
}
```

5.10 Missing Coin Sum Queries

You have n coins with positive integer values. The coins are numbered $1, 2, \ldots, n$. Your task is to process q queries of the form: "if you can use coins $a \ldots b$, what is the smallest sum you cannot produce?"

Input
The first input line has two integers n and q: the number of coins and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin. Finally, there are q lines that describe the queries. Each line has two values a and b: you can use coins $a \ldots b$.

Output
Print the answer for each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$ • $1 \le a \le b \le n$

```cpp
using vi = vector<int>;
using ll = long long;

const int maxx = 1e9;

class PersistentSegtree{
    private:

    struct Node{
        ll sum;
        int mini, maxi, l, r;
        Node( ll sum = 0, int mini = maxx, int maxi = -maxx ) : sum(sum), mini(mini), maxi(maxi), l(0), r(0) {}
        void merge( Node L, Node R ){
            sum = L.sum + R.sum;
            mini = min( L.mini, R.mini );
            maxi = max( L.maxi, R.maxi );
        }
    };

    Node *seg;
    vi rt;
    int MIN, MAX, id;

    int create(){
        return id++;
    }

    int copy( int node ){
        int cp = create();
        return seg[cp] = seg[node], cp;
    }

    int update( int node, int ti, int tf, int id, int x ){
        int cp = copy(node);
        if( ti == tf ){
            seg[cp].merge(seg[node], Node( x, x, x ) );
            return cp;
        }

        int tm = (ti + tf)>>1, &l = seg[cp].l, &r = seg[cp].r;
        if( id <= tm ) l = update( seg[node].l, ti, tm, id, x );
        else r = update( seg[node].r, tm + 1, tf, id, x );

        seg[cp].merge( seg[l], seg[r] );
        return cp;
    }

    ll query( int n1, int n2, int ti, int tf, ll qi, ll qf ){
        if( qi > seg[n2].maxi || seg[n2].mini > qf || n1 == n2 ) return 0;
        if( qi <= seg[n2].mini && seg[n2].maxi <= qf ) return seg[n2].sum - seg[n1].sum;
        int tm = (ti + tf)>>1;
        return query( seg[n1].l, seg[n2].l, ti, tm, qi, qf ) + query( seg[n1].r, seg[n2].r, tm + 1, tf, qi, qf );
    }

    public:
```

```
    PersistentSegtree( int n, int MIN, int MAX ) : MIN(MIN),
         MAX(MAX), rt(1), id(0) {
        seg = new Node[n*__lg(MAX - MIN + 1) + 2*n];
        create();
    }

    void update( int id, int x ){
        rt.push_back(update(rt.back(), MIN, MAX, id, x ) );
    }

    ll query( int L, int R, ll D, ll U ){
        return query( rt[L], rt[R + 1], MIN, MAX, D, U );
    }
};

void solve(){
    int n, q; cin >> n >> q;

    vi v(n);
    for( int &x : v ) cin >> x;

    vi c = v;
    sort(all(c));
    c.erase(unique(all(c)), c.end());

    PersistentSegtree seg( n, 0, sz(c) - 1 );
    for( int &x : v ){
        int id = lower_bound(all(c), x) - c.begin();
        seg.update( id, x );
    }

    while( q-- ){
        int l, r; cin >> l >> r; l--; r--;

        ll ans = 0;
        for( ll new_ans = seg.query(l, r, 1, ans + 1);
             new_ans != ans; new_ans = seg.query(l, r, 1,
             ans + 1) )
            ans = new_ans;

        cout << ans + 1 << '\n';
    }
}
```

5.11 Movie Festival Queries

In a movie festival, n movies will be shown. You know the starting and ending time of each movie. Your task is to process q queries of the form: if you arrive and leave the festival at specific times, what is the maximum number of movies you can watch? You can watch two movies if the first movie ends before or exactly when the second movie starts. You can start the first movie exactly when you arrive and leave exactly when the last movie ends.

Input
The first input line has two integers n and q: the number of movies and queries. After this, there are n lines describing the movies. Each line has two integers a and b: the starting and ending time of a movie. Finally, there are q lines describing the queries. Each line has two integers a and b: your arrival and leaving time.

Output
Print the maximum number of movies for each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a < b \le 10^6$

```
const int maxn = 2e5 + 10;
const int logn = 20;
const int inf = 1e9;

int pai[logn][maxn], ans[maxn];

struct intervalo{

    int l, r, id;
    intervalo( int l, int r ) : l(l), r(r) {}

    bool operator < ( intervalo i ){
        if( r == i.r ) return l < i.l;
        return r < i.r;
    }
};

struct query{
    int l, r, id, ub = -1;
    query( int l, int r, int id ) : l(l), r(r), id(id) {}
    bool operator < ( query q ){ return l < q.l; }
};

vector<intervalo> I;
queue<intervalo> fila;
vector<query> queries;

int lca( int a, int x ){

    int resp = 0;
    for( int k = logn - 1; k >= 0; k-- ){
        if( I[ pai[k][a] ].r <= x ){
            resp += ( 1<<k );
            a = pai[k][a];
        }
    }

    return resp + 1;
}

int main(){

    int n, q; cin >> n >> q;

    for( int i = 0; i < n; i++ ){

        int l, r; cin >> l >> r;
        I.push_back( intervalo( l, r ) );
    }
    sort( I.begin(), I.end() );
    I.push_back( intervalo( inf, inf ) );

    for( int i = 1; i <= q; i++ ){

        int l, r; cin >> l >> r;
        queries.push_back( query( l, r, i ) );
    }
    sort( queries.begin(), queries.end() );

    int cont = 0, p = 0;
    for( intervalo& cur : I ){
        cur.id = cont++;

        while( p < queries.size() && queries[p].l <= cur.l ){
            queries[p].ub = cur.id;
            p++;
        }

        if( !fila.empty() ){
            while( !fila.empty() && fila.front().r <= cur.l )
                {
                pai[0][ fila.front().id ] = cur.id;
                fila.pop();
            }
        }
        fila.push(cur);
    }
    pai[0][n] = n;
    for( int k = 1; k < logn; k++ )
        for( int i = 0; i <= n; i++ ) pai[k][i] = pai[k - 1][
            pai[k - 1][i] ];

    for( query cur : queries ){

        if( I[cur.ub].r > cur.r ){
            ans[ cur.id ] = 0;
            continue;
        }
        ans[cur.id] = lca( cur.ub, cur.r );
    }

    for( int i = 1; i <= q; i++ ) cout << ans[i] << endl;
}
```

5.12 Pizzeria Queries

There are n buildings on a street, numbered $1, 2, \ldots, n$. Each building has a pizzeria and an apartment. The pizza price in building k is p_k. If you order a pizza from building a to building b, its price (with delivery) is $p_a + |a - b|$. Your task is to process two types of queries:
The pizza price p_k in building k becomes x. You are in building k and want to order a pizza. What is the minimum price?

Input
The first input line has two integers n and q: the number of buildings and queries. The second line has n integers $p_1, p_2, \ldots, p_n$: the initial pizza price in each building. Finally, there are q lines that describe the queries. Each line is either "1 k x" or "2 k".

Output
Print the answer for each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le p_i, x \le 10^9$ • $1 \le k \le n$

```
const int MAX = 2e5;
const int INF = 2e9;
int v[MAX], n, q;

namespace seg {
    pair<ll, ll> seg[4*MAX];

    pair<ll, ll> build(int p=1, int l=0, int r=n-1) {
        if (l == r) return seg[p] = {v[l] - l, v[l] + l};
        int m = (l+r)/2;
```

```
        auto esq = build(2*p, l, m), dir = build(2*p+1, m+1,
            r);
        return seg[p] = {min(esq.first, dir.first), min(esq.
            second, dir.second)};
    }
    pair<ll, ll> query(int a, int b, int p=1, int l=0, int r=
        n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return {INF, INF};
        int m = (l+r)/2;
        auto esq = query(a, b, 2*p, l, m), dir = query(a, b,
            2*p+1, m+1, r);
        return {min(esq.first, dir.first), min(esq.second,
            dir.second)};
    }
    void update(int a, int x, int p=1, int l=0, int r=n-1) {
        if (l == r) {
            seg[p] = {x - l, x + l};
        }
        else {
            int m = (l+r)/2;
            if (a <= m)
                update(a, x, p*2, l, m);
            else
                update(a, x, p*2+1, m+1, r);
            auto esq = seg[p*2], dir = seg[p*2+1];
            seg[p] = {min(esq.first, dir.first), min(esq.
                second, dir.second)};
        }
    }
};

void solve(){
    cin >> n >> q;

    for(int i = 0; i < n; i++) {
        cin >> v[i];
    }

    seg::build();

    for(int i = 0; i < q; i++) {
        int qi; cin >> qi;
        if(qi == 1) {
            int k, x; cin >> k >> x; k--;
            seg::update(k, x);
        }
        else {
            int k; cin >> k; k--;
            int esq = seg::query(0, k).first + k;
            int dir = seg::query(k, n - 1).second - k;
            cout << min(esq, dir) << '\n';
        }
    }
}
```

5.13 Polynomial Queries

Your task is to maintain an array of n values and efficiently process the following types of queries:
Increase the first value in range $[a, b]$ by 1, the second value by 2, the third value by 3, and so on. Calculate the sum of values in range $[a, b]$.

Input
The first input line has two integers n and q: the size of the array and the number of queries. The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array. Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output
Print the answer to each sum query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le t_i \le 10^6$ • $1 \le a \le b \le n$

```
const int MAX = 2e5 + 20;
const ll LINF = 4e18;

namespace seg {
    ll seg[4*MAX];
    pair<ll, ll> lazy[4*MAX];
    ll n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = {0, 0};
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = build(2*p, l, m) + build(2*p+1, m+1,
            r);
    }
    void build(int n2, ll* v2) {
        n = n2, v = v2;
        build();
    }
    void prop(int p, int l, int r) {
        ll ini = lazy[p].first, pulo = lazy[p].second, len =
            r - l + 1;
        seg[p] += ini * len;
        seg[p] += pulo * (len - 1) * len / 2;
        if (l != r) {
            lazy[2*p].first += ini;
            lazy[2*p].second += pulo;

            int len_esq = (r - l) / 2 + 1;
            lazy[2*p + 1].first += ini + pulo * len_esq;
            lazy[2*p + 1].second += pulo;
        }
        lazy[p] = {0, 0};
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return 0;
        int m = (l+r)/2;
        return query(a, b, 2*p, l, m) + query(a, b, 2*p+1, m
            +1, r);
    }
    ll update(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) {
            lazy[p].first += (l - a + 1);
            lazy[p].second += 1;
            prop(p, l, r);
            return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = update(a, b, 2*p, l, m) + update(a, b
            , 2*p+1, m+1, r);
    }
};

void solve() {
    int n, q; cin >> n >> q;
    ll t[n + 1]; t[0] = 0;
    for(int i = 1; i <= n; i++) {
        cin >> t[i];
    }

    seg::build(n + 1, t);

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        int a, b; cin >> a >> b;

        if(c == 1) {
            seg::update(a, b);
        }
        else {
            cout << seg::query(a, b) << '\n';
        }

    }
}
```

5.14 Prefix Sum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the maximum prefix sum in range $[a, b]$?
Empty prefixes (with sum 0) are allowed.

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $-10^9 \le x_i, u \le 10^9$ • $1 \le k \le n$ • $1 \le a \le b \le n$

```
const int MAX = 2e5+20;
const ll LINF = 4e18;

namespace seg {
    ll seg[4*MAX], lazy[4*MAX];
    ll n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = 0;
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = max(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    void build(int n2, ll* v2) {
        n = n2, v = v2;
        build();
    }
    void prop(int p, int l, int r) {
```

```
        seg[p] += lazy[p];
        if (l != r) lazy[2*p] += lazy[p], lazy[2*p+1] += lazy
            [p];
        lazy[p] = 0;
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return -LINF;
        int m = (l+r)/2;
        return max(query(a, b, 2*p, l, m), query(a, b, 2*p+1,
            m+1, r));
    }
    ll update(int a, int b, int x, int p=1, int l=0, int r=n
        -1) {
        prop(p, l, r);
        if (a <= l and r <= b) {
            lazy[p] += x;
            prop(p, l, r);
            return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = max(update(a, b, x, 2*p, l, m),
            update(a, b, x, 2*p+1, m+1, r));
    }
};

void solve() {
    int n, q; cin >> n >> q;
    ll x[n + 1], pref[n + 1]; x[0] = 0; pref[0] = 0;
    for(int i = 1; i <= n; i++) {
        cin >> x[i]; pref[i] = pref[i - 1] + x[i];
    }

    seg::build(n + 1, pref);

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        int a, b; cin >> a >> b;

        if(c == 1) {
            seg::update(a, n, b - x[a]);
            x[a] = b;
        }
        else {
            ll ans = seg::query(a, b) - seg::query(a - 1, a -
                1);
            cout << max(ans, 0LL) << '\n';
        }

    }

}
```

5.15 Range Interval Queries

Given an array x of n integers, your task is to process q queries of the form: how many integers i satisfy $a \leq i \leq b$ and $c \leq x_i \leq d$?

Input
The first line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has four integers a, b, c and d: how many integers i satisfy $a \leq i \leq b$ and $c \leq x_i \leq d$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$ • $1 \leq c \leq d \leq 10^9$

```
struct Evento{
  int val, pos, id; Evento( int val, int pos, int id ) : val(
      val), pos(pos), id(id) {}
  bool operator < ( Evento e ){
    return (( val == e.val ) ? abs(id) < abs(e.id) : val < e.
        val );
  }
};
vector<Evento> line;

int main(){
  int n, q; cin >> n >> q;
  for( int i = 1; i <= n; i++ ){
    int x; cin >> x;
    line.push_back( Evento( x, i, 0 ) );
  }

  for( int i = 1; i <= q; i++ ){
    int l, r, mini, maxi; cin >> l >> r >> mini >> maxi;
    line.push_back( Evento( mini - 1, l - 1, i ) );
    line.push_back( Evento( mini - 1, r, -i ) );
    line.push_back( Evento( maxi, l - 1, -i ) );
    line.push_back( Evento( maxi, r, i ) );
  }

  vector<int> resp(q + 1);

  sort( line.begin(), line.end() );

  vector<int> bit( n + 1 );
  auto update = [&]( int id, int val ){
    for( int i = id; i < bit.size(); i += i&-i ) bit[i] +=
        val;
  };

  auto query = [&]( int id ){
    int sum = 0;
    for( int i = id; i > 0; i -= i&-i ) sum += bit[i];
    return sum;
  };

  for( auto evento : line ){
    if( evento.id == 0 ) update( evento.pos, 1 );
    else if( evento.id < 0 ) resp[-evento.id] -= query(
        evento.pos );
    else resp[evento.id] += query( evento.pos );
  }

  for( int i = 1; i <= q; i++ ) cout << resp[i] << endl;
}
```

5.16 Range Queries And Copies

Your task is to maintain a list of arrays which initially has a single array. You have to process the following types of queries: Set the value a in array k to x. Calculate the sum of values in range $[a, b]$ in array k. Create a copy of array k and add it to the end of the list.

Input
The first input line has two integers n and q: the array size and the number of queries. The next line has n integers $t_1, t_2, \ldots, t_n$: the initial contents of the array. Finally, there are q lines describing the queries. The format of each line is one of the following: "1 k a x", "2 k a b" or "3 k".

Output
Print the answer to each sum query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq t_i, x \leq 10^9$ • $1 \leq a \leq b \leq n$

```
using ll = long long;

class PersistentSegmentTree{
private:
  ll min_id, max_id;

  struct Node{
    ll val; int l, r;
    Node() : val(0), l(0), r(0) {}
  };
  vector<Node> seg;

  int create(){
    seg.emplace_back();
    return seg.size() - 1;
  }

  int l( int pos ){
    if( seg[pos].l == 0 ){ int aux = create(); seg[pos].l =
        aux; }
    return seg[pos].l;
  }

  int r( int pos ){
    if( seg[pos].r == 0 ){ int aux = create(); seg[pos].r =
        aux; }
    return seg[pos].r;
  }

  int update( int pos, ll ini, ll fim, ll id, ll val ){
    int novo = create();
    if( ini == fim ){ seg[novo].val = val; return novo; }

    l(pos); r(pos);
    ll mid = (ini + fim)>>1;
    if( id <= mid ){
      seg[novo].l = update( l(pos), ini, mid, id, val );
      seg[novo].r = seg[pos].r;
    }
    else{
      seg[novo].r = update( r(pos), mid + 1, fim, id, val );
      seg[novo].l = seg[pos].l;
    }

    seg[novo].val = seg[seg[novo].l].val + seg[seg[novo].r].
        val;
    return novo;
  }

  ll query( int pos, ll ini, ll fim, ll ki, ll kf ){
    if( pos == 0 || ki > fim || ini > kf ) return 0;
    if( ki <= ini && fim <= kf ) return seg[pos].val;
```

```
    ll mid = (ini + fim)>>1;
    return query( seg[pos].l, ini, mid, ki, kf ) + query( seg
        [pos].r, mid + 1, fim, ki, kf );
  }
public:
  PersistentSegmentTree( ll min_id, ll max_id ) : min_id(
      min_id), max_id(max_id){ create(); create(); }

  int update( int root, ll id, ll val ){ return update( root,
       min_id, max_id, id, val ); }
  ll query( int root, ll l, ll r ){ return query( root,
      min_id, max_id, l, r ); }
};

int main(){
    int n, q; cin >> n >> q;
    PersistentSegmentTree seg( 0, n - 1 );
    vector<int> root(1, 1);

    for( int i = 0; i < n; i++ ){
        int x; cin >> x;
        root[0] = seg.update( root[0], i, x );
    }

    while( q-- ){
        int t; cin >> t;
        if( t == 1 ){
            int ver, id, x; cin >> ver >> id >> x;
            ver--; id--;
            root[ver] = seg.update( root[ver], id, x );
        }
        if( t == 2 ){
            int ver, l, r; cin >> ver >> l >> r;
            ver--; l--; r--;
            cout << seg.query( root[ver], l, r ) << endl;
        }
        if( t == 3 ){
            int ver; cin >> ver;
            ver--;
            root.push_back(root[ver]);
        }
    }

}
```

5.17 Range Update Queries

Given an array of n integers, your task is to process q queries of the following types:
increase each value in range $[a, b]$ by u what is the value at position k?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 a b u" or "2 k".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    FT bit(n + 2);

    for(int i = 1; i <= n; i++) {
        int a; cin >> a;
        bit.update(i, a);
        bit.update(i + 1, -a);
    }

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        if(c == 1) {
            int a, b, u; cin >> a >> b >> u;
            bit.update(a, u);
            bit.update(b + 1, -u);
        }
        else {
            int k; cin >> k;
            cout << bit.query(k + 1) << '\n';
        }
    }
}
```

5.18 Range Updates And Sums

Your task is to maintain an array of n values and efficiently process the following types of queries:
Increase each value in range $[a, b]$ by x. Set each value in range $[a, b]$ to x. Calculate the sum of values in range $[a, b]$.

Input
The first input line has two integers n and q: the array size and the number of queries. The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array. Finally, there are q lines describing the queries. The format of each line is one of the following: "1 a b x", "2 a b x", or "3 a b".

Output
Print the answer to each sum query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq t_i, x \leq 10^6$ • $1 \leq a \leq b \leq n$

```
const int maxn = 2e5 + 10;
ll v[maxn];

struct node{
    ll sum = 0, lzs = 0, lzi = 0; node( ll sum = 0 ) : sum(
        sum) {}
    node operator + ( node n ){ return node( sum + n.sum ); }
} seg[4*maxn];

void build( int pos, int ini, int fim ){
    if( ini == fim ){ seg[pos] = node( v[ini] ); return; }
    int mid = ( ini + fim )/2;
    int l = 2*pos, r = 2*pos + 1;
    build( l, ini, mid ); build( r, mid + 1, fim );
    seg[pos] = seg[l] + seg[r];
}

void refresh( int pos, ll ini, ll fim ){
    if( !seg[pos].lzi && !seg[pos].lzs ) return;
    ll s = seg[pos].lzs; seg[pos].lzs = 0;
    ll i = seg[pos].lzi; seg[pos].lzi = 0;
    if( s ) seg[pos] = ( fim - ini + 1 )*s;
    seg[pos].sum += ( fim - ini + 1 )*i ;
    if( ini == fim ) return;
    int l = 2*pos, r = 2*pos + 1;
    if( s ){
        seg[l].lzi = 0; seg[l].lzs = s;
        seg[r].lzi = 0; seg[r].lzs = s;
    }
    seg[l].lzi += i;
    seg[r].lzi += i;
}

void update( int pos, int ini, int fim, int ki, int kf, ll
    val, int t ){
    refresh( pos, ini, fim );
    if( ki > fim || ini > kf ) return;
    if( ki <= ini && fim <= kf ){
        if( t == 1 ) seg[pos].lzi += val;
        else seg[pos].lzi = 0, seg[pos].lzs = val;
        refresh( pos, ini, fim ); return;
    }
    int mid = ( ini + fim )/2;
    int l = 2*pos, r = 2*pos + 1;
    update( l, ini, mid, ki, kf, val, t );
    update( r, mid + 1, fim, ki, kf , val, t );
    seg[pos] = seg[l] + seg[r];
}

ll query( int pos, int ini, int fim, int ki, int kf ){
    refresh( pos, ini, fim );
    if( ki > fim || ini > kf ) return 0;
    if( ki <= ini && fim <= kf ) return seg[pos].sum;
    int mid = ( ini + fim )/2;
    int l = 2*pos, r = 2*pos + 1;
    return query( l, ini, mid, ki, kf ) + query( r, mid + 1,
        fim, ki, kf );
}

int main(){
    int n, m; cin >> n >> m;
    for( int i = 1; i <= n; i++ ) cin >> v[i];
    build( 1, 1, n );
    while( m-- ){
        int t; cin >> t;
        if( t != 3 ){
            ll l, r, val; cin >> l >> r >> val;
            update( 1, 1, n, l, r, val, t );
        }
        else{
            ll l, r; cin >> l >> r;
            cout << query( 1, 1, n, l, r ) << endl;
        }
    }
}
```

5.19 Range Xor Queries

Given an array of n integers, your task is to process q queries of the form: what is the xor sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the xor sum of values in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```cpp
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] ^= dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res ^= s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n);
    FT bit(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
        bit.update(i, v[i]);
    }

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b; a--; b--;
        cout << (bit.query(b + 1) ^ bit.query(a)) << '\n';
    }
}
```

5.20 Salary Queries

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Input
The first input line contains two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$. The next line has n integers $p_1, p_2, \ldots, p_n$: each employee's salary. After this, there are q lines describing the queries. Each line has one of the following forms:
• ! k x: change the salary of employee k to x • ? a b: count the number of employees whose salary is between $a \ldots b$

Output
Print the answer to each ? query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq p_i \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq x \leq 10^9$
• $1 \leq a \leq b \leq 10^9$

```cpp
const int MAX = 6e5+20;

struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    ll query(int a, int b) {
        return query(b + 1) - query(a);
    }
};

void solve() {
    int n, q; cin >> n >> q;
    vector<int> p(n);
    vector<tuple<int, int, int>> queries(q);
    for(int i = 0; i < n; i++) cin >> p[i];

    for(int i = 0; i < q; i++) {
        char c; cin >> c;
        int a, b; cin >> a >> b;
        queries[i] = {c, a, b};
    }

    vector<int> all_salaries;
    for(auto x : p) all_salaries.push_back(x);
    for(auto [c, a, b] : queries) {
        all_salaries.push_back(b);
        if(c == '?') all_salaries.push_back(a);
    }

    sort(all_salaries.begin(), all_salaries.end());
    all_salaries.erase(unique(all_salaries.begin(),
         all_salaries.end()), all_salaries.end());

    for(int i = 0; i < n; i++) p[i] = lower_bound(all(
         all_salaries), p[i]) - all_salaries.begin();
    for(int i = 0; i < q; i++) {
        get<2>(queries[i]) = lower_bound(all(all_salaries),
             get<2>(queries[i])) - all_salaries.begin();
        if(get<0>(queries[i]) == '?')
            get<1>(queries[i]) = lower_bound(all(all_salaries
                 ), get<1>(queries[i])) - all_salaries.begin
                 ();
    }

    FT bit(MAX);
    for(auto x : p) bit.update(x, 1);

    for(auto [c, a, b] : queries) {
        if(c == '!') {
            bit.update(b, 1);
            bit.update(p[a - 1], -1);
            p[a - 1] = b;
        }
        else {
            int ans = bit.query(a, b);
```

```
            cout << ans << '\n';
        }
    }
}
```

5.21 Static Range Minimum Queries

Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the minimum value in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```
template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k
             ) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j +
                      pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
    }
    RMQ st(v);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << st.query(a - 1, b) << '\n';
    }
}
```

5.22 Static Range Sum Queries

Given an array of n integers, your task is to process q queries of the form: what is the sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the sum of values in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```
void solve(){
    int n, q; cin >> n >> q;
    vector<ll> v(n + 1), pref(n + 1);
    for(int i = 1; i <= n; i++) {
        cin >> v[i];
        pref[i] = v[i] + pref[i - 1];
    }

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << pref[b] - pref[a - 1] << '\n';
    }
}
```

5.23 Subarray Sum Queries

There is an array consisting of n integers. Some values of the array will be updated, and after each update, your task is to report the maximum subarray sum in the array.

Input
The first input line contains integers n and m: the size of the array and the number of updates. The array is indexed $1, 2, \ldots, n$. The next line has n integers: $x_1, x_2, \ldots, x_n$: the initial contents of the array. Then there are m lines describing the changes. Each line has two integers k and x: the value at position k becomes x.

Output
After each update, print the maximum subarray sum. Empty subarrays (with sum 0) are allowed.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $-10^9 \leq x_i \leq 10^9$ • $1 \leq k \leq n$ • $-10^9 \leq x \leq 10^9$

```
const int MAXN = 2e5 + 5, INF = 2e9, M = 1e9 + 7;

struct node {
    ll best;
    ll bestprefix;
    ll bestsufix;
    ll sum;
};

node t[4*MAXN];

node combine(node a, node b) {
    node c;
    c.sum = a.sum + b.sum;
    c.best = max(max(a.best, b.best), a.bestsufix + b.
         bestprefix);
    c.bestprefix = max(a.bestprefix, a.sum + b.bestprefix);
    c.bestsufix = max(b.bestsufix, b.sum + a.bestsufix);
    return c;
}

void build(ll a[], ll v, ll tl, ll tr) {
    if (tl == tr) {
        t[v].sum = a[tl];
        t[v].best = max(a[tl], 0LL);
        t[v].bestprefix = max(a[tl], 0LL);
        t[v].bestsufix = max(a[tl], 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(ll v, ll tl, ll tr, ll pos, ll new_val) {
    if (tl == tr) {
        t[v].sum = new_val;
        t[v].best = max(new_val, 0LL);
        t[v].bestprefix = max(new_val, 0LL);
        t[v].bestsufix = max(new_val, 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void solve(){
    ll n, m; cin >> n >> m;
    ll x[n + 1];
    x[0] = 0;
    for(ll i = 1; i <= n; i++) cin >> x[i];

    build(x, 1, 0, n);

    for(ll i = 0; i < m; i++)
    {
        ll k, a; cin >> k >> a;
        update(1, 0, n, k, a);
        cout << t[1].best << '\n';
    }

}
```

5.24 Subarray Sum Queries II

You are given an array of n integers and q queries. In each query, your task is to calculate the maximum subarray sum in the range $[a, b]$. Empty subarrays (with sum 0) are allowed.

Input

The first line contains two integers n and q: the number of elements and the number of queries. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Finally there are q lines that describe the queries. Each line has two integers a and b.

Output
Print the answer for each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $-10^9 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```
const int MAXN = 2e5 + 5, INF = 2e9, M = 1e9 + 7;

struct node {
    ll best;
    ll bestprefix;
    ll bestsufix;
    ll sum;
};

node t[4*MAXN];

node combine(node a, node b) {
    node c;
    c.sum = a.sum + b.sum;
    c.best = max(max(a.best, b.best), a.bestsufix + b.
         bestprefix);
    c.bestprefix = max(a.bestprefix, a.sum + b.bestprefix);
    c.bestsufix = max(b.bestsufix, b.sum + a.bestsufix);
    return c;
}

void build(ll a[], ll v, ll tl, ll tr) {
    if (tl == tr) {
        t[v].sum = a[tl];
        t[v].best = max(a[tl], 0LL);
        t[v].bestprefix = max(a[tl], 0LL);
        t[v].bestsufix = max(a[tl], 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(ll v, ll tl, ll tr, ll pos, ll new_val) {
    if (tl == tr) {
        t[v].sum = new_val;
        t[v].best = max(new_val, 0LL);
        t[v].bestprefix = max(new_val, 0LL);
        t[v].bestsufix = max(new_val, 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

node query(int v, int tl, int tr, int l, int r) {
    if (l > r) return {0, 0, 0, 0};
    if (l == tl && r == tr) return t[v];
    int tm = (tl + tr) / 2;
    node left  = query(v*2, tl, tm, l, min(r, tm));
    node right = query(v*2+1, tm+1, tr, max(l, tm+1), r);
    return combine(left, right);
}

void solve(){
    ll n, m; cin >> n >> m;
    ll x[n + 1];
    x[0] = 0;
    for(ll i = 1; i <= n; i++) cin >> x[i];

    build(x, 1, 0, n);

    for(ll i = 0; i < m; i++)
    {
        ll a, b; cin >> a >> b;
        cout << query(1, 0, n, a, b).best << '\n';
    }
}
```

5.25 Visible Buildings Queries

There are n buildings in a row numbered $1, 2, \ldots, n$ from left to right. You are standing to the left of the first building. You can see a building if it is taller than all buildings to its left. Your task is to process q queries: If only buildings in range $[a, b]$ existed, how many buildings would you see?

Input
The first line has two integers n and q: the number of buildings and queries. The second line has n integers $h_1, h_2, \ldots, h_n$: the heights of the buildings. Finally, there are q lines describing the queries. Each line has two integers a and b.

Output
For each query, print one integer: the number of visible buildings.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq q \leq 2 \cdot 10^5$ • $1 \leq h_i \leq 10^9$ • $1 \leq a \leq b \leq n$

```
const int MAX = 1e5, INF = 2e9;
int n, q;
vector<int> next_h, h;

void next_higher() {
    next_h.resize(n, n);
    stack<int> st;

    for (int i = n - 1; i >= 0; --i) {
        while (!st.empty() && h[st.top()] <= h[i])
            st.pop();
        if (!st.empty())
            next_h[i] = st.top();
        st.push(i);
    }
}

namespace seg {
    // (answer, pos of the biggest)
    pair<int, int> seg[4*MAX];

    pair<int, int> query(int a, int b, int p=1, int l=0, int
         r=n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return {0, -1};
        int m = (l+r)/2;

        auto [esq_ans, esq_big] = query(a, b, 2*p, l, m);
        if (esq_big == -1) {
            return query(a, b, 2*p+1, m+1, r);
        }
        int nxt = next_h[esq_big];
        if(nxt > b || nxt > r) return {esq_ans, esq_big};

        auto [dir_ans, dir_big] = query(nxt, b, 2*p+1, m+1, r
             );
        return {dir_ans + esq_ans, dir_big};
    }

    pair<int, int> build(int p=1, int l=0, int r=n-1) {
        if (l == r) return seg[p] = {1, l};
        int m = (l+r)/2;

        auto [esq_ans, esq_big] = build(2*p, l, m); build(2*p
             +1, m+1, r);

        int nxt = next_h[esq_big];
        if(nxt > r) return seg[p] = {esq_ans, esq_big};

        auto [dir_ans, dir_big] = query(nxt, r, 2*p+1, m+1, r
             );
        return seg[p] = {dir_ans + esq_ans, dir_big};
    }
};

void solve(){
    cin >> n >> q;
    h.resize(n);
    for(int i = 0; i < n; i++) cin >> h[i];
    next_higher();

    seg::build();

    for(int i = 0; i < q; i++)
    {
        int l, r; cin >> l >> r; l--; r--;
        cout << seg::query(l, r).first << '\n';
    }
}
```

6 Tree Algorithms

6.1 Company Queries I

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x's boss k levels higher up in the hierarchy?

Input
The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \ldots, e_n$: for each employee $2, 3, \ldots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k: who is employee x's boss k levels higher up?

Output
Print the answer for each query. If such a boss does not exist, print -1.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le e_i \le i-1$ • $1 \le x \le n$ • $1 \le k \le n$

```cpp
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int x, k; cin >> x >> k;
        int at = 0;
        while(k) {
            if(k % 2 == 1) {
                x = up[x][at];
            }
            k >>= 1;
            at++;
        }
        if(x == 0) cout << -1 << '\n';
        else cout << x << '\n';
    }
}
```

6.2 Company Queries II

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is the lowest common boss of employees a and b in the hierarchy?

Input
The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \ldots, e_n$: for each employee $2, 3, \ldots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers a and b: who is the lowest common boss of employees a and b?

Output
Print the answer for each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le e_i \le i-1$ • $1 \le a, b \le n$

```cpp
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << get_lca(a, b) << '\n';
    }
}
```

6.3 Counting Paths

You are given a tree consisting of n nodes, and m paths in the tree. Your task is to calculate for each node the number of paths containing that node.

Input
The first input line contains integers n and m: the number of nodes and paths. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are m lines describing the paths. Each line contains two integers a and b: there is a path between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the number of paths containing that node.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int MAX = 2e5 + 5;
vector<vi> G(MAX);
int ans[MAX];

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k
            ) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
```

```
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j +
                    pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

struct LCA {
    int T = 0;
    vi time, path, ret, pai;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), pai(sz(C)), rmq((dfs(C
        ,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        pai[v] = par;
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
};

void dfs(int v, int p) {
    for(auto x : G[v]) {
        if(x == p) continue;
        dfs(x, v);
        ans[v] += ans[x];
    }
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    LCA lca(G);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        int c = lca.lca(a, b);

        if(c == b) swap(a, b);
        if(c == a) {
            ans[b]++;
        }
        else {
            ans[a]++; ans[b]++; ans[c]--;
        }

        if(lca.pai[c] >= 0) ans[lca.pai[c]]--;
    }

    dfs(0, -1);

    for(int i = 0; i < n; i++) cout << ans[i] << ' ';
    cout << '\n';

}
```

6.4 Distance Queries

You are given a tree consisting of n nodes. Your task is to process q queries of the form: what is the distance between nodes a and b?

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integer a and b: what is the distance between nodes a and b?

Output
Print q integers: the answer to each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}
```

```
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << dist(a, b) << '\n';
    }
}
```

6.5 Distinct Colors

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a color. Your task is to determine for each node the number of distinct colors in the subtree of the node.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. The next line consists of n integers $c_1, c_2, \ldots, c_n$: the color of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the number of distinct colors.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c_i \le 10^9$

```
const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
pair<pii, int> query[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve_DistinctValuesQueries(int n, int q) {
    vector<int> is_first(n), next_eq(n);
    map<int, int> M;

```

```
    for(int i = n - 1; i >= 0; i--) {
        auto it = M.find(v[i]);
        if(it == M.end()) {
            is_first[i] = 1;
            next_eq[i] = n;
        }
        else {
            is_first[i] = 1;
            is_first[it -> second] = 0;
            next_eq[i] = it -> second;
        }
        M[v[i]] = i;
    }

    sort(query, query + q);
    vector<int> ans(q);

    FT bit(n);
    for(int i = 0; i < n; i++) if(is_first[i]) bit.update(i,
         1);

    int at = 0;
    for(int i = 0; i < q; i++) {
        int a = query[i].F.F, b = query[i].F.S;
        for(; at < a; at++) if(next_eq[at] <= n - 1) bit.
             update(next_eq[at], 1);
        ans[query[i].S] = bit.query(b + 1) - bit.query(a);
    }

    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

void dfs(int s, int p) {
    at++;
    query[s].S = s;
    query[s].F.F = at; v[at] = s;
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
    at++;
    query[s].F.S = at; v[at] = s;
}

void solve(){
    int n; cin >> n;
    vector<int> c(n);
    for(int i = 0; i < n; i++) cin >> c[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1);

    for(int i = 0; i < 2 * n; i++) v[i] = c[v[i]];

    solve_DistinctValuesQueries(2 * n, n);
}
```

6.6 Finding A Centroid

Given a tree of n nodes, your task is to find a centroid, i.e., a node such that when it is appointed the root of the tree, each subtree has at most $\lfloor n/2 \rfloor$ nodes.

Input

The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: a centroid node. If there are several possibilities, you can choose any of them.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

```
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, subsize[MAX];
vector<int> g[MAX];

void dfs(int k, int p=-1) {
    subsize[k] = 1;
    for (int i : g[k]) if (i != p) {
        dfs(i, k);
        subsize[k] += subsize[i];
    }
}

int centroid(int k, int p=-1, int size=-1) {
    if (size == -1) size = subsize[k];
    for (int i : g[k]) if (i != p) if (subsize[i] > size/2)
        return centroid(i, k, size);
    return k;
}

pair<int, int> centroids(int k=0) {
    dfs(k);
    int i = centroid(k), i2 = i;
    for (int j : g[i]) if (2*subsize[j] == subsize[k]) i2 = j
        ;
    return {i, i2};
}

void solve(){
    cin >> n;
    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    auto ans = centroids();
    cout << ans.first + 1 << '\n';
}
```

6.7 Fixed-Length Paths I

Given a tree of n nodes, your task is to count the number of distinct paths that consist of exactly k edges.

Input

The first input line contains two integers n and k: the number of nodes and the path length. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output

Print one integer: the number of paths.

Constraints

- $1 \le k \le n \le 2 \cdot 10^5$
- $1 \le a, b \le n$

```
const int maxn = 200010;
vector<int> adj[maxn];
int sub[maxn];
bool removed[maxn];
ll resp;
int f[maxn];
int n, k;
vector<int> freq;

void dfsInit( int cur, int pai ){
    sub[cur] = 1;
    for( int i = 0; i < adj[cur].size(); i++ ){
        int viz = adj[cur][i];
        if( viz == pai ) continue;
        if( removed[viz] ) continue;
        dfsInit( viz, cur );
        sub[cur] += sub[viz];
    }
}

int findCentroid( int cur, int pai, int size ){
    for( int i = 0; i < adj[cur].size(); i++ ){
        int viz = adj[cur][i];
        if( viz == pai ) continue;
        if( removed[viz] ) continue;
        if( sub[viz] > size/2 ) return findCentroid( viz, cur
             , size );
    }
    return cur;
}

void dfs( int cur, int pai, int nivel ){
    freq.pb(nivel);
    if( nivel <= k ) resp += f[k - nivel];
    for( int i = 0; i < adj[cur].size(); i++ ){
        int viz = adj[cur][i];
        if( viz == pai ) continue;
        if( removed[viz] ) continue;
        dfs( viz, cur, nivel + 1 );
    }
}

void decompose( int node ){
    dfsInit( node, node );
    int centroid = findCentroid( node, node, sub[node] );
    // conquer

    f[0]++;
    for( int i = 0; i < adj[centroid].size(); i++ ){
        int viz = adj[centroid][i];
        if( removed[viz] ) continue;
        dfs( viz, centroid, 1 );
        for( int i = 0; i < freq.size(); i++ ) f[ freq[i]
             ]++;
        freq.clear();
    }

    for( int i = 0; i <= sub[node]; i++ ) f[i] = 0;

```

```
    removed[centroid] = true;

    // divide
    for( int i = 0; i < adj[centroid].size(); i++ ){
        int viz = adj[centroid][i];
        if( removed[viz] ) continue;
        decompose( viz );
    }
}

int main(){
    scanf("%d %d", &n, &k );
    for( int i = 1; i < n; i++ ){
        int a, b; scanf("%d %d", &a, &b );
        adj[a].pb(b);
        adj[b].pb(a);
    }
    decompose( 1 );
    printf("%lld", resp );
}
```

6.8 Fixed-Length Paths II

Given a tree of n nodes, your task is to count the number of distinct paths that have at least k_1 and at most k_2 edges.

Input
The first input line contains three integers n, k_1 and k_2: the number of nodes and the path lengths. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the number of paths.

Constraints
• $1 \le k_1 \le k_2 \le n \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int maxn = 2e5 + 10;
vector<ll> adj[maxn], v[maxn], lazy( maxn );

ll dfs( int cur, int pai, int k1, int k2 ){
    ll resp = 0;
    for( ll& viz : adj[cur] ) if( viz != pai ){
        resp += dfs( viz, cur, k1, k2 );
        if( adj[cur][0] == pai || (int)v[viz].size() > (int)v
            [adj[cur][0]].size() ) swap( adj[cur][0], viz )
            ;
    }

    if( adj[cur].empty() || adj[cur][0] == pai ){
        v[cur].push_back(-lazy[cur]); lazy[cur]++;
        return 0;
    }
    swap( v[adj[cur][0]], v[cur] ); swap( lazy[cur], lazy[adj
        [cur][0]] );

    v[cur].push_back(-lazy[cur]);
    int t = (int)v[cur].size();
    int p2 = max( t - k2 - 1, 0 ), p1 = max( t - k1, 0 );
    resp += ( v[cur][p2] - v[cur][p1] );

    lazy[cur]++;
    v[cur].push_back(-lazy[cur]);
    t++;

    for( int viz : adj[cur] ) if( viz != adj[cur][0] && viz
        != pai ){
        v[viz].push_back(-lazy[viz]);
        reverse( v[viz].begin(), v[viz].end() );
        for( int dist = 1; dist < (int)v[viz].size(); dist++
            ){
            ll qtd = v[viz][dist] - v[viz][dist - 1];

            int p2 = max( t - (k2 - dist) - 2, 0 ), p1 = max(
                t - (k1 - dist) - 1, 0 );
            p2 = min( p2, t - 1 ); p1 = min( p1, t - 1 );

            resp += qtd*( v[cur][p2] - v[cur][p1]);
        }

        lazy[cur] += v[viz].back() + lazy[viz];
        v[cur].back() = -lazy[cur];
        for( int dist = 0; dist < (int)v[viz].size(); dist++
            ) v[cur][t - dist - 2] += v[viz][dist] - v[viz
            ].back();
    }

    v[cur].pop_back();

    return resp;
}

int main(){
    int n, k1, k2; cin >> n >> k1 >> k2;
    for( int i = 1; i < n; i++ ){
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    cout << dfs( 1, 1, k1, k2 ) << endl;
}
```

6.9 Path Queries

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a value. Your task is to process following types of queries: change the value of node s to x calculate the sum of values on the path from the root to node s

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 s".

Output
Print the answer to each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b, s \le n$ • $1 \le v_i, x \le 10^9$

```
const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
pii tempo[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void dfs(int s, int p, const vector<int> &t) {
    at++;
    tempo[s].F = at; v[at] = t[s];
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s, t);
    }
    at++;
    tempo[s].S = at; v[at] = -t[s];
}

void solve(){
    int n, q; cin >> n >> q;
    vector<int> t(n);
    for(int i = 0; i < n; i++) cin >> t[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1, t);

    FT bit(2 * n);
    for(int i = 0; i < 2 * n; i++) bit.update(i, v[i]);

    for(int i = 0; i < q; i++) {
        int a; cin >> a;
        if(a == 1) {
            int s, x; cin >> s >> x; s--;
            auto [tin, tout] = tempo[s];
            bit.update(tin, x - v[tin]);
            bit.update(tout, v[tin] - x);
            v[tin] = x; v[tout] = -x;
        }
        else {
            int s; cin >> s; s--;
            auto [tin, tout] = tempo[s];
            cout << bit.query(tout) << '\n';
        }
    }
}
```

6.10 Path Queries II

You are given a tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$. Each node has a value. Your task is to process following types of queries:
change the value of node s to x find the maximum value on the

path between nodes a and b.

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 a b".

Output
Print the answer to each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b, s \le n$ • $1 \le v_i, x \le 10^9$

```cpp
const int MAX = 2e5 + 5;
int v[MAX];
vector<vi> G;

namespace seg {
    int tree[2 * MAX];
    int n;

    void build(int n2) {
        n = n2;
        for(int i = 0; i <= 2 * n; i++) tree[i] = 0;
    }

    void update(int pos, int x) {
        for (tree[pos += n] = x; pos > 1; pos >>= 1)
            tree[pos >> 1] = max(tree[pos], tree[pos ^ 1]);
    }

    int query(int l, int r) {
        int res = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = max(res, tree[l++]);
            if (r & 1) res = max(res, tree[--r]);
        }
        return res;
    }
}

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N),pos(N){ dfsSz(0); dfsHld(0); seg::build(N);}
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (;; v = par[rt[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void updateNode(int u, int val) {
        seg::update(pos[u], val);
    }

    int queryPath(int u, int v_node) {
        int res = 0;
        process(u, v_node, [&](int l, int r) {
            res = max(res, seg::query(l, r));
        });
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    G.resize(n);
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    HLD<false> hld(G);
    for(int i = 0; i < n; i++) {
        hld.updateNode(i, v[i]);
    }

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        if(c == 1) {
            int s, x; cin >> s >> x; s--;
            hld.updateNode(s, x);
            v[s] = x;
        }
        else {
            int a, b; cin >> a >> b; a--; b--;
            cout << hld.queryPath(a, b) << '\n';
        }
    }
}
```

6.11 Subordinates

Given the structure of a company, your task is to calculate for each employee the number of their subordinates.

Input
The first input line has an integer n: the number of employees. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director of the company. After this, there are $n-1$ integers: for each employee $2, 3, \ldots, n$ their direct boss in the company.

Output
Print n integers: for each employee $1, 2, \ldots, n$ the number of their subordinates.

Constraints
• $1 \le n \le 2 \cdot 10^5$

```cpp
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int subtree[MAX];
vi adj[MAX];

void dfs(int s, int p) {
    subtree[s] = 1;
    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
        subtree[s] += subtree[x];
    }
}

void solve(){
    int n; cin >> n;
    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);
    for(int i = 1; i <= n; i++) cout << subtree[i] - 1 << ' ';
    cout << '\n';
}
```

6.12 Subtree Queries

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a value. Your task is to process following types of queries: change the value of node s to x calculate the sum of values in the subtree of node s

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 s".

Output
Print the answer to each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b, s \le n$ • $1 \le v_i, x \le 10^9$

```cpp
const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
```

```
pii tempo[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void dfs(int s, int p, const vector<int> &t) {
    at++;
    tempo[s].F = at; v[at] = t[s];
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s, t);
    }
    at++;
    tempo[s].S = at; v[at] = t[s];
}

void solve(){
    int n, q; cin >> n >> q;
    vector<int> t(n);
    for(int i = 0; i < n; i++) cin >> t[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1, t);

    FT bit(2 * n);
    for(int i = 0; i < 2 * n; i++) bit.update(i, v[i]);

    for(int i = 0; i < q; i++) {
        int a; cin >> a;
        if(a == 1) {
            int s, x; cin >> s >> x; s--;
            auto [tin, tout] = tempo[s];
            bit.update(tin, x - v[tin]);
            bit.update(tout, x - v[tin]);
            v[tin] = x; v[tout] = x;
        }
        else {
            int s; cin >> s; s--;
            auto [tin, tout] = tempo[s];
            cout << (bit.query(tout + 1) - bit.query(tin)) /
                 2 << '\n';
        }
    }
}
```

6.13 Tree Diameter

You are given a tree consisting of n nodes. The diameter of a tree is the maximum distance between two nodes. Your task is to determine the diameter of the tree.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the diameter of the tree.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int dist[MAX];
vi adj[MAX];

void dfs(int s, int p) {
    dist[s] = dist[p] + 1;
    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

void solve(){
    int n; cin >> n;
    for(int i = 1; i < n; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    dist[0] = -1;
    dfs(1, 0);
    int maxi = -1, at = 0;
    for(int i = 1; i <= n; i++) {
        if(maxi < dist[i]) {
            at = i;
            maxi = dist[i];
        }
    }

    dfs(at, 0);
    maxi = -1;
    for(int i = 1; i <= n; i++) {
        if(maxi < dist[i]) maxi = dist[i];
    }

    cout << maxi << '\n';
}
```

6.14 Tree Distances I

You are given a tree consisting of n nodes. Your task is to determine for each node the maximum distance to another node.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the maximum distance to another node.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 2e5 + 5;

int n;
vi G[MAX];
int d[MAX], par[MAX];
int df, c1, c2 = -1;

// queremos contar pra cima e pra baixo
void center(){
    int f;
    function<void(int)> dfs = [&] (int v) {
        if(d[v] > df) f = v, df = d[v];
        for (int u : G[v]) if (u != par[v])
            d[u] = d[v] + 1, par[u] = v, dfs(u);
    };

    f = df = par[0] = -1, d[0] = 0;
    dfs(0);

    int root = f;
    f = df = par[root] = -1, d[root] = 0;
    dfs(root);

    vector<int> c;
    while(f != -1) {
        if(d[f] == df / 2 or d[f] == (df + 1) / 2) c.
            push_back(f);
        f = par[f];
    }

    c1 = c[0];
    if(c.size() > 1) c2 = c[1];
    d[c1] = 0; par[c1] = -1;
    dfs(c1);
}

int check[MAX], ans[MAX];

void dfs(int s, int p) {
    if(s == c2) check[s] = 1;
    for(auto x : G[s]) {
        if(x == p) continue;
        if(check[s]) check[x] = 1;
        dfs(x, s);
    }

    if(check[s]) ans[s] = df / 2 + d[s];
    else ans[s] = (df + 1) / 2 + d[s];
}

void solve(){
    cin >> n;

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }
    center();
    dfs(c1, -1);

    for(int i = 0; i < n; i++) {
        cout << ans[i] << ' ';
    }
```

```
63      cout << '\n';
64  }
```

6.15 Tree Distances II

You are given a tree consisting of n nodes. Your task is to determine for each node the sum of the distances from the node to all other nodes.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the sum of the distances.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```
1  int n;
2  graph G;
3  vi sz, down, up;
4  int ans = 0;
5
6  // queremos contar pra cima e pra baixo
7  void dfs_down(int s, int p){
8      sz[s] = 1;
9      down[s] = 0;
10     for(auto x : G[s]){
11         if(x == p) continue;
12         dfs_down(x, s);
13         down[s] += down[x];
14         sz[s] += sz[x];
15     }
16     down[s] += sz[s] - 1;
17 }
18
19 void dfs_up(int s, int p){
20     if(p != -1)
21         up[s] = up[p] + (down[p] - down[s] - sz[s]) + (n - sz
               [s]);
22
23     for(auto x : G[s]){
24         if(x == p) continue;
25         dfs_up(x, s);
26     }
27 }
28
29 void solve(){
30     cin >> n;
31
32     G.resize(n + 1);
33     sz.resize(n + 1);
34     down.resize(n + 1);
35     up.resize(n + 1);
36
37     for(int i = 0; i < n - 1; i++) {
38         int a, b; cin >> a >> b; a--; b--;
39         G[a].pb(b); G[b].pb(a);
40     }
41
42     dfs_down(0, -1);
43     dfs_up(0, -1);
44
45     for(int i = 0; i < n; i++) {
46         cout << up[i] + down[i] << ' ';
47     }
48     cout << '\n';
49 }
```

6.16 Tree Matching

You are given a tree consisting of n nodes. A matching is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the maximum number of pairs.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```
1  const int MAX = 2e5 + 5;
2  vi G[MAX];
3  int dp[MAX][2];
4
5  void dfs(int s, int p) {
6      for(auto x : G[s]) {
7          if(x == p) continue;
8          dfs(x, s);
9      }
10     for(auto x : G[s]) {
11         if(x == p) continue;
12         dp[s][0] += max(dp[x][0], dp[x][1]);
13     }
14     for(auto x : G[s]) {
15         if(x == p) continue;
16         dp[s][1] = max(dp[s][1], 1 + dp[s][0] - max(dp[x][0],
                dp[x][1]) + dp[x][0]);
17     }
18 }
19
20 void solve(){
21     int n; cin >> n;
22
23     for(int i = 0; i < n - 1; i++) {
24         int a, b; cin >> a >> b; a--; b--;
25         G[a].pb(b); G[b].pb(a);
26     }
27
28     dfs(0, -1);
29
30     cout << max(dp[0][0], dp[0][1]) << '\n';
31 }
```

7 Mathematics

7.1 Another Game

There are n heaps of coins and two players who move alternately. On each move, a player selects some of the nonempty heaps and removes one coin from each heap. The player who removes the last coin wins the game. Your task is to find out who wins if both players play optimally.

Input
The first input line contains an integer t: the number of tests. After this, t test cases are described: The first line contains an integer n: the number of heaps. The next line has n integers $x_1, x_2, \ldots, x_n$: the number of coins in each heap.

Output
For each test case, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints
• $1 \leq t \leq 2 \cdot 10^5$ • $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • the sum of all n is at most $2 \cdot 10^5$

```
void solve() {
    int n; cin >> n;
    int x, cont = 0;
    for(int i = 0; i < n; i++){
        cin >> x;
        if(x % 2 == 1) cont++;
    }

    if(cont > 0) cout << "first\n";
    else cout << "second\n";
}
```

7.2 Binomial Coefficients

Your task is to calculate n binomial coefficients modulo 10^9+7. A binomial coefficient $\binom{a}{b}$ can be calculated using the formula $\frac{a!}{b!(a-b)!}$. We assume that a and b are integers and $0 \leq b \leq a$.

Input
The first input line contains an integer n: the number of calculations. After this, there are n lines, each of which contains two integers a and b.

Output
Print each binomial coefficient modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^5$ • $0 \leq b \leq a \leq 10^6$

```
const int maxn = 1e6+6, mod = 1e9+7;

int fat[maxn], invfat[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void solve() {
    int n; cin >> n;
    int a, b;
    for(int i=0; i<n; i++) {
        cin >> a >> b;
        cout << (((fat[a] * invfat[b]) % mod ) * invfat[a-b]
             ) % mod << '\n';
    }
}

int32_t main() {
    calcfat();
    calcinvfat();
    solve();
    return 0;
}
```

7.3 Bracket Sequences I

Your task is to calculate the number of valid bracket sequences of length n. For example, when $n = 6$, there are 5 sequences:
• ()()() • ()(()) • (())() • ((())) • (()())

Input
The only input line has an integer n.

Output
Print the number of sequences modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

```
const int maxn = 2e6+6, mod = 1e9+7;

int fat[maxn], invfat[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void solve() {
    int n; cin >> n;
    if(n % 2 == 1){
        cout << "0\n";
        return;
    }
    else n = n/2;

    cout << (((fat[2*n] * invfat[n+1]) % mod) * invfat[n]) %
         mod << '\n';
}

int32_t main() {
    calcfat();
    calcinvfat();
    solve();

    return 0;
}
```

7.4 Bracket Sequences II

Your task is to calculate the number of valid bracket sequences of length n when a prefix of the sequence is given.

Input
The first input line has an integer n. The second line has a string of k characters: the prefix of the sequence.

Output
Print the number of sequences modulo $10^9 + 7$.

Constraints
• $1 \leq k \leq n \leq 10^6$

```cpp
const int maxn = 2e6+6, mod = 1e9+7;
int fat[2 * maxn], invfat[2 * maxn], catalan[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void calccatalan(){
    for(int i=0; i<maxn; i++){
        catalan[i] = (((fat[2*i] * invfat[i+1]) % mod) *
             invfat[i]) % mod;
    }
}

void solve() {
    int n; cin >> n;
    if(n % 2 == 1){
        cout << "0\n";
        return;
    }

    string s; cin >> s;
    int k = 0;
    for(auto x : s){
        if(x == '(') k ++;
        else if (x == ')') k --;
        if(k < 0){
            cout << "0\n";
            return;
        }
    }

    n = (n - s.size() - k) / 2;
    int fator = 1, soma = 0;

    for(int i = 0; k-2*i >= 0; i++){
        fator = (((fat[k-i] * invfat[i]) % mod) * invfat[k-2*
             i]) % mod;

        if(i % 2 == 1) fator = -fator;
        fator = (fator + mod) % mod;
        soma = (soma + (fator * catalan[n + k - i]) % mod ) %
              mod;
    }

    cout << soma << '\n';
}

int32_t main() {
    calcfat();
    calcinvfat();
    calccatalan();
    solve();

    return 0;
}
```

7.5 Candy Lottery

There are n children, and each of them independently gets a random integer number of candies between 1 and k. What is the expected maximum number of candies a child gets?

Input
The only input line contains two integers n and k.

Output
Print the expected number rounded to six decimal places (rounding half to even).

Constraints
• $1 \leq n \leq 100$ • $1 \leq k \leq 100$

```cpp
int main() {
    ios::sync_with_stdio(0), cout.tie(0);
    int n, k; cin >> n >> k;
    ld total = 0;

    for(int i = 1; i <= k; i++){
        total += 1 - pow((i-1) / (ld)k, (ld)n);
    }

    cout << setprecision(6) << fixed;
    cout << total << '\n';

    return 0;
}
```

7.6 Christmas Party

There are n children at a Christmas party, and each of them has brought a gift. The idea is that everybody will get a gift brought by someone else. In how many ways can the gifts be distributed?

Input
The only input line has an integer n: the number of children.

Output
Print the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

```cpp
const int maxn = 2e6+6, mod = 1e9+7;

int fat[maxn], invfat[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void solve() {
    int n; cin >> n;
    int total = 0;
    for(int i=0; i<=n; i++){
        if(i % 2 == 0) total += invfat[i];
        else total -= invfat[i];
        total = (total + mod) % mod;
    }

    cout << (total * fat[n]) % mod << '\n';
}

int32_t main() {
    calcfat();
    calcinvfat();
    solve();
    return 0;
}
```

7.7 Common Divisors

You are given an array of n positive integers. Your task is to find two integers such that their greatest common divisor is as large as possible.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the maximum greatest common divisor.

Constraints
• $2 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^6$

```cpp
const int MAXN = 1e6+5;

void solve() {
    int n, x[MAXN];
    for(int i=0; i<MAXN; i++) x[i] = 0;

    cin >> n;
    for(int i = 0; i < n; i++){
        int k; cin >> k;
        x[k] += 1;
    }
    int mdc = 1;

    for(int i = 1; i < MAXN; i++){
        int cont = 0;
        for(int j = i; j < MAXN; j+=i){
            cont += x[j];
            if(cont >= 2) break;
        }
        if(cont >= 2) mdc = i;
    }

    cout << mdc << '\n';
}
```

7.8 Counting Coprime Pairs

Given a list of n positive integers, your task is to count the number of pairs of integers that are coprime (i.e., their greatest common divisor is one).

Input
The first input line has an integer n: the number of elements. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list.

Output
Print one integer: the answer for the task.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq x_i \leq 10^6$

```cpp
const int maxn = 1e6+5, maxlogn = 20, maxpot = 1048576, inf =
      2e9, M = 1e9 + 7;

int mobius[maxn], menorprimo[maxn], freq[maxn];

void calc_menorprimo(){
    for(int i=2; i<maxn; i++){
        if(menorprimo[i] != 0) continue;

        for(int j=i; j<maxn; j+=i){
            if(menorprimo[j] == 0) menorprimo[j] = i;
        }
    }
}

void calc_mobius(){
    for(int i=2; i<maxn; i++){
        if(menorprimo[i] == i){
            mobius[i] = -1;
            continue;
        }

        if(i % (menorprimo[i] * menorprimo[i]) == 0){
            mobius[i] = 0;
            continue;
        }

        mobius[i] = -mobius[i / menorprimo[i]];
    }
}

void solve() {
    ll n; cin >> n;
    ll x, maxX = 0;
    for(int i = 0; i < n; i++){
        cin >> x;
        freq[x] ++;
        maxX = max(x, maxX);
    }

    ll total = n * (n - 1) / 2;
    for(int i = 2; i <= maxX; i++){
        if(mobius[i] == 0) continue;
        ll d = 0;
        for(int j = i; j <= maxX; j+=i){
            d += freq[j];
        }

        total += mobius[i] * d * (d - 1) / 2;
    }

    cout << total << '\n';
}

int main() {
    calc_menorprimo();
    calc_mobius();
    solve();
    return 0;
}
```

7.9 Counting Divisors

Given n integers, your task is to report for each integer the number of its divisors. For example, if $x = 18$, the correct answer is 6 because its divisors are $1, 2, 3, 6, 9, 18$.

Input
The first input line has an integer n: the number of integers.

After this, there are n lines, each containing an integer x.

Output
For each integer, print the number of its divisors.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq x \leq 10^6$

```cpp
const int MAX = 1000005;
int divi[MAX];

void calc(){
    divi[0] = 0;
    for(int i=1; i<=MAX; i++){
        for(int j=i; j<=MAX; j+=i){
            divi[j] ++;
        }
    }
}

void solve(){
    int x; cin >> x;
    cout << divi[x] << '\n';
    return;
}

int main(){
    calc();
    int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

7.10 Counting Grids

Your task is to count the number of different $n \times n$ grids whose each square is black or white. Two grids are considered to be different if it is not possible to rotate one of them so that they look the same.

Input
The only input line has an integer n: the size of the grid.

Output
Print one integer: the number of grids modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^9$

```cpp
const int maxn = 2e6+6, n = 8, m = 64, mod = 1e9+7;

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
```

```
}

int inv(int n){
    return fexp(n, mod-2);
}

void solve() {
    int n; cin >> n;
    int ans = 0;

    if(n % 2 == 0){
        ans += fexp(2, n * n);
        ans += fexp(2, n * n / 4 + 1);
        ans += fexp(2, n * n / 2);
    }
    else{
        ans += fexp(2, n * n);
        ans += fexp(2, (n * n + 3) / 4 + 1);
        ans += fexp(2, (n * n + 1) / 2);
    }

    ans = ans % mod;
    ans = (ans * inv(4)) % mod;

    cout << ans << '\n';
}
```

7.11 Counting Necklaces

Your task is to count the number of different necklaces that consist of n pearls and each pearl has m possible colors. Two necklaces are considered to be different if it is not possible to rotate one of them so that they look the same.

Input
The only input line has two numbers n and m: the number of pearls and colors.

Output
Print one integer: the number of different necklaces modulo $10^9 + 7$.

Constraints
• $1 \le n, m \le 10^6$

```
const int maxn = 2e6+6, n = 8, m = 64, mod = 1e9+7;

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void solve() {
    int n, m; cin >> n >> m;
    int total = 0;
    for(int k=0; k<n; k++)
        total = (total + fexp(m, __gcd(k, n))) % mod;

    total = (total * inv(n)) % mod;

    cout << total << '\n';
}
```

7.12 Creating Strings II

Given a string, your task is to calculate the number of different strings that can be created using its characters.

Input
The only input line has a string of length n. Each character is between a–z.

Output
Print the number of different strings modulo $10^9 + 7$.

Constraints
• $1 \le n \le 10^6$

```
const int maxn = 1e6+6, mod = 1e9+7;

int fat[maxn], invfat[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void solve() {
    string n; cin >> n;
    int x=1, total;
    int a, b;
    sort(all(n));
    total = fat[n.size()];
    for(int i=1; i<n.size(); i++){
        if(n[i]==n[i-1]) x++;
        else{
            total = (total * invfat[x]) % mod;
            x=1;
        }
    }
    total = (total * invfat[x]) % mod;
    cout << total << '\n';
}

int32_t main() {
    calcfat();
    calcinvfat();
    solve();
    return 0;
}
```

7.13 Dice Probability

You throw a dice n times, and every throw produces an outcome between 1 and 6. What is the probability that the sum of outcomes is between a and b?

Input
The only input line contains three integers n, a and b.

Output
Print the probability rounded to six decimal places (rounding half to even).

Constraints
• $1 \le n \le 100$ • $1 \le a \le b \le 6n$

```
vector<double> Prod(vector<double>& a, vector<double>& b) {
    vector<double> c (a.size() + b.size() - 1);
    for(int i = 0; i < c.size(); i++){
        c[i] = 0;
        for(int j = 0; j < a.size(); j++){
            if(i - j >= b.size() || i - j < 0) continue;
            c[i] += a[j] * b[i - j];
        }
    }
    return c;
}

vector<double> Exp(vector<double>& a, int exp){
    if(exp == 0) return {1};
    if(exp == 1) return a;

    vector<double> c = Exp(a, exp / 2);
    c = Prod(c, c);

    if(exp % 2 == 1) return Prod(c, a);
    else return c;
}

int main() {
    ios::sync_with_stdio(0), cout.tie(0);

    int n, a, b; cin >> n >> a >> b;

    double x = (double)1/6;
    vector<double> r = {0, x, x, x, x, x, x}, s = {0, x, x, x
        , x, x, x};
    r = Exp(r, n);

    double parcial = 0;
```

```
    for(int i = a; i <= b; i++) {
        parcial += r[i];
    }

    cout << fixed << setprecision(6);
    cout << parcial << '\n';

    return 0;
}
```

7.14 Distributing Apples

There are n children and m apples that will be distributed to them. Your task is to count the number of ways this can be done. For example, if $n = 3$ and $m = 2$, there are 6 ways: $[0, 0, 2]$, $[0, 1, 1]$, $[0, 2, 0]$, $[1, 0, 1]$, $[1, 1, 0]$ and $[2, 0, 0]$.

Input
The only input line has two integers n and m.

Output
Print the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n, m \leq 10^6$

```
const int maxn = 2e6+6, mod = 1e9+7;

int fat[maxn], invfat[maxn];

void calcfat(){
    fat[0] = 1;
    for(int i=1; i<maxn; i++){
        fat[i] = (fat[i-1] * i) % mod;
    }
}

int fexp(int n, int exp){
    if(exp == 0) return 1;
    if(exp == 1) return n;
    int x = fexp(n, exp / 2);
    x = (x * x) % mod;
    if(exp % 2 == 1){
        return (n * x) % mod;
    }
    return x;
}

int inv(int n){
    return fexp(n, mod-2);
}

void calcinvfat(){
    for(int i=0; i<maxn; i++){
        invfat[i] = inv(fat[i]);
    }
}

void solve() {
    int n, m; cin >> n >> m;
    cout << (((fat[n+m-1] * invfat[m]) % mod) * invfat[n-1])
         % mod << '\n';
}

int32_t main() {
    calcfat();
    calcinvfat();
    solve();

    return 0;
}
```

7.15 Divisor Analysis

Given an integer, your task is to find the number, sum and product of its divisors. As an example, let us consider the number 12:
• the number of divisors is 6 (they are 1, 2, 3, 4, 6, 12) • the sum of divisors is $1+2+3+4+6+12 = 28$ • the product of divisors is $1 \cdot 2 \cdot 3 \cdot 4 \cdot 6 \cdot 12 = 1728$
Since the input number may be large, it is given as a prime factorization.

Input
The first line has an integer n: the number of parts in the prime factorization. After this, there are n lines that describe the factorization. Each line has two numbers x and k where x is a prime and k is its power.

Output
Print three integers modulo $10^9 + 7$: the number, sum and product of the divisors.

Constraints
• $1 \leq n \leq 10^5$ • $2 \leq x \leq 10^6$ • each x is a distinct prime • $1 \leq k \leq 10^9$

```
const int M = 1e9 + 7;

vector<pair<int, int>> p;
bool tem_impar = false;

int fexp(int a, int b) {
    if (b == 0) return 1;
    int f = fexp(a, b >> 1);
    f = ((ll) f * f) % M;
    if (b & 1) f = ((ll) f * a) % M;
    return f;
}

int inv(int a) {
    return fexp(a, M-2);
}

int div(){
    int d = 1;
    for(int i=0; i<p.size(); i++)
        d = ((ll)d * (p[i].second + 1)) % M;

    return d;
}

int soma(){
    int d = 1, f = 1;
    for(int i=0; i<p.size(); i++){
        f = fexp(p[i].first, p[i].second + 1) - 1;
        f = (M + f) % M;
        f = ((ll)f * inv(p[i].first - 1)) % M;

        d = ((ll)d * (f)) % M;
    }
    return d;
}

int prod(){
    int d = 1, n = 1;

    if(tem_impar){
        bool flag = true;
        for(int i=0; i<p.size(); i++){
            if(p[i].second % 2 == 1 && flag){
                d = ((ll)d * ((p[i].second + 1)/2)) % (M-1);
                flag = false;
            }
            else d = ((ll)d * (p[i].second + 1)) % (M-1);
        }

        for(int i=0; i<p.size(); i++)
            n = ((ll)n * fexp(p[i].first, p[i].second)) % M;
    }
    else{
        for(int i=0; i<p.size(); i++)
            d = ((ll)d * (p[i].second + 1)) % (M-1);

        for(int i=0; i<p.size(); i++)
            n = ((ll)n * fexp(p[i].first, p[i].second/2)) % M
                ;
    }

    return fexp(n, d);
}

void solve() {
    int n; cin >> n;
    for(int i = 0; i < n; i++){
        int x, y; cin >> x >> y;
        if(y % 2 == 1) tem_impar = true;
        p.push_back({x, y});
    }

    cout << div() << '\n';
    cout << soma() << '\n';
    cout << prod() << '\n';
}
```

7.16 Exponentiation

Your task is to efficiently calculate values a^b modulo $10^9 + 7$. Note that in this task we assume that $0^0 = 1$.

Input
The first input line contains an integer n: the number of calculations. After this, there are n lines, each containing two integers a and b.

Output
Print each value a^b modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq a, b \leq 10^9$

```
int fexp(int a, int b, int M) {
```

```
    if (b == 0) return 1;
    int f = fexp(a, b >> 1, M);
    f = ((ll) f * f) % M;
    if (b & 1) f = ((ll) f * a) % M;
    return f;
}

void solve() {
    int a, b; cin >> a >> b;
    int p = 1e9 + 7;
    cout << fexp(a, b, p) << '\n';
}
```

7.17 Exponentiation II

Your task is to efficiently calculate values a^{b^c} modulo 10^9+7. Note that in this task we assume that $0^0 = 1$.

Input
The first input line has an integer n: the number of calculations. After this, there are n lines, each containing three integers a, b and c.

Output
Print each value a^{b^c} modulo 10^9+7.

Constraints
• $1 \le n \le 10^5$ • $0 \le a, b, c \le 10^9$

```
int fexp(int a, int b, int M) {
    if (b == 0) return 1;
    int f = fexp(a, b >> 1, M);
    f = ((ll) f * f) % M;
    if (b & 1) f = ((ll) f * a) % M;
    return f;
}

void solve() {
    int a, b, c; cin >> a >> b >> c;
    int p = 1e9 + 7;
    cout << fexp(a, fexp(b, c, p-1), p) << '\n';
}
```

7.18 Fibonacci Numbers

The Fibonacci numbers can be defined as follows:
• $F_0 = 0$ • $F_1 = 1$ • $F_n = F_{n-2} + F_{n-1}$
Your task is to calculate the value of F_n for a given n.

Input
The only input line has an integer n.

Output
Print the value of F_n modulo 10^9+7.

Constraints
• $0 \le n \le 10^{18}$

```
const int maxn = 2e6+6, mod = 1e9+7;

void multi(int a[2][2], int b[2][2]){
    int x1 = ( (a[0][0] * b[0][0]) % mod + (a[0][1] * b
        [1][0]) % mod ) % mod;
    int x2 = ( (a[0][0] * b[0][1]) % mod + (a[0][1] * b
        [1][1]) % mod ) % mod;
    int x3 = ( (a[1][0] * b[0][0]) % mod + (a[1][1] * b
        [1][0]) % mod ) % mod;
    int x4 = ( (a[1][0] * b[0][1]) % mod + (a[1][1] * b
        [1][1]) % mod ) % mod;

    a[0][0] = x1; a[0][1] = x2; a[1][0] = x3; a[1][1] = x4;
}

void fexp(int n[2][2], int exp){
    if(exp == 0){
        n[0][0] = 1; n[0][1] = 0; n[1][0] = 0; n[1][1] = 1;
        return;
    }
    if(exp == 1) return;

    int m[2][2] = {{n[0][0], n[0][1]}, {n[1][0], n[1][1]}};
    fexp(n, exp / 2);
    multi(n, n);

    if(exp % 2 == 1) multi(n, m);
}

void solve() {
    int n; cin >> n;
    int m[2][2] = {{1, 1}, {1, 0}};
    fexp(m, n);
    cout << m[0][1] << '\n';
}
```

7.19 Graph Paths I

Consider a directed graph that has n nodes and m edges. Your task is to count the number of paths from node 1 to node n with exactly k edges.

Input
The first input line contains three integers n, m and k: the number of nodes and edges, and the length of the path. The nodes are numbered $1, 2, \ldots, n$. Then, there are m lines describing the edges. Each line contains two integers a and b: there is an edge from node a to node b.

Output
Print the number of paths modulo 10^9+7.

Constraints
• $1 \le n \le 100$ • $1 \le m \le n(n-1)$ • $1 \le k \le 10^9$ • $1 \le a, b \le n$

```
const int maxn = 2e6+6, n = 101, mod = 1e9+7;

int grafo[n][n];

void mult(int a[n][n], int b[n][n]){
    int c[n][n];

    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            c[i][j] = 0;
            for(int k=0; k<n; k++){
                c[i][j] = (c[i][j] + (a[i][k] * b[k][j]) %
                    mod ) % mod;
            }
        }
    }

    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            a[i][j] = c[i][j];
}

void fexp(int a[n][n], int exp){
    if(exp == 0){
        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){
                if(i == j) a[i][j] = 1;
                else a[i][j] = 0;
            }
        }
        return;
    }
    if(exp == 1) return;

    int b[n][n];
    if(exp % 2 == 1)
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) b[i][j]
            = a[i][j];

    fexp(a, exp / 2);
    mult(a, a);

    if(exp % 2 == 1) mult(a, b);
}

void solve() {
    int n1, m, k; cin >> n1 >> m >> k;

    for(int i=0; i<m; i++){
        int a, b; cin >> a >> b;
        grafo[a][b]++;
    }

    fexp(grafo, k);

    cout << grafo[1][n1] << '\n';
}
```

7.20 Graph Paths II

Consider a directed weighted graph having n nodes and m edges. Your task is to calculate the minimum path length from node 1 to node n with exactly k edges.

Input
The first input line contains three integers n, m and k: the number of nodes and edges, and the length of the path. The nodes are numbered $1, 2, \ldots, n$. Then, there are m lines describing the edges. Each line contains three integers a, b and c: there is an edge from node a to node b with weight c.

Output
Print the minimum path length. If there are no such paths,

print -1.

Constraints
• $1 \le n \le 100$ • $1 \le m \le n(n-1)$ • $1 \le k \le 10^9$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
const int maxn = 2e6+6, n = 101, mod = 1e9+7, inf = 9e18;

int grafo[n][n];

void mult(int a[n][n], int b[n][n]){
    int c[n][n];

    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            c[i][j] = inf;
            for(int k=0; k<n; k++){
                if(a[i][k] == 0 || b[k][j] == 0) continue;
                c[i][j] = min(c[i][j], a[i][k] + b[k][j]);
            }
            if(c[i][j] == inf) c[i][j] = 0;
        }
    }

    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            a[i][j] = c[i][j];
}

void fexp(int a[n][n], int exp){
    if(exp == 0){
        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){
                if(i == j) a[i][j] = 1;
                else a[i][j] = 0;
            }
        }
        return;
    }
    if(exp == 1) return;

    int b[n][n];
    if(exp % 2 == 1)
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) b[i][j]
            = a[i][j];

    fexp(a, exp / 2);
    mult(a, a);

    if(exp % 2 == 1) mult(a, b);
}

void solve() {
    int n1, m, k; cin >> n1 >> m >> k;

    for(int i=0; i<m; i++){
        int a, b; cin >> a >> b;
        int peso; cin >> peso;
        if(grafo[a][b] == 0) grafo[a][b] = inf;

        grafo[a][b] = min(grafo[a][b], peso);
    }

    fexp(grafo, k);

    if(grafo[1][n1] == 0) cout << "-1\n";
    else cout << grafo[1][n1] << '\n';
}
```

7.21 Grundy's Game

There is a heap of n coins and two players who move alternately. On each move, a player chooses a heap and divides into two nonempty heaps that have a different number of coins. The player who makes the last move wins the game. Your task is to find out who wins if both players play optimally.

Input
The first input line contains an integer t: the number of tests. After this, there are t lines that describe the tests. Each line has an integer n: the number of coins in the initial heap.

Output
For each test case, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints
• $1 \le t \le 10^5$ • $1 \le n \le 10^6$

```
const int maxn = 1e4+2;
int gnumbers[maxn];

void grundy() {
    array<bool, maxn/2+1> excluded;
    for (int i = 0; i <= maxn; ++i) {
        auto e_begin = excluded.begin();
        auto e_end = e_begin + i/2;
        fill(e_begin, e_end, false);
        for (int j = 1; j < (i+1)/2; ++j) {
            int const k = i - j;
            excluded[gnumbers[j] ^ gnumbers[k]] = true;
        }
        gnumbers[i] = find(e_begin, e_end, false) - e_begin;
    }
}

void solve() {
    int n; cin >> n;
    if(n < maxn) {
        if(gnumbers[n] == 0) cout << "second\n";
        else cout << "first\n";
    }
    else {
        cout << "first\n";
    }
}

int main() {
    grundy();
    int t; cin >> t; while(t--)
    solve();

    return 0;
}
```

7.22 Inversion Probability

An array has n integers $x_1, x_2, \ldots, x_n$, and each of them has been randomly chosen between 1 and r_i. An inversion is a pair (a, b) where $a<b$ and $x_a>x_b$. What is the expected number of inversions in the array?

Input
The first input line contains an integer n: the size of the array. The second line contains n integers $r_1, r_2, \ldots, r_n$: the range of possible values for each array position.

Output
Print the expected number of inversions rounded to six decimal places (rounding half to even).

Constraints
• $1 \le n \le 100$ • $1 \le r_i \le 100$

```
struct bint {
    static const int BASE = 1e9;
    vector<int> v;
    bool neg;

    bint() : neg(0) {}
    bint(int val) : bint() { *this = val; }
    bint(long long val) : bint() { *this = val; }

    void trim() {
        while (v.size() and v.back() == 0) v.pop_back();
        if (!v.size()) neg = 0;
    }

    bint& operator=(const bint& val) { v = val.v, neg = val.
        neg; return *this; }
    bint& operator=(long long val) {
        v.clear(), neg = 0;
        if (val < 0) neg = 1, val *= -1;
        for (; val; val /= BASE) v.push_back(val % BASE);
        return *this;
    }

    int cmp(const bint& r) const {
        if (neg != r.neg) return neg ? -1 : 1;
        if (v.size() != r.v.size()) {
            int ret = v.size() < r.v.size() ? -1 : 1;
            return neg ? -ret : ret;
        }
        for (int i = int(v.size())-1; i >= 0; i--) {
            if (v[i] != r.v[i]) {
                int ret = v[i] < r.v[i] ? -1 : 1;
                return neg ? -ret : ret;
            }
        }
        return 0;
    }
    friend bool operator<(const bint& l, const bint& r) {
        return l.cmp(r) == -1; }
    friend bool operator>=(const bint& l, const bint& r) {
        return l.cmp(r) >= 0; }
    friend bool operator!=(const bint& l, const bint& r) {
        return l.cmp(r) != 0; }
    friend bool operator==(const bint& l, const bint& r) {
        return l.cmp(r) == 0; }
```

```

    friend bint operator-(bint val) {
        if (val != 0) val.neg ^= 1;
        return val;
    }

    bint& operator +=(const bint& r) {
        if (!r.v.size()) return *this;
        if (neg != r.neg) return *this -= -r;
        for (int i = 0, c = 0; i < r.v.size() or c; i++) {
            if (i == v.size()) v.push_back(0);
            v[i] += c + (i < r.v.size() ? r.v[i] : 0);
            if ((c = v[i] >= BASE)) v[i] -= BASE;
        }
        return *this;
    }
    friend bint operator+(bint a, const bint& b) { return a
         += b; }

    bint& operator -=(const bint& r) {
        if (!r.v.size()) return *this;
        if (neg != r.neg) return *this += -r;
        if ((!neg and *this < r) or (neg and r < *this)) {
            *this = r - *this;
            neg ^= 1;
            return *this;
        }
        for (int i = 0, c = 0; i < r.v.size() or c; i++) {
            v[i] -= c + (i < r.v.size() ? r.v[i] : 0);
            if ((c = v[i] < 0)) v[i] += BASE;
        }
        trim();
        return *this;
    }
    friend bint operator-(bint a, const bint& b) { return a
         -= b; }

    bint& operator *=(int val) {
        if (val < 0) val *= -1, neg ^= 1;
        for (int i = 0, c = 0; i < v.size() or c; i++) {
            if (i == v.size()) v.push_back(0);
            long long at = (long long) v[i] * val + c;
            v[i] = at % BASE;
            c = at / BASE;
        }
        trim();
        return *this;
    }

    bint& operator/=(int val) {
        if (val < 0) neg ^= 1, val *= -1;
        for (int i = int(v.size())-1, c = 0; i >= 0; i--) {
            long long at = v[i] + c * (long long) BASE;
            v[i] = at / val;
            c = at % val;
        }
        trim();
        return *this;
    }

    friend bint abs(bint val) {
        val.neg = 0;
        return val;
    }
    friend bint operator *(bint a, int b) { return a *= b; }
    friend bint operator /(bint a, int b) { return a /= b; }

    friend pair<bint, bint> divmod(const bint& a_, const bint
         & b_) { // O(n^2)
        if (a_ == 0) return {0, 0};
        int norm = BASE / (b_.v.back() + 1);
        bint a = abs(a_) * norm;
        bint b = abs(b_) * norm;
        bint q, r;
        for (int i = a.v.size() - 1; i >= 0; i--) {
            r *= BASE, r += a.v[i];
            long long upper = b.v.size() < r.v.size() ? r.v[b
                 .v.size()] : 0;
            int lower = b.v.size() - 1 < r.v.size() ? r.v[b.v
                 .size() - 1] : 0;
            int d = (upper * BASE + lower) / b.v.back();
            r -= b*d;
            while (r < 0) r += b, d--; // roda O(1) vezes
            q.v.push_back(d);
        }
        reverse(q.v.begin(), q.v.end());
        q.neg = a_.neg ^ b_.neg;
        r.neg = a_.neg;
        q.trim(), r.trim();
        return {q, r / norm};
    }
    bint operator/(const bint& val) { return divmod(*this,
         val).first; }
    bint& operator/=(const bint& val) { return *this = *this
         / val; }
    bint operator%(const bint& val) { return divmod(*this,
         val).second; }
    bint& operator%=(const bint& val) { return *this = *this
         % val; }
};

pair<int, int> prob(int ri, int rj) {
    int num = 0, den = 1;
    if(rj >= ri) {
        num = ri - 1;
        den = 2 * rj;
    }
    else {
        num = rj - 1 + 2 * ri - 2 * rj;
        den = 2 * ri;
    }
    return {num, den};
}

vector<int> calc_primes(int max) {
    vector<int> divi(max + 1);
    vector<int> primes;

    divi[1] = 1;
    for (int i = 2; i <= max; i++) {
        if (divi[i] == 0) divi[i] = i, primes.push_back(i);
        for (int j : primes) {
            if (j > divi[i] or i*j > max) break;
            divi[i*j] = j;
        }
    }

    return primes;
}

void solve() {
    int n; cin >> n;
    vector<int> r(n);
    for(auto &x : r) cin >> x;

    bint MMC = 1;
    auto primes = calc_primes(200);

    for(int p : primes) {
        int q = p;
        while(q * p <= 200) q *= p;
        MMC *= q;
    }

    bint NUM = 0;
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            auto [a, b] = prob(r[i], r[j]);
            NUM += (MMC / b) * a;
        }
    }

    NUM *= 1'000'000;
    bint ANS = NUM/MMC;
    bint res = NUM%MMC;

    res *= 2;
    int comp = res.cmp(MMC);

    if (comp > 0) ANS += bint(1);
    else if (comp == 0 && ANS.v[0] % 2 != 0) {
        ANS += bint(1);
    }

    ll final_ans = 0;
    if (ANS.v.size() > 0) final_ans += ANS.v[0];
    if (ANS.v.size() > 1) final_ans += 1LL * ANS.v[1] * bint
         ::BASE;

    cout << final_ans / 1000000 << ".";
    string final_res = "";
    for(int i = 0; i < 6; i++) {
        final_res.push_back(final_ans % 10 + '0');
        final_ans /= 10;
    }
    reverse(final_res.begin(), final_res.end());
    cout << final_res << '\n';
}
```

7.23 Josephus Queries

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, every second child is removed from the circle, until there are no children left. Your task is to process q queries of the form: "when there are n children, who is the kth child that will be removed?"

Input
The first input line has an integer q: the number of queries. After this, there are q lines that describe the queries. Each line has two integers n and k: the number of children and the position of the child.

Output
Print q integers: the answer for each query.

Constraints
• $1 \leq q \leq 10^5$ • $1 \leq k \leq n \leq 10^9$

```
void solve(){
    int k, n, exp = 1, soma = 0, M = 0, tam = 0;
    cin >> n >> k;

    while(true){
        M = 2*(n % exp);
        M = (M + exp + 1) % (2*exp);
        tam = (n- M)/(2*exp) - ceil(double(1-M) / (2*exp)) +
             1;

        if(k > soma + tam){
            soma += tam;
```

```
            if(soma == n-1){
                cout << (M + exp) % (2*exp) <<'\n';
                break;
            }
            exp *= 2;
            continue;
        }

        if(soma == 0) soma = -1;
        cout << exp*2*(k - soma - 1) + M << '\n';
        break;
    }

    return;
}
```

7.24 Moving Robots

Each square of an 8×8 chessboard has a robot. Each robot independently moves k steps, and there can be many robots on the same square. On each turn, a robot moves one step left, right, up or down, but not outside the board. It randomly chooses a direction among those where it can move. Your task is to calculate the expected number of empty squares after k turns.

Input
The only input line has an integer k.

Output
Print the expected number of empty squares rounded to six decimal places (rounding half to even).

Constraints
- $1 \le k \le 100$

```
const int maxn = 2e6+6, n = 8, m = 64, mod = 1e9+7;

double base[m][m];
vector<pair<int, int>> macaco = {{1, 0}, {0, 1}, {-1, 0}, {0, -1}};

int matriz(int i, int j){
    if(i < 0 || i >= n || j < 0 || j >= n) return -1;
    return i*n + j;
}

void mult(double a[m][m], double b[m][m]){
    double c[m][m];

    for(int i=0; i<m; i++) {
        for(int j=0; j<m; j++) {
            c[i][j] = 0;
            for(int k=0; k<m; k++) c[i][j] += a[i][k] * b[k][j];
        }
    }

    for(int i=0; i<m; i++)
        for(int j=0; j<m; j++)
            a[i][j] = c[i][j];
}

void fexp(double a[m][m], int exp){
    if(exp == 0) {
        for(int i=0; i<m; i++) {
            for(int j=0; j<m; j++) {
                if(i == j) a[i][j] = 1;
                else a[i][j] = 0;
            }
        }
        return;
    }
    if(exp == 1) return;

    double b[m][m];
    if(exp % 2 == 1)
        for(int i=0; i<m; i++) for(int j=0; j<m; j++) b[i][j] = a[i][j];

    fexp(a, exp / 2);
    mult(a, a);

    if(exp % 2 == 1) mult(a, b);
}

void solve() {
    int k; cin >> k;
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            bool a = i == 0 || i == n-1;
            bool b = j == 0 || j == n-1;

            if(a && b) {
                for(auto [x, y] : macaco)
                    if(matriz(i+x, j+y) != -1) base[matriz(i, j)][matriz(i+x, j+y)] = (double) 1/2;
            }

            else if(a || b) {
                for(auto [x, y] : macaco)
                    if(matriz(i+x, j+y) != -1) base[matriz(i, j)][matriz(i+x, j+y)] = (double) 1/3;
            }

            else {
                for(auto [x, y] : macaco)
                    if(matriz(i+x, j+y) != -1) base[matriz(i, j)][matriz(i+x, j+y)] = (double) 1/4;
            }
        }
    }

    fexp(base, k);
    double ans = 0;
    for(int j=0; j<m; j++){
        double total = 1;
        for(int i=0; i<m; i++){
            total *= 1 - base[i][j];
        }
        ans += total;
    }

    cout << fixed << setprecision(6);
    cout << ans << '\n';
}
```

7.25 Next Prime

Given a positive integer n, find the next prime number after it.

Input
The first line has an integer t: the number of tests. After that, each line has a positive integer n.

Output
For each test, print the next prime after n.

Constraints
- $1 \le t \le 20$
- $1 \le n \le 10^{12}$

```
ll mul(ll a, ll b, ll m) {
    ll ret = a*b - ll((long double)1/m*a*b+0.5)*m;
    return ret < 0 ? ret+m : ret;
}

ll pow(ll x, ll y, ll m) {
    if (!y) return 1;
    ll ans = pow(mul(x, x, m), y/2, m);
    return y%2 ? mul(x, ans, m) : ans;
}

bool prime(ll n) {
    if (n < 2) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0) return 0;
    ll r = __builtin_ctzll(n - 1), d = n >> r;

    // com esses primos, o teste funciona garantido para n <= 2^64
    // funciona para n <= 3*10^24 com os primos ate 41
    for (int a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
        ll x = pow(a, d, n);
        if (x == 1 or x == n - 1 or a % n == 0) continue;

        for (int j = 0; j < r - 1; j++) {
            x = mul(x, x, n);
            if (x == n - 1) break;
        }
        if (x != n - 1) return 0;
    }
    return 1;
}

void solve()
{
    ll n; cin >> n;
    for(int i = 1; i <= 10000; i++) {
        if(prime(n + i)) {
            cout << n + i << '\n';
            break;
        }
    }
}
```

7.26 Nim Game I

There are n heaps of sticks and two players who move alternately. On each move, a player chooses a non-empty heap and removes any number of sticks. The player who removes the last stick wins the game. Your task is to find out who wins if both players play optimally.

Input

The first input line contains an integer t: the number of tests. After this, t test cases are described: The first line contains an integer n: the number of heaps. The next line has n integers $x_1, x_2, \ldots, x_n$: the number of sticks in each heap.

Output
For each test case, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints
• $1 \leq t \leq 2 \cdot 10^5$ • $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • the sum of all n is at most $2 \cdot 10^5$

```cpp
const int maxn = 2e6+5;
int v[maxn];

void solve() {
    int n; int x; int result = 0;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> x;
        result ^= x;
    }

    if(result == 0) cout << "second\n";
    else cout << "first\n";
}

int main() {
    ios::sync_with_stdio(0), cout.tie(0);
    int t; cin >> t; while(t--) solve();

    return 0;
}
```

7.27 Nim Game II

There are n heaps of sticks and two players who move alternately. On each move, a player chooses a non-empty heap and removes 1, 2, or 3 sticks. The player who removes the last stick wins the game. Your task is to find out who wins if both players play optimally.

Input
The first input line contains an integer t: the number of tests. After this, t test cases are described: The first line contains an integer n: the number of heaps. The next line has n integers $x_1, x_2, \ldots, x_n$: the number of sticks in each heap.

Output
For each test case, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints
• $1 \leq t \leq 2 \cdot 10^5$ • $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • the sum of all n is at most $2 \cdot 10^5$

```cpp
const int maxn = 2e6+5;
int v[maxn];

void solve() {
    int n; int x; int result = 0;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> x;
        x = x % 4;
        result ^= x;
    }

    if(result == 0) cout << "second\n";
    else cout << "first\n";
}
```

7.28 Permutation Order

Let $p(n, k)$ denote the kth permutation (in lexicographical order) of $1 \ldots n$. For example, $p(4, 1) = [1, 2, 3, 4]$ and $p(4, 2) = [1, 2, 4, 3]$. Your task is to process two types of tests:
Given n and k, find $p(n, k)$ Given n and $p(n, k)$, find k

Input
The first line has an integer t: the number of tests. Each test is either "1 n k" or "2 n $p(n, k)$".

Output
For each test, print the answer according to the example.

Constraints
• $1 \leq t \leq 1000$ • $1 \leq n \leq 20$ • $1 \leq k \leq n!$

```cpp
template<class T> using ordered_set = tree<T, null_type, less
    <T>, rb_tree_tag, tree_order_statistics_node_update>;

ll fat[MAX];

void calc(){
    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = fat[i - 1] * i;
}

bool comp(vi a, vi b) {
    int flag = false;
    for(int i = 0; i < min(a.size(), b.size()); i++) {
        if(a[i] > b[i]) {
            flag = false;
            break;
        }

        if(a[i] < b[i]) {
            flag = true;
            break;
        }
    }

    return flag;
}

vi solve1(ll n, ll k) {
    ordered_set<int> s;
    for(int i = 1; i <= n; i++) s.insert(i);
    vi p;

    while(!s.empty()) {
        ll at = (k - 1) / fat[s.size() - 1];
        k -= at * fat[s.size() - 1];

        auto it = s.find_by_order(at);
        p.push_back(*it);
        s.erase(it);
    }

    return p;
}

void solve2() {
    int n; cin >> n;
    vi p(n);
    for(int i = 0; i < n; i++) cin >> p[i];

    ll l = 0, r = fat[n];
    while(r - l > 1) {
        ll m = (l + r) / 2;
        if(comp(solve1(n, m), p)) l = m;
        else r = m;
    }

    cout << r << '\n';
}

signed main() {
    calc();
    int t; cin >> t; while(t--) {
        int a; cin >> a;
        if(a == 1) {
            ll n, k; cin >> n >> k;
            for(auto x : solve1(n, k)) cout << x << ' '; cout
                << '\n';
        }
        else solve2();
    }

    return 0;
}
```

7.29 Permutation Rounds

There is a sorted array $[1, 2, \ldots, n]$ and a permutation $p_1, p_2, \ldots, p_n$. On each round, all elements move according to the permutation: the element at position i moves to position p_i. After how many rounds is the array sorted again for the first time?

Input
The first line has an integer n. The next line contains n integers $p_1, p_2, \ldots, p_n$.

Output

Print the number of rounds modulo $10^9 + 7$.

Constraints
• $1 \le n \le 2 \cdot 10^5$

```cpp
int divi[MAX];

int fexp(int x, int y) {
    int ret = 1;
    while (y) {
        if (y & 1) ret = (ret * x) % M;
        y >>= 1;
        x = (x * x) % M;
    }
    return ret;
}

void crivo(int lim) {
    for (int i = 1; i <= lim; i++) divi[i] = 1;

    for (int i = 2; i <= lim; i++) if (divi[i] == 1)
        for (int j = i; j <= lim; j += i) divi[j] = i;
}

void solve() {
    int n; cin >> n;
    vi p(n + 1), mark(n + 1), ciclos;
    for(int i = 1; i <= n; i++) cin >> p[i];

    int tam = 0, at;
    for(int ini = 1; ini <= n; ini++) {
        if(mark[ini] == 1) continue;

        at = ini; tam = 0;
        while(mark[at] == 0) {
            tam++;
            mark[at] = 1;
            at = p[at];
        }
        ciclos.push_back(tam);
    }

    crivo(MAX - 1);
    unordered_map<int, int> max_primos;
    for(auto c : ciclos) {
        while(c != 1) {
            int p = divi[c], count = 0;
            while(c % p == 0) {
                c /= p;
                count++;
            }
            max_primos[p] = max(max_primos[p], count);
        }
    }

    int ans = 1;
    for(auto x : max_primos) {
        ans = (ans * fexp(x.first, x.second)) % M;
    }

    cout << ans << '\n';
}
```

7.30 Prime Multiples

You are given k distinct prime numbers $a_1, a_2, \ldots, a_k$ and an integer n. Your task is to calculate how many of the first n positive integers are divisible by at least one of the given prime numbers.

Input
The first input line has two integers n and k. The second line has k prime numbers $a_1, a_2, \ldots, a_k$.

Output
Print one integer: the number integers within the interval $1, 2, \ldots, n$ that are divisible by at least one of the prime numbers.

Constraints
• $1 \le n \le 10^{18}$ • $1 \le k \le 20$ • $2 \le a_i \le n$

```cpp
int main() {
    ll n, k, cont = 1, soma = 0;
    cin >> n >> k;
    ll a[k];
    for(int i=0; i<k; i++){
        cin >> a[i];
        cont *= 2;
    }
    for(cont--; cont > 0; cont--){
        ll aux = cont;
        ll bit, total = 0;
        ll num = n;
        for(int j=0; j<k; j++){
            bit = aux % 2;
            if(bit == 1){
                total += bit;
                num = num / a[j];
            }
            aux = aux >> 1;
        }
        if(total % 2 == 1) soma += num;
        else soma -= num;
    }
    cout << soma << '\n';
}
```

7.31 Stair Game

There is a staircase consisting of n stairs, numbered $1, 2, \ldots, n$. Initially, each stair has some number of balls. There are two players who move alternately. On each move, a player chooses a stair k where $k \neq 1$ and it has at least one ball. Then, the player moves any number of balls from stair k to stair $k-1$. The player who moves last wins the game. Your task is to find out who wins the game when both players play optimally. Note that if there are no possible moves at all, the second player wins.

Input
The first input line has an integer t: the number of tests. After this, t test cases are described: The first line contains an integer n: the number of stairs. The next line has n integers $p_1, p_2, \ldots, p_n$: the initial number of balls on each stair.

Output
For each test, print "first" if the first player wins the game and "second" if the second player wins the game.

Constraints
• $1 \le t \le 2 \cdot 10^5$ • $1 \le n \le 2 \cdot 10^5$ • $0 \le p_i \le 10^9$ • the sum of all n is at most $2 \cdot 10^5$

```cpp
const int maxn = 2e6+5;
int v[maxn];

void solve() {
    int n; int x; int result = 0;
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> x;
        if(i % 2 == 1) result ^= x;
    }

    if(result == 0) cout << "second\n";
    else cout << "first\n";
}
```

7.32 Stick Game

Consider a game where two players remove sticks from a heap. The players move alternately, and the player who removes the last stick wins the game. A set $P = \{p_1, p_2, \ldots, p_k\}$ determines the allowed moves. For example, if $P = \{1, 3, 4\}$, a player may remove 1, 3 or 4 sticks. Your task is find out for each number of sticks $1, 2, \ldots, n$ if the first player has a winning or losing position.

Input
The first input line has two integers n and k: the number of sticks and moves. The next line has k integers $p_1, p_2, \ldots, p_k$ that describe the allowed moves. All integers are distinct, and one of them is 1.

Output
Print a string containing n characters: W means a winning position, and L means a losing position.

Constraints
• $1 \le n \le 10^6$ • $1 \le k \le 100$ • $1 \le p_i \le n$

```
const int maxn = 2e6+5;
int v[maxn];

int main() {
    ios::sync_with_stdio(0), cout.tie(0);
    int n, k; cin >> n >> k;
    vector<int> p(k);
    for(int i = 0; i < k; i ++) cin >> p[i];

    for(int i = 0; i <= n; i++){
        if(v[i] == 0){
            for(int j = 0; j < k; j++){
                v[i + p[j]] = 1;
            }
        }
    }

    for(int i=1; i<=n; i++){
        if(v[i] == 0) cout << "L";
        else cout << "W";
    }
    cout << '\n';

    return 0;
}
```

7.33 Sum Of Divisors

Let $\sigma(n)$ denote the sum of divisors of an integer n. For example, $\sigma(12) = 1 + 2 + 3 + 4 + 6 + 12 = 28$. Your task is to calculate the sum $\sum_{i=1}^{n} \sigma(i)$ modulo $10^9 + 7$.

Input
The only input line has an integer n.

Output
Print $\sum_{i=1}^{n} \sigma(i)$ modulo $10^9 + 7$.

Constraints
• $1 \le n \le 10^{12}$

```
const int M = 1e9 + 7;

void solve() {
    ll n; cin >> n;
    ll sum = 0, raiz = sqrt(n);
    for(ll d = 1; d*d <= n; d++){
        sum = (sum + d * ((n/d) - raiz)) % M;
    }
    for(ll m = 1; m*m <= n; m++){
        ll p = (n/m) % M;
        sum = (sum + p*(p+1)/2) % M;
    }

    cout << sum << '\n';
}
```

7.34 Sum Of Four Squares

A well known result in number theory is that every non-negative integer can be represented as the sum of four squares of non-negative integers. You are given a non-negative integer n. Your task is to find four non-negative integers a, b, c and d such that $n = a^2 + b^2 + c^2 + d^2$.

Input
The first line has an integer t: the number of test cases. Each of the next t lines has an integer n.

Output
For each test case, print four non-negative integers a, b, c and d that satisfy $n = a^2 + b^2 + c^2 + d^2$.

Constraints
• $1 \le t \le 1000$ • $0 \le n \le 10^7$ • the sum of all n is at most 10^7

```
void solve() {
    int n; cin >> n;
    vector<int> quad;
    int at = 0;
    for(int i = 1; at <= n; i++) {
        quad.push_back(at);
        at += 2 * i - 1;
    }

    bitset<MAX> sumsquare;
    for(auto x : quad)
        for(auto y : quad)
            if(x + y < MAX) sumsquare.set(x + y);

    int a[2] = {0, 0};

    for(int i = 0; i <= n; i++) {
        if(sumsquare[i] == 1 && sumsquare[n - i] == 1) {
            a[0] = i;
            a[1] = n - i;
            break;
        }
    }

    for(auto x : a)
        for(int i = 0; i * i <= x; i++) {
            int d = x - i * i;
            int s = round(sqrt((double) d));
            if(s * s == d) {
                cout << i << ' ' << s << ' ';
                break;
            }
        }

    cout << '\n';
}
```

7.35 System Of Linear Equations

You are given $n \cdot (m+1)$ coefficients $a_{i,j}$ and b_i which form the following n linear equations:

• $a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,m}x_m = b_1 \pmod{10^9+7}$ • $a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,m}x_m = b_2 \pmod{10^9+7}$ • $\ldots$ • $a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,m}x_m = b_n \pmod{10^9+7}$
Your task is to find any m integers $x_1, x_2, \ldots, x_m$ that satisfy the given equations.

Input
The first line has two integers n and m: the number of equations and variables. The next n lines each have $m+1$ integers $a_{i,1}, a_{i,2}, \ldots, a_{i,m}, b_i$: the coefficients of the i-th equation.

Output
Print m integers $x_1, x_2, \ldots, x_m$: the values of the variables that satisfy the equations. The values must also satisfy $0 \le x_i < 10^9 + 7$. You can print any valid solution. If no solution exists print only -1.

Constraints
• $1 \le n, m \le 500$ • $0 \le a_{i,j}, b_i < 10^9 + 7$

```
const double eps = 1e-12;

ll pow(ll x, ll y, ll m) {
    ll ret = 1;
    while (y) {
        if (y & 1) ret = (ret * x) % m;
        y >>= 1;
        x = (x * x) % m;
    }
    return ret;
}

ll inv(ll a) {
    return pow(a, MOD - 2, MOD);
}

int solveLinear(vector<vi>& A, vi& b, vi& x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    vi col(m); iota(all(col), 0);

    rep(i,0,n) {
        int v, bv = 0;
        rep(r,i,n) rep(c,i,m)
            if ((v = fabs(A[r][c])) > bv)
                br = r, bc = c, bv = v;
        if (bv <= eps) {
            rep(j,i,n) if (fabs(b[j]) > eps) return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j,0,n) swap(A[j][i], A[j][bc]);
        bv = inv(A[i][i]);
        rep(j,i+1,n) {
            int fac = (A[j][i] * bv) % MOD;
            b[j] = (b[j] + MOD - (fac * b[i] % MOD)) % MOD;
            rep(k,i+1,m) A[j][k] = (A[j][k] + MOD - (fac*A[i
                ][k]) % MOD) % MOD;
        }
        rank++;
    }

    x.assign(m, 0);
    for (int i = rank; i--;) {
```

```
        b[i] = (b[i] * inv(A[i][i])) % MOD;
        x[col[i]] = b[i];
        rep(j,0,i) b[j] = (b[j] + MOD - (A[j][i] * b[i]) %
             MOD) % MOD;
    }
    return rank;
}

void solve() {
    int n, m; cin >> n >> m;
    vector<vector<int>> a;
    vector<int> b;
    int x;
    for(int i = 0; i < n; i++) {
        a.push_back({});
        for(int j = 0; j < m; j++) {
            cin >> x;
            a.back().push_back(x);
        }
        cin >> x;
        b.push_back(x);
    }

    vector<int> y(m);
    int ans = solveLinear(a, b, y);

    if(ans == -1) {
        cout << ans << '\n';
        return;
    }

    for(auto k : y) cout << k << ' '; cout << '\n';
}
```

7.36 Throwing Dice

Your task is to calculate the number of ways to get a sum n by throwing dice. Each throw yields an integer between $1 \ldots 6$. For example, if $n = 10$, some possible ways are $3+3+4$, $1+4+1+4$ and $1+1+6+1+1$.

Input
The only input line contains an integer n.

Output
Print the number of ways modulo 10^9+7.

Constraints
• $1 \le n \le 10^{18}$

```
const int maxn = 2e6+6, m = 6, mod = 1e9+7;

int base[m][m] = {
    {1, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 1, -1},
    {0, 0, 0, 1, -1, 0},
    {0, 0, 1, -1, 0, 0},
    {0, 1, -1, 0, 0, 0}
};

int multiplicador[m][m] = {
    {1, 1, 0, 0, 0, 0},
    {1, 0, 1, 0, 0, 0},
    {1, 0, 0, 1, 0, 0},
    {1, 0, 0, 0, 1, 0},
    {1, 0, 0, 0, 0, 1},
    {1, 0, 0, 0, 0, 0}
};

void mult(int a[m][m], int b[m][m]){
    int c[m][m];

    for(int i=0; i<m; i++){
        for(int j=0; j<m; j++){
            c[i][j] = 0;
            for(int k=0; k<m; k++){
                c[i][j] = (c[i][j] + (a[i][k] * b[k][j]) %
                     mod) % mod;
            }
        }
    }

    for(int i=0; i<m; i++)
        for(int j=0; j<m; j++)
            a[i][j] = c[i][j];
}

void fexp(int a[m][m], int exp){
    if(exp == 0){
        for(int i=0; i<m; i++){
            for(int j=0; j<m; j++){
                if(i == j) a[i][j] = 1;
                else a[i][j] = 0;
            }
        }
        return;
    }
    if(exp == 1) return;

    int b[m][m];
    if(exp % 2 == 1)
        for(int i=0; i<m; i++) for(int j=0; j<m; j++) b[i][j]
             = a[i][j];

    fexp(a, exp / 2);
    mult(a, a);

    if(exp % 2 == 1) mult(a, b);
}

void solve() {
    int n; cin >> n;
    fexp(multiplicador, n);
    mult(base, multiplicador);
    cout << base[0][0] << '\n';
}
```

7.37 Triangle Number Sums

A triangle number is a positive integer of the form $1+2+\cdots+k$. The first triangle numbers are 1, 3, 6, 10 and 15. Every positive integer can be represented as a sum of triangle numbers. For example, $42 = 21+21$ and $1337 = 1326+10+1$. Given a positive integer n, determine the smallest number of triangle numbers that sum to n.

Input
The first line has an integer t: the number of tests. After that, each line has a positive integer n.

Output
For each test, print the smallest number of triangle numbers.

Constraints
• $1 \le t \le 100$ • $1 \le n \le 10^{12}$

```
vi triang;

void solve() {
    int n; cin >> n;
    for(auto x : triang) {
        if(x == n) {
            cout << 1 << '\n';
            return;
        }
    }

    int i = 0, j = triang.size() - 1;

    while(i <= j) {
        if(triang[i] + triang[j] == n) {
            cout << 2 << '\n';
            return;
        }
        else if(triang[i] + triang[j] >= n) j--;
        else i++;
    }

    cout << 3 << '\n';
    return;
}

signed main() {
    for(int i = 0; i < MAX; i++) triang.push_back((i * (i +
         1)) / 2);

    int t; cin >> t; while(t--)
    solve();

    return 0;
}
```

8 String Algorithms

8.1 All Palindromes

Given a string, calculate for each position the length of the longest palindrome that ends at that position.

Input
The only line contains a string of length n. Each character is one of a–z.

Output
Print n numbers: the length of each palindrome.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$

```cpp
using vi = vector<int>;
using pii = pair<int, int>;

array<vi, 2> manacher( const string &s ){
    int n = sz(s);
    array<vi, 2> p = { vi(n + 1), vi(n) };
    FOR(z, 0, 2) for( int i = 0, l = 0, r = 0; i < n; i++ ){
        int t = r - i + !z;
        if(i < r ) p[z][i] = min( t, p[z][l + t] );
        int L = i - p[z][i], R = i + p[z][i] - !z;
        while( L >= 1 && R + 1 < n && s[L - 1] == s[R + 1] )
            p[z][i]++, L--, R++;
        if( R > r ) l = L, r = R;
    }
    return p;
}

void solve(){
    string s; cin >> s;
    auto p = manacher(s);

    int n = sz(s);

    set<int> ids[2];
    vector<vector<pii>> undo(n);

    FOR(i, 0, n){
        if( p[0][i] ){
            ids[0].insert(i);
            undo[i + p[0][i] - 1].push_back({ i, 0 });
        }
        ids[1].insert(i);
        undo[i + p[1][i]].push_back({ i, 1 });

        int ans = 2*(i - *ids[1].begin()) + 1;
        if( !ids[0].empty() ) ans = max( ans, 2*(i - *ids[0].
            begin() + 1) );

        for( auto [j, t] : undo[i] ) ids[t].erase(j);

        cout << ans << "␣";
    }
    cout << '\n';
}
```

8.2 Counting Patterns

Given a string and patterns, count for each pattern the number of positions where it appears in the string.

Input
The first input line has a string of length n. The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns. The string and the patterns consist of characters a–z.

Output
For each pattern, print the number of positions.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq k \leq 5 \cdot 10^5$ • the total length of the patterns is at most $5 \cdot 10^5$

```cpp
vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260LL);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n,
            cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] =
            nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r +=
            ra[sa[i]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%
                n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}

string s;
bool complb(int p, const string& t) {
    return s.compare(p, t.size(), t) < 0;
}
bool compup(const string& t, int p) {
    return s.compare(p, t.size(), t) > 0;
}

void solve() {
    cin >> s;
    int k; cin >> k;

    vector<int> sa = suffix_array(s);

    for(int i = 0; i < k; i++) {
        string t; cin >> t;
        auto ub = upper_bound(sa.begin(), sa.end(), t, compup
            );
        auto lb = lower_bound(sa.begin(), sa.end(), t, complb
            );
        cout << ub - lb << '\n';
    }

}
```

8.3 Distinct Subsequences

You are given a string. You can remove any number of characters from it, but you cannot change the order of the remaining characters. How many different strings can you generate?

Input
The first input line contains a string of size n. Each character is one of a–z.

Output
Print one integer: the number of strings modulo $10^9 + 7$.

Constraints
- $1 \leq n \leq 5 \cdot 10^5$

```
const int alphabet = 26;
const int mod = 1e9 + 7;

int main(){
  string s; cin >> s;
  int n = s.size();

  auto sum = [&]( int a, int b ){
    return (a + b)%mod;
  };

  vector<vector<int>> last( n, vector<int>(alphabet, -1));
  for( int i = 0; i < n; i++ ){
    if( i > 0 ) last[i] = last[i - 1];
    last[i][s[i] - 'a'] = i;
  }

  vector<int> dp(n);

  for( int j : last.back() ) if( j != -1 ) dp[j] = 1;
  for( int i = n - 1; i >= 0; i-- )
    if( i > 0 ) for( int j : last[i - 1] ) if( j != -1 ) dp[j
        ] = sum( dp[j], dp[i] );

  cout << accumulate(dp.begin(), dp.end(), 0LL)%mod << endl;
}
```

8.4 Distinct Substrings

Count the number of distinct substrings that appear in a string.

Input
The only input line has a string of length n that consists of characters a–z.

Output
Print one integer: the number of substrings.

Constraints
- $1 \leq n \leq 10^5$

```
vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260LL);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n,
            cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] =
            nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r +=
            ra[sa[i]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%
                n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
    int n = s.size(), k = 0;
    vector<int> ra(n), lcp(n);
    for (int i = 0; i < n; i++) ra[sa[i]] = i;

    for (int i = 0; i < n; i++, k -= !!k) {
        if (ra[i] == n-1) { k = 0; continue; }
        int j = sa[ra[i]+1];
        while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[ra[i]] = k;
    }
    return lcp;
}

void solve() {
    string s; cin >> s;
    int n = s.size();

    vector<int> sa = suffix_array(s);
    vector<int> lcp = kasai(s, sa);

    int ans = 0;
    for(auto x : lcp) ans += x;
    cout << n * (n + 1) / 2 - ans << '\n';
}
```

8.5 Finding Borders

A border of a string is a prefix that is also a suffix of the string but not the whole string. For example, the borders of abcababcab are ab and abcab. Your task is to find all border lengths of a given string.

Input
The only input line has a string of length n consisting of characters a–z.

Output
Print all border lengths of the string in increasing order.

Constraints
- $1 \leq n \leq 10^6$

```
vector<int> get_z(string s) {
    int n = s.size();
    vector<int> z(n, 0);

    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n and s[z[i]] == s[i + z[i]]) z[i
            ]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }

    return z;
}

void solve() {
    string s; cin >> s;
    int n; n = s.size();

    auto v = get_z(s);
    for(int i = n - 1; i >= 0; i--) {
        if(n - i == v[i]) cout << v[i] << ' ';
    }
    cout << '\n';
}
```

8.6 Finding Patterns

Given a string and patterns, check for each pattern if it appears in the string.

Input
The first input line has a string of length n. The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns. The string and the patterns consist of characters a–z.

Output
For each pattern, print "YES" if it appears in the string and "NO" otherwise.

Constraints
- $1 \leq n \leq 10^5$
- $1 \leq k \leq 5 \cdot 10^5$
- the total length of the patterns is at most $5 \cdot 10^5$

```
const int alphabet = 26;

class AhoCorasick{
private:
  struct Node{
    int suffix_link, output, pai, pai_ch;
    vector<int> go, adj;

    Node( int pai = 0, int pai_ch = 0 ) : pai(pai), pai_ch(
        pai_ch), suffix_link(-1), output(false){
      go = adj = vector<int>( alphabet, -1 );
    };
  };
  vector<Node> t;

  int create( int pai, int pai_ch ){
    t.emplace_back( pai, pai_ch );
    return (int)t.size() - 1;
  }
```

```
19
20    int get_adj( int node, int c ){
21      if( t[node].adj[c] == -1 ){ int aux = create( node, c );
            t[node].adj[c] = aux; }
22      return t[node].adj[c];
23    }
24  public:
25    AhoCorasick(){ create(0, 0); }
26
27    int add_string( string &s ){
28      int node = 0;
29      for( char c : s ) node = get_adj( node, c - 'a' );
30      t[node].output = true;
31      return node;
32    }
33
34    int suffix_link( int node ){
35      if( t[node].suffix_link != -1 ) return t[node].
            suffix_link;
36      if( node == 0 || t[node].pai == 0 ) return t[node].
            suffix_link = 0;
37      return t[node].suffix_link = go( suffix_link(t[node].pai)
            , t[node].pai_ch );
38    }
39
40    int go( int node, int c ){
41      if( t[node].go[c] != -1 ) return t[node].go[c];
42      if( t[node].adj[c] != -1 ) return t[node].go[c] = t[node
            ].adj[c];
43      return t[node].go[c] = (( node == 0 ) ? 0 : go(
            suffix_link(node), c ) );
44    }
45
46    int size(){ return t.size(); }
47  };
48
49  int main(){
50    string s; cin >> s;
51    int q; cin >> q;
52
53    AhoCorasick aho;
54
55    vector<int> queries(q);
56
57    for( int i = 0; i < q; i++ ){
58      string t; cin >> t;
59      queries[i] = aho.add_string(t);
60    }
61
62    vector<int> marc(aho.size());
63
64    int node = 0;
65    for( char c : s ){
66      node = aho.go( node, c - 'a' );
67      for( int cur = node; !marc[cur]; cur = aho.suffix_link(
            cur) )
68        marc[cur] = true;
69    }
70
71    for( int x : queries ) cout << (( marc[x] ) ? "YES" : "NO"
          ) << endl;
72  }
```

8.7 Finding Periods

A period of a string is a prefix that can be used to generate the whole string by repeating the prefix. The last repetition may be partial. For example, the periods of abcabca are abc, abcabc and abcabca. Your task is to find all period lengths of a string.

Input
The only input line has a string of length n consisting of characters a–z.

Output
Print all period lengths in increasing order.

Constraints
- $1 \le n \le 10^6$

```
1  vector<int> z_function( string &s ){
2    int n = s.size();
3    vector<int> z(n);
4    for( int i = 1, l = 0, r = 0; i < n; i++ ){
5      if( i < r ) z[i] = min( r - i, z[i - l] );
6      while( i + z[i] < n && s[z[i]] == s[i + z[i]] ) z[i]++;
7      if( i + z[i] > r ) l = i, r = i + z[i];
8    }
9    return z;
10 }
11
12 int main(){
13   string s; cin >> s;
14   vector<int> z = z_function(s);
15   int n = s.size();
16   for( int i = 1; i < n; i++ ) if( i + z[i] == n ) cout << i
         << " ";
17   cout << n << endl;
18 }
```

8.8 Inverse Suffix Array

Given a suffix array of a string, your task is to reconstruct the string. The suffix array of a string of length n is a permutation of numbers $1, 2, \ldots, n$ that presents the lexicographical order of the suffixes.

Input
The first line has an integer n: the length of the string. The next line has n integers: the suffix array.

Output
Print a string that corresponds to the suffix array. The string must consist of characters a–z. If there are several possible strings, you can print any of them. If no string corresponds to the suffix array, print -1.

Constraints
- $1 \le n \le 10^5$

```
1 const int inf = 2e9 + 10;
2
3 // 4 1 3 5 6 7 2
4 // como 4 vem primeiro podemos colocar a nele
5 // a4a5a6a7 <= a1a2a3a4a5a6a7 e a5a6a7 <= a2a3a4a5a6a7
6 // então podemos fazer a1 = a4. Caso contrário, teriamos que
      colocar a1 = a4 + 1
7 // e analogo pra tds os outros
8
9 void solve(){
10     int n; cin >> n;
11     vector<int> a(n), pos(n + 2);
12     for(int i = 0; i < n; i++) {
13         cin >> a[i];
14         pos[a[i]] = i;
15     }
16     pos[n + 1] = 0;
17
18     char at = 'a';
19     vector<char> s(n + 1, '$');
20     for(int i = 0; i < n - 1; i++)  {
21         if(pos[a[i] + 1] < pos[a[i + 1] + 1])  {
22             s[a[i]] = at;
23             s[a[i + 1]] = at;
24         }
25         else {
26             s[a[i]] = at;
27             s[a[i + 1]] = at + 1;
28             at++;
29         }
30     }
31
32     for(int i = 1; i < s.size(); i++) {
33         if(s[i] > 'z' || s[i] < 'a') {
34             cout << -1 << '\n';
35             return;
36         }
37     }
38
39     for(int i = 1; i < s.size(); i++) {
40         cout << s[i];
41     }
42     cout << '\n';
43 }
```

8.9 Longest Palindrome

Given a string, your task is to determine the longest palindromic substring of the string. For example, the longest palindrome in aybabtu is bab.

Input
The only input line contains a string of length n. Each character is one of a–z.

Output
Print the longest palindrome in the string. If there are several solutions, you may print any of them.

Constraints
- $1 \le n \le 10^6$

```
1 array<vi, 2> manacher(const string& s) {
2     int n = sz(s);
3     array<vi,2> p = {vi(n+1), vi(n)};
4     rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
5         int t = r-i+!z;
6         if (i<r) p[z][i] = min(t, p[z][l+t]);
7         int L = i-p[z][i], R = i+p[z][i]-!z;
8         while (L>=1 && R+1<n && s[L-1] == s[R+1])
```

```
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}

void solve(){
    string s; cin >> s;
    auto ans = manacher(s);

    int q[2] = {-1, -1}, pos[2] = {-1, -1};
    for(int j = 0; j < 2; j++) {
        for(int i = 0; i < ans[j].size(); i++) {
            if(q[j] < ans[j][i]) {
                q[j] = ans[j][i];
                pos[j] = i;
            }
        }
    }

    int bst = 0;
    if(q[0] * 2 < q[1] * 2 + 1) bst = 1;

    if(pos[bst] == 0) {
        cout << s[0] << '\n';
        return;
    }

    for(int i = pos[bst] - q[bst]; i < pos[bst] + q[bst] +
         bst; i++) {
        cout << s[i];
    }
    cout << '\n';

}
```

8.10 Minimal Rotation

A rotation of a string can be generated by moving characters one after another from beginning to end. For example, the rotations of acab are acab, caba, abac, and baca. Your task is to determine the lexicographically minimal rotation of a string.

Input
The only input line contains a string of length n. Each character is one of a–z.

Output
Print the lexicographically minimal rotation.

Constraints
- $1 \leq n \leq 10^6$

```
const int alphabet = 26;

class SuffixAutomaton{
private:
  struct State{
    int link, len;
    vector<int> next;
    State() : link(-1), len(0){
      next.resize(alphabet, -1);
    }
  };

  vector<State> st;
  int last;

  int create(){
    st.emplace_back();
    return (int)st.size() - 1;
  }
public:
  SuffixAutomaton( string &s ){
    last = create();
    for( int i = 0, cur = 0; i < s.size(); i++, last = cur ){
      cur = create();
      st[cur].len = st[last].len + 1;

      int p = last, c = s[i] - 'a';
      while( p != -1 && st[p].next[c] == -1 ){
        st[p].next[c] = cur;
        p = st[p].link;
      }

      if( p == -1 ){
        st[cur].link = 0;
        continue;
      }

      int q = st[p].next[c];
      if( st[p].len + 1 == st[q].len ){
        st[cur].link = q;
        continue;
      }

      int clone = create();
      st[clone] = st[q];
      st[clone].len = st[p].len + 1;

      while( p != -1 && st[p].next[c] == q ){
        st[p].next[c] = clone;
        p = st[p].link;
      }

      st[cur].link = st[q].link = clone;
    }
  }

  string minimum_with_lenght( int n ){
    string resp;
    for( int state = 0, i = 0; i < n; i++ ){
      for( int c = 0; c < alphabet; c++ ) if( st[state].next[
           c] != -1 ){
        resp.push_back(c + 'a');
        state = st[state].next[c];
        break;
      }
    }
    return resp;
  }
};

int main(){
  string s; cin >> s;
  s += s;

  SuffixAutomaton aut(s);

  cout << aut.minimum_with_lenght((int)s.size()/2) << endl;
}
```

8.11 Palindrome Queries

You are given a string that consists of n characters between a–z. The positions of the string are indexed $1, 2, \ldots, n$. Your task is to process m operations of the following types:
Change the character at position k to x Check if the substring from position a to position b is a palindrome

Input
The first input line has two integers n and m: the length of the string and the number of operations. The next line has a string that consists of n characters. Finally, there are m lines that describe the operations. Each line is of the form "1 k x" or "2 a b".

Output
For each operation 2, print YES if the substring is a palindrome and NO otherwise.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

```cpp
using ll = long long;

const int mod = 1e9 + 7;
const int base = 43;
const int maxn = 2e5 + 10;

struct Node{
  ll hash, inv_hash, pot;
  Node( char c = 'a' ) : pot(base), hash(c - 'a'), inv_hash(c - 'a') {}

  Node operator + ( Node n ){
    Node resp;
    resp.hash = (hash + pot*n.hash)%mod;
    resp.inv_hash = (n.inv_hash + n.pot*inv_hash)%mod;
    resp.pot = (pot*n.pot)%mod;

    return resp;
  }
} seg[4*maxn];

void build( int pos, int ini, int fim, string &s ){

  if( ini == fim ){
    seg[pos] = Node( s[ini] );
    return;
  }
  int l = 2*pos, r = 2*pos + 1, mid = (ini + fim)/2;
  build( l, ini, mid, s); build( r, mid + 1, fim, s );
  seg[pos] = seg[l] + seg[r];
}

void update( int pos, int ini, int fim, int id, char c ){
  if( ini > id || id > fim ) return;
  if( ini == fim ){
    seg[pos] = Node(c);
    return;
  }
  int l = 2*pos, r = 2*pos + 1, mid = (ini + fim)/2;
  update( l, ini, mid, id, c ); update( r, mid + 1, fim, id, c );
  seg[pos] = seg[l] + seg[r];
}

Node query( int pos, int ini, int fim, int ki, int kf ){
  if( ki > fim || ini > kf ) return seg[0];
  if( ki <= ini && fim <= kf ) return seg[pos];
  int l = 2*pos, r = 2*pos + 1, mid = (ini + fim)/2;
  return query( l, ini, mid, ki, kf ) + query( r, mid + 1, fim, ki, kf );
}

void update( int id, char c, int n ){
  update( 1, 0, n - 1, id, c );
}

bool query( int l, int r, int n ){
  Node x = query( 1, 0, n - 1, l, r );
  return x.hash == x.inv_hash;
}

int main(){
  int n, q; cin >> n >> q;
  string s; cin >> s;

  build( 1, 0, n - 1, s );
  seg[0].pot = 1;

  while( q-- ){
    int t; cin >> t;
    if( t == 1 ){
      int id; char c; cin >> id >> c;
      update( id - 1, c, n );
    }
    else{
      int l, r; cin >> l >> r;
      cout << (( query( l - 1, r - 1, n ) ) ? "YES" : "NO") << '\n';
    }
  }
}
```

8.12 Pattern Positions

Given a string and patterns, find for each pattern the first position (1-indexed) where it appears in the string.

Input
The first input line has a string of length n. The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns. The string and the patterns consist of characters a–z.

Output
Print the first position for each pattern (or -1 if it does not appear at all).

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq k \leq 5 \cdot 10^5$ • the total length of the patterns is at most $5 \cdot 10^5$

```cpp
using pii = pair<int, int>;

const int alphabet = 26;

class AhoCorasick{
private:
  struct Node{
    int suffix_link, output, pai, pai_ch;
    vector<int> go, adj;

    Node( int pai = 0, int pai_ch = 0 ) : pai(pai), pai_ch(pai_ch), suffix_link(-1), output(false){
      go = adj = vector<int>( alphabet, -1 );
    };
  };
  vector<Node> t;

  int create( int pai, int pai_ch ){
    t.emplace_back( pai, pai_ch );
    return (int)t.size() - 1;
  }

  int get_adj( int node, int c ){
    if( t[node].adj[c] == -1 ){ int aux = create( node, c ); t[node].adj[c] = aux; }
    return t[node].adj[c];
  }
public:
  AhoCorasick(){ create(0, 0); }

  int add_string( string &s ){
    int node = 0;
    for( char c : s ) node = get_adj( node, c - 'a' );
    t[node].output = true;
    return node;
  }

  int suffix_link( int node ){
    if( t[node].suffix_link != -1 ) return t[node].suffix_link;
    if( node == 0 || t[node].pai == 0 ) return t[node].suffix_link = 0;
    return t[node].suffix_link = go( suffix_link(t[node].pai), t[node].pai_ch );
  }

  int go( int node, int c ){
    if( t[node].go[c] != -1 ) return t[node].go[c];
    if( t[node].adj[c] != -1 ) return t[node].go[c] = t[node].adj[c];
    return t[node].go[c] = (( node == 0 ) ? 0 : go( suffix_link(node), c ) );
  }

  int size(){ return t.size(); }
};

int main(){
  string s; cin >> s;
  int q; cin >> q;

  AhoCorasick aho;

  vector<pii> queries(q);

  for( int i = 0; i < q; i++ ){
    string t; cin >> t;
    queries[i] = pii(aho.add_string(t), t.size());
  }

  vector<int> marc(aho.size(), -1e9 );

  int node = 0;
  for( int i = 0; i < s.size(); i++ ){
    node = aho.go( node, s[i] - 'a' );
    for( int cur = node; marc[cur] == -1e9; cur = aho.suffix_link(cur) )
      marc[cur] = i;
  }

  for( auto [id, tam] : queries ) cout << max( -1, marc[id] - tam + 2 ) << endl;
```

```
74 }
```

8.13 Repeating Substring

A repeating substring is a substring that occurs in two (or more) locations in the string. Your task is to find the longest repeating substring in a given string.

Input
The only input line has a string of length n that consists of characters a–z.

Output
Print the longest repeating substring. If there are several possibilities, you can print any of them. If there is no repeating substring, print -1.

Constraints
• $1 \le n \le 10^5$

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) {
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
             = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p -
                     1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                    s[i + k] == s[j + k]; k++);
    }
};

void solve() {
    string s; cin >> s;
    SuffixArray sa(s);

    int tot = 0; int loc = 0;
    for(int i = 0; i < sz(sa.lcp); i++) {
        tot = max(tot, sa.lcp[i]);
        if(tot == sa.lcp[i]) loc = sa.sa[i];
    }

    if(tot == 0) {
        cout << -1 << '\n';
        return;
    }

    for(int i = 0; i < tot; i++) {
        cout << s[i + loc];
    }
    cout << '\n';
```

```
43 }
```

8.14 Required Substring

Your task is to calculate the number of strings of length n having a given pattern of length m as their substring. All strings consist of characters A–Z.

Input
The first input line has an integer n: the length of the final string. The second line has a pattern of length m.

Output
Print the number of strings modulo $10^9 + 7$.

Constraints
• $1 \le n \le 1000$ • $1 \le m \le 100$

```
const int alphabet = 26;
const int mod = 1e9 + 7;

vector<int> prefix_function( string &s ){
  int n = s.size();
  vector<int> pi(n);
  for( int i = 1; i < n; i++ ){
    int j = pi[i - 1];
    while( j > 0 && s[i] != s[j] ) j = pi[j - 1];
    pi[i] = j + (int)(s[i] == s[j]);
  }
  return pi;
}

vector<vector<int>> build_automaton( string &s ){
  s += '#';
  vector<int> pi = prefix_function(s);

  int n = s.size();
  vector<vector<int>> aut( n, vector<int>(alphabet));
  for( int i = 0; i < n; i++ ){
    for( int c = 0; c < alphabet; c++ ){
      if( i > 0 && s[i] - 'A' != c ) aut[i][c] = aut[pi[i -
           1]][c];
      else aut[i][c] = i + (int)(s[i] - 'A' == c);
    }
  }
  return aut;
}

int main(){
  int n; cin >> n;
  string s; cin >> s;
  vector<vector<int>> aut = build_automaton(s);
  vector<vector<int>> dp(2, vector<int>(s.size()));

  dp[0][0] = 1;
  for( int i = 0; i < n; i++ ){
    vector<vector<int>> next_dp(2, vector<int>(s.size()));
    for( int state = 0; state < s.size(); state++ ){
      for( int c = 0; c < alphabet; c++ ){
        int new_state = aut[state][c];
        next_dp[1][new_state] += dp[1][state];
        next_dp[1][new_state] %= mod;

        next_dp[(new_state == (int)s.size() - 1)][new_state]
             += dp[0][state];
        next_dp[(new_state == (int)s.size() - 1)][new_state]
             %= mod;
      }
    }
    swap( dp, next_dp );
  }

  cout << accumulate( dp[1].begin(), dp[1].end(), 0LL )%mod
       << endl;
}
```

8.15 String Functions

We consider a string of n characters, indexed $1, 2, \ldots, n$. Your task is to calculate all values of the following functions:
• $z(i)$ denotes the maximum length of a substring that begins at position i and is a prefix of the string. In addition, $z(1) = 0$.
• $\pi(i)$ denotes the maximum length of a substring that ends at position i, is a prefix of the string, and whose length is at most $i - 1$.
Note that the function z is used in the Z-algorithm, and the function π is used in the KMP algorithm.

Input
The only input line has a string of length n. Each character is between a–z.

Output
Print two lines: first the values of the z function, and then the values of the π function.

Constraints
• $1 \le n \le 10^6$

```
void pi(const string& s) {
    vi p(sz(s));
    for(int i = 1; i < sz(s); i++) {
        int g = p[i - 1];
        while(g && s[i] != s[g]) g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }

    for(auto x : p) cout << x << ' ';
    cout << '\n';
}

void Z(const string& s) {
    vi z(sz(s));
    int l = -1, r = -1;
    for(int i = 1; i < sz(s); i++) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while(i + z[i] < sz(s) && s[i + z[i]] == s[z[i]]) z[i
             ]++;
        if(i + z[i] > r) l = i, r = i + z[i];
    }
    for(auto x : z) cout << x << ' ';
    cout << '\n';
}

void solve() {
    string s; cin >> s;
    Z(s); pi(s);
}
```

8.16 String Matching

Given a string and a pattern, your task is to count the number of positions where the pattern occurs in the string.

Input
The first input line has a string of length n, and the second input line has a pattern of length m. Both of them consist of characters a–z.

Output
Print one integer: the number of occurrences.

Constraints
• $1 \leq n, m \leq 10^6$

```
const int maxn = 1e6 + 5, inf = 2e9, P = 1e9 + 7;
const ll  linf = 4e18, M = 2305843009213693951;

ll pot[maxn];

ll hsh(string s, int n){
    ll h = 0;
    for(int i = 0; i < n; i++){
        h = (h + (__int128 (pot[n - 1 - i]) * s[i]) % M) % M;
    }

    return h;
}

void calcpot(){
    pot[0] = 1;
    for(int i = 1; i < maxn; i++){
        pot[i] = (__int128 (pot[i - 1]) * P) % M;
    }
}

void solve() {
    string s, t; cin >> s >> t;
    int tot = 0;

    if(t.size() > s.size()){
        cout << "0\n";
        return;
    }

    ll at = hsh(s, t.size()), h = hsh(t, t.size());

    if(at == h) tot++;
    for(int i = 0; i < (int)s.size() - (int)t.size(); i++){
        at = ((__int128 (at) * pot[1] - __int128 (s[i]) * pot
             [t.size()] + s[i + t.size()]) % M + M) % M;
        if(at == h) tot++;
    }

    cout << tot << '\n';
}

int main() {
    calcpot();
    solve();
    return 0;
}
```

8.17 String Transform

Consider the following string transformation:
append the character # to the string (we assume that # is lexicographically smaller than all other characters of the string) generate all rotations of the string sort the rotations in increasing order based on this order, construct a new string that contains the last character of each rotation
For example, the string babc becomes babc#. Then, the sorted list of rotations is #babc, abc#b, babc#, bc#ba, and c#bab. This yields a string cb#ab.

Input
The only input line contains the transformed string of length $n + 1$. Each character of the original string is one of a–z.

Output
Print the original string of length n.

Constraints
• $1 \leq n \leq 10^6$

```
void solve() {
    string s;
    vector<pair<char, int>> t;
    cin >> s;

    for(int i = 0; i < sz(s); i++) {
        t.push_back({s[i], i});
    }
    sort(t.begin(), t.end());

    char c; int p = 0;
    for(int i = 0; i < sz(s); i++) {
        tie(c, p) = t[p];
        if(c != '#') cout << c;
    }
    cout << '\n';
}
```

8.18 Substring Distribution

You are given a string of length n. For every integer between $1 \ldots n$ you need to print the number of distinct substrings of that length.

Input
The only input line has a string of length n that consists of characters a–z.

Output
For each integer between $1 \ldots n$ print the number of distinct substrings of that length.

Constraints
• $1 \leq n \leq 10^5$

```
const int alphabet = 26;

class SuffixAutomaton{
public:
  struct State{
    int link, len;
    vector<int> adj;
    State( int link = -1, int len = 0 ) : link(link), len(len
         ) {
      adj.resize(alphabet, -1);
    }
  };
  vector<State> st;

  int last;

  int create(){
    st.emplace_back();
    return (int)st.size() - 1;
  }

  SuffixAutomaton( const string &s ){
    last = create();
    for( int i = 0, cur = 0; i < (int)s.size(); i++, last =
         cur ){
      cur = create();
      st[cur].len = st[last].len + 1;
      int p = last, c = s[i] - 'a';
      while( p != -1 && st[p].adj[c] == -1 ){
        st[p].adj[c] = cur;
        p = st[p].link;
      }

      if( p == -1 ){
        st[cur].link = 0;
        continue;
      }

      int q = st[p].adj[c];
      if( st[p].len + 1 == st[q].len ){
        st[cur].link = q;
        continue;
      }

      int clone = create();
      st[clone] = st[q];
      st[clone].len = st[p].len + 1;

      while( p != -1 && st[p].adj[c] == q ){
        st[p].adj[c] = clone;
        p = st[p].link;
      }

      st[q].link = st[cur].link = clone;
    }
  }
};

int main(){
  string s; cin >> s;

  SuffixAutomaton aut(s);

  vector<int> resp(s.size());

  for( int i = 1; i < aut.st.size(); i++ ){
    resp[aut.st[aut.st[i].link].len]++;
    if( aut.st[i].len < s.size() ) resp[aut.st[i].len]--;
  }

  for( int i = 0; i < resp.size(); i++ ){
    if(i) resp[i] += resp[i - 1];
    cout << resp[i] << "␣";
  } cout << endl;
```

```
73 }
```

8.19 Substring Order I

You are given a string of length n. If all of its distinct substrings are ordered lexicographically, what is the kth smallest of them?

Input
The first input line has a string of length n that consists of characters a–z. The second input line has an integer k.

Output
Print the kth smallest distinct substring in lexicographical order.

Constraints
• $1 \le n \le 10^5$ • $1 \le k \le \frac{n(n+1)}{2}$ • It is guaranteed that k does not exceed the number of distinct substrings.

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
             = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p -
                     1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                    s[i + k] == s[j + k]; k++);
    }
};

void solve() {
    string s; cin >> s;
    int n = sz(s);
    ll k; cin >> k;

    SuffixArray SA(s);
    ll cnt = 0;

    string ans;

    for(int i = 0; i <= n; i++) {
        if(cnt + (n - SA.sa[i]) - SA.lcp[i] < k) {
            cnt += (n - SA.sa[i]) - SA.lcp[i];
            continue;
        }

        ll posfim = SA.lcp[i] + k - cnt;

        for(int j = SA.sa[i]; j < SA.sa[i] + posfim; j++) ans
             += s[j];
        break;
    }

    cout << ans << '\n';

}
```

8.20 Substring Order II

You are given a string of length n. If all of its substrings (not necessarily distinct) are ordered lexicographically, what is the kth smallest of them?

Input
The first input line has a string of length n that consists of characters a–z. The second input line has an integer k.

Output
Print the kth smallest substring in lexicographical order.

Constraints
• $1 \le n \le 10^5$ • $1 \le k \le \frac{n(n+1)}{2}$

```
const int MAX = 1e5 + 10;

namespace sam {
    int cur, sz, len[2*MAX], link[2*MAX], acc[2*MAX];
    ll cnt[2 * MAX];
    int nxt[2*MAX][26];

    void add(int c) {
        int at = cur;
        cnt[sz] = 1;
        len[sz] = len[cur]+1, cur = sz++;
        while (at != -1 and !nxt[at][c]) nxt[at][c] = cur, at
             = link[at];
        if (at == -1) { link[cur] = 0; return; }
        int q = nxt[at][c];
        if (len[q] == len[at]+1) { link[cur] = q; return; }
        int qq = sz++;
        cnt[qq] = 0;
        len[qq] = len[at]+1, link[qq] = link[q];
        for (int i = 0; i < 26; i++) nxt[qq][i] = nxt[q][i];
        while (at != -1 and nxt[at][c] == q) nxt[at][c] = qq,
             at = link[at];
        link[cur] = link[q] = qq;
    }
    void build(string& s) {
        cur = 0, sz = 0, len[0] = 0, link[0] = -1, sz++;
        for (auto i : s) add(i-'a');
        int at = cur;
        while (at) acc[at] = 1, at = link[at];
    }

    void calc_rep() {
        vector<pair<int, int>> v;
        for (int i = 1; i < sz; i++) {
            v.push_back({len[i], i});
        }
        sort(v.rbegin(), v.rend());

        for (auto [l, u] : v) {
            if (link[u] != -1) {
                cnt[link[u]] += cnt[u];
            }
        }
        cnt[0] = 0;
    }

    ll dp[2*MAX];
    ll paths(int i) {
        auto& x = dp[i];
        if (x) return x;
        x = cnt[i];
        for (int j = 0; j < 26; j++) if (nxt[i][j]) x +=
             paths(nxt[i][j]);
        return x;
    }
    void kth_substring(ll k, int at=0) { // k=1 : menor
         substring lexicog.
        if(at == 0) calc_rep();
        if(k <= cnt[at]) {
            cout << '\n';
            return;
        }
        k -= cnt[at];
        for (int i = 0; i < 26; i++) if (k and nxt[at][i]) {
            if (paths(nxt[at][i]) >= k) {
                cout << char('a'+i);
                kth_substring(k, nxt[at][i]);
                return;
            }
            k -= paths(nxt[at][i]);
        }
    }
};

void solve() {
    string s; cin >> s;
    int n = sz(s);
    ll k; cin >> k;

    sam::build(s);
    sam::kth_substring(k);
}
```

8.21 Word Combinations

You are given a string of length n and a dictionary containing k words. In how many ways can you create the string using the words?

Input
The first input line has a string containing n characters between a–z. The second line has an integer k: the number of words in the dictionary. Finally there are k lines describing the words. Each word is unique and consists of characters a–z.

Output
Print the number of ways modulo $10^9 + 7$.

Constraints
• $1 \le n \le 5000$ • $1 \le k \le 10^5$ • the total length of the words is at most 10^6

```
const int alphabet = 26;
const int mod = 1e9 + 7;
```

```
3
4  class Trie{
5  private:
6    struct Node{
7      int output;
8      vector<int> adj;
9      Node() : output(false) {
10       adj.resize(alphabet, -1);
11     }
12   };
13   vector<Node> t;
14
15   int create(){
16     t.emplace_back();
17     return (int)t.size() - 1;
18   }
19
20   int get_adj( int node, int c ){
21     if( t[node].adj[c] == -1 ){ int aux = create(); t[node].
           adj[c] = aux; }
22     return t[node].adj[c];
23   }
24
25 public:
26
27   Trie(){ create(); }
28
29   void add_string( string &s ){
30     int node = 0;
31     for( char c : s ) node = get_adj( node, c - 'a' );
32     t[node].output = true;
33   }
34
35   int go( int node, int c ){
36     return t[node].adj[c];
37   }
38
39   bool is_output( int node ){
40     return t[node].output;
41   }
42 };
43
44 int main(){
45   string s; cin >> s;
46   int n = s.size();
47
48   Trie trie;
49
50   int m; cin >> m;
51   while( m-- ){
52     string t; cin >> t;
53     trie.add_string(t);
54   }
55
56   vector<int> dp(n + 1);
57   dp[0] = 1;
58
59   for( int i = 0; i < n; i++ ){
60     for( int j = i, node = 0; j < n; j++ ){
61       node = trie.go( node, s[j] - 'a' );
62       if( node == -1 ) break;
63       if( trie.is_output(node) ){
64         dp[j + 1] += dp[i];
65         dp[j + 1] %= mod;
66       }
67     }
68   }
69
70   cout << dp.back() << endl;
71 }
```

9 Geometry

9.1 All Manhattan Distances

Given a set of points, calculate the sum of all Manhattan distances between two point pairs.

Input
The first line has an integer n: the number of points. The following n lines describe the points. Each line has two integers x and y. You can assume that each point is distinct.

Output
Print the sum of all Manhattan distances.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $-10^9 \leq x, y \leq 10^9$

```cpp
using i128 = __int128;
using ll = long long;

// podemos tratar x e y separadamente
// x1 x2 x3 x4 ... xk
// k -> (xk - xk-1) + (xk - xk-2) + ... + (xk - x1) = k * xk
      - pref(k)

void print_i128(i128 x) {
    if (x == 0) { cout << '0'; return; }
    if (x < 0) { cout << '-'; x = -x; }
    string s;
    while (x > 0) {
        s.push_back('0' + int(x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}

void solve(){
    int n; cin >> n;
    vector<int> px(n + 1), py(n + 1);
    for(int i = 1; i <= n; i++){
        cin >> px[i] >> py[i];
    }
    sort(px.begin() + 1, px.end());
    sort(py.begin() + 1, py.end());

    vector<i128> prefx(n + 1), prefy(n + 1);
    for(int i = 1; i <= n; i++){
        prefx[i] = prefx[i - 1] + px[i];
        prefy[i] = prefy[i - 1] + py[i];
    }

    i128 ans = 0;

    for(int i = 1; i <= n; i++){
        ans += (ll)i * px[i] - prefx[i];
        ans += (ll)i * py[i] - prefy[i];
    }

    print_i128(ans);
    cout << '\n';
}
```

9.2 Area Of Rectangles

Given n rectangles, your task is to determine the total area of their union.

Input
The first line has an integer n: the number of rectangles. After that, there are n lines describing the rectangles. Each line has four integers x_1, y_1, x_2 and y_2: a rectangle begins at point (x_1, y_1) and ends at point (x_2, y_2).

Output
Print the total area covered by the rectangles.

Constraints
• $1 \leq n \leq 10^5$ • $-10^6 \leq x_1 < x_2 \leq 10^6$ • $-10^6 \leq y_1 < y_2 \leq 10^6$

```cpp
const int maxn = 1e6 + 10;

using ll = long long;

struct Event{
  int x, y1, y2, val;
  Event( int x, int y1, int y2, int val ) : x(x), y1(y1), y2(
       y2), val(val) {}

  bool operator < ( Event e ){
    return x < e.x;
  }
};

class SegTree{
private:
  struct Node{
    int mini, qtd, lazy;
    Node( int mini = 0, int qtd = 0 ) : mini(mini), qtd(qtd),
         lazy(0) {}

    Node operator + ( Node n ){
      if( mini == n.mini ) return Node( mini, qtd + n.qtd );
      if( mini < n.mini ) return *this;
      return n;
    }
  };

  int minx, maxx;
  vector<Node> seg;

  void build( int pos, int ini, int fim ){
    if( ini == fim ){ seg[pos] = Node( 0, 1 ); return; }
    int l = 2*pos, r = 2*pos + 1, mid = (ini + fim)>>1;
    build( l, ini, mid ); build( r, mid + 1, fim );
    seg[pos] = seg[l] + seg[r];
  }

  void refresh( int pos, int ini, int fim ){
    if( seg[pos].lazy == 0 ) return;
    int x = seg[pos].lazy; seg[pos].lazy = 0;
    seg[pos].mini += x;
    if( ini == fim ) return;
    int l = 2*pos, r = 2*pos + 1;
    seg[l].lazy += x;
    seg[r].lazy += x;
  }

  void update( int pos, int ini, int fim, int ki, int kf, int
       val ){
    refresh( pos, ini, fim );
    if( ki > fim || ini > kf ) return;
    if( ki <= ini && fim <= kf ){
      seg[pos].lazy = val;
      refresh( pos, ini, fim );
      return;
    }
    int l = 2*pos, r = 2*pos + 1, mid = (ini + fim)>>1;
    update( l, ini, mid, ki, kf, val ); update( r, mid + 1,
         fim, ki, kf, val );
    seg[pos] = seg[l] + seg[r];
  }
public:
  SegTree( int minx, int maxx ) : minx(minx), maxx(maxx) {
    seg.resize(4*(maxx - minx + 1));
    build( 1, minx, maxx );
  }

  void update( int l, int r, int val ){
    update( 1, minx, maxx, l, r, val );
  }

  ll query(){
    return maxx - minx + 1 - (( seg[1].mini == 0 ) ? seg[1].
         qtd : 0 );
  }
};

int main(){
  int n; cin >> n;

  SegTree seg( -maxn, maxn );

  vector<Event> line;
  while( n-- ){
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    if( x1 > x2 ) swap( x1, x2 );
    if( y1 > y2 ) swap( y1, y2 );

    line.emplace_back( x1, y1, y2 - 1, 1 );
    line.emplace_back( x2, y1, y2 - 1, -1 );
  }

  sort( line.begin(), line.end() );

  ll area = 0;
  int prev_x = -maxn;
  for( auto evento : line ){
    area += seg.query()*(evento.x - prev_x);
    seg.update( evento.y1, evento.y2, evento.val );
    prev_x = evento.x;
  }

  cout << area << endl;
}
```

9.3 Convex Hull

Given a set of n points in the two-dimensional plane, your task is to determine the convex hull of the points.

Input
The first input line has an integer n: the number of points. After this, there are n lines that describe the points. Each line has two integers x and y: the coordinates of a point. You may assume that each point is distinct, and the area of the hull is positive.

Output
First print an integer k: the number of points in the convex hull. After this, print k lines that describe the points. You can print the points in any order. Print all points that lie on the convex hull.

Constraints
• $3 \le n \le 2 \cdot 10^5$ • $-10^9 \le x, y \le 10^9$

```cpp
using ll = long long;

struct Point{
  ll x, y;
  Point( ll x = 0, ll y = 0 ) : x(x), y(y) {}

  ll operator ^ ( Point p ){
    return x*p.y - y*p.x;
  }

  Point operator - ( Point p ){
    return Point( x - p.x, y - p.y );
  }

  bool operator < ( Point p ) {
    if( x == p.x ) return y < p.y;
    return x < p.x;
  }
};

vector<Point> build( vector<Point> &points ){
  vector<Point> upper_hull;
  for( auto p : points ){
    while( upper_hull.size() > 1 ){
      Point last = upper_hull.back();
      Point sec_last = upper_hull[upper_hull.size() - 2];
      if( ((last - sec_last)^(p - sec_last)) > 0 ) upper_hull
          .pop_back();
      else break;
    }

    upper_hull.push_back(p);
  }

  vector<Point> lower_hull;
  for( auto p : points ){
    while( lower_hull.size() > 1 ){
      Point last = lower_hull.back();
      Point sec_last = lower_hull[lower_hull.size() - 2];
      if( ((last - sec_last)^(p - sec_last)) < 0 ) lower_hull
          .pop_back();
      else break;
    }

    lower_hull.push_back(p);
  }

  lower_hull.insert(lower_hull.end(), upper_hull.begin() + 1,
       upper_hull.end() - 1 );
  return lower_hull;
}

int main(){
  int n; cin >> n;

  vector<Point> points(n);
  for( auto &p : points ) cin >> p.x >> p.y;
  sort( points.begin(), points.end() );

  vector<Point> convex_hull = build(points);

  cout << convex_hull.size() << endl;
  for( auto p : convex_hull ) cout << p.x << " " << p.y <<
       endl;
}
```

9.4 Intersection Points

Given n horizontal and vertical line segments, your task is to calculate the number of their intersection points. You can assume that no parallel line segments intersect, and no endpoint of a line segment is an intersection point.

Input
The first line has an integer n: the number of line segments. Then there are n lines describing the line segments. Each line has four integers: x_1, y_1, x_2 and y_2: a line segment begins at point (x_1, y_1) and ends at point (x_2, y_2).

Output
Print the number of intersection points.

Constraints
• $1 \le n \le 10^5$ • $-10^6 \le x_1 \le x_2 \le 10^6$ • $-10^6 \le y_1 \le y_2 \le 10^6$ • $(x_1, y_1) \ne (x_2, y_2)$

```cpp
using ll = long long;

const int maxn = 1e6 + 10;

struct Event{
  int x, y1, y2;
  Event( int x, int y1, int y2 ) : x(x), y1(y1), y2(y2) {}

  bool operator < ( Event e ){
    return x < e.x;
  }
};

int main(){
  int n; cin >> n;

  vector<Event> line;
  for( int i = 0; i < n; i++ ){
    int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
    x1 += maxn;
    x2 += maxn;
    y1 += maxn;
    y2 += maxn;
    if( y1 > y2 ) swap( y1, y2 );
    if( x1 > x2 ) swap( x1, x2 );

    if( x1 == x2 ) line.emplace_back( x1, y1, y2 );
    else{
      line.emplace_back( x1, y1, 1 );
      line.emplace_back( x2, y1, -1 );
    }
  }

  sort( line.begin(), line.end() );

  vector<int> BIT( 2*maxn );

  auto update = [&]( int i, int val ){
    for(; i < BIT.size(); i += i&-i ) BIT[i] += val;
  };

  auto query = [&]( int i ){
    int resp = 0;
    for(; i > 0; i -= i&-i ) resp += BIT[i];
    return resp;
  };

  ll resp = 0;

  for( auto evento : line ){
    if( abs(evento.y2) == 1 ) update( evento.y1, evento.y2 );
    else resp += query( evento.y2 ) - query( evento.y1 );
  }

  cout << resp << endl;
}
```

9.5 Line Segment Intersection

There are two line segments: the first goes through the points (x_1, y_1) and (x_2, y_2), and the second goes through the points (x_3, y_3) and (x_4, y_4). Your task is to determine if the line segments intersect, i.e., they have at least one common point.

Input
The first input line has an integer t: the number of tests. After this, there are t lines that describe the tests. Each line has eight integers x_1, y_1, x_2, y_2, x_3, y_3, x_4 and y_4.

Output
For each test, print "YES" if the line segments intersect and "NO" otherwise.

Constraints
• $1 \le t \le 10^5$ • $-10^9 \le x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4 \le 10^9$ • $(x_1, y_1) \ne (x_2, y_2)$ • $(x_3, y_3) \ne (x_4, y_4)$

```cpp
struct P {
    ll x, y;
    void read() { cin >> x >> y; }
    ll operator*(P b) const {
        return 1LL*x*b.y - 1LL*b.x*y;
    }
    P operator-(P b) {
        return P{x-b.x, y-b.y};
    }
    void operator-=(P b) {
```

```
        *this = *this - b;
    }
    ll triangle(P b, P c) {
        return (b - *this) * (c - *this);
    }
    bool operator<(P b) {
        return make_pair(x, y) < make_pair(b.x, b.y);
    }
    void show() {
        cout << x << ' ' << y << endl;
    }
};

// 1 = esquerda, -1 = direita, 0 = na reta
int local(P a, P b, P c){
    ll k = (c.y - a.y) * (b.x - a.x);
    ll l = (c.x - a.x) * (b.y - a.y);
    if (k > l)
        return 1;
    else if (k < l)
        return -1;
    else
        return 0;
}

bool seg_int(P a, P b, P c, P d){
    //lugar relativo de a e b por cd e de c e d por ab
    int k1 = local(c, d, a);
    int k2 = local(c, d, b);
    int k3 = local(a, b, c);
    int k4 = local(a, b, d);

    if(!k1 && !k2 && !k3 && !k4){
        //normalizar por a
        b-=a; c-=a; d-=a;
        ll z3, z4;

        //analisar c e d entre 0 e b
        if(b.x*c.x + b.y*c.y < 0) z3 = -1;
        else if(b.x*c.x + b.y*c.y < b.x*b.x + b.y*b.y) z3 =
             0;
        else z3 = 1;

        if(b.x*d.x + b.y*d.y < 0) z4 = -1;
        else if(b.x*d.x + b.y*d.y < b.x*b.x + b.y*b.y) z4 =
             0;
        else z4 = 1;

        return z3 * z4 != 1;
    }

    return k1*k2 <= 0 && k3*k4 <= 0;
}

int main(){
    P a, b, c, d;
    int t;
    cin >> t;
    while (t--) {
        a.read(); b.read(); c.read(); d.read();
        if(seg_int(a, b, c, d)) cout << "YES\n";
        else cout << "NO\n";
    }
}
```

9.6 Line Segments Trace I

There are n line segments whose endpoints have integer coordinates. The left x-coordinate of each segment is 0 and the right x-coordinate is m. The slope of each segment is an integer. For each x-coordinate $0, 1, \ldots, m$, find the maximum point in any line segment.

Input
The first line has two integers n and m: the number of line segments and the maximum x-coordinate. The next n lines describe the line segments. Each line has two integers y_1 and y_2: there is a line segment between points $(0, y_1)$ and (m, y_2).

Output
Print $m+1$ integers: the maximum points for $x = 0, 1, \ldots, m$.

Constraints
• $1 \le n, m \le 10^5$ • $0 \le y_1, y_2 \le 10^9$

```
// Li Chao tree

const int maxn = 1e5;
const long long INF = 4e18;

ftype dot(point a, point b) {
    return (conj(a) * b).x();
}

ftype f(point a,  ftype x) {
    return dot(a, {x, 1});
}

point line[4 * maxn];

void gen() {
    for (int i = 0; i < 4 * maxn; i++) {
        line[i] = {0, INF};
    }
}

void add_line(point nw, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    bool lef = f(nw, l) < f(line[v], l);
    bool mid = f(nw, m) < f(line[v], m);
    if(mid) {
        swap(line[v], nw);
    }
    if(r - l == 1) {
        return;
    } else if(lef != mid) {
        add_line(nw, 2 * v, l, m);
    } else {
        add_line(nw, 2 * v + 1, m, r);
    }
}

ftype get(int x, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    if(r - l == 1) {
        return f(line[v], x);
    } else if(x < m) {
        return min(f(line[v], x), get(x, 2 * v, l, m));
    } else {
        return min(f(line[v], x), get(x, 2 * v + 1, m, r));
    }
}

void solve() {
    gen();
    int n, m; cin >> n >> m;
    for(int i = 0; i < n; i++) {
        int y1, y2; cin >> y1 >> y2;
        int a = (y2 - y1) / m, b = y1;
        add_line({-a, -b});
    }

    for(int i = 0; i <= m; i++)
        cout << -get(i) << ' ';
    cout << '\n';
}
```

9.7 Line Segments Trace II

There are n line segments whose endpoints have integer coordinates. Each x-coordinate is between 0 and m. The slope of each segment is an integer. For each x-coordinate $0, 1, \ldots, m$, find the maximum point in any line segment. If there is no segment at some point, the maximum is -1.

Input
The first line has two integers n and m: the number of line segments and the maximum x-coordinate. The next n lines describe the line segments. Each line has four integers x_1, y_1, x_2 and y_2: there is a line segment between points (x_1, y_1) and (x_2, y_2).

Output
Print $m+1$ integers: the maximum points for $x = 0, 1, \ldots, m$.

Constraints
• $1 \le n, m \le 10^5$ • $0 \le x_1 < x_2 \le m$ • $0 \le y_1, y_2 \le 10^9$

```
// Li Chao tree

const int maxn = 2e5;
const long long INF = 4e18;

ftype dot(point a, point b) {
    return (conj(a) * b).x();
}

ftype f(pair<point, pair<ll, ll>> a,  ftype x) {
    auto p = a.first;
    auto limits = a.second;
    if(x < limits.first || x > limits.second)
        return INF;
    return dot(p, {x, 1});
}

pair<point, pair<ll, ll>> line[4 * maxn];

void gen() {
    for (int i = 0; i < 4 * maxn; i++) {
        line[i] = {{0, INF}, {-INF, INF}};
    }
}
```

```
25
26 void add_line(pair<point, pair<ll, ll>> nw, int v = 1, int l
       = 0, int r = maxn) {
27     int m = (l + r) / 2;
28     int L = nw.second.first, R = nw.second.second;
29
30     if (R < l || r <= L) return;
31
32     if (L <= l && r <= R + 1) {
33         bool lef = f(nw, l) < f(line[v], l);
34         bool mid = f(nw, m) < f(line[v], m);
35         if (mid) swap(line[v], nw);
36
37         if (r - l == 1) return;
38         if (lef != mid)
39             add_line(nw, 2 * v, l, m);
40         else
41             add_line(nw, 2 * v + 1, m, r);
42     } else {
43         if (r - l == 1) return;
44         add_line(nw, 2 * v, l, m);
45         add_line(nw, 2 * v + 1, m, r);
46     }
47 }
48
49 ftype get(int x, int v = 1, int l = 0, int r = maxn) {
50     int m = (l + r) / 2;
51     if(r - l == 1) {
52         return f(line[v], x);
53     } else if(x < m) {
54         return min(f(line[v], x), get(x, 2 * v, l, m));
55     } else {
56         return min(f(line[v], x), get(x, 2 * v + 1, m, r));
57     }
58 }
59
60 void solve() {
61     gen();
62     int n, m; cin >> n >> m;
63     for(int i = 0; i < n; i++) {
64         int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
65         int a = (y2 - y1) / (x2 - x1);
66         int b = y1 - a * x1;
67         add_line({{-a, -b}, {x1, x2}});
68     }
69     for(int i = 0; i <= m; i++) {
70         ll ans = -get(i);
71         if(ans == -INF)
72             cout << -1 << ' ';
73         else
74             cout << ans << ' ';
75     }
76     cout << '\n';
77 }
```

9.8 Lines And Queries I

Your task is to efficiently process the following types of queries: Add a line $ax + b$ Find the maximum point in any line at position x

Input
The first line has an integer n: the number of queries. The following n lines describe the queries. The format of each line is either "1 a b" or "2 x". You may assume that the first query is of type 1.

Output
Print the answer for each query of type 2.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le a, b \le 10^9$ • $0 \le x \le 10^5$

```
1  // Li Chao tree
2
3  const int maxn = 2e5;
4  const long long INF = 4e18;
5
6  ftype dot(point a, point b) {
7      return (conj(a) * b).x();
8  }
9
10 ftype f(point a,  ftype x) {
11     return dot(a, {x, 1});
12 }
13
14 point line[4 * maxn];
15
16 void gen() {
17     for (int i = 0; i < 4 * maxn; i++) {
18         line[i] = {0, INF};
19     }
20 }
21
22 void add_line(point nw, int v = 1, int l = 0, int r = maxn) {
23     int m = (l + r) / 2;
24     bool lef = f(nw, l) < f(line[v], l);
25     bool mid = f(nw, m) < f(line[v], m);
26     if(mid) {
27         swap(line[v], nw);
28     }
29     if(r - l == 1) {
30         return;
31     } else if(lef != mid) {
32         add_line(nw, 2 * v, l, m);
33     } else {
34         add_line(nw, 2 * v + 1, m, r);
35     }
36 }
37
38 ftype get(int x, int v = 1, int l = 0, int r = maxn) {
39     int m = (l + r) / 2;
40     if(r - l == 1) {
41         return f(line[v], x);
42     } else if(x < m) {
43         return min(f(line[v], x), get(x, 2 * v, l, m));
44     } else {
45         return min(f(line[v], x), get(x, 2 * v + 1, m, r));
46     }
47 }
48
49 void solve() {
50     gen();
51     int n; cin >> n;
52     for(int i = 0; i < n; i++) {
53         int q; cin >> q;
54         if(q == 1) {
55             int a, b; cin >> a >> b;
56             add_line({-a, -b});
57         }
58         else {
59             int x; cin >> x;
60             cout << -get(x) << '\n';
61         }
62     }
63
64 }
```

9.9 Lines And Queries II

Your task is to efficiently process the following types of queries: Add a line $ax+b$ that is active in range $[l, r]$ Find the maximum point in any active line at position x

Input
The first line has an integer n: the number of queries. The following n lines describe the queries. The format of each line is either "1 a b l r" or "2 x".

Output
Print the answer for each query of type 2. If no line is active, print NO.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le a, b \le 10^9$ • $0 \le x \le 10^5$ • $0 \le l \le r \le 10^5$

```
1  // Li Chao tree
2
3  const int maxn = 2e5;
4  const long long INF = 4e18;
5
6  ftype dot(point a, point b) {
7      return (conj(a) * b).x();
8  }
9
10 ftype f(pair<point, pair<ll, ll>> a,  ftype x) {
11     auto p = a.first;
12     auto limits = a.second;
13     if(x < limits.first || x > limits.second)
14         return INF;
15     return dot(p, {x, 1});
16 }
17
18 pair<point, pair<ll, ll>> line[4 * maxn];
19
20 void gen() {
21     for (int i = 0; i < 4 * maxn; i++) {
22         line[i] = {{0, INF}, {-INF, INF}};
23     }
24 }
25
26 void add_line(pair<point, pair<ll, ll>> nw, int v = 1, int l
       = 0, int r = maxn) {
27     int m = (l + r) / 2;
28     int L = nw.second.first, R = nw.second.second;
29
30     if (R < l || r <= L) return;
31
32     if (L <= l && r <= R + 1) {
33         bool lef = f(nw, l) < f(line[v], l);
34         bool mid = f(nw, m) < f(line[v], m);
35         if (mid) swap(line[v], nw);
36
37         if (r - l == 1) return;
38         if (lef != mid)
39             add_line(nw, 2 * v, l, m);
40         else
41             add_line(nw, 2 * v + 1, m, r);
42     } else {
43         if (r - l == 1) return;
44         add_line(nw, 2 * v, l, m);
45         add_line(nw, 2 * v + 1, m, r);
46     }
```

```
}

ftype get(int x, int v = 1, int l = 0, int r = maxn) {
    int m = (l + r) / 2;
    if(r - l == 1) {
        return f(line[v], x);
    } else if(x < m) {
        return min(f(line[v], x), get(x, 2 * v, l, m));
    } else {
        return min(f(line[v], x), get(x, 2 * v + 1, m, r));
    }
}

void solve() {
    gen();
    int n; cin >> n;
    for(int i = 0; i < n; i++) {
        int q; cin >> q;
        if(q == 1) {
            int a, b; cin >> a >> b;
            int l, r; cin >> l >> r;
            add_line({{-a, -b}, {l, r}});
        }
        else {
            int x; cin >> x;
            ll ans = -get(x);
            if(ans == -INF)
                cout << "NO\n";
            else
                cout << ans << '\n';
        }
    }
}
```

9.10 Maximum Manhattan Distances

A set is initially empty and n points are added to it. Calculate the maximum Manhattan distance of two points after each addition.

Input
The first line has an integer n: the number of points. The following n lines describe the points. Each line has two integers x and y. You can assume that each point is distinct.

Output
After each addition, print the maximum distance.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le x, y \le 10^9$

```
void solve(){
    int n; cin >> n;
    set<int> px;
    set<int> py;
    for(int i = 0; i < n; i++){
        int x1, y1; cin >> x1 >> y1;
        // Chebyshev distance
        int x = x1 + y1;
        int y = x1 - y1;

        px.insert(x);
        py.insert(y);
        int ansx = *px.rbegin() - *px.begin();
        int ansy = *py.rbegin() - *py.begin();
        cout << max(ansx, ansy) << '\n';
    }
}
```

9.11 Minimum Euclidean Distance

Given a set of points in the two-dimensional plane, your task is to find the minimum Euclidean distance between two distinct points. The Euclidean distance of points (x_1, y_1) and (x_2, y_2) is $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

Input
The first input line has an integer n: the number of points. After this, there are n lines that describe the points. Each line has two integers x and y. You may assume that each point is distinct.

Output
Print one integer: d^2 where d is the minimum Euclidean distance (this ensures that the result is an integer).

Constraints
• $2 \le n \le 2 \cdot 10^5$ • $-10^9 \le x, y \le 10^9$

```
using ll = long long;

const ll inf = 8e18 + 10;

struct Point{
  ll x, y;
  Point( ll x = 0, ll y = 0 ) : x(x), y(y) {}

  ll squared_distance( Point p ){
    return (x - p.x)*(x - p.x) + (y - p.y)*(y - p.y);
  }

  bool operator < ( Point p ){
    return y < p.y;
  }
};

ll solve( ll xl, ll xr, vector<Point> &points ){
  if( points.size() <= 1 ) return inf;
  if( xl == xr ){
    ll resp = inf;
    for( int i = 1; i < points.size(); i++ ) resp = min( resp
        , points[i].squared_distance( points[i - 1] ) );
    return resp;
  }

  vector<Point> pl, pr;
  ll mid = ( xl + xr )>>1;
  for( auto p : points ){
    if( p.x <= mid ) pl.push_back(p);
    else pr.push_back(p);
  }

  ll resp = min( solve( xl, mid, pl ), solve( mid + 1, xr, pr
      ) );
  ll D = ceil(sqrt(resp));

  pl.clear(), pr.clear();

  for( auto p : points ) if( (p.x - mid)*(p.x - mid) <= resp
      ) {
    if( p.x <= mid ) pl.push_back(p);
    else pr.push_back(p);
  }

  int p1 = 0, p2 = 0;
  for( auto p : pl ){
    while( p1 < pr.size() && (pr[p1].y - p.y)*(pr[p1].y - p.y
        ) > resp && pr[p1].y < p.y ) p1++;
    while( p2 < pr.size() && ( (pr[p2].y - p.y)*(pr[p2].y - p
        .y) <= resp || pr[p2].y < p.y ) ) p2++;

    for( int i = p1; i < p2; i++ ) resp = min( resp, p.
        squared_distance( pr[i] ) );
  }

  return resp;
}

int main(){
  int n; cin >> n;
  vector<Point> points(n);

  for( auto &p : points ) cin >> p.x >> p.y;
  sort( points.begin(), points.end() );

  cout << solve( -1e9, 1e9, points ) << endl;
}
```

9.12 Point In Polygon

You are given a polygon of n vertices and a list of m points. Your task is to determine for each point if it is inside, outside or on the boundary of the polygon. The polygon consists of n vertices $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. The vertices (x_i, y_i) and (x_{i+1}, y_{i+1}) are adjacent for $i = 1, 2, \ldots, n-1$, and the vertices (x_1, y_1) and (x_n, y_n) are also adjacent.

Input
The first input line has two integers n and m: the number of vertices in the polygon and the number of points. After this, there are n lines that describe the polygon. The ith such line has two integers x_i and y_i. You may assume that the polygon is simple, i.e., it does not intersect itself. Finally, there are m lines that describe the points. Each line has two integers x and y.

Output
For each point, print "INSIDE", "OUTSIDE" or "BOUNDARY".

Constraints
• $3 \le n, m \le 1000$ • $1 \le m \le 1000$ • $-10^9 \le x_i, y_i \le 10^9$ • $-10^9 \le x, y \le 10^9$

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
```

```
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()=1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << "," << p.y << ")"; }
};

template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}

template<class P>
int inPolygon(vector<P> &p, P a) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) {
            return 2;
        }
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}

int main(){
    int n, m; cin >> n >> m;
    vector<Point<ll>> ponto(n+1);
    for(int i=0; i<n; i++) {
        int x, y; cin >> x >> y;
        ponto[i] = Point<ll>(x, y);
    }
    ponto[n] = ponto[0];

    string s[3] = {"OUTSIDE\n", "INSIDE\n", "BOUNDARY\n"};

    while(m--){
        int total = 0;
        int x, y; cin >> x >> y;
        Point<ll> p = Point<ll>(x, y);
        cout << s[inPolygon(ponto, p)];
    }
}
```

9.13 Point Location Test

There is a line that goes through the points $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$. There is also a point $p_3 = (x_3, y_3)$. Your task is to determine whether p_3 is located on the left or right side of the line or if it touches the line when we are looking from p_1 to p_2.

Input
The first input line has an integer t: the number of tests. After this, there are t lines that describe the tests. Each line has six integers: x_1, y_1, x_2, y_2, x_3 and y_3.

Output
For each test, print "LEFT", "RIGHT" or "TOUCH".

Constraints
• $1 \le t \le 10^5$ • $-10^9 \le x_1, y_1, x_2, y_2, x_3, y_3 \le 10^9$ • $x_1 \neq x_2$ or $y_1 \neq y_2$

```
int main() {
    ll x1, y1, x2, y2, x3, y3;
    int t;
    cin >> t;
    while (t--) {
        cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
        ll k = (y3 - y1) * (x2 - x1);
        ll l = (x3 - x1) * (y2 - y1);
        if (k > l)
            cout << "LEFT\n";
        else if (k < l)
            cout << "RIGHT\n";
        else
            cout << "TOUCH\n";
    }
}
```

9.14 Polygon Area

Your task is to calculate the area of a given polygon. The polygon consists of n vertices $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. The vertices (x_i, y_i) and (x_{i+1}, y_{i+1}) are adjacent for $i = 1, 2, \ldots, n-1$, and the vertices (x_1, y_1) and (x_n, y_n) are also adjacent.

Input
The first input line has an integer n: the number of vertices. After this, there are n lines that describe the vertices. The ith such line has two integers x_i and y_i. You may assume that the polygon is simple, i.e., it does not intersect itself.

Output
Print one integer: $2a$ where the area of the polygon is a (this ensures that the result is an integer).

Constraints
• $3 \le n \le 1000$ • $-10^9 \le x_i, y_i \le 10^9$

```
struct P {
    ll x, y;
    void read() { cin >> x >> y; }
    ll operator*(P b) const {
        return 1LL*x*b.y - 1LL*b.x*y;
    }
    P operator-(P b) {
        return P{x-b.x, y-b.y};
    }
    ll triangle(P b, P c) {
        return (b - *this) * (c - *this);
    }
    bool operator<(P b) {
        return make_pair(x, y) < make_pair(b.x, b.y);
    }
    void show() {
        cout << x << ' ' << y << endl;
    }
};

ll area(P pontos[], int n){
    ll total = 0;
    for(int i=0; i<n; i++){
        total += (pontos[i].y + pontos[i+1].y) * (pontos[i].x - pontos[i+1].x);
    }
    return abs(total);
}

int main() {
    int n; cin >> n;
    P pontos[n+1];
    for(int i=0; i<n; i++)
        pontos[i].read();
    pontos[n] = pontos[0];

    cout << area(pontos, n) << '\n';
    return 0;
}
```

9.15 Polygon Lattice Points

Given a polygon, your task is to calculate the number of lattice points inside the polygon and on its boundary. A lattice point is a point whose coordinates are integers. The polygon consists of n vertices $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. The vertices (x_i, y_i) and (x_{i+1}, y_{i+1}) are adjacent for $i = 1, 2, \ldots, n-1$, and the vertices (x_1, y_1) and (x_n, y_n) are also adjacent.

Input
The first input line has an integer n: the number of vertices. After this, there are n lines that describe the vertices. The ith such line has two integers x_i and y_i. You may assume that the polygon is simple, i.e., it does not intersect itself.

Output
Print two integers: the number of lattice points inside the polygon and on its boundary.

Constraints
• $3 \le n \le 10^5$ • $-10^9 \le x_i, y_i \le 10^9$

```
struct P {
    ll x, y;
    void read() { cin >> x >> y; }
    ll operator*(P b) const {
        return 1LL*x*b.y - 1LL*b.x*y;
    }
    P operator-(P b) {
        return P{x-b.x, y-b.y};
    }
    ll triangle(P b, P c) {
        return (b - *this) * (c - *this);
    }
    bool operator<(P b) {
        return make_pair(x, y) < make_pair(b.x, b.y);
    }
    void show() {
        cout << x << ' ' << y << endl;
    }
};

ll area(P ponto[], int n){
    ll total = 0;
    for(int i=0; i<n; i++){
        total += (ponto[i].y + ponto[i+1].y) * (ponto[i].x - ponto[i+1].x);
    }

    return abs(total);
}

ll boun(P ponto[], int n){
    ll B = 0;
    for(int i=0; i<n; i++){
        B += abs(__gcd(ponto[i+1].x - ponto[i].x, ponto[i+1].y - ponto[i].y));
    }

    return B;
}

ll ins(P ponto[], int n){
    ll A = area(ponto, n);
    ll B = boun(ponto, n);
    ll C = (A - B)/2 + 1;

    return C;
}

int main() {
    int n; cin >> n;
    P ponto[n+1];
    for(int i=0; i<n; i++)
        ponto[i].read();
    ponto[n] = ponto[0];

    cout << ins(ponto, n) << ' ' << boun(ponto, n) << '\n';
}
```

9.16 Robot Path

You are given a description of a robot's path. The robot begins at point $(0, 0)$ and performs n commands. Each command moves the robot some distance up, down, left or right. The robot will stop when it has performed all commands, or immediately when it returns to a point that it has already visited. Your task is to calculate the total distance the robot moves.

Input
The first line has an integer n: the number of commands. After that, there are n lines describing the commands. Each line has a character d and an integer x: the robot moves the distance x to the direction d. Each direction is U (up), D (down), L (left), or R (right).

Output
Print the total distance the robot moves.

Constraints
• $1 \le n \le 10^5$ • $1 \le x \le 10^6$

```
using ll = long long;
using vi = vector<int>;
using pii = pair<ll, ll>;

const int maxn = 1e5 + 10;
const ll inf = 1e11 + 10;

struct Point{
    ll x, y;
    Point( ll x = 0, ll y = 0 ) : x(x), y(y) {}
    Point operator + ( const Point &o ) const { return Point( x + o.x, y + o.y ); }
    Point operator * ( const int c ) const { return Point( x*c, y*c ); }
    bool operator == ( const Point &o ) const { return x == o.x && y == o.y; }
    ll dist( const Point &o ){ return abs(x - o.x) + abs(y - o.y); }
} pos[maxn], dir[maxn];

Point direction( char c ){
    if( c == 'U' ) return Point( 0, 1 );
    if( c == 'R' ) return Point( 1, 0 );
    if( c == 'D' ) return Point( 0, -1 );
    return Point( -1, 0 );
}

struct Event{
    ll y, x1, x2, val, id;
    bool operator < ( const Event &o ) const {
        return ((y == o.y) ? val > o.val : y < o.y );
    }
} events[2*maxn];

ll total[maxn];

bool check( int n, int q, int p ){
    set<pii> s;
    auto update = [&]( ll l, ll r, int op ){
        if( op == -1 ){ s.erase({ r, l }); return false; }

        auto it = s.lower_bound({ l, -1 });
        if( it != s.end() && it->second <= r ) return true;

        s.insert({ r, l });
        return false;
    };

    int t = 0;
    for( int i = 0; i < q; i++ )
        if( events[i].id <= p && update( events[i].x1, events[i].x2, events[i].val ) )
            return true;
    return false;
}

int bs( int n, int q ){
    int l = 0, r = n - 1;
    while( l < r ){
        int m = (l + r)/2;
        if( check( n, q, m ) ) r = m;
        else l = m + 1;
    }
    return r;
}

bool inter( Point a1, Point a2, Point b1, Point b2 ){
    if( a1.x > a2.x ) swap( a1.x, a2.x );
    if( a1.y > a2.y ) swap( a1.y, a2.y );
    if( b1.x > b2.x ) swap( b1.x, b2.x );
    if( b1.y > b2.y ) swap( b1.y, b2.y );

    if( a2.x < b1.x || b2.x < a1.x ) return false;
    if( a2.y < b1.y || b2.y < a1.y ) return false;
    return true;
}

void check_max_len( int i, int j, ll &len ){
    while( len > 0 ){
        Point i1 = pos[i] + dir[i], i2 = pos[i] + dir[i] * len;
        Point j1 = pos[j], j2 = pos[j + 1];
        if( !inter( i1, i2, j1, j2 ) ) break;
        len--;
    }
}

void solve(){
    int n; cin >> n;

    int q = 0;
    auto create_line = [&]( ll x1, ll x2, ll y1, ll y2, int id ){
        if( x1 > x2 ) swap( x1, x2 );
        if( y1 > y2 ) swap( y1, y2 );

        events[q++] = { y1, x1, x2,  1, id };
        events[q++] = { y2, x1, x2, -1, id };
    };

    create_line( -inf, -inf, -inf, -inf, -1 );
    create_line( 0, 0, 0, 0, -1 );
```

```
96      create_line( inf, inf, inf, inf, -1 );
97
98      FOR(i, 0, n){
99          char c; int d; cin >> c >> d;
100         dir[i] = direction(c);
101         if( i > 0 && dir[i - 1] + dir[i] == Point(0, 0) ){ n
                = i; break; }
102         total[i] += d; total[i + 1] += total[i];
103
104         Point p1 = pos[i] + dir[i];
105         pos[i + 1] = pos[i] + dir[i]*d;
106
107         create_line( p1.x, pos[i + 1].x, p1.y, pos[i + 1].y,
                i );
108     }
109
110     sort( events, events + q );
111
112     int id = bs( n, q );
113
114     ll ans = (id == 0) ? 0 : total[id - 1];
115     ll len = pos[id].dist(pos[id + 1]) - 1;
116     FOR(i, 0, id) check_max_len( id, i, len );
117
118     cout << ans + len + 1 << '\n';
119 }
120
121 int main(){
122     solve();
```

10 Advanced Techniques

10.1 Apples And Bananas

There are n apples and m bananas, and each of them has an integer weight between $1 \dots k$. Your task is to calculate, for each weight w between $2 \dots 2k$, the number of ways we can choose an apple and a banana whose combined weight is w.

Input
The first input line contains three integers k, n and m: the number k, the number of apples and the number of bananas. The next line contains n integers $a_1, a_2, \dots, a_n$: weight of each apple. The last line contains m integers $b_1, b_2, \dots, b_m$: weight of each banana.

Output
For each integer w between $2 \dots 2k$ print the number of ways to choose an apple and a banana whose combined weight is w.

Constraints
• $1 \le k, n, m \le 2 \cdot 10^5$ • $1 \le a_i \le k$ • $1 \le b_i \le k$

```cpp
using cd = complex<long double>;
using ll = long long;
using ld = long double;
const double PI = acos(-1);

void fft( vector<cd> &a, bool invert ){

  int n = a.size();
  for( int i = 1, j = 0; i < n; i++ ){
    int bit = (n>>1);
    for(; bit&j; bit >>= 1 ) j ^= bit;
    j ^= bit;

    if( i < j ) swap( a[i], a[j] );
  }

  for( int len = 2; len <= n; len <<= 1 ){
    ld ang = ((invert) ? -1.0 : 1.0 )*2*PI/len;
    cd wlen( cos(ang), sin(ang) );
    for( int i = 0; i < n; i += len ){
      cd w(1);
      for( int j = 0; j < len/2; j++ ){
        cd u = a[i + j], v = a[i + j + len/2]*w;
        a[i + j] = u + v;
        a[i + j + len/2] = u - v;
        w *= wlen;
      }
    }
  }

  if( invert ) for( auto &x : a ) x /= n;
}

vector<ll> mult( vector<ll> &a, vector<ll> &b ){
  vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
  int n = 1;
  while( n < a.size() + b.size() ) n <<= 1;

  fa.resize(n); fb.resize(n);

  fft( fa, false ); fft( fb, false );
  for( int i = 0; i < n; i++ ) fa[i] *= fb[i];
  fft( fa, true );

  vector<ll> resp(n);
  for( int i = 0; i < n; i++ ) resp[i] = round(fa[i].real());
  return resp;
}

int main(){
  int k, n, m; cin >> k >> n >> m;
  vector<ll> a(k + 1), b(k + 1);

  for( int i = 0; i < n; i++ ){ int x; cin >> x; a[x]++; }
  for( int i = 0; i < m; i++ ){ int x; cin >> x; b[x]++; }

  vector<ll> c = mult( a, b );

  for( int i = 2; i <= 2*k; i++ ) cout << c[i] << "␣"; cout
       << endl;

}
```

10.2 Corner Subgrid Check

You are given a grid of letters. Your task is to find subgrids whose height and width is at least two and all the corners have the same letter. For each letter, check if there is a valid subgrid whose corners have that letter.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print k lines: for each letter, YES if there is a valid subgrid and NO otherwise.

Constraints
• $1 \le n \le 3000$ • $1 \le k \le 26$

```cpp
using ll = long long;
using vi = vector<int>;
using pii = pair<int, int>;

const int maxn = 3e3;
const int alphabet = 26;

int ptr[alphabet], p[alphabet][maxn];
bitset<maxn*maxn> marc[alphabet];
bool ok[alphabet];

void solve(){
    int n, q; cin >> n >> q;

    FOR(i, 0, n){
        fill(ptr, ptr + q, 0);
        FOR(j, 0, n){
            char c; cin >> c;
            p[c - 'A'][ptr[c - 'A']++] = j;
        }

        FOR(c, 0, q) if( !ok[c] ){
            FOR(j, 0, ptr[c] ){
                FOR(k, j + 1, ptr[c])
                    if( marc[c][p[c][j]*n + p[c][k]] ){ ok[c]
                        = true; break; }
                    else marc[c][p[c][j]*n + p[c][k]] = true;
                if( ok[c] ) break;
            }
        }
    }
    FOR(c, 0, q) cout << ((ok[c]) ? "YES" : "NO" ) << '\n';
}
```

10.3 Corner Subgrid Count

You are given an $n \times n$ grid whose each square is either black or white. A subgrid is called beautiful if its height and width is at least two and all of its corners are black. How many beautiful subgrids are there within the given grid?

Input
The first input line has an integer n: the size of the grid. Then there are n lines describing the grid: 1 means that a square is black and 0 means it is white.

Output
Print the number of beautiful subgrids.

Constraints
• $1 \le n \le 3000$

```cpp
using ll = long long;

const int maxn = 3e3;

void solve(){
    int n; cin >> n;
    vector<bitset<maxn>> v(n);
    ll ans = 0;

    for( int i = 0; i < n; i++ ){
        cin >> v[i];

        for( int j = 0; j < i; j++ ){
            ll tot = (v[j]&v[i]).count();
            ans += tot*(tot - 1)/2;
        }
    }

    cout << ans << '\n';
}
```

10.4 Cut And Paste

Given a string, your task is to process operations where you cut a substring and paste it to the end of the string. What is the final string after all the operations?

Input
The first input line has two integers n and m: the length of the string and the number of operations. The characters of the string are numbered $1, 2, \ldots, n$. The next line has a string of length n that consists of characters A–Z. Finally, there are m lines that describe the operations. Each line has two integers a and b: you cut a substring from position a to position b.

Output
Print the final string after all the operations.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a \le b \le n$

```cpp
struct Node{
  char c; int val, tam = 0;
  Node *r = nullptr, *l = nullptr;
  Node( char c = 0, int val = 0 ) : c(c), val(val), tam(1) {}
};

int tam( Node* x ){ return (( x == nullptr ) ? 0 : x->tam );
    }

Node* merge( Node *a, Node *b ){
  if( a == nullptr ) return b;
  if( b == nullptr ) return a;

  if( a->val > b->val ){
    a->r = merge( a->r, b );
    a->tam = tam(a->l) + tam(a->r) + 1;
    return a;
  }
  else{
    b->l = merge( a, b->l );
    b->tam = tam(b->l) + tam(b->r) + 1;
    return b;
  }
}

void print( Node *root ){
  if( root == nullptr ) return;
  print( root->l );
  cout << root->c;
  print( root->r );
}

pair<Node*, Node*> split( Node* root, int k ){
  if( root == nullptr ) return make_pair( nullptr, nullptr );
  if( k <= tam(root->l) ){
    auto [a, b] = split( root->l, k );
    root->l = b;
    root->tam = tam(root->l) + tam(root->r) + 1;
    return make_pair( a, root );
  }
  else{
    auto [a, b] = split( root->r, k - tam(root->l) - 1 );
    root->r = a;
    root->tam = tam(root->l) + tam(root->r) + 1;
    return make_pair( root, b );
  }
}

tuple<Node*, Node*, Node*> split( Node* root, int l, int r ){
  auto [a, b] = split( root, r );
  auto [c, d] = split( a, l - 1 );
  return make_tuple(c, d, b);
}

int main(){
  int n, m; cin >> n >> m;
  mt19937 rng(chrono::steady_clock::now().time_since_epoch().
      count());
  vector<Node> v(n);

  Node *root = nullptr;
  for( int i = 0; i < n; i++ ){
    char c; cin >> c;
    v[i] = Node( c, rng() );
  }

  for( auto& x : v ) root = merge( root, &x );

  while( m-- ){
    int l, r; cin >> l >> r;
    auto [a, b, c] = split( root, l, r );

    root = merge( merge( a, c ), b );
  }

  print(root);
}
```

10.5 Distinct Routes II

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n. You can use each teleporter at most once during the game. You want to play the game for exactly k days. Every time you use any teleporter you have to pay one coin. What is the minimum number of coins you have to pay during k days if you play optimally?

Input
The first input line has three integers n, m and k: the number of rooms, the number of teleporters and the number of days you play the game. The rooms are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b. There are no two teleporters whose starting and ending room are the same.

Output
First print one integer: the minimum number of coins you have to pay if you play optimally. Then, print k route descriptions according to the example. You can print any valid solution. If it is not possible to play the game for k days, print only -1.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le k \le n-1$ • $1 \le a, b \le n$

```cpp
const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N
        ) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost
            ,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost
            ,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.
                            to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.
                            to });
                }
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e
            .flow;
        return {totflow, totcost/2};
    }
};

void solve(){
    int n, m, k; cin >> n >> m >> k;

```

```
    MCMF g(n + 1);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        g.addEdge(a, b, 1, 1);
    }

    g.addEdge(0, 1, k, 0);
    auto ans = g.maxflow(0, n);

    if(ans.first == k) {
        cout << ans.second << '\n';

        for(int i = 0; i < k; i++) {
            vector<int> path;
            int curr = 1;
            path.push_back(curr);

            while(curr != n) {
                for(auto& e : g.ed[curr]) {
                    if(e.cap == 1 && e.flow == 1) {
                        e.flow = 0;
                        curr = e.to;
                        path.push_back(curr);
                        break;
                    }
                }
            }

            cout << path.size() << '\n';
            for(int node : path) {
                cout << node << "␣";
            }
            cout << '\n';
        }

    }
    else {
        cout << -1 << '\n';
    }
}
```

10.6 Dynamic Connectivity

Consider an undirected graph that consists of n nodes and m edges. There are two types of events that can happen:
A new edge is created between nodes a and b. An existing edge between nodes a and b is removed.
Your task is to report the number of components after every event.

Input
The first input line has three integers n, m and k: the number of nodes, edges and events. After this there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. There is at most one edge between any pair of nodes. Then there are k lines describing the events. Each line has the form "t a b" where t is 1 (create a new edge) or 2 (remove an edge). A new edge is always created between two nodes that do not already have an edge between them, and only existing edges can get removed.

Output
Print $k+1$ integers: first the number of components before the first event, and after this the new number of components after each event.

Constraints
• $2 \le n \le 10^5$ • $1 \le m, k \le 10^5$ • $1 \le a, b \le n$

```
using pii = pair<int, int>;

const int maxn = 1e5 + 10;

struct UnionFind{
    int *p, *sz, *stk;
    int comp, top;

    UnionFind( int n = 1, int q = 1 ) : p(new int[n]), sz(new
         int[n]), stk(new int[q]), comp(n), top(-1) {
        fill( sz, sz + n, 1 );
        iota( p, p + n, 0 );
    }

    int find( int u ){
        while( u != p[u] ) u = p[u];
        return u;
    }

    void join( int u, int v ){
        u = find(u); v = find(v);
        if( u == v ){ stk[++top] = -1; return; }
        if( sz[u] < sz[v] ) swap( u, v );

        p[v] = u;
        sz[u] += sz[v];

        stk[++top] = v;
        comp--;
    }

    void rollback(){
        if( top < 0 ) return;
        int v = stk[top--]; if( v == -1 ) return;

        sz[p[v]] -= sz[v];
        p[v] = v;
        comp++;
    }
};

struct DynamicConnectivity{
    vector<pii> *v;
    UnionFind dsu;

    int k;
    DynamicConnectivity( int n, int m, int k ) : dsu(
         UnionFind(n, m + k)), v(new vector<pii>[4*(k + 1)])
         , k(k) {}

    void update( int node, int ti, int tf, int qi, int qf,
         pii p ){
        if( qi > tf || ti > qf ) return;
        if( qi <= ti && tf <= qf ){
            v[node].push_back(p);
            return;
        }
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        update( l, ti, tm, qi, qf, p );
        update( r, tm + 1, tf, qi, qf, p );
    }

    void update( int l, int r, pii p ){
        update( 1, 0, k, l, r, p );
    }

    void solve( int node, int ti, int tf ){
        for( auto &[u, v] : v[node] ) dsu.join( u, v );
        if( ti == tf ) cout << dsu.comp << "␣";
        else{
            int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
            solve( l, ti, tm ); solve( r, tm + 1, tf );
        }
        for( auto &[u, v] : v[node] ) dsu.rollback();
    }

    void solve(){
        solve( 1, 0, k );
    }
};

void solve(){
    int n, m, k; cin >> n >> m >> k;

    DynamicConnectivity DC( n, m, k );

    map<pii, int> mp;
    for( int i = 0; i < m; i++ ){
        int u, v; cin >> u >> v; u--; v--;
        if( u > v ) swap( u, v );
        mp[pii(u, v)] = 0;
    }

    for( int i = 1; i <= k; i++ ){
        int t, u, v; cin >> t >> u >> v; u--; v--;
        if( u > v ) swap( u, v );
        if( t == 1 ) mp[pii(u, v)] = i;
        else{
            DC.update( mp[pii(u, v)], i - 1, pii(u, v) );
            mp.erase(pii(u, v));
        }
    }

    for( auto &[par, id] : mp ) DC.update( id, k, par );

    DC.solve();
    cout << "\n";
}
```

10.7 Eulerian Subgraphs

You are given an undirected graph that has n nodes and m edges. We consider subgraphs that have all nodes of the original graph and some of its edges. A subgraph is called Eulerian if each node has even degree. Your task is to count the number of Eulerian subgraphs modulo $10^9 + 7$.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the edges. Each line has two integers a and b: there is an edge between nodes a and b. There is at most one edge between two nodes, and each edge connects two distinct nodes.

Output
Print the number of Eulerian subgraphs modulo $10^9 + 7$.

Constraints

• $1 \le n \le 10^5$ • $0 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 1e5+5, MOD = 1e9+7;
vector<int> G[MAX];
bool vis[MAX];

void dfs(int s) {
    vis[s] = 1;
    for(auto x : G[s]) if(!vis[x]) dfs(x);
}

ll fexp(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % MOD, e /= 2)
        if (e & 1) ans = ans * b % MOD;
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    int comp = 0;
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            dfs(i);
            comp++;
        }
    }

    int exp = m - (n - comp);
    cout << fexp(2, exp) << '\n';
}
```

10.8 Hamming Distance

The Hamming distance between two strings a and b of equal length is the number of positions where the strings differ. You are given n bit strings, each of length k and your task is to calculate the minimum Hamming distance between two strings.

Input
The first input line has two integers n and k: the number of bit strings and their length. Then there are n lines each consisting of one bit string of length k.

Output
Print the minimum Hamming distance between two strings.

Constraints
• $2 \le n \le 2 \cdot 10^4$ • $1 \le k \le 30$

```
int main(){
  int n, k; cin >> n >> k;
  vector<int> v(n);
  int resp = k;
  for( int i = 0; i < n; i++ ){
    for( int j = 0; j < k; j++ ){
      char c; cin >> c;
      v[i] <<= 1;
      if( c == '1' ) v[i]++;
    }
    for( int j = 0; j < i; j++ ) resp = min( resp,
        __builtin_popcount(v[i]^v[j]) );
  }
  cout << resp << endl;
}
```

10.9 Houses And Schools

There are n houses on a street, numbered $1, 2, \ldots, n$. The distance of houses a and b is $|a - b|$. You know the number of children in each house. Your task is to establish k schools in such a way that each school is in some house. Then, each child goes to the nearest school. What is the minimum total walking distance of the children if you act optimally?

Input
The first input line has two integers n and k: the number of houses and the number of schools. The houses are numbered $1, 2 \ldots, n$. After this, there are n integers $c_1, c_2, \ldots, c_n$: the number of children in each house.

Output
Print the minimum total distance.

Constraints
• $1 \le k \le n \le 3000$ • $1 \le c_i \le 10^9$

```
const int MAX = 3e3 + 3;
const int LINF = 4000000000000000000;

int dp[MAX][MAX];

int n, K;
int cur_j;
int c[MAX], pref[MAX], pref1[MAX];

int cost(int i, int k) {
    int m = (k + i) / 2;
    int p1 = (pref1[m] - pref1[k - 1]) - k * (pref[m] - pref[
        k - 1]);
    int p2 = i * (pref[i] - pref[m]) - (pref1[i] - pref1[m]);
    return p1 + p2;
}

struct DP {
    int lo(int ind) { return cur_j - 1; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[k][cur_j - 1] + cost(ind
        , k); }
    void store(int ind, int k, ll v) { dp[ind][cur_j] = v; }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, 0, n + 1); }
};

void solve(){
    cin >> n >> K;
    for(int i = 1; i <= n; i++) {
        cin >> c[i];
        pref[i] = pref[i - 1] + c[i];
        pref1[i] = pref1[i - 1] + c[i] * i;
    }

    DP dcdp;

    for(int i = 1; i <= n; i++) dp[i][1] = pref[i] * i -
        pref1[i];

    for(int j = 2; j <= K; j++) {
        cur_j = j;
        dcdp.solve(1, n + 1);
    }

    int ans = LINF;

    for(int i = K; i <= n; i++) {
        ans = min(ans, dp[i][K] + (pref1[n] - pref1[i - 1]) -
            i * (pref[n] - pref[i - 1]));
    }

    cout << ans << '\n';
}
```

10.10 Knuth Division

Given an array of n numbers, your task is to divide it into n subarrays, each of which has a single element. On each move, you may choose any subarray and split it into two subarrays. The cost of such a move is the sum of values in the chosen subarray. What is the minimum total cost if you act optimally?

Input
The first input line has an integer n: the array size. The array elements are numbered $1, 2, \ldots, n$. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the minimum total cost.

Constraints
• $1 \le n \le 5000$ • $1 \le x_i \le 10^9$

```
using ll = long long;

const int maxn = 5e3;
const ll inf = 1e18;

ll dp[maxn][maxn], sum[maxn];
int opt[maxn][maxn];

ll cost( int i, int j ){
    return sum[j] - ((i > 0) ? sum[i - 1] : 0);
}
```

```
12
13 void solve(){
14     int n; cin >> n;
15
16     for( int i = 0; i < n; i++ ){
17         int x; cin >> x;
18         sum[i] += x;
19         sum[i + 1] = sum[i];
20
21         fill( dp[i], dp[i] + maxn, inf );
22     }
23
24     for( int len = 1; len <= n; len++ )
25         for( int l = 0, r = len - 1; r < n; l++, r++ ){
26             if( len == 1 ){
27                 dp[l][r] = 0; opt[l][r] = l; continue;
28             }
29
30             dp[l][r] = inf;
31             opt[l][r] = l;
32             for( int m = opt[l][r - 1]; m <= opt[l + 1][r]; m
                    ++ )
33                 if( dp[l][m] + dp[m + 1][r] < dp[l][r] ){
34                     dp[l][r] = dp[l][m] + dp[m + 1][r];
35                     opt[l][r] = m;
36                 }
37             dp[l][r] += cost( l, r );
38         }
39
40     cout << dp[0][n - 1] << '\n';
41 }
```

10.11 Meet In The Middle

You are given an array of n numbers. In how many ways can you choose a subset of the numbers with sum x?

Input
The first input line has two numbers n and x: the array size and the required sum. The second line has n integers $t_1, t_2, \ldots, t_n$: the numbers in the array.

Output
Print the number of ways you can create the sum x.

Constraints
• $1 \le n \le 40$ • $1 \le x \le 10^9$ • $1 \le t_i \le 10^9$

```
1 using ll = long long;
2
3 const int maxn = 40;
4 const int inf = 1e9 + 10;
5
6 int v[2][(1<<(maxn/2))];
7
8 void solve(){
9     int n, x; cin >> n >> x;
10     int n0 = n/2, n1 = n - n0;
11
12     for( int i = 0; i < n0; i++ ){
13         int t; cin >> t;
14         for( int j = 0; j < (1<<i); j++ ) v[0][j + (1<<i)] =
               min( inf, v[0][j] + t );
15     }
16     sort( v[0], v[0] + (1<<n0) );
17
18     for( int i = 0; i < n1; i++ ){
19         int t; cin >> t;
20         for( int j = 0; j < (1<<i); j++ ) v[1][j + (1<<i)] =
               min( inf, v[1][j] + t );
21     }
22     sort( v[1], v[1] + (1<<n1), greater<int>() );
23
24     ll ans = 0;
25     for( int i = 0, j = 0, k = 0; i < (1<<n1); i++ ){
26         while( j < (1<<n0) && v[1][i] + v[0][j] < x ) j++;
27         k = max( k, j );
28         while( k < (1<<n0) && v[1][i] + v[0][k] <= x ) k++;
29
30         ans += k - j;
31     }
32
33     cout << ans << '\n';
34 }
```

10.12 Monster Game I

You are playing a game that consists of n levels. Each level has a monster. On levels $1, 2, \ldots, n-1$, you can either kill or escape the monster. However, on level n you must kill the final monster to win the game. Killing a monster takes sf time where s is the monster's strength and f is your skill factor (lower skill factor is better). After killing a monster, you get a new skill factor. What is the minimum total time in which you can win the game?

Input
The first input line has two integers n and x: the number of levels and your initial skill factor. The second line has n integers $s_1, s_2, \ldots, s_n$: each monster's strength. The third line has n integers $f_1, f_2, \ldots, f_n$: your new skill factor after killing a monster.

Output
Print one integer: the minimum total time to win the game.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x \le 10^6$ • $1 \le s_1 \le s_2 \le \cdots \le s_n \le 10^6$
• $x \ge f_1 \ge f_2 \ge \cdots \ge f_n \ge 1$

```
1 const int MAX = 2e5+1;
2
3 // matar custa s_i * f tempo e vc recebe a forca f_i
4 // dp[i] = resposta matando o monstro i
5 // dp[n] = 0
6 // dp[k] = min(s[l] * f[k] + dp[l]); k + 1 <= l <= n.
7 // CHT em s[l] * x + dp[l]
8
9 int dp[MAX], s[MAX], f[MAX];
10 int n, x;
11
12 struct Line {
13     mutable ll k, m, p;
14     bool operator<(const Line& o) const { return k < o.k; }
15     bool operator<(ll x) const { return p < x; }
16 };
17
18 struct LineContainer : multiset<Line, less<>> {
19     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
20     static const ll inf = LLONG_MAX;
21     ll div(ll a, ll b) { // floored division
22         return a / b - ((a ^ b) < 0 && a % b); }
23     bool isect(iterator x, iterator y) {
24         if (y == end()) return x->p = inf, 0;
25         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
26         else x->p = div(y->m - x->m, x->k - y->k);
27         return x->p >= y->p;
28     }
29     void add(ll k, ll m) {
30         auto z = insert({k, m, 0}), y = z++, x = y;
31         while (isect(y, z)) z = erase(z);
32         if (x != begin() && isect(--x, y)) isect(x, y = erase
               (y));
33         while ((y = x) != begin() && (--x)->p >= y->p)
34             isect(x, erase(y));
35     }
36     ll query(ll x) {
37         assert(!empty());
38         auto l = *lower_bound(x);
39         return l.k * x + l.m;
40     }
41 };
42
43 void solve(){
44     cin >> n >> x; dp[n] = 0;
45     for(int i = 1; i <= n; i++) cin >> s[i]; s[0] = 0;
46     for(int i = 1; i <= n; i++) cin >> f[i]; f[0] = x;
47
48     LineContainer cht;
49     cht.add(-s[n], -dp[n]);
50
51     for(int at = n - 1; at >= 0; at--) {
52         dp[at] = -cht.query(f[at]);
53         cht.add(-s[at], -dp[at]);
54     }
55
56     cout << dp[0] << '\n';
57 }
```

10.13 Monster Game II

You are playing a game that consists of n levels. Each level has a monster. On levels $1, 2, \ldots, n-1$, you can either kill or escape the monster. However, on level n you must kill the final monster to win the game. Killing a monster takes sf time where s is the monster's strength and f is your skill factor. After killing a monster, you get a new skill factor (lower skill factor is better). What is the minimum total time in which you can win the game?

Input
The first input line has two integers n and x: the number of levels and your initial skill factor. The second line has n integers $s_1, s_2, \ldots, s_n$: each monster's strength. The third line has n integers $f_1, f_2, \ldots, f_n$: your new skill factor after killing a monster.

Output
Print one integer: the minimum total time to win the game.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x \le 10^6$ • $1 \le s_i, f_i \le 10^6$

```cpp
const int MAX = 2e5+1;

// matar custa s_i * f tempo e vc recebe a forca f_i
// dp[i] = resposta matando o monstro i
// dp[n] = 0
// dp[k] = min(s[l] * f[k] + dp[l]); k + 1 <= l <= n.
// CHT em s[l] * x + dp[l]

int dp[MAX], s[MAX], f[MAX];
int n, x;

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase
            (y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

void solve(){
    cin >> n >> x; dp[n] = 0;
    for(int i = 1; i <= n; i++) cin >> s[i]; s[0] = 0;
    for(int i = 1; i <= n; i++) cin >> f[i]; f[0] = x;

    LineContainer cht;
    cht.add(-s[n], -dp[n]);

    for(int at = n - 1; at >= 0; at--) {
        dp[at] = -cht.query(f[at]);
        cht.add(-s[at], -dp[at]);
    }

    cout << dp[0] << '\n';
}
```

10.14 Necessary Cities

There are n cities and m roads between them. There is a route between any two cities. A city is called necessary if there is no route between some other two cities after removing that city (and adjacent roads). Your task is to find all necessary cities.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output
First print an integer k: the number of necessary cities. After that, print a list of k cities. You may print the cities in any order.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
vector<vi> G;

struct block_cut_tree {
    vector<vector<int>> g, blocks, tree;
    vector<vector<pair<int, int>>> edgblocks;
    stack<int> s;
    stack<pair<int, int>> s2;
    vector<int> id, art, pos;

    block_cut_tree(vector<vector<int>> g_) : g(g_) {
        int n = g.size();
        id.resize(n, -1), art.resize(n), pos.resize(n);
        build();
    }

    int dfs(int i, int& t, int p = -1) {
        int lo = id[i] = t++;
        s.push(i);

        if (p != -1) s2.emplace(i, p);
        for (int j : g[i]) if (j != p and id[j] != -1) s2.
            emplace(i, j);

        for (int j : g[i]) if (j != p) {
            if (id[j] == -1) {
                int val = dfs(j, t, i);
                lo = min(lo, val);

                if (val >= id[i]) {
                    art[i]++;
                    blocks.emplace_back(1, i);
                    while (blocks.back().back() != j)
                        blocks.back().push_back(s.top()), s.
                            pop();

                    edgblocks.emplace_back(1, s2.top()), s2.
                        pop();
                    while (edgblocks.back().back() != pair(j,
                        i))
                        edgblocks.back().push_back(s2.top()),
                            s2.pop();
                }
                // if (val > id[i]) aresta i-j eh ponte
            }
            else lo = min(lo, id[j]);
        }

        if (p == -1 and art[i]) art[i]--;
        return lo;
    }

    void build() {
        int t = 0;
        for (int i = 0; i < g.size(); i++) if (id[i] == -1)
            dfs(i, t, -1);

        tree.resize(blocks.size());
        for (int i = 0; i < g.size(); i++) if (art[i])
            pos[i] = tree.size(), tree.emplace_back();

        for (int i = 0; i < blocks.size(); i++) for (int j :
            blocks[i]) {
            if (!art[j]) pos[j] = i;
            else tree[i].push_back(pos[j]), tree[pos[j]].
                push_back(i);
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;

    G.resize(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    block_cut_tree BCT(G);

    vector<int> ans;
    for(int i = 0; i < n; i++) if(BCT.art[i] >= 1) ans.
        push_back(i + 1);
    cout << ans.size() << '\n';
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}
```

10.15 Necessary Roads

There are n cities and m roads between them. There is a route between any two cities. A road is called necessary if there is no route between some two cities after removing that road. Your task is to find all necessary roads.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output
First print an integer k: the number of necessary roads. After that, print k lines that describe the roads. You may print the roads in any order.

Constraints
• $2 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```cpp
vector<vi> G;

struct block_cut_tree {
    vector<vector<int>> g, blocks, tree;
    vector<vector<pair<int, int>>> edgblocks;
    stack<int> s;
    stack<pair<int, int>> s2;
    vector<int> id, art, pos;

    block_cut_tree(vector<vector<int>> g_) : g(g_) {
        int n = g.size();
        id.resize(n, -1), art.resize(n), pos.resize(n);
        build();
    }

    int dfs(int i, int& t, int p = -1) {
        int lo = id[i] = t++;
        s.push(i);

        if (p != -1) s2.emplace(i, p);
        for (int j : g[i]) if (j != p and id[j] != -1) s2.
            emplace(i, j);

        for (int j : g[i]) if (j != p) {
            if (id[j] == -1) {
                int val = dfs(j, t, i);
                lo = min(lo, val);

                if (val >= id[i]) {
                    art[i]++;
                    blocks.emplace_back(1, i);
                    while (blocks.back().back() != j)
                        blocks.back().push_back(s.top()), s.
                            pop();

                    edgblocks.emplace_back(1, s2.top()), s2.
                        pop();
                    while (edgblocks.back().back() != pair(j,
                        i))
                        edgblocks.back().push_back(s2.top()),
                            s2.pop();
                }
                // if (val > id[i]) aresta i-j eh ponte
            }
            else lo = min(lo, id[j]);
        }

        if (p == -1 and art[i]) art[i]--;
        return lo;
    }

    void build() {
        int t = 0;
        for (int i = 0; i < g.size(); i++) if (id[i] == -1)
            dfs(i, t, -1);

        tree.resize(blocks.size());
        for (int i = 0; i < g.size(); i++) if (art[i])
            pos[i] = tree.size(), tree.emplace_back();

        for (int i = 0; i < blocks.size(); i++) for (int j :
            blocks[i]) {
            if (!art[j]) pos[j] = i;
            else tree[i].push_back(pos[j]), tree[pos[j]].
                push_back(i);
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;

    G.resize(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    block_cut_tree BCT(G);

    vector<pii> ans;

    for(auto& edge_block : BCT.edgblocks) {
        if(edge_block.size() == 1) {
            ans.push_back(edge_block[0]);
        }
    }

    cout << ans.size() << '\n';

    for(auto x : ans) {
        cout << x.first + 1 << ' ' << x.second + 1 << '\n';
    }
}
```

10.16 New Roads Queries

There are n cities in Byteland but no roads between them. However, each day, a new road will be built. There will be a total of m roads. Your task is to process q queries of the form: "after how many days can we travel from city a to city b for the first time?"

Input
The first input line has three integers n, m and q: the number of cities, roads and queries. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the roads in the order they are built. Each line has two integers a and b: there will be a road between cities a and b. Finally, there are q lines that

describe the queries. Each line has two integers a and b: we want to travel from city a to city b.

Output
For each query, print the number of days, or -1 if it is never possible.

Constraints
• $1 \le n, m, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int maxn = 2e5 + 10;
const int inf = 1e9;
int pai[maxn], peso[maxn], h[maxn];

void init( int n ){ for( int i = 0; i <= n; i++ ) pai[i] = i,
      peso[i] = inf; }

int find( int a ){ return ( a == pai[a] ) ? a : find( pai[a]
    ); }
void join( int a, int b, int p ){
    a = find(a), b = find(b);
    if( a == b ) return;
    if( h[a] < h[b] + 1 ) swap( a, b );
    pai[b] = a; peso[b] = p;
    h[a] = max( h[a], h[b] + 1 );
}

int query( int a, int b ){
    int resp = 0;
    while( a != b ){
        if( peso[a] == inf && peso[b] == inf ) return -1;
        if( peso[a] < peso[b] ) resp = max( resp, peso[a] ),
            a = pai[a];
        else resp = max( resp, peso[b] ), b = pai[b];
    }
    return resp;
}

int main(){
    int n, m, q; cin >> n >> m >> q;
    init(n);
    for( int i = 1; i <= m; i++ ){
        int a, b; cin >> a >> b;
        join( a, b, i );
    }
    for( int i = 0; i < q; i++ ){
        int a, b; cin >> a >> b;
        cout << query( a, b ) << endl;
    }
}
```

10.17 One Bit Positions

You are given a binary string of length n. Your task is to calculate, for every k between $1 \dots n-1$, the number of ways we can choose two positions i and j such that $i - j = k$ and there is a one-bit at both positions.

Input
The only input line has a string that consists only of characters 0 and 1.

Output
For every distance k between $1 \dots n-1$ print the number of ways we can choose two such positions.

Constraints
• $2 \le n \le 2 \cdot 10^5$

```
using vi = vector<int>;

void fft( vector<C> &a ){
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for(static int k = 2; k < n; k *= 2 ){
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L)/k);
        FOR(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i
            /2];
    }

    vi rev(n);
    FOR(i, 0, n) rev[i] = (rev[i/2] | (i&1) << L )/2;
    FOR(i, 0, n) if( i < rev[i] ) swap( a[i], a[rev[i]] );

    for( int k = 1; k < n; k *= 2 )
        for( int i = 0; i < n; i += 2*k ) FOR(j, 0, k ){
            auto x = (double *)&rt[j + k], y = (double *)&a[i
                + j + k];
            C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]
                );
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}

vd conv( const vd &a, const vd &b ){
    if( a.empty() || b.empty() ) return {};
    vd res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1<<L;
    vector<C> in(n), out(n);
    copy(all(a), in.begin());
    FOR(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for( C& x : in ) x *= x;
    FOR(i, 0, n) out[i] = in[-i&(n - 1)] - conj(in[i]);
    fft(out);
    FOR(i, 0, sz(res)) res[i] = imag(out[i])/(4*n);
    return res;
}

void solve(){
    string s; cin >> s;
    vd a(sz(s));
    FOR(i, 0, sz(s)) a[i] = s[i] - '0';
    vd b(a.rbegin(), a.rend());

    vd ans = conv( a, b );

    for( int i = sz(s); i < sz(ans); i++ ) cout << abs(round(
        ans[i])) << "␣"; cout << '\n';
}
```

10.18 Parcel Delivery

There are n cities and m routes through which parcels can be carried from one city to another city. For each route, you know the maximum number of parcels and the cost of a single parcel. You want to send k parcels from Syrjälä to Lehmälä. What is the cheapest way to do that?

Input
The first input line has three integers n, m and k: the number of cities, routes and parcels. The cities are numbered $1, 2, \dots, n$. City 1 is Syrjälä and city n is Lehmälä. After this, there are m lines that describe the routes. Each line has four integers a, b, r and c: there is a route from city a to city b, at most r parcels can be carried through the route, and the cost of each parcel is c.

Output
Print one integer: the minimum total cost or -1 if there are no solutions.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le k \le 100$ • $1 \le a, b \le n$ • $1 \le r, c \le 1000$

```
const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N
        ) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost
            ,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost
            ,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
```

```
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.
                            to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.
                            to });
                }
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e
            .flow;
        return {totflow, totcost/2};
    }
};

void solve(){
    int n, m, k; cin >> n >> m >> k;

    MCMF G(n + 1);
    G.addEdge(0, 1, k, 0);

    for(int i = 0; i < m; i++) {
        int a, b, r, c; cin >> a >> b >> r >> c;
        G.addEdge(a, b, r, c);
    }

    auto ans = G.maxflow(0, n);
    if(ans.first == k) cout << ans.second << '\n';
    else cout << -1 << '\n';
}
```

10.19 Reachability Queries

A directed graph consists of n nodes and m edges. The edges are numbered $1, 2, \ldots, n$. Your task is to answer q queries of the form "can you reach node b from node a?"

Input
The first input line has three integers n, m and q: the number of nodes, edges and queries. Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b. Finally there are q lines describing the queries. Each line consists of two integers a and b: "can you reach node b from node a?"

Output

Print the answer for each query: either "YES" or "NO".

Constraints
• $1 \le n \le 5 \cdot 10^4$ • $1 \le m, q \le 10^5$

```
using vi = vector<int>;
using ll = long long;

const int maxn = 5e4;

struct Kosaraju{
    vector<vector<vi>> adj;
    vector<vi> adjc;
    vector<bitset<maxn>> b;
    vi scc;

    Kosaraju( vector<vector<vi>> adj ) : adj(adj) {
        int n = sz(adj[0]);
        scc = vi(n, -1);
        adjc.resize(n);
        b.resize(n);

        kosaraju();
    }

    void dfs( int u, int t, vi &o ){
        for( int v : adj[t][u] ) if( scc[v] == -1 ){
            scc[v] = scc[u]; dfs( v, t, o );
        }
        if(!t) o.push_back(u);
    }

    void kosaraju(){
        int n = sz(adj[0]);

        vi o;
        FOR(i, 0, n) if( scc[i] == -1 ){
            scc[i] = i; dfs( i, 0, o );
        }

        fill(all(scc), -1);
        reverse(all(o));

        for( int i : o ){
            if( scc[i] == -1 ){
                scc[i] = i; dfs( i, 1, o );
            }
            b[scc[i]][i] = 1;
        }

        reverse(all(o));

        for( int i : o ){
            for( int v : adj[0][i] ) if( scc[i] != scc[v] )
                adjc[scc[i]].push_back(scc[v]);

            sort(all(adjc[i]));
            adjc[i].erase(unique(all(adjc[i])), adjc[i].end()
                );

            for( int v : adjc[i] ) b[i] |= b[v];
        }
    }
};

void solve(){
    int n, m, q; cin >> n >> m >> q;
    vector<vector<vi>> adj(2, vector<vi>(n));
    while( m-- ){
        int u, v; cin >> u >> v; u--; v--;
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    Kosaraju kos(adj);

    while( q-- ){
        int u, v; cin >> u >> v; u--; v--;
        int scc = kos.scc[u];
        cout << ((kos.b[scc][v]) ? "YES" : "NO" ) << '\n';
    }
}
```

10.20 Reachable Nodes

A directed acyclic graph consists of n nodes and m edges. The nodes are numbered $1, 2, \ldots, n$. Calculate for each node the number of nodes you can reach from that node (including the node itself).

Input
The first input line has two integers n and m: the number of nodes and edges. Then there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge from node a to node b.

Output
Print n integers: for each node the number of reachable nodes.

Constraints
• $1 \le n \le 5 \cdot 10^4$ • $1 \le m \le 10^5$

```
using vi = vector<int>;
using ll = long long;

const int maxn = 5e4;

struct Kosaraju{
    vector<vector<vi>> adj;
    vector<vi> adjc;
    vector<bitset<maxn>> b;
    vi scc;

    Kosaraju( vector<vector<vi>> adj ) : adj(adj) {
        int n = sz(adj[0]);
        scc = vi(n, -1);
        adjc.resize(n);
        b.resize(n);

        kosaraju();
    }

    void dfs( int u, int t, vi &o ){
        for( int v : adj[t][u] ) if( scc[v] == -1 ){
            scc[v] = scc[u]; dfs( v, t, o );
        }
        if(!t) o.push_back(u);
    }

    void kosaraju(){
        int n = sz(adj[0]);

        vi o;
        FOR(i, 0, n) if( scc[i] == -1 ){
```

```
            scc[i] = i; dfs( i, 0, o );
        }

        fill(all(scc), -1);
        reverse(all(o));

        for( int i : o ){
            if( scc[i] == -1 ){
                scc[i] = i; dfs( i, 1, o );
            }
            b[scc[i]][i] = 1;
        }

        reverse(all(o));

        for( int i : o ){
            for( int v : adj[0][i] ) if( scc[i] != scc[v] )
                adjc[scc[i]].push_back(scc[v]);

            sort(all(adjc[i]));
            adjc[i].erase(unique(all(adjc[i])), adjc[i].end()
                 );

            for( int v : adjc[i] ) b[i] |= b[v];
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;
    vector<vector<vi>> adj(2, vector<vi>(n));
    while( m-- ){
        int u, v; cin >> u >> v; u--; v--;
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    Kosaraju kos(adj);

    FOR(i, 0, n){
        int scc = kos.scc[i];
        cout << kos.b[scc].count() << "␣";
    }
    cout << '\n';
}
```

10.21 Reversals And Sums

Given an array of n integers, you have to process following operations:
reverse a subarray calculate the sum of values in a subarray

Input
The first input line has two integers n and m: the size of the array and the number of operations. The array elements are numbered $1, 2, \ldots, n$. The next line as n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Finally, there are m lines that describe the operations. Each line has three integers t, a and b. If $t = 1$, you should reverse a subarray from a to b. If $t = 2$, you should calculate the sum of values from a to b.

Output
Print the answer to each operation where $t = 2$.

Constraints

- $1 \le n \le 2 \cdot 10^5$
- $1 \le m \le 10^5$
- $0 \le x_i \le 10^9$
- $1 \le a \le b \le n$

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());

class Treap{
private:
  struct Node{
    ll sum, val; int prior, lazy, sub;
    Node *l = nullptr, *r = nullptr;
    Node( int x = 0 ) : sum(x), val(x), prior(rng()), sub(1),
        lazy(0) {}
  };
  vector<Node> v;
  Node *root = nullptr;

public:
  int get_sub( Node* x ){ return (( x ) ? x->sub : 0 ); }
  ll get_sum( Node* x ){ return (( x ) ? x->sum : 0 ); }
  Node* update( Node* x ){
    x->sub = get_sub(x->l) + get_sub(x->r) + 1;
    x->sum = get_sum(x->l) + get_sum(x->r) + x->val;
    return x;
  }

  void refresh( Node* x ){
    if( x == nullptr || x->lazy == 0 ) return;
    x->lazy = 0;
    swap( x->l, x->r );
    if( x->l ) x->l->lazy ^= 1;
    if( x->r ) x->r->lazy ^= 1;
  }

  Node* merge( Node *a, Node *b ){
    refresh(a); refresh(b);

    if( a == nullptr ) return b;
    if( b == nullptr ) return a;

    if( a->prior > b->prior ){
      a->r = merge( a->r, b );
      return update(a);
    }
    else{
      b->l = merge( a, b->l );
      return update(b);
    }
  }

  pair<Node*, Node*> split( Node* root, int k ){
    refresh( root );
    if( root == nullptr ) return make_pair( nullptr, nullptr
        );
    if( k <= get_sub(root->l) ){
      auto [a, b] = split( root->l, k );
      root->l = b;
      return make_pair(a, update(root));
    }
    else{
      auto [a, b] = split( root->r, k - get_sub(root->l) - 1
          );
      root->r = a;
      return make_pair(update(root), b);
    }
  }

  tuple<Node*, Node*, Node*> split( Node* root, int l, int r
      ){
    auto [a, b] = split( root, r );
    auto [c, d] = split( a, l - 1 );
    return make_tuple( c, d, b );
  }

  void reverse( int l, int r ){
    auto [a, b, c] = split( root, l, r );
    b->lazy ^= 1; refresh(b);
    root = merge( merge( a, b ), c );
  }

  ll query( int l, int r ){
    auto [a, b, c] = split( root, l, r );
    ll resp = get_sum(b);
    root = merge( merge( a, b ), c );
    return resp;
  }

  Treap( vector<int>& xs ){
    v.resize(xs.size());
    for( int i = 0; i < xs.size(); i++ ){
      v[i] = Node(xs[i]);
      root = merge( root, &v[i] );
    }
  }
};

int main(){
  int n, q; cin >> n >> q;
  vector<int> v(n);
  for( int &x : v ) cin >> x;

  Treap treap(v);

  while( q-- ){
    int t, l, r; cin >> t >> l >> r;
    if( t == 1 ) treap.reverse( l, r );
    else cout << treap.query( l, r ) << endl;
  }

}
```

10.22 Signal Processing

You are given two integer sequences: a signal and a mask. Your task is to process the signal by moving the mask through the signal from left to right. At each mask position calculate the sum of products of aligned signal and mask values in the part where the signal and the mask overlap.

Input
The first input line consists of two integers n and m: the length of the signal and the length of the mask. The next line consists of n integers $a_1, a_2, \ldots, a_n$ defining the signal. The last line consists of m integers $b_1, b_2, \ldots, b_m$ defining the mask.

Output
Print $n + m - 1$ integers: the sum of products of aligned values at each mask position from left to right.

Constraints

- $1 \le n, m \le 2 \cdot 10^5$
- $1 \le a_i, b_i \le 100$

```
const ll mod = (17LL << 27) + 1, root = 3; // = 2281701377
// const ll mod = (119 << 23) + 1, root = 62; // = 998244353
```

```cpp
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}

void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

void solve(){
    int n, m; cin >> n >> m;
    vector<int> a(n), b(m);
    for(auto &x : a) cin >> x;
    for(auto &x : b) cin >> x;
    reverse(all(b));

    vl ans = conv(a, b);

    for(auto x : ans) cout << x << ' ';
    cout << endl;
}
```

10.23 Subarray Squares

Given an array of n elements, your task is to divide into k subarrays. The cost of each subarray is the square of the sum of the values in the subarray. What is the minimum total cost if you act optimally?

Input
The first input line has two integers n and k: the array elements and the number of subarrays. The array elements are numbered $1, 2, \ldots, n$. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the minimum total cost.

Constraints
• $1 \le k \le n \le 3000$ • $1 \le x_i \le 10^5$

```cpp
const int MAX = 3001;

int n, k;
ll dp[MAX], x[MAX], psum[MAX];
ll res[MAX];

struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int fim) {
        ll at = psum[ind] - psum[fim];
        return at * at + dp[fim];
    }
    void store(int ind, int k, ll v) { res[ind] = v; }
    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};

void solve(){
    cin >> n >> k;
    for(int i = 1; i <= n; i++) {
        cin >> x[i];
        psum[i] = psum[i - 1] + x[i];
        dp[i] = psum[i] * psum[i];
    }

    DP dcdp;

    for(int j = 1; j < k; j++) {
        dcdp.solve(1, n + 1);
        for(int i = 0; i <= n; i++) dp[i] = res[i];
    }

    cout << dp[n] << '\n';
}
```

10.24 Substring Reversals

Given a string, your task is to process operations where you reverse a substring of the string. What is the final string after all the operations?

Input
The first input line has two integers n and m: the length of the string and the number of operations. The characters of the string are numbered $1, 2, \ldots, n$. The next line has a string of length n that consists of characters A–Z. Finally, there are m lines that describe the operations. Each line has two integers a and b: you reverse a substring from position a to position b.

Output
Print the final string after all the operations.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a \le b \le n$

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

class Treap{
private:
  struct Node{
    char c; int prior, sub, lazy = 0;
    Node *l = nullptr, *r = nullptr;
    Node( char c = '#' ) : c(c), prior(rng()), sub(1), lazy(0) {}
  };
  vector<Node> v;
  Node *root = nullptr;
public:
  int get_sub( Node *x ){ return (( x != nullptr ) ? x->sub : 0 ); }
  Node* update( Node *x ){ x->sub = get_sub(x->l) + get_sub(x->r) + 1; return x; }

  void print( Node *x ){
    if( x == nullptr ) return;
    refresh( x );
    print( x->l );
    cout << x->c;
    print( x->r );
  }

  void print(){ print( root ); cout << endl; }

  void refresh( Node *x ){
    if( x == nullptr || x->lazy == 0 ) return;
    x->lazy = 0;
    swap( x->l, x->r );
    if( x->l != nullptr ) x->l->lazy ^= 1;
    if( x->r != nullptr ) x->r->lazy ^= 1;
  }

  Node* merge( Node *a, Node *b ){
    refresh(a); refresh(b);

    if( a == nullptr ) return b;
    if( b == nullptr ) return a;

    if( a->prior > b->prior ){
      a->r = merge( a->r, b );
      return update(a);
    }
    else{
      b->l = merge( a, b->l );
      return update(b);
    }
  }

  pair<Node*, Node*> split( Node *root, int k ){
    if( root == nullptr ) return make_pair( nullptr, nullptr );
    refresh( root );
```

```
    if( k <= get_sub(root->l) ){
      auto [a, b] = split( root->l, k );
      root->l = b;
      return make_pair(a, update(root));
    }
    else{
      auto [a, b] = split( root->r, k - get_sub(root->l) - 1
          );
      root->r = a;
      return make_pair(update(root), b);
    }
  }

  tuple<Node*, Node*, Node*> split( Node* root, int l, int r
      ){
    auto [a, b] = split( root, r );
    auto [c, d] = split( a, l - 1 );
    return make_tuple( c, d, b );
  }

  void reverse( int l, int r ){
    auto [a, b, c] = split( root, l, r );
    b->lazy ^= 1;
    refresh( b );
    root = merge( merge( a, b ), c );
  }

  Treap( string &s ){
    v.resize(s.size());
    for( int i = 0; i < v.size(); i++ ){
      v[i] = Node(s[i]);
      root = merge( root, &v[i] );
    }
  }
};

int main(){
  int n, q; cin >> n >> q;
  string s; cin >> s;
  Treap treap( s );

  while( q-- ){
    int l, r; cin >> l >> r;
    treap.reverse( l, r );
  }
  treap.print();
}
```

10.25 Task Assignment

A company has n employees and there are n tasks that need to be done. We know for each employee the cost of carrying out each task. Every employee should be assigned to exactly one task. What is the minimum total cost if we assign the tasks optimally and how could they be assigned?

Input
The first input line has one integer n: the number of employees and the number of tasks that need to be done. After this, there are n lines each consisting of n integers. The ith line consists of integers $c_{i1}, c_{i2}, \ldots, c_{in}$: the cost of each task when it is assigned to the ith employee.

Output
First print the minimum total cost. Then print n lines each consisting of two integers a and b: you assign the bth task to the ath employee. If there are multiple solutions you can print any of them.

Constraints
• $1 \le n \le 200$ • $1 \le c_{ij} \le 1000$

```
const int maxn = 400 + 2;
const int inf = 2e9;

int cost[maxn][maxn], capacity[maxn][maxn], dist[maxn], pai[
    maxn];
vector<int> adj[maxn];

void dijkstra( int source ){
  fill( dist, dist + maxn, inf );
  fill( pai, pai + maxn, -1 );

  set<pair<int, int>> s;
  s.insert({ 0, source });
  dist[source] = 0;

  while( !s.empty() ){
    int cur = s.begin()->second; s.erase(s.begin());
    for( int viz : adj[cur] ) if( capacity[cur][viz] && dist[
        viz] > dist[cur] + cost[cur][viz] ){
      s.erase({ dist[viz], viz });
      dist[viz] = dist[cur] + cost[cur][viz];
      s.insert({ dist[viz], viz });

      pai[viz] = cur;
    }
  }
}

int min_cost_flow( int n ){
  int source = 2*n;
  int sink = 2*n + 1;

  for( int i = 0; i < n; i++ ){
    capacity[source][i] = 1;
    adj[source].push_back(i);
    adj[i].push_back(source);
  }

  for( int i = n; i < 2*n; i++ ){
    capacity[i][sink] = 1;
    adj[i].push_back(sink);
    adj[sink].push_back(i);
  }

  int flow = 0, cost = 0;
  while( flow < n ){
    dijkstra(source);

    int new_flow = n - flow;
    for( int cur = sink; cur != source; cur = pai[cur] )
      new_flow = min( new_flow, capacity[pai[cur]][cur] );

    flow += new_flow;
    cost += new_flow*dist[sink];

    for( int cur = sink; cur != source; cur = pai[cur] ){
      capacity[pai[cur]][cur] -= new_flow;
      capacity[cur][pai[cur]] += new_flow;
    }
  }
  return cost;
}

int main(){
  int n; cin >> n;

  for( int i = 0; i < n; i++ )
    for( int j = n; j < 2*n; j++ ){
      cin >> cost[i][j];
      cost[j][i] = -cost[i][j];

      capacity[i][j] = 1;

      adj[i].push_back(j);
      adj[j].push_back(i);
    }

  int min_cost = min_cost_flow(n);

  cout << min_cost << endl;
  for( int i = 0; i < n; i++ )
    for( int j = n; j < 2*n; j++ ) if( capacity[i][j] == 0 )
        cout << i + 1 << " " << j - n + 1 << endl;
}
```

11 Construction Problems

11.1 Chess Tournament

There will be a chess tournament of n players. Each player has announced the number of games they want to play. Each pair of players can play at most one game. Your task is to determine which games will be played so that everybody will be happy.

Input
The first input line has an integer n: the number of players. The players are numbered $1, 2, \ldots, n$. The next line has n integers $x_1, x_2, \ldots, x_n$: for each player, the number of games they want to play.

Output
First print an integer k: the number of games. Then, print k lines describing the games. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \leq n \leq 10^5$ • $\sum_{i=1}^{n} x_i \leq 2 \cdot 10^5$

```cpp
void solve(){
    int n; cin >> n;
    priority_queue<pii> pq;
    for(int i = 0; i < n; i++) {
        int d; cin >> d;
        if(d != 0) pq.push({d, i + 1});
    }

    vector<pii> ans;

    while(pq.size() > 0) {
        vector<pii> aux;
        auto [at, pos] = pq.top(); pq.pop();

        while(at > 0) {
            if(pq.size() == 0) {
                cout << "IMPOSSIBLE\n";
                return;
            }

            auto [nxt, nxt_pos] = pq.top(); pq.pop();
            aux.push_back({nxt - 1, nxt_pos});
            ans.push_back({pos, nxt_pos});
            at--;
        }

        for(auto [x, y] : aux) if(x > 0) pq.push({x, y});
    }

    cout << ans.size() << '\n';
    for(auto [x, y] : ans) cout << x << ' ' << y << '\n';
}
```

11.2 Distinct Sums Grid

Create an $n \times n$ grid that fulfills the following requirements: Each integer $1 \ldots n$ appears n times in the grid. If we create a set that consists of all sums in rows and columns, there are $2n$ distinct values.

Input
The only line has an integer n.

Output
Print a grid that fulfills the requirements. You can print any valid solution. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 1000$

```cpp
const int MAX = 1024;
int v[MAX][MAX];

int randRange(int hi) {
    return rand() % hi;
}

struct pii {
    int elem;
    int qtt;

    bool operator<(const pii& other) const {
        return elem < other.elem;
    }
};

void adc(set<pii> &sums, int elem, int qtt) {
    auto it = sums.find({elem, 0});
    if(it == sums.end()) {
        if(qtt > 0) sums.insert({elem, qtt});
    }
    else {
        int old_qtt = it -> qtt;
        sums.erase(it);
        if(old_qtt + qtt > 0) sums.insert({elem, old_qtt +
            qtt});
    }
}

void solve(){
    int n; cin >> n;
    if(n <= 3) {
        cout << "IMPOSSIBLE\n";
        return;
    }

    vector<int> vert(n), hor(n);
    set<pii> sums;
    sums.insert({n * (n + 1) / 2, n});
    for(int i = 0; i < n; i++) {
        sums.insert({n * (i + 1), 1});
        hor[i] = n * (i + 1);
        vert[i] = n * (n + 1) / 2;
        for(int j = 0; j < n; j++) {
            v[i][j] = i + 1;
        }
    }

    while(true) {
        int x1 = randRange(n), y1 = randRange(n);
        int x2 = randRange(n), y2 = randRange(n);

```

```
        if(n >= 10){
            vector<int> new_elem = {
                hor[x1] + v[x2][y2] - v[x1][y1],
                hor[x2] + v[x1][y1] - v[x2][y2],
                vert[y1] + v[x2][y2] - v[x1][y1],
                vert[y2] + v[x1][y1] - v[x2][y2]
            };

            bool flag = false;
            for(auto x : new_elem) {
                if(sums.find({x, 0}) != sums.end()) flag =
                    true;
            }
            if(flag) continue;
        }

        adc(sums, hor[x1], -1);
        hor[x1] += v[x2][y2] - v[x1][y1];
        adc(sums, hor[x1], +1);

        adc(sums, hor[x2], -1);
        hor[x2] += v[x1][y1] - v[x2][y2];
        adc(sums, hor[x2], +1);

        adc(sums, vert[y1], -1);
        vert[y1] += v[x2][y2] - v[x1][y1];
        adc(sums, vert[y1], +1);

        adc(sums, vert[y2], -1);
        vert[y2] += v[x1][y1] - v[x2][y2];
        adc(sums, vert[y2], +1);

        swap(v[x1][y1], v[x2][y2]);
        if(sums.size() == 2 * n) break;
    }

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            cout << v[i][j] << '␣';
        }
        cout << '\n';
    }
}
```

11.3 Filling Trominos

Your task is to fill an $n \times m$ grid using L-trominos (three squares that have an L-shape). For example, here is one way to fill a 4×6 grid:

Input
The first input line has an integer t: the number of tests. After that, there are t lines that describe the tests. Each line has two integers n and m.

Output
For each test, print YES if there is a solution, and NO otherwise. If there is a solution, also print n lines that each contain m letters between A–Z. Adjacent squares must have the same letter exactly when they belong to the same tromino. You can print any valid solution.

Constraints
• $1 \le t \le 100$ • $1 \le n, m \le 100$

```
using Matrix = vector<string>;

Matrix transposed( Matrix a ){
    Matrix at(sz(a[0]), string(sz(a), '#'));
    FOR(i, 0, sz(a)) FOR(j, 0, sz(a[0])) at[j][i] = a[i][j];
    return at;
}

Matrix solve( int n, int m, bool corner ){
    if( n <= 0 || m <= 0 ) return Matrix{};
    assert(n > 1 && m > 1);
    if( m%3 != 0 ) return transposed(solve(m, n, false));
    if( n%2 == 0 ){
        Matrix ans;
        for( int i = 0; i < n; i += 4 ){
            ans.emplace_back();
            FOR(j, 0, m/3) ans.back() += "AAB";
            ans.emplace_back();
            FOR(j, 0, m/3) ans.back() += "ABB";

            if( i + 2 == n ) break;

            ans.emplace_back();
            FOR(j, 0, m/3) ans.back() += "BAA";
            ans.emplace_back();
            FOR(j, 0, m/3) ans.back() += "BBA";
        }

        if( corner ) FOR(i, 0, n) FOR(j, 0, m) ans[i][j] +=
            24;

        return ans;
    }

    if( m%2 == 0 ){
        Matrix ans = solve(n - 3, m, corner);
        FOR(i, 0, 3) ans.emplace_back();

        for( int i = 0; i < m; i += 4 ){
            ans[n - 3] += "CC";
            ans[n - 2] += "CD";
            ans[n - 1] += "DD";

            if( i + 2 == m ) break;

            ans[n - 3] += "DD";
            ans[n - 2] += "CD";
            ans[n - 1] += "CC";
        }
        return ans;
    }

    assert(m >= 9);

    Matrix ans = solve(n - 5, m, false);
    FOR(i, 0, 5) ans.emplace_back();

    Matrix aux = solve(5, m - 9, true);

    ans[n - 5] = "EELLMNNRR" + ((m > 9) ? aux[0] : "");
    ans[n - 4] = "EFLMMNORS" + ((m > 9) ? aux[1] : "");
    ans[n - 3] = "FFHJJOOSS" + ((m > 9) ? aux[2] : "");
    ans[n - 2] = "GHHIJKPQQ" + ((m > 9) ? aux[3] : "");
    ans[n - 1] = "GGIIKKPPQ" + ((m > 9) ? aux[4] : "");

    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    if( (n*m)%3 != 0 || n == 1 || m == 1 ){ cout << "NO\n";
        return; }
    if( (n == 3 && m%2) || (m == 3 && n%2) ){ cout << "NO\n";
        return; }

    cout << "YES\n";
    Matrix ans = solve( n, m, false );

    FOR(i, 0, n){
        FOR(j, 0, m) cout << ans[i][j]; cout << '\n';
    }
}
```

11.4 Grid Path Construction

Given an $n \times m$ grid and two squares $a = (y_1, x_1)$ and $b = (y_2, x_2)$, create a path from a to b that visits each square exactly once. For example, here is a path from $a = (1, 3)$ to $b = (3, 6)$ in a 4×7 grid:

Input
The first input line has an integer t: the number of tests. After this, there are t lines that describe the tests. Each line has six integers n, m, y_1, x_1, y_2 and x_2. In all tests $1 \le y_1, y_2 \le n$ and $1 \le x_1, x_2 \le m$. In addition, $y_1 \ne y_2$ or $x_1 \ne x_2$.

Output
Print YES, if it is possible to construct a path, and NO otherwise. If there is a path, also print its description which consists of characters U (up), D (down), L (left) and R (right). If there are several paths, you can print any of them.

Constraints
• $1 \le t \le 100$ • $1 \le n \le 50$ • $1 \le m \le 50$

```
// NOT FINISHED

bool is_possible(int n, int m, int x1, int y1, int x2, int y2
    ) {
    if (n == 1) return (min(y1, y2) == 0 && max(y1, y2) == m
        - 1);
    if (m == 1) return (min(x1, x2) == 0 && max(x1, x2) == n
        - 1);
    int c1 = (x1 + y1) % 2;
    int c2 = (x2 + y2) % 2;
    if ((n * m) % 2 != 0) return c1 == c2 && c1 == 0;
    return c1 != c2;
}

void contra(string &s, char L, char R, char U, char D) {
    reverse(all(s));
    for(int i = 0; i < sz(s); i++) {
        if(s[i] == L) s[i] = R;
        else if(s[i] == R) s[i] = L;
        else if(s[i] == U) s[i] = D;
        else if(s[i] == D) s[i] = U;
    }
}

string solve_from_0(int n, int m, int x, int y, char L, char
    R, char U, char D) {
    string ans = "";

    if (n == 1) {
        for(int i = 0; i < m - 1; i++) ans += R;
        return ans;
    }
```

```
    if (m == 1) {
        for(int i = 0; i < n - 1; i++) ans += D;
        return ans;
    }

    if(n == 2 || m == 2) {
        if(m < n) return solve_from_0(m, n, y, x, U, D, L, R)
            ;

        for(int i = 0; i < y; i++) {
            if(i % 2) ans += U;
            else ans += D;

            ans += R;
        }

        if(y % 2) {
            for(int i = y + 1; i < m; i++) ans += R;
            ans += U;
            for(int i = y + 1; i < m; i++) ans += L;
        }
        else {
            for(int i = y + 1; i < m; i++) ans += R;
            ans += D;
            for(int i = y + 1; i < m; i++) ans += L;
        }

        return ans;
    }

    if(x != 0) {
        for(int i = 0; i < m - 1; i++) ans += R;
        ans += D;
        ans += solve_from_0(m, n - 1, (m - 1) - y, x - 1, U,
            D, R, L);
    }
    else {
        for(int i = 0; i < n - 1; i++) ans += D;
        ans += R;
        ans += solve_from_0(m - 1, n, y - 1, (n - 1) - x, D,
            U, L, R);
    }
    return ans;
}

string solve_from_any(int n, int m, int x1, int y1, int x2,
     int y2, char L, char R, char U, char D) {

}

void solve(){
    int n, m, x1, y1, x2, y2;
    cin >> n >> m >> x1 >> y1 >> x2 >> y2;
    x1--; y1--; x2--; y2--;

    if(!is_possible(n,m,x1,y1,x2,y2)) {
        cout << "NO\n";
        return;
    }

    string ans = solve_from_any(n, m, x1, y1, x2, y2, 'L', 'R
        ', 'U', 'D');
    if(sz(ans) == n * m - 1) {
        cout << "YES\n";
        cout << ans << endl;
    }
    else {
        cout << "NO\n";
    }
}
```

11.5 Inverse Inversions

Your task is to create a permutation of numbers $1, 2, \ldots, n$ that has exactly k inversions. An inversion is a pair (a, b) where $a<b$ and $p_a>p_b$ where p_i denotes the number at position i in the permutation.

Input
The only input line has two integers n and k.

Output
Print a line that contains the permutation. You can print any valid solution.

Constraints
• $1 \le n \le 10^6$ • $0 \le k \le \frac{n(n-1)}{2}$

```
void solve(){
    ll n, m; cin >> n >> m;
    vi ans;
    deque<int> list;

    for(int i = 1; i <= n; i++) list.push_back(i);

    for(int i = n; i >= 1; i--) {
        int add = 0;
        if(m >= i - 1) {
            add = list.back(); list.pop_back();
            m -= i - 1;
        }
        else {
            add = list.front(); list.pop_front();
        }
        ans.push_back(add);
    }

    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}
```

11.6 Monotone Subsequences

Your task is to create a permutation of numbers $1, 2, \ldots, n$ whose longest monotone subsequence has exactly k elements. A monotone subsequence is either increasing or decreasing. For example, some monotone subsequences in $[2, 1, 4, 5, 3]$ are $[2, 4, 5]$ and $[4, 3]$.

Input
The first input line has an integer t: the number of tests. After this, there are t lines. Each line has two integers n and k.

Output
For each test, print a line that contains the permutation. You can print any valid solution. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \le t \le 1000$ • $1 \le k \le n \le 100$

```cpp
void solveTest(){
  int n, k; cin >> n >> k;
  if( k*k < n ){ cout << "IMPOSSIBLE" << endl; return; }
  for( int i = 1; i <= ((n + k - 1)/k); i++ ) for( int j =
       min(i*k, n); j > (i - 1)*k; j-- ) cout << j << "␣";
  cout << endl;
}

int main(){
  int t; cin >> t;
  while( t-- ) solveTest();
}
```

11.7 Permutation Prime Sums

Given n, create two permutations a and b of size n such that $a_i + b_i$ is prime for $i = 1, 2, \ldots, n$.

Input
The only line has an integer n.

Output
Print two permutations. You can print any valid solution. If there are no solutions, print IMPOSSIBLE.

Constraints
- $1 \leq n \leq 10^5$

```cpp
// teorema: existe ao menos um primo p entre n e 2*n

int main(){
  int n; cin >> n;
  // crivo de eratostenes
  vector<bool> primo(2*n + 1, true);
  for( int i = 2; i < primo.size(); i++ ) if( primo[i] )
    for( int j = 2*i; j < primo.size(); j += i ) primo[j] =
         false;

  vector<int> a, b;

  while( n > 0 ){
    int p = n;
    while( !primo[n + p] ) p--;
    for( int j = p; j <= n; j++ ) a.push_back(j), b.push_back
         (n + p - j);
    n = p - 1;
  }

  for( int x : a ) cout << x << "␣"; cout << endl;
  for( int x : b ) cout << x << "␣"; cout << endl;

}
```

11.8 Third Permutation

You are given two permutations a and b such that $a_i \neq b_i$ in every position. Create a third permutation c such that $a_i \neq c_i$ and $b_i \neq c_i$ in every position.

Input
The first line has an integer n: the permutation size. The second line has n integers $a_1, a_2, \ldots, a_n$. The third line has n integers $b_1, b_2, \ldots, b_n$.

Output
Print n integers $c_1, c_2, \ldots, c_n$. You can print any valid solution. If there are no solutions, print IMPOSSIBLE.

Constraints
- $2 \leq n \leq 10^5$

```cpp
int main(){
  int n; cin >> n;
  vector<int> a(n), b(n), pos_a(n);
  for( int i = 0; i < n; i++ ){
    cin >> a[i];
    pos_a[--a[i]] = i;
  }
  for( int i = 0; i < n;i ++ ){
    cin >> b[i]; b[i]--;
  }

  if( n == 2 ){ cout << "IMPOSSIBLE" << endl; return 0; }

  set<int> all(a.begin(), a.end()), free;

  auto find = [&]( set<int>& s, int x ){ return s.find(x) !=
       s.end(); };

  vector<int> c(n, -1);

  for( int i = 0; i < n; i++ ) if( a[i] == b[pos_a[b[i]]] &&
       i < pos_a[b[i]] ){
    bool b1 = find( all, a[i] ), b2 = find( all, b[i] );
    all.erase(a[i]); all.erase(b[i]);

    set<int> &s = ((free.empty()) ? all : free);

    int v1 = *s.begin(); s.erase(s.begin());
    int v2 = *s.begin(); s.erase(s.begin());

    if( b1 ) free.insert(a[i]);
    if( b2 ) free.insert(b[i]);
    c[i] = v1; c[pos_a[b[i]]] = v2;
  }

  for( int i = 0; i < n; i++ ) if( c[i] == -1 ){
    int val = b[pos_a[b[i]]];
    if( find( all, val ) ){ c[i] = val; all.erase(val); }
    else{ c[i] = *free.begin(); free.erase(free.begin()); }
  }

  for( int x : c ) cout << x + 1 << "␣"; cout << endl;
}
```

12 Advanced Graph Problems

12.1 Acyclic Graph Edges

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is acyclic.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge between nodes a and b.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```
void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        cout << min(a, b) << ' ' << max(a, b) << '\n';
    }
}
```

12.2 Bus Companies

There are n cities and m bus companies. Each bus company operates in specific cities and sells tickets for a specific price. Buying a ticket from a bus company allows you to travel between any two cities that the company operates in. Determine the cost of the cheapest route from Syrjälä to every city.

Input
The first line has two integers n and m: the number of cities and bus companies. The cities are numbered $1, 2, \ldots, n$, and city 1 is Syrjälä. The next line has m integers $c_1, c_2, \ldots, c_m$: the ticket costs for each bus company. After that, there are m pairs of lines describing the cities for each bus company. The first line of each pair has a single integer k: the number of cities the bus company operates in. The second line of each pair has k distinct integers $a_1, a_2, \ldots, a_k$: the cities the bus company operates in. You can assume that it is possible to travel from Syrjälä to all other cities.

Output
Print n integers: the cheapest route costs from Syrjälä to cities $1, 2, \ldots, n$.

Constraints
• $1 \leq n, m \leq 10^5$ • $1 \leq c \leq 10^9$ • $2 \leq k \leq n$ • $1 \leq a \leq n$ • the sum of all k is at most $2 \cdot 10^5$

```
using ll = long long;

const ll inf = 1e18;

int main(){
  int n, m; cin >> n >> m;
  vector<int> cost(m);
  vector<vector<int>> adj(n + m);
  for( int &x : cost ) cin >> x;
  for( int i = 0; i < m; i++ ){
    int k; cin >> k;
    while( k-- ){
      int id; cin >> id;
      id--;
      adj[id].push_back(n + i);
      adj[n + i].push_back(id);
    }
  }

  vector<ll> dist(n + m, inf);

  auto dijkstra = [&]( int source ){
    set<pair<ll, int>> s;
    s.insert({ 0, source });
    dist[source] = 0;
    while( !s.empty() ){
      int cur = s.begin()->second; s.erase(s.begin());
      for( int viz : adj[cur] ){
        ll d = ((cur >= n) ? cost[cur - n] : 0 );
        if( dist[viz] > dist[cur] + d ){
          s.erase({ dist[viz], viz });
          dist[viz] = dist[cur] + d;
          s.insert({ dist[viz], viz });
        }
      }
    }
  };

  dijkstra(0);

  for( int i = 0; i < n; i++ ) cout << dist[i] << "␣"; cout
       << endl;
}
```

12.3 Course Schedule II

You want to complete n courses that have requirements of the form "course a has to be completed before course b". You want to complete course 1 as soon as possible. If there are several ways to do this, you want then to complete course 2 as soon as possible, and so on. Your task is to determine the order in which you complete the courses.

Input
The first input line has two integers n and m: the number of courses and requirements. The courses are numbered $1, 2, \ldots, n$. Then, there are m lines describing the requirements. Each line has two integers a and b: course a has to be completed before course b. You can assume that there is at least one valid schedule.

Output
Print one line having n integers: the order in which you complete the courses.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```
const int MAX = 1e5 + 10;
vector<int> G[MAX], ans;
int deg_in[MAX];

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[b].push_back(a);
        deg_in[a]++;
    }

    priority_queue<int> pq;

    for(int i = n; i >= 1; i--) {
        if(!deg_in[i]) pq.push(i);
    }

    while(!pq.empty()) {
        int at = pq.top(); pq.pop();
        ans.push_back(at);
        for(auto x : G[at]) {
            deg_in[x]--;
            if(deg_in[x] == 0) pq.push(x);
        }
    }

    reverse(all(ans));
    for(auto x : ans) cout << x << '␣';
    cout << '\n';
}
```

12.4 Creating Offices

There are n cities and $n-1$ roads between them. There is a unique route between any two cities, and their distance is the number of roads on that route. A company wants to have offices in some cities, but the distance between any two offices has to be at least d. What is the maximum number of offices they can have?

Input
The first input line has two integers n and d: the number of cities and the minimum distance. The cities are numbered $1, 2, \ldots, n$. After this, there are $n-1$ lines describing the roads. Each line has two integers a and b: there is a road between cities a and b.

Output
First print an integer k: the maximum number of offices. After that, print the cities which will have offices. You can print any

valid solution.

Constraints
• $1 \le n, d \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
using vi = vector<int>;

struct CentroidTree{
    vector<vi> adj, lvl;
    vi sub, opt, cpai, clvl;

    CentroidTree( vector<vi> adj ){
        int n = sz(adj); this->adj = adj;
        sub = opt = cpai = clvl = vi(n, 0);
        lvl = vector<vi>(__lg(n) + 1, vi(n));

        build( 0, -1, 0 );
    }

    void dfs( int u, int p, int d, int clvl ){
        lvl[clvl][u] = d;
        sub[u] = 1;
        for( int v : adj[u] ) if( v != p && !opt[v] ) { dfs( v, u, d + 1, clvl ); sub[u] += sub[v]; }
    }

    int find( int u, int p, int t ){
        for( int v : adj[u] ) if( v != p && !opt[v] && sub[v] > t/2 ) return find( v, u, t );
        return u;
    }

    void build( int u, int p, int d  ){
        dfs(u, u, 0, d);
        int c = find( u, u, sub[u] );
        dfs(c, c, 0, d);

        cpai[c] = p; clvl[c] = d; opt[c] = sz(adj);
        for( int v : adj[c] ) if( !opt[v] ) build( v, c, d + 1 );
    }

    void enable( int u ){
        for( int c = u; c != -1; c = cpai[c] ) opt[c] = min( opt[c], lvl[clvl[c]][u] );
    }

    int closest( int u ){
        int ans = opt[u];
        for( int c = u; c != -1; c = cpai[c] ) ans = min( ans, opt[c] + lvl[clvl[c]][u] );
        return ans;
    }
};

void solve(){
    int n, k; cin >> n >> k;
    vector<vi> adj(n);
    for( int i = 1; i < n; i++ ){
        int u, v; cin >> u >> v; u--; v--;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    CentroidTree tree(adj);

    vi ordem(n);
    iota( all(ordem), 0 );
    sort( all(ordem), [&]( int u, int v ){
        return tree.lvl[0][u] > tree.lvl[0][v];
    });

    vi ans;
    for( int u : ordem ) if( tree.closest(u) >= k ){
        tree.enable(u);
        ans.push_back(u);
    }

    cout << sz(ans) << '\n';
    for( int x : ans ) cout << x + 1 << " "; cout << '\n';
}
```

12.5 Critical Cities

There are n cities and m flight connections between them. A city is called a critical city if it appears on every route from a city to another city. Your task is to find all critical cities from Syrjälä to Lehmälä.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. Then, there are m lines describing the connections. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way. You may assume that there is a route from Syrjälä to Lehmälä.

Output
First print an integer k: the number of critical cities. After this, print k integers: the critical cities in increasing order.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
using vi = vector<int>;
using pii = pair<int, int>;

const int maxn = 1e5;

vi adj[2][maxn];

vi bfs( int n ){
    vi lvl(n, n);
    lvl[0] = 0;
    queue<int> q;
    for( q.push(0); !q.empty(); q.pop() ){
        int u = q.front();
        for( int v : adj[0][u] ) if( lvl[v] > lvl[u] + 1 )
            lvl[v] = lvl[u] + 1, q.push(v);
    }

    set<pii> s;

    vi marc(n), ans;

    marc[n - 1] = true;
    for( s.insert({ lvl[n - 1], n - 1}); !s.empty(); ){
        int u = s.rbegin()->second; s.erase(prev(s.end()));
        if( s.empty() ) ans.push_back(u);

        for( int v : adj[1][u] ) if( !marc[v] )
            marc[v] = true, s.insert({ lvl[v], v });
    }

    sort(all(ans));

    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    while( m-- ){
        int u, v; cin >> u >> v; u--; v--;
        if( u == n - 1 || v == 0 ) continue;
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    vi ans = bfs(n);

    cout << sz(ans) << '\n';
    for( int x : ans ) cout << x + 1 << " "; cout << '\n';
}
```

12.6 Even Outdegree Edges

Given an undirected graph, your task is to choose a direction for each edge so that in the resulting directed graph each node has an even outdegree. The outdegree of a node is the number of edges coming out of that node.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. You may assume that the graph is simple, i.e., there is at most one edge between any two nodes and every edge connects two distinct nodes.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution. If there are no solutions, only print IMPOSSIBLE.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int MAX = 1e5 + 1;

vector<pii> G[MAX];
vector<pair<pii, bool>> edg;
bool vis[MAX];

int deg = 0;

void dfs(int at, int pai) {
    vis[at] = 1;
    for(auto [x, e] : G[at]) {
        if(!vis[x]) {
            dfs(x, at);
        }
    }
    deg = 0;
    int e_pai = -1;
```

```
    for(auto [x, e] : G[at]) {
        if(!edg[e].second && x != pai) {
            edg[e].second = 1;
            edg[e].first = {at, x};
            deg++;
        }
        else if(edg[e].second) {
            deg += (edg[e].first.first == at);
        }
        else {
            e_pai = e;
        }
    }

    if(e_pai == -1) return;

    if(deg % 2 == 0) edg[e_pai].first = {pai, at};
    else edg[e_pai].first = {at, pai};

    edg[e_pai].second = 1;
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back({b, i});
        G[b].push_back({a, i});
        edg.push_back({{a, b}, 0});
    }

    for(int i = 1; i <= n; i++) {
        if(!vis[i]) dfs(i, 0);
        if(deg % 2) {
            cout << "IMPOSSIBLE\n";
            return;
        }
    }

    for(auto [par, def] : edg) {
        cout << par.first << ' ' << par.second << '\n';
    }

}
```

12.7 Fixed Length Walk Queries

You are given an undirected graph with n nodes and m edges. The graph is simple and connected. You start at a specific node, and on each turn you must move through an edge to another node. Your task is to answer q queries of the form: "is it possible to start at node a and end up on node b after exactly x turns?"

Input
The first line contains three integers n, m and q: the number of nodes, edges and queries. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines which describe the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines, each describing a query. Each line contains three integers a, b and x.

Output
For each query, print the answer (YES or NO) on its own line.

Constraints
• $2 \le n \le 2500$ • $1 \le m \le 5000$ • $1 \le q \le 10^5$ • $0 \le x \le 10^9$

```
vector<vector<int>> adj;
vector<int> dist;

void bfs( int source ){
  fill( dist.begin(), dist.end(), 1e9 );
  queue<int> fila;
  fila.push(2*source);
  dist[2*source] = 0;
  while( !fila.empty() ){
    int cur = fila.front(); fila.pop();
    for( int viz : adj[cur/2] ){
      int id = (dist[cur] + 1)%2;
      if( dist[2*viz + id] > dist[cur] + 1 ){ fila.push(2*viz
          + id); dist[2*viz + id] = dist[cur] + 1; }
    }
  }
}

int main(){
  int n, m, q; cin >> n >> m >> q;
  adj.resize(n);
  dist.resize(2*n);
  while( m-- ){
    int a, b; cin >> a >> b;
    a--; b--;
    adj[a].push_back(b);
    adj[b].push_back(a);
  }

  vector<vector<tuple<int, int, int>>> queries(n);
  vector<bool> resp(q);
  for( int i = 0; i < q; i++ ){
    int a, b, c; cin >> a >> b >> c;
    a--; b--;
    queries[a].emplace_back( b, c, i );
  }

  for( int i = 0; i < n; i++ ){
    bfs(i);
    for( auto [b, x, id] : queries[i] ){
      int parity = x%2;
      resp[id] = (( dist[2*b + parity] <= x ) ? true : false
          );
    }
  }

  for( bool x : resp ) cout << (( x ) ? "YES" : "NO" ) <<
      endl;
}
```

12.8 Flight Route Requests

There are n cities with airports but no flight connections. You are given m requests which routes should be possible to travel. Your task is to determine the minimum number of one-way flight connections which makes it possible to fulfil all requests.

Input
The first input line has two integers n and m: the number of cities and requests. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines describing the requests. Each line has two integers a and b: there has to be a route from city a to city b. Each request is unique.

Output
Print one integer: the minimum number of flight connections.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
using vi = vector<int>;

const int maxn = 1e5;
vi adj[2][maxn];
int in[maxn];
bool marc[maxn];

void dfs( int u, vi &c ){
    marc[u] = true;
    c.push_back(u);
    for( int t = 0; t < 2; t++ )
    for( int v : adj[t][u] ) if( !marc[v] ) dfs(v, c);
}

bool check_cycle( int node ){
    vi c;
    dfs( node, c );

    queue<int> q;
    for( int u : c ) if( !in[u] ) q.push(u);

    for(;!q.empty(); q.pop() ){
        int u = q.front();
        for( int v : adj[0][u] ) if( --in[v] == 0 ) q.push(v)
            ;
    }

    for( int u : c ) if( in[u] ) return true;
    return false;
}

void solve(){
    int n, m; cin >> n >> m;
    while( m-- ){
        int u, v; cin >> u >> v; u--; v--;
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
        in[v]++;
    }

    int ans = n;
    for( int i = 0; i < n; i++ ) if( !marc[i] && !check_cycle
        (i) ) ans--;
    cout << ans << endl;
}
```

12.9 Forbidden Cities

There are n cities and m roads between them. Kaaleppi is currently in city a and wants to travel to city b. However, there is a problem: Kaaleppi has recently robbed a bank in city c and can't enter the city, because the local police would catch him. Your task is to find out if there is a route from city a to city b that does not visit city c. As an additional challenge, you have

to process q queries where a, b and c vary.

Input
The first input line has three integers n, m and q: the number of cities, roads and queries. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the roads. Each line has two integers a and b: there is a road between cities a and b. Each road is bidirectional. Finally, there are q lines describing the queries. Each line has three integers a, b and c: is there a route from city a to city b that does not visit city c? You can assume that there is a route between any two cities.

Output
For each query, print "YES", if there is such a route, and "NO" otherwise.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le q \le 10^5$ • $1 \le a, b, c \le n$

```cpp
using vi = vector<int>;

const int maxn = 1e5;
const int logn = 18;

int pai[maxn][logn], tin[maxn], tout[maxn], low[maxn];
int dfs_time;

vi adj[maxn];

void dfs( int u, int p ){

    pai[u][0] = p;
    for( int i = 1; i < logn; i++ ) pai[u][i] = pai[pai[u][i - 1]][i - 1];

    tin[u] = low[u] = ++dfs_time;

    int cnt = 0;
    for( int v : adj[u] ) if( v != p ){
        if( !tin[v] ){
            cnt++;

            dfs( v, u );
            low[u] = min( low[u], low[v] );
        }
        else low[u] = min( low[u], tin[v] );
    }
    tout[u] = dfs_time;
}

bool upper( int u, int v ){
    return tin[u] <= tin[v] && tout[v] <= tout[u];
}

int lca( int u, int v ){
    if( upper( u, v ) ) return u;
    if( upper( v, u ) ) return v;

    for( int i = logn - 1; i >= 0; i-- ) if( !upper( pai[u][i], v ) ) u = pai[u][i];
    return pai[u][0];
}

int lift( int u, int v ){
    assert(upper( v, u ));
    for( int i = logn - 1; i >= 0; i-- ) if( !upper( pai[u][i], v ) ) u = pai[u][i];
    return u;
}

bool check_path( int a, int b, int c ){

    if( a == c || b == c ) return true;
    if( !upper( lca(a, b), c ) ) return false;

    if( upper( c, a ) && tin[c] <= low[lift( a, c )] ) return true;
    if( upper( c, b ) && tin[c] <= low[lift( b, c )] ) return true;
    return false;
}

void solve(){
    int n, m, q; cin >> n >> m >> q;
    for( int i = 0; i < m; i++ ){
        int u, v; cin >> u >> v; u--; v--;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    dfs( 0, 0 );

    while( q-- ){
        int a, b, c; cin >> a >> b >> c;
        a--; b--; c--;

        if( !check_path( a, b, c ) ) cout << "YES\n";
        else cout << "NO\n";
    }
}
```

12.10 Graph Coloring

You are given a simple graph with n nodes and m edges. Your task is to use the minimum possible number of colors to color each node such that no edge connects two nodes of the same color.

Input
The first line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. Then there are m lines describing the edges. Each line has two integers a and b: there is an edge connecting nodes a and b.

Output
First, print an integer k: the minimum number of colors. After this, print n integers $c_1, c_2, \ldots, c_n$: the colors of the nodes. The colors should satisfy $1 \le c_i \le k$. You may print any valid solution.

Constraints
• $1 \le n \le 16$ • $0 \le m \le \frac{n(n-1)}{2}$

```cpp
const int MAX = 16;

vi G[MAX];
pii dp[1 << MAX];

bool check(int mask) {
    for(int i = 0; i < MAX; i++) {
        if(!((1 << i) & mask)) continue;
        for(auto x : G[i])
        if((1 << x) & mask)
            return false;
    }
    return true;
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a - 1].push_back(b - 1);
        G[b - 1].push_back(a - 1);
    }

    for(int mask = 1; mask < (1 << n); mask++) {
        if(check(mask)) {
            dp[mask] = {1, mask};
            continue;
        }

        for (int x = mask; x; ) {
            --x &= mask;
            if(dp[x].first != 1) continue;
            int novo = dp[x].first + dp[mask ^ x].first;
            if(dp[mask].first == 0 || dp[mask].first > novo) {
                dp[mask].first = novo;
                dp[mask].second = x;
            }
        }
    }

    int k = (1 << n) - 1;
    cout << dp[k].first << '\n';
    vector<int> ans(MAX, 0);
    int c = 1;
    while(k != 0) {
        for(int i = 0; i < MAX; i++) {
            if((1 << i) & dp[k].second) ans[i] = c;
        }
        k = dp[k].second ^ k;
        c++;
    }
    for(int i = 0; i < n; i++) {
        cout << ans[i] << ' ';
    }
    cout << '\n';

}
```

12.11 Graph Girth

Given an undirected graph, your task is to determine its girth, i.e., the length of its shortest cycle.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. You may assume that there is at most one edge between each two nodes.

Output
Print one integer: the girth of the graph. If there are no cycles, print -1.

Constraints
• $1 \leq n \leq 2500$ • $1 \leq m \leq 5000$

```
const int maxn = 2510;

vector<int> adj[maxn];

int nivel[maxn], pai[maxn];
bool marc[maxn];

queue<int> fila;

int resp = maxn;

void reseta(){

    for( int i = 0; i < maxn; i++ ){
        marc[i] = false;
        pai[i] = 0;
        nivel[i] = 0;
    }

}

void add( int p, int x, int n ){

    nivel[x] = n;
    marc[x] = true;

    fila.push(x);

    pai[x] = p;
}

void bfs( int source ){

    add( 0, source, 0 );

    while( !fila.empty() ){

        int cur = fila.front();
        fila.pop();

        for( int i = 0; i < adj[cur].size(); i++ ){

            int viz = adj[cur][i];

            if( !marc[viz] ) add( cur, viz, nivel[cur] + 1 );
            else if( viz != pai[cur] ){

                resp = min( resp, nivel[viz] + nivel[cur] + 1
                    );

            }
        }
    }
}

int main(){

    int n, m; scanf("%d %d", &n, &m );

    for( int i = 0; i < m; i++ ){

        int a, b; scanf("%d %d", &a, &b );

        adj[a].pb(b);
        adj[b].pb(a);
    }

    for( int i = 1; i <= n; i++ ){

        bfs(i);

        reseta();
    }

    if( resp == maxn ) resp = -1;

    printf("%d", resp );
}
```

12.12 MST Edge Cost

Given an undirected weighted graph, determine for each edge the minimum spanning tree cost if the edge must be included in the spanning tree.

Input
The first line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. The following m lines describe the edges. Each line has three integers a, b, w: there is an edge between nodes a and b with weight w. You can assume that the graph is connected and simple and each edge appears at most once in the graph.

Output
For each edge in the input order, print the minimum spanning tree cost when the edge is included.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$ • $1 \leq w \leq 10^9$

```
using vi = vector<int>;
using trio = tuple<int, int, int>;
using ll = long long;
using pii = pair<int, int>;

struct BinaryLifting{
    vector<vector<pii>> adj;
    vector<vi> pai, maxi;
    vi tin, tout;

    int t = -1;

    BinaryLifting( vector<vector<pii>> adj ) : adj(adj){
        int n = sz(adj);
        pai = maxi = vector<vi>(n, vi(__lg(n) + 1));
        tin = tout = vi(n);

        dfs(0, 0, 0);
    }

    void dfs( int u, int p, int pw ){
        pai[u][0] = p;
        maxi[u][0] = pw;
        for( int i = 1; i < sz(maxi[u]); i++ ){
            pai[u][i] = pai[pai[u][i - 1]][i - 1];
            maxi[u][i] = max( maxi[u][i - 1], maxi[pai[u][i -
                1]][i - 1] );
        }

        tin[u] = ++t;
        for( auto [v, w] : adj[u] ) if( v != p ) dfs( v, u, w
            );
        tout[u] = t;
    }

    bool upper( int u, int v ){
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }

    int query( int u, int v ){
        int ans = -1;
        for( int i = sz(pai[0]) - 1; i >= 0; i-- ){
            if( !upper( pai[u][i], v ) ){
                ans = max( ans, maxi[u][i] );
                u = pai[u][i];
            }
            if( !upper( pai[v][i], u ) ){
                ans = max( ans, maxi[v][i] );
                v = pai[v][i];
            }
        }

        if( !upper( u, v ) ) ans = max( ans, maxi[u][0] );
        if( !upper( v, u ) ) ans = max( ans, maxi[v][0] );

        return ans;
    }

    int lca( int u, int v ){
        if( upper( u, v ) ) return u;
        if( upper( v, u ) ) return v;

        for( int i = sz(pai[0]); i >= 0; i-- ) if( !upper(
            pai[u][i], v ) ) u = pai[u][i];
        return pai[u][0];
    }
};

struct DSU{
    vi pai;
    DSU( int n ) : pai(n){
        iota(all(pai), 0);
    }

    int find( int u ){
        return (( u == pai[u] ) ? u : pai[u] = find(pai[u]));
    }

    void join( int u, int v ){
        pai[find(u)] = find(v);
    }
};

void solve(){
    int n, m; cin >> n >> m;

    vector<trio> edges(m);
    for( auto &[w, u, v] : edges ){
        cin >> u >> v >> w;
        u--; v--;
    }

    vector<trio> sorted = edges;
    sort(all(sorted));

    DSU dsu(n);
    vector<vector<pii>> adj(n);

    ll cost = 0;
```

```
    for( auto [w, u, v] : sorted ) if( dsu.find(u) != dsu.
        find(v) ){
      cost += w;
      dsu.join(u, v);

      adj[u].push_back({ v, w });
      adj[v].push_back({ u, w });
    }

    BinaryLifting tree(adj);
    for( auto [w, u, v] : edges ) cout << cost + w - tree.
        query(u, v) << '\n';
}
```

12.13 Mst Edge Check

Given an undirected weighted graph, determine for each edge if it can be included in a minimum spanning tree.

Input
The first line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. The following m lines describe the edges. Each line has three integers a, b, w: there is an edge between nodes a and b with weight w. You can assume that the graph is connected and simple and each edge appears at most once in the graph.

Output
For each edge in the input order, print YES if it can be included in the minimum spanning tree and NO otherwise.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$ • $1 \leq w \leq 10^9$

```
using quadrupla = tuple<int, int, int, int>;

int main(){
  int n, m; cin >> n >> m;
  vector<quadrupla> arestas;
  for( int i = 0; i < m; i++ ){
    int a, b, c; cin >> a >> b >> c;
    a--; b--;
    arestas.emplace_back( c, a, b, i );
  }
  sort( arestas.begin(), arestas.end() );

  vector<int> pai(n); iota( pai.begin(), pai.end(), 0 );

  function<int(int)> find = [&]( int a ){ return (( pai[a] ==
      a ) ? a : pai[a] = find(pai[a])); };
  auto join = [&]( int a, int b ){
    pai[find(a)] = find(b);
  };

  vector<bool> resp(m);
  for( int l = 0, r = 0; l < m; l = r ){
    while( (r < m) && (get<0>(arestas[r]) == get<0>(arestas[l
        ])) ) r++;
    for( int i = l; i < r; i++ ){
      auto [c, a, b, id] = arestas[i];
      if( find(a) != find(b) ) resp[id] = true;
    }
    for( int i = l; i < r; i++ ) join( get<1>(arestas[i]),
        get<2>(arestas[i]) );
  }
  for( bool x : resp ) cout << (( x ) ? "YES" : "NO" ) <<
      endl;
}
```

12.14 Mst Edge Set Check

Given an undirected weighted graph and edge sets, determine for each set if the edges can be included in a minimum spanning tree.

Input
The first line has three integers n, m and q: the number of nodes, edges and edge sets. The nodes are numbered $1, 2, \ldots, n$. The following m lines describe the edges. Each line has three integers a, b, w: there is an edge between nodes a and b with weight w. The edges are numbered $1, 2, \ldots, m$ in the input order. The following $2q$ lines describe the edge sets. For each set, the first line contains its size and the second line contains its edges. The total number of edges in all sets is at most m. You can assume that the graph is connected and simple and each edge appears at most once in the graph.

Output
For each edge set, print YES if the edges can be included in the minimum spanning tree and NO otherwise.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m, q \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$ • $1 \leq w \leq 10^9$

```
using quadrupla = tuple<int, int, int, int>;

class unionFind{
private:
  vector<int> pai, tam;
  stack<int> pilha;
  int find( int a, int t ){
    if( a == pai[a] ) return a;
    if( t == 0 ) return find(pai[a], t);
    return pai[a] = find(pai[a], t);
  }
public:
  unionFind( int n ) { pai.resize(n); iota( pai.begin(), pai.
      end(), 0 ); tam.resize(n, 1); }

  void rollback(){
    int id = pilha.top(); pilha.pop();
    if( id != -1 ){ tam[pai[id]] -= tam[id]; pai[id] = id; }
  }

  void join( int a, int b, int t ){
    a = find(a, t);
    b = find(b, t);
    if( a == b ){ pilha.push(-1); return; }
    if( tam[a] < tam[b] ) swap( a, b );
    pai[b] = a;
    pilha.push(b);
  }

  bool check( int a, int b ){ return find(a, 0) == find(b, 0)
      ; }
};

int main(){
  int n, m, q; cin >> n >> m >> q;

  unionFind union_find(n);

  vector<quadrupla> arestas;
  for( int i = 0; i < m; i++ ){
    int a, b, c; cin >> a >> b >> c;
    a--; b--;
    arestas.emplace_back( c, a, b, i );
  }
  sort( arestas.begin(), arestas.end() );

  vector<vector<int>> queries(m);

  for( int i = 0; i < q; i++ ){
    int k; cin >> k;
    while( k-- ){
      int x; cin >> x;
      queries[--x].push_back(i);
    }
  }

  vector<bool> resp(q, true);
  vector<vector<int>> cur_edges(q);

  for( int l = 0, r = 0; l < m; l = r ){
    while( (r < m) && (get<0>(arestas[r]) == get<0>(arestas[l
        ])) ) r++;
    for( int i = l; i < r; i++ ) for( int x : queries[get<3>(
        arestas[i])] ) cur_edges[x].clear();

    vector<int> id_queries;
    for( int i = l; i < r; i++ ){
      auto [c, a, b, id] = arestas[i];
      for( int x : queries[id] ){
        cur_edges[x].push_back(i);
        if( cur_edges[x].size() == 1 ) id_queries.push_back(x
            );
      }
    }

    for( int x : id_queries ) if( resp[x] ){
      bool ok = true;
      int cont = 0;
      for( int i : cur_edges[x] ){
        auto [c, a, b, id] = arestas[i];
        if( union_find.check( a, b ) ){ ok = false; break; }
        cont++;
        union_find.join( a, b, 0 );
      }
      while( cont-- ) union_find.rollback();

      resp[x] = resp[x]&&ok;
    }

    for( int i = l; i < r; i++ ) union_find.join( get<1>(
        arestas[i]), get<2>(arestas[i]), 1 );
  }

  for( bool x : resp ) cout << (( x ) ? "YES" : "NO" ) <<
      endl;
}
```

12.15 Nearest Shops

There are n cities and m roads. Each road is bidirectional and connects two cities. It is also known that k cities have an anime shop. If you live in a city, you of course know the local anime shop well if there is one. You would like to find the nearest anime shop that is not in your city. For each city, determine the minimum distance to another city that has an anime shop.

Input
The first line has three integers n, m and k: the number of cities, roads and anime shops. The cities are numbered $1, 2, \ldots, n$. The next line contains k integers: the cities that have an anime shop. Finally, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b.

Output
Print n integers: for each city, the minimum distance to another city with an anime shop. If there is no such city, print -1 instead.

Constraints
• $1 \le k \le n \le 10^5$ • $0 \le m \le 2 \cdot 10^5$

```cpp
struct res {
    int start1 = -1;
    int dst1 = -1;
    int start2 = -1;
    int dst2 = -1;
};

vector<vector<int>> G;
vector<int> start;
vector<res> ans;
int k, n, m;

// faz uma bfs com todos os inicios e salva os dois melhores (primeiros que aparecem)
void bfs()
{
    ans.resize(n + 1);
    queue<pair<int, int>> fila;

    for(auto x : start) {
        fila.push({x, 1});
        ans[x].start1 = x;
        ans[x].dst1 = 0;
    }

    while(!fila.empty()) {
        auto [at, level] = fila.front(); fila.pop();
        for(auto viz : G[at]) {
            if(ans[viz].start1 == -1 && level == 1) {
                ans[viz].start1 = ans[at].start1;
                ans[viz].dst1 = ans[at].dst1 + 1;
                fila.push({viz, 1});
            }
            else if(ans[viz].start2 == -1) {
                if(ans[at].start1 != ans[viz].start1 && level == 1) {
                    ans[viz].start2 = ans[at].start1;
                    ans[viz].dst2 = ans[at].dst1 + 1;
                    fila.push({viz, 2});
                }
                else if(ans[at].start2 != -1 && level == 2) {
                    ans[viz].start2 = ans[at].start2;
                    ans[viz].dst2 = ans[at].dst2 + 1;
                    fila.push({viz, 2});
                }
            }
        }
    }
}

void solve() {
    cin >> n >> m >> k;
    G.resize(n + 1);
    for(int i = 0; i < k; i++) {
        int a; cin >> a;
        start.push_back(a);
    }

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    bfs();

    for(int i = 1; i <= n; i++) {
        if(ans[i].dst1 <= 0) cout << ans[i].dst2 << ' ';
        else cout << ans[i].dst1 << ' ';
    }
    cout << '\n';
}
```

12.16 Network Breakdown

Syrjälä's network has n computers and m connections between them. The network consists of components of computers that can send messages to each other. Nobody in Syrjälä understands how the network works. For this reason, if a connection breaks down, nobody will repair it. In this situation a component may be divided into two components. Your task is to calculate the number of components after each connection breakdown.

Input
The first input line has three integers n, m and k: the number of computers, connections and breakdowns. The computers are numbered $1, 2, \ldots, n$. Then, there are m lines describing the connections. Each line has two integers a and b: there is a connection between computers a and b. Each connection is between two different computers, and there is at most one connection between two computers. Finally, there are k lines describing the breakdowns. Each line has two integers a and b: the connection between computers a and b breaks down.

Output
After each breakdown, print the number of components.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le k \le m$ • $1 \le a, b \le n$

```cpp
const int maxn = 100010;

vector< pair< int, int > > connections, breakdowns;
vector<int> resp;

map< int, map< int, bool > > bd;

int edge[maxn], h[maxn];
bool marc[maxn];

int find( int a ){

    if( a == edge[a] ) return a;

    return edge[a] = find( edge[a] );
}

void join( int a, int b ){

    a = find(a);
    b = find(b);

    if( a == b ) return;

    if( h[a] < h[b] ) swap( a, b );

    edge[b] = a;

    h[a] = max( h[a], h[b] + 1);

}

void reseta(){

    for( int i = 0; i < maxn; i++){

        h[i] = 0;
```

```
38
39          edge[i] = i;
40      }
41  }
42
43  int main(){
44
45      reseta();
46
47      int n, m, k; scanf("%d␣%d␣%d", &n, &m, &k);
48
49      for( int i = 0; i < m; i++){
50
51          int a, b; scanf("%d␣%d", &a, &b);
52
53          connections.push_back({ a, b });
54      }
55
56      for( int i = 0; i < k; i++){
57
58          int a, b; scanf("%d␣%d", &a, &b);
59
60          bd[a][b] = true;
61          bd[b][a] = true;
62
63          breakdowns.push_back({ a, b });
64      }
65
66      for( int i = 0; i < m; i++){
67
68          pair<int, int> p = connections[i];
69
70          if( !bd[p.first][p.second] ) join( p.first, p.second
                );
71
72      }
73
74      for( int i = 0; i < maxn; i++) marc[i] = false;
75
76      int componentes = 0;
77
78      for( int i = 1; i <= n; i++){
79
80          int a = find(i);
81
82          if( !marc[a] ){
83
84              marc[a] = true;
85
86              componentes++;
87
88          }
89      }
90
91      while( breakdowns.size() > 0 ){
92
93          pair< int, int > p = breakdowns.back();
94          breakdowns.pop_back();
95
96          resp.push_back(componentes);
97
98          int a = find( p.first );
99          int b = find( p.second );
100
101         if( a != b ) componentes--;
102
103         join( a, b );
104     }
105
106     while( resp.size() > 0 ){
107
108         printf("%d\n", resp.back() );
109         resp.pop_back();
110     }
111 }
```

12.17 Network Renovation

Syrjälä's network consists of n computers and $n-1$ connections between them. It is possible to send data between any two computers. However, if any connection breaks down, it will no longer be possible to send data between some computers. Your task is to add the minimum number of new connections in such a way that you can still send data between any two computers even if any single connection breaks down.

Input
The first input line has an integer n: the number of computers. The computers are numbered $1, 2, \ldots, n$. After this, there are $n-1$ lines describing the connections. Each line has two integers a and b: there is a connection between computers a and b.

Output
First print an integer k: the minimum number of new connections. After this, print k lines describing the connections. You can print any valid solution.

Constraints
• $3 \le n \le 10^5$ • $1 \le a, b \le n$

```
1  int main(){
2    int n; cin >> n;
3    vector<vector<int>> adj(n);
4    for( int i = 0; i < n - 1; i++ ){
5      int a, b; cin >> a >> b;
6      a--; b--;
7      adj[a].push_back(b);
8      adj[b].push_back(a);
9    }
10   vector<int> folhas;
11   function<void(int, int)> dfs = [&]( int cur, int pai ){
12     for( int viz : adj[cur] ) if( viz != pai ) dfs( viz, cur
           );
13     if( adj[cur].size() == 1 ) folhas.push_back(cur);
14   };
15
16   dfs( 0, 0 );
17
18   int k = folhas.size();
19   cout << (k + 1)/2 << endl;
20   for( int i = 0; i < k/2; i++ )
21     cout << folhas[i] + 1 << "␣" << folhas[i + k/2] + 1 <<
         endl;
22   if( k%2 ) cout << folhas[0] + 1 << "␣" << folhas.back() + 1
         << endl;
23 }
```

12.18 New Flight Routes

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output
First print an integer k: the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
1  using vi = vector<int>;
2  using pii = pair<int, int>;
3
4  struct SCC{
5      vector<vector<vi>> adj;
6      vector<vi> adjc;
7      vi scc, inc, outc;
8
9      SCC( vector<vector<vi>> adj ) : adj(adj) {
10         int n = sz(adj[0]);
11         inc = outc = vi(n);
12         adjc.resize(n);
13         kosaraju();
14     }
15
16     void dfs( int u, int t, vi &o ){
17         for( int v : adj[t][u] ) if( scc[v] == -1 ){
18             scc[v] = scc[u]; dfs( v, t, o );
19         }
20         if(!t) o.push_back(u);
21     }
22
23     void kosaraju(){
24         int n = sz(adj[0]);
25
26         vi o;
27         scc.assign(n, -1);
28         FOR(i, 0, n) if( scc[i] == -1 ){
29             scc[i] = i;
30             dfs( i, 0, o );
31         }
32
33         scc.assign(n, -1);
34         reverse(all(o));
35
36         for( int i : o ) if( scc[i] == -1 ){
37             scc[i] = i;
38             dfs( i, 1, o );
39         }
40
41         reverse(all(o));
```

```
        FOR(i, 0, n) for( int v : adj[0][i] ) if( scc[i] !=
             scc[v] ){
            inc[scc[v]]++;
            outc[scc[i]]++;
            adjc[scc[i]].push_back(scc[v]);
        }

        FOR(i, 0, n){
            sort(all(adjc[i]));
            adjc[i].erase(unique(all(adjc[i])), adjc[i].end()
                 );
        }
    }
};

const int maxn = 1e5 + 10;
bool marc[maxn];

int dfs( int u, SCC &scc ){
    marc[u] = true;
    if( scc.outc[u] == 0 ) return u;

    for( int v : scc.adjc[u] ) if( !marc[v] ){
        int x = dfs( v, scc );
        if( x != -1 ) return x;
    }
    return -1;
}

int calc( SCC &scc ){
    int n = sz(scc.scc);
    int in0 = 0, out0 = 0, tot = 0;

    FOR(i, 0, n) if( scc.scc[i] == i ){
        in0 += (scc.inc[i] == 0);
        out0 += (scc.outc[i] == 0);
        tot++;
    }

    return (tot == 1) ? 0 : max(in0, out0);
}

vector<pii> build_edges( SCC &scc ){
    int n = sz(scc.scc);

    vi in, out;
    vector<pii> edges;

    int ci = -1, cf = -1;

    FOR(i, 0, n) if( scc.scc[i] == i && scc.inc[i] == 0 ){
        int o = dfs( i, scc );
        if( o == -1 ){ in.push_back(i); continue; }
        if( ci == -1 ) tie(ci, cf) = pii( i, o );
        else{
            edges.push_back({ cf, i }); cf = o;
        }
    }

    edges.push_back({ cf, ci });

    FOR(i, 0, n) if( scc.scc[i] == i && scc.outc[i] == 0 && !
         marc[i] ) out.push_back(i);

    while( !in.empty() && !out.empty() ){
        int a = in.back(); in.pop_back();
        int b = out.back(); out.pop_back();

        edges.push_back({ b, a });
    }

    for( int u : out ) edges.push_back({ u, ci });
    for( int u : in ) edges.push_back({ ci, u });

    return edges;
}

void solve(){
    int n, m; cin >> n >> m;
    vector<vector<vi>> adj(2, vector<vi>(n));

    while( m-- ){
        int u, v; cin >> u >> v; u--; v--;
        adj[0][u].push_back(v);
        adj[1][v].push_back(u);
    }

    SCC scc(adj);

    int ans = calc(scc);
    cout << ans << '\n';

    if( ans == 0 ) return;

    vector<pii> edges = build_edges(scc);
    for( auto [a, b] : edges ) cout << a + 1 << " " << b + 1
         << '\n';
}
```

12.19 Prufer Code

A Prüfer code of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree. The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbor to the code, and remove the leaf from the tree. Given a Prüfer code of a tree, your task is to construct the original tree.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. The second line contains $n-2$ integers: the Prüfer code.

Output
Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b: there is an edge between nodes a and b. You can print the edges in any order.

Constraints
• $3 \le n \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
int main(){
  int n; cin >> n;
  vector<int> grau(n), v(n - 2);
  for( int &x : v ){ cin >> x; grau[--x]++; }
  set<int> s;
  for( int i = 0; i < n; i++ ) if( grau[i] == 0 ) s.insert(i)
       ;
  for( int i = 0; i < n - 2; i++ ){
    int cur = *s.begin(); s.erase(s.begin());
    cout << cur + 1 << " " << v[i] + 1 << endl;
    if( --grau[v[i]] == 0 ) s.insert(v[i]);
  }
  cout << *s.begin() + 1 << " " << *s.rbegin() + 1 << endl;
}
```

12.20 Split Into Two Paths

You are given an acyclic directed graph with n nodes and m edges. Determine whether two paths can be formed in the graph such that each node of the graph appears in exactly one of the paths. Note that all edges of the graph do not need to appear in the paths.

Input
The first line has two integers n and m: the number of nodes and the number of edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines which describe the edges. Each line has two integers a and b: there is an edge in the graph from node a to node b.

Output
First print the line YES if the paths can be formed, or NO otherwise. If the paths can be formed, print them on the following two lines. At the beginning of both lines, print the amount of nodes in the path and then the nodes of the path in order. There must be an edge in the graph between subsequent nodes. One of the paths may contain zero nodes. If there exist multiple solutions, you can print any solution.

Constraints
• $2 \le n \le 2 \cdot 10^5$ • $0 \le m \le 5 \cdot 10^5$

```
vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), q;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
    rep(j,0,sz(q)) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}

void solve(){
    int n, m; cin >> n >> m;
    vector<vi> G(n+1), Grev(n+1);

    for(int i = 1; i <= n; i++) {
        G[0].push_back(i);
        Grev[i].push_back(0);
    }

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        Grev[b].push_back(a);
    }

    auto top = topoSort(G);
    vi rec(n+1);
    set<int> S = {0};

    vector<bool> liga(n + 1);
    for(int i = 1; i <= n; i++) {
        int v = top[i], u = top[i-1];
        for(auto x : Grev[v]) {
            if(x == u) {
                liga[i] = true;
                break;
            }
```

```
        }
    }

    for(int i = 2; i <= n; i++) {
        int v = top[i], u = top[i-1];
        int q = -1;

        for(auto x : Grev[v]) {
            auto it = S.find(x);
            if(it != S.end()) {
                q = *it;
                break;
            }
        }

        if(!liga[i]) {
            if(q != -1) {
                rec[i] = q;
                S.clear();
                S.insert(u);
            }
            else {
                cout << "NO\n";
                return;
            }
        }
        else {
            if(q != -1) {
                rec[i] = q;
                S.insert(u);
            }
        }
    }

    cout << "YES\n";
    vi color(n + 1);
    color[top[n]] = 1;
    for(int i = n; i >= 1; i--) {
        int v = top[i];
        int u = top[i-1];

        if(color[u] == 0 && liga[i]) {
            color[u] = color[v];
        }
        else if(color[u] == 0 && !liga[i]) {
            color[u] = 3 - color[v];
        }

        if(color[u] != color[v]) {
            color[rec[i]] = color[v];
        }
    }

    vi c[2];
    for(int i = 1; i <= n; i++) {
        c[color[top[i]] - 1].push_back(top[i]);
    }

    for(int q = 0; q <= 1; q++){
        cout << c[q].size() << ' ';
        for(auto x : c[q]) cout << x << ' '; cout << '\n';
    }
}
```

12.21 Strongly Connected Edges

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is strongly connected.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. You may assume that the graph is simple, i.e., there are at most one edge between two nodes and every edge connects two distinct nodes.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution. If there are no solutions, only print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

```
const int MAX = 1e5 + 1;

vi G[MAX];
int vis[MAX];
int pai[MAX];

int cnt = 0;

void dfs(int at) {
    cnt++; vis[at] = cnt;
    for(auto x : G[at]) {
        if(!vis[x]) {
            pai[x] = at;
            dfs(x);
        }
    }
}

vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ?: dfs(e,g,f));
    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}

void solve(){
    int n, m; cin >> n >> m;
    vector<pii> edg(m);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
        edg[i] = {a, b};
    }

    dfs(1);
    if(cnt != n) {
        cout << "IMPOSSIBLE\n";
        return;
    }

    vector<vi> G_aux(n + 1);

    for(auto [a, b] : edg) {
        if(pai[a] == b) {
            G_aux[b].push_back(a);
            continue;
        }
        if(pai[b] == a) {
            G_aux[a].push_back(b);
            continue;
        }
        if(vis[a] > vis[b]) swap(a, b);
        G_aux[b].push_back(a);
    }

    int res = 0;

    scc(G_aux, [&](vi& c) {
        res = max(res, sz(c));
    });

    if (res == n) {
        for(int i = 1; i <= n; i++) {
            for(auto x : G_aux[i]) {
                cout << i << ' ' << x << '\n';
            }
        }
    } else {
        cout << "IMPOSSIBLE\n";
    }

}
```

12.22 Transfer Speeds Sum

A computer network has n computers and $n-1$ connections between two computers. Information can be exchanged between every pair of computers using the connections. Each connection has a certain transfer speed. Let $d(a, b)$ denote the transfer speed between computers a and b, which is the speed of the slowest connection on the route between a and b. Your task is to compute the sum of transfer speeds between all pairs of computers.

Input
The first line contains the integer n: the number of computers. The computers are numbered $1, 2, \ldots, n$. After this, there are $n-1$ lines, which describe the connections. Each line has three integers a, b and x: there is a connection between computers a and b with transfer speed x.

Output
Print one integer: the sum of transfer speeds.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x \le 10^6$

```cpp
using ll = long long;

int main(){
  int n; cin >> n;
  vector<tuple<int, int, int>> arestas;
  for( int i = 0; i < n - 1; i++ ){
    int a, b, c; cin >> a >> b >> c;
    a--; b--;
    arestas.emplace_back(c, a, b);
  }

  sort( arestas.rbegin(), arestas.rend() );

  vector<int> pai(n), tam(n, 1); iota( pai.begin(), pai.end()
       , 0 );

  function<int(int)> find = [&]( int a ){ return (( a == pai[
       a] ) ? a : pai[a] = find(pai[a])); };
  auto join = [&]( int a, int b ){
    a = find(a); b = find(b);
    if( tam[a] < tam[b] ) swap( a, b );
    pai[b] = a;
    tam[a] += tam[b];
  };

  ll resp = 0;
  for( auto [c, a, b] : arestas ){
    resp += 1LL*tam[find(a)]*tam[find(b)]*c;
    join( a, b );
  }

  cout << resp << endl;
}
```

12.23 Tree Coin Collecting I

You are given a tree with n nodes. Some nodes contain a coin. Your task is to answer q queries of the form: what is the shortest length of a path from node a to node b that visits a node with a coin?

Input
The first line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The second line contains n integers $c_1, c_2, \ldots, c_n$. If $c_i = 1$, node i has a coin. If $c_i = 0$, node i doesn't have a coin. You can assume at least one node has a coin. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integers a and b: the start and end nodes.

Output
Print q integers: the answers to the queries.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```cpp
const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
bool coin[MAX];
int tam = 0;
vi adj[MAX];

int up[MAX][LOG];
int dist_to_coin[MAX][LOG];
int depth[MAX];

void calc_dist_to_coin() {
    queue<int> q;

    for(int i = 1; i <= n; i++) {
        dist_to_coin[i][0] = INF;
        if(coin[i]) {
            q.push(i);
            dist_to_coin[i][0] = 0;
        }
    }

    while(!q.empty()) {
        int at = q.front(); q.pop();
        for(auto x : adj[at]) {
            if(dist_to_coin[x][0] == INF) {
                dist_to_coin[x][0] = dist_to_coin[at][0] + 1;
                q.push(x);
            }
        }
    }
}

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
        dist_to_coin[s][i] = min(dist_to_coin[s][i-1],
             dist_to_coin[up[s][i-1]][i-1]);
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}

int min_dist(int a, int b) {
    int ans = INF;
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) {
            ans = min(ans, dist_to_coin[a][i]);
            a = up[a][i];
        }
    }

    if(a == b) {
        return min(ans, dist_to_coin[a][0]);
    }

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            ans = min(ans, dist_to_coin[a][i]);
            ans = min(ans, dist_to_coin[b][i]);
            a = up[a][i];
            b = up[b][i];
        }
    }

    ans = min({ans, dist_to_coin[a][0], dist_to_coin[b][0],
          dist_to_coin[up[a][0]][0]});

    return ans;
}

void solve(){
    cin >> n >> q;
    int root = 0;
    for(int i = 1; i <= n; i++) {
        cin >> coin[i];
        if(coin[i] == 1) root = i;
    }

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    calc_dist_to_coin();
    dfs(root, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        int ans = dist(a, b);
        ans += 2 * min_dist(a, b);
        cout << ans << '\n';
    }
}
```

12.24 Tree Coin Collecting II

You are given a tree with n nodes. Some nodes contain a coin. Your task is to answer q queries of the form: what is the shortest length of a path from node a to node b that visits all nodes with coins?

Input
The first line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The second line contains n integers $c_1, c_2, \ldots, c_n$. If $c_i = 1$, node

i has a coin. If $c_i = 0$, node i doesn't have a coin. You can assume at least one node has a coin. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integers a and b: the start and end nodes.

Output
Print q integers: the answers to the queries.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

```
const int MAX = 2e5 + 5;
const int LOG = 19;

bool coin[MAX];
int pai[MAX];
int subtree[MAX];
int tam = 0;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

int calc_subtree(int s, int p) {
    subtree[s] = coin[s];

    for(auto x : adj[s]) {
        if(x == p) continue;
        subtree[s] += calc_subtree(x, s);
    }

    if(subtree[s]) tam++;
    return subtree[s];
}

void dfs(int s, int p) {
    if(subtree[s]) pai[s] = s;
    else pai[s] = pai[p];

    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}

void solve(){
    int n, q; cin >> n >> q;
    int root = 0;
    for(int i = 1; i <= n; i++) {
        cin >> coin[i];
        if(coin[i] == 1) root = i;
    }

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    calc_subtree(root, 0);
    dfs(root, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        int ans = 2 * (tam - 1);
        ans += dist(a, pai[a]);
        ans += dist(b, pai[b]);
        ans -= dist(pai[a], pai[b]);
        cout << ans << '\n';
    }
}
```

12.25 Tree Isomorphism I

Given two rooted trees, your task is to find out if they are isomorphic, i.e., it is possible to draw them so that they look the same.

Input
The first input line has an integer t: the number of tests. Then, there are t tests described as follows: The first line has an integer n: the number of nodes in both trees. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree.

Output
For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Constraints
• $1 \le t \le 1000$ • $2 \le n \le 10^5$ • the sum of all values of n is at most 10^5

```
map<vector<int>, int> mp;
vector<vector<vector<int>>> adj;
int cont = 0;

int converte( vector<int> &v ){
  if( mp[v] == 0 ) mp[v] = ++cont;
  return mp[v];
}

int dfs( int cur, int pai, int t ){
  vector<int> v;
  for( int viz : adj[t][cur] ) if( viz != pai ) v.push_back(
      dfs( viz, cur, t ) );
  sort( v.begin(), v.end() );
  return converte(v);
}

void solve(){
  mp.clear();
  cont = 0;
  adj.clear();
  int n; cin >> n;
  adj.resize(2, vector<vector<int>>(n));

  for( int i = 0; i < n - 1; i++ ){
    int a, b; cin >> a >> b;
    a--; b--;
    adj[0][a].push_back(b);
    adj[0][b].push_back(a);
  }

  for( int i = 0; i < n - 1; i++ ){
    int a, b; cin >> a >> b;
    a--; b--;
    adj[1][a].push_back(b);
    adj[1][b].push_back(a);
  }

  cout << (( dfs( 0, 0, 0 ) == dfs( 0, 0, 1 ) ) ? "YES" : "NO
      " ) << endl;
}
```

12.26 Tree Isomorphism II

Given two (not rooted) trees, your task is to find out if they are isomorphic, i.e., it is possible to draw them so that they look the same.

Input
The first input line has an integer t: the number of tests. Then, there are t tests described as follows: The first line has an integer n: the number of nodes in both trees. The nodes are numbered $1, 2, \ldots, n$. Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree.

Output
For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Constraints
• $1 \le t \le 1000$ • $2 \le n \le 10^5$ • the sum of all values of n is at most 10^5

```
using pii = pair<int, int>;

map<vector<int>, int> mp;
vector<vector<vector<int>>> adj;
vector<vector<int>> sub;
```

```
int cont = 0;

int converte( vector<int> &v ){
  if( mp[v] == 0 ) mp[v] = ++cont;
  return mp[v];
}

int dfs( int cur, int pai, int t ){
  vector<int> v;
  for( int viz : adj[t][cur] ) if( viz != pai ) v.push_back(
       dfs( viz, cur, t ));
  sort( v.begin(), v.end() );
  return converte(v);
}

void dfs_init( int cur, int pai, int t ){
  sub[t][cur] = 1;
  for( int viz : adj[t][cur] ) if( viz != pai ){ dfs_init(
       viz, cur, t ); sub[t][cur] += sub[t][viz]; }
}

int find_centroid( int cur, int pai, int t ){
  for( int viz : adj[t][cur] ) if( viz != pai && sub[t][viz]
       > sub[t][0]/2 ) return find_centroid( viz, cur, t );
  return cur;
}

pii get_centroids( int t ){
  dfs_init( 0, 0, t );
  int centroid = find_centroid( 0, 0, t );
  pii big( 0, 0 );
  for( int viz : adj[t][centroid] ) if( sub[t][viz] <= sub[t
       ][centroid] ) big = max( big, pii( sub[t][viz], viz )
        );
  return pii( centroid, big.second );
}

void solve(){
  mp.clear(); adj.clear(); sub.clear();
  cont = 0;
  int n; cin >> n;
  adj.resize(2, vector<vector<int>>(n));
  sub.resize(2, vector<int>(n));

  for( int i = 0; i < n - 1; i++ ){
    int a, b; cin >> a >> b;
    a--; b--;
    adj[0][a].push_back(b);
    adj[0][b].push_back(a);
  }

  auto [a1, a2] = get_centroids(0);

  for( int i = 0; i < n - 1; i++ ){
    int a, b; cin >> a >> b;
    a--; b--;
    adj[1][a].push_back(b);
    adj[1][b].push_back(a);
  }

  auto [b1, b2] = get_centroids(1);

  cout << (( dfs( a1, a1, 0 ) == dfs( b1, b1, 1 ) || dfs( a2,
       a2, 0 ) == dfs( b1, b1, 1 ) ) ? "YES" : "NO" ) <<
       endl;
}
```

12.27 Tree Traversals

There are three common ways to traverse the nodes of a binary tree:
• Preorder: First process the root, then the left subtree, and finally the right subtree. • Inorder: First process the left subtree, then the root, and finally the right subtree. • Postorder: First process the left subtree, then the right subtree, and finally the root.
There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.

Input
The first input line has an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers. You can assume that the input corresponds to a binary tree.

Output
Print the postorder traversal of the tree.

Constraints
• $1 \leq n \leq 10^5$

```
struct node {
    int id = -1;
    node *left, *right;
};

vector<int> preord, inord, inord_rev;
int n;
int preord_pos = 0;

node* process(int left, int right) {
    if(left > right) return nullptr;
    int at = preord[preord_pos++];
    int pos = inord_rev[at];

    node* novo = new node();
    novo->id = at;
    novo->left = process(left, pos - 1);
    novo->right = process(pos + 1, right);

    return novo;
}

void answer(node* tree) {
    if(tree == nullptr) return;
    answer(tree->left);
    answer(tree->right);
    cout << tree->id << ' ';
}

void solve(){
    cin >> n;
    preord.resize(n); inord.resize(n); inord_rev.resize(n +
         1);

    for(int i = 0; i < n; i++) cin >> preord[i];
    for(int i = 0; i < n; i++) {
        cin >> inord[i];
        inord_rev[inord[i]] = i;
    }

```

```
    int root = preord[0];
    node* tree = process(0, n-1);

    answer(tree);
    cout << '\n';
}
```

12.28 Visiting Cities

You want to travel from Syrjälä to Lehmälä by plane using a minimum-price route. Which cities will you certainly visit?

Input
The first input line contains two integers n and m: the number of cities and the number of flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. After this, there are m lines describing the flights. Each line has three integers a, b, and c: there is a flight from city a to city b with price c. All flights are one-way flights. You may assume that there is a route from Syrjälä to Lehmälä.

Output
First print an integer k: the number of cities that are certainly in the route. After this, print the k cities sorted in increasing order.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

```
using ll = long long;

const ll inf = 1e18;

int main(){
  int n, m; cin >> n >> m;

  vector<vector<pair<int, int>>> adj[2];
  adj[0].resize(n);
  adj[1].resize(n);

  while( m-- ){
    int a, b, c; cin >> a >> b >> c;
    a--; b--;
    adj[0][a].push_back({ b, c });
    adj[1][b].push_back({ a, c });
  }

  vector<ll> dist;

  auto dijkstra = [&]( int source ){
    dist.resize(n, inf);
    set<pair<ll, int>> s;

    s.insert({ 0, source });
    dist[source] = 0;

    while( !s.empty() ){
      int u = s.begin()->second;
      s.erase(s.begin());

      for( auto [v, d] : adj[1][u] )
        if( dist[v] > dist[u] + d ){
          dist[v] = dist[u] + d;
          s.insert({ dist[v], v });
        }
    }
  };

  dijkstra( n - 1 );

  auto solve = [&](){
    vector<int> ans;

    set<pair<ll, int>> s;

    s.insert({ 0, 0 });

    while( !s.empty() ){
      auto [dist_u, u] = *s.begin();
      s.erase(s.begin());

      if( s.empty() ) ans.push_back(u);

      for( auto [v, d] : adj[0][u] )
        if( dist_u + d + dist[v] == dist[0] )
          s.insert({ dist_u + d, v });
    }

    sort( ans.begin(), ans.end() );
    return ans;
  };

  vector<int> ans = solve();

  cout << ans.size() << endl;
  for( int x : ans ) cout << x + 1 << "␣";
  cout << endl;
}
```

13 Counting Problems

13.1 All Letter Subgrid Count I

You are given a grid of letters. Your task is to calculate the number of square subgrids that contain all the letters.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print the number of subgrids.

Constraints
• $1 \leq n \leq 3000$ • $1 \leq k \leq 26$

```cpp
const int MAXN = 3005;
const int MAXK = 26;

// maior quad com canto em (i, j) sem letra X
int dp[2][MAXN][MAXK];
int mat[MAXN][MAXN];

void solve() {
    int n, k; cin >> n >> k;

    if(k == 1) {
        cout << (ll)n * (n + 1LL) * (2 * n + 1LL) / 6 << '\n';
        return;
    }

    for(int i = 0; i < n; i++) {
        string s; cin >> s;
        for(int j = 0; j < n; j++) {
            mat[i][j] = s[j] - 'A';
        }
    }

    for(int j = 0; j < n; j++) {
        for(int c = 0; c < k; c++) {
            if(mat[0][j] != c) dp[0][j][c] = 1;
        }
    }

    ll tot = 0;

    for(int i = 1; i < n; i++) {
        for(int c = 0; c < k; c++)
            if(mat[i][0] != c) dp[1][0][c] = 1;

        for(int j = 1; j < n; j++) {
            int aux = 0;
            for(int c = 0; c < k; c++) {
                if(mat[i][j] == c) dp[1][j][c] = 0;
                else dp[1][j][c] = min({dp[1][j - 1][c], dp[0][j][c], dp[0][j - 1][c]}) + 1;
                aux = max(aux, dp[1][j][c]);
            }
            tot += min(i, j) - aux + 1;
        }

        for(int j = 0; j < n; j++) {
            for(int c = 0; c < k; c++) {
                dp[0][j][c] = dp[1][j][c];
                dp[1][j][c] = 0;
            }
        }
    }

    cout << tot << '\n';
}
```

13.2 All Letter Subgrid Count II

You are given a grid of letters. Your task is to calculate the number of rectangle subgrids that contain all the letters.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print the number of subgrids.

Constraints
• $1 \leq n \leq 500$ • $1 \leq k \leq 26$

```cpp
const int MAXN = 505;
const int MAXLG = 9;

int jmp[MAXLG][MAXLG][MAXN][MAXN];
int lg[MAXN];
int pw[MAXLG];

inline int query(int r1, int c1, int r2, int c2) __attribute__((always_inline));
inline int query(int r1, int c1, int r2, int c2) {
    int depR = lg[r2 - r1 + 1];
    int depC = lg[c2 - c1 + 1];
    int pwR = pw[depR];
    int pwC = pw[depC];

    return jmp[depR][depC][r1][c1] |
           jmp[depR][depC][r1][c2 - pwC + 1] |
           jmp[depR][depC][r2 - pwR + 1][c1] |
           jmp[depR][depC][r2 - pwR + 1][c2 - pwC + 1];
}

void solve() {
    int n, k; cin >> n >> k;
    int bst = (1 << k) - 1;

    if(k == 1) {
        cout << ((ll)n * (n + 1LL) / 2) * ((ll)n * (n + 1LL) / 2) << '\n';
        return;
    }

    for(int i = 0; i < n; i++) {
        string s; cin >> s;
        for(int j = 0; j < n; j++) {
            jmp[0][0][i][j] = (1 << (s[j] - 'A'));
        }
    }

    // build
    for (int kc = 1; kc < MAXLG; ++kc) {
        int lenC = pw[kc-1];
        if(lenC > n) break;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= n - pw[kc]; j++) {
                jmp[0][kc][i][j] = (jmp[0][kc - 1][i][j] | jmp[0][kc - 1][i][j + lenC]);
            }
        }
    }

    for (int kr = 1; kr < MAXLG; ++kr) {
        int lenR = pw[kr-1];
        if(lenR > n) break;
        for (int kc = 0; kc < MAXLG; ++kc) {
            if(pw[kc] > n) break;
            for (int i = 0; i <= n - pw[kr]; i++) {
                for (int j = 0; j <= n - pw[kc]; j++) {
                    jmp[kr][kc][i][j] = (jmp[kr - 1][kc][i][j] | jmp[kr - 1][kc][i + lenR][j]);
                }
            }
        }
    }
    // end build

    ll tot = 0;

    for(int i = 0; i < n; i++) {
        ll aux = 0;
        for(int j = 0; j < n; j++) {
            int r = n - 1, c = j;

            for(; r >= i; r--) {
                for(; c < n; c++)
                    if(query(i, j, r, c) == bst) break;
                if(c == n) break;
                aux += n - c;
            }
        }
        tot += aux;
    }

    cout << tot << '\n';
}

signed main() {

    lg[1] = 0;
    for(int i = 2; i < MAXN; i++) lg[i] = lg[i/2] + 1;
    for(int i = 0; i < MAXLG; i++) pw[i] = 1 << i;

    solve();
    return 0;
}

// Pikachu :

using pii = pair<int, int>;
using ll = long long;

const int maxn = 500 + 10;

int v[maxn], mat[maxn][maxn];

pii front[maxn], back[maxn];
int pf = -1, pb = -1;

inline void refresh(){
  while( pb >= 0 ){
    int x = back[pb].first;
    int OR = (( pf < 0 ) ? 0 : front[pf].second );
```

```
    front[++pf] = pii( x, OR|x );
    pb--;
  }
}
inline void push( int x ){
  int OR = (( pb < 0 ) ? 0 : back[pb].second );
  back[++pb] = pii( x, OR|x );
}

inline void pop(){
  if( pf < 0 ) refresh();
  if( pf >= 0 ) pf--;
}

inline int get_or(){
  return ((( pf < 0 ) ? 0 : front[pf].second )|(( pb < 0 ) ?
       0 : back[pb].second ));
}

inline bool empty(){
  return pf < 0 && pb < 0;
}

inline void clear(){
  while( !empty() ) pop();
}

int main(){
  int n, k; cin >> n >> k;

  for( int i = 0; i < n; i++ )
    for( int j = 0; j < n; j++ ){
      char c; cin >> c;
      mat[i][j] = (1<<(c - 'A'));
    }

  ll resp = 0;

  for( int i1 = 0; i1 < n; i1++ ){
    fill( v, v + n, 0 );
    for( int i2 = i1; i2 < n; i2++ ){
      clear();
      for( int j = 0; j < n; j++ ) v[j] |= mat[i2][j];

      for( int j1 = 0, j2 = -1; j1 < n; j1++ ){
        while( j2 + 1 < n && get_or() != (1<<k) - 1 ) push(v
             [++j2]);
        if( get_or() != (1<<k) - 1 ) break;

        resp += n - j2;
        pop();
      }
    }
  }

  cout << resp << endl;

}
```

13.3 Border Subgrid Count I

You are given a grid of letters. Your task is to calculate, for each letter, the number of square subgrids whose border consists of that letter.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print k lines: for each letter, the number of subgrids.

Constraints
• $1 \leq n \leq 3000$ • $1 \leq k \leq 26$

```
// TLE !!!

using pii = pair<int, int>;
using ll = long long;

const int maxn = 3e3 + 10;

int L[maxn][maxn], R[maxn][maxn], U[maxn][maxn], D[maxn][maxn
    ];
char mat[maxn][maxn];

struct BIT{
    int *bit, n;
    BIT( int n = maxn ) : n(n), bit(new int[n]) {}

    void update( int i, int x ){
        for(i++; i - 1 < n; i += i&-i) bit[i - 1] += x;
    }

    int query( int i ){
        int ans = 0;
        for(i++; i - 1 >= 0; i -= i&-i ) ans += bit[i - 1];
        return ans;
    }

    int query( int l, int r ){
        return query(r) - query(l - 1);
    }
} bit;

ll calc( vector<pii> &v ){
    vector<pii> undo;
    for( auto [i, j] : v ) undo.push_back({ j + min( R[i][j],
         D[i][j] ), j });
    sort(all(undo));

    int p = 0;
    ll ans = 0;
    for( auto [i, j] : v ){
        while( p < sz(undo) && undo[p].first <= j ) bit.
            update( undo[p++].second, -1 );

        bit.update(j, 1);
        ans += bit.query( j - min( L[i][j], U[i][j] ) +  1, j
            );
    }

    while( p < sz(undo) ) bit.update( undo[p++].second, -1 );
    return ans;
}

ll solve( vector<pii> &v ){
    sort(all(v), []( pii a, pii b ){
        if( a.ff - a.ss == b.ff - b.ss ) return a.ff > b.ff;
        return a.ff - a.ss < b.ff - b.ss;
    });

    int n = sz(v);

    ll ans = 0;
    while( !v.empty() ){
        vector<pii> cur;
        while( cur.empty() || ( !v.empty() && v.back().ff - v
            .back().ss == cur.back().ff - cur.back().ss ) )
            {
            cur.push_back(v.back());
            v.pop_back();
        }

        ans += calc(cur);
    }

    return ans;
}

void solve(){
    int n, k; cin >> n >> k;
    vector<vector<pii>> cells(k);

    FOR(i, 0, n) FOR(j, 0, n){
        cin >> mat[i][j];
        cells[mat[i][j] - 'A'].push_back({ i, j });

        L[i][j] = U[i][j] = 1;
        if( i > 0 && mat[i - 1][j] == mat[i][j] ) U[i][j] +=
            U[i - 1][j];
        if( j > 0 && mat[i][j - 1] == mat[i][j] ) L[i][j] +=
            L[i][j - 1];
    }

    iFOR(i, 0, n) iFOR(j, 0, n){
        R[i][j] = D[i][j] = 1;
        if( i + 1 < n && mat[i + 1][j] == mat[i][j] ) D[i][j]
            += D[i + 1][j];
        if( j + 1 < n && mat[i][j + 1] == mat[i][j] ) R[i][j]
            += R[i][j + 1];
    }

    for( auto &v : cells ) cout << solve(v) << '\n';
}
```

13.4 Border Subgrid Count II

You are given a grid of letters. Your task is to calculate, for each letter, the number of rectangular subgrids whose border consists of that letter.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print k lines: for each letter, the number of subgrids.

Constraints
• $1 \leq n \leq 500$ • $1 \leq k \leq 26$

```
using vi = vector<int>;
using pii = pair<int, int>;
using ll = long long;

const int maxn = 500;
```

```
const int alpha = 26;
int mat[maxn][maxn], h[maxn];

void solve(){
    int n, k; cin >> n >> k;
    FOR(i, 0, n) FOR(j, 0, n){
        char c; cin >> c;
        mat[i][j] = c - 'A';
    }

    vector<ll> ans(k);

    FOR(i1, 0, n){
        FOR(i2, i1, n ){
            int cnt = 0;
            FOR(j, 0, n){
                if( i2 == i1 || mat[i2][j] != mat[i2 - 1][j]
                    ) h[j] = 0;
                h[j]++;

                if( j == 0 || mat[i1][j] != mat[i1][j - 1] ||
                     mat[i2][j] != mat[i2][j - 1] ) cnt =
                    0;
                if( h[j] == i2 - i1 + 1 ) ans[mat[i1][j]] +=
                    ++cnt;
            }
        }
    }

    for( auto x : ans ) cout << x << '\n';
}
```

13.5 Collecting Numbers Distribution

You are given an array that contains each number between $1 \dots n$ exactly once. You collect the numbers in increasing order from 1 to n. On each round, you traverse the array from left to right and collect as many consecutive numbers as possible, starting from the smallest number that has not been collected yet. Your task is to determine, for each $k = 1, 2, \dots, n$, the number of arrays that require exactly k rounds to collect all the numbers.

Input
The only line has an integer n.

Output
Print n numbers: for each $k = 1, 2, \dots, n$, the answer modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 5000$

```
const int MAX = 5008;
const int MOD = 1e9 + 7;

/*
dp[n][k] = k * dp[n - 1][k] + (n - k + 1) * dp[n - 1][k - 1]
considera o vetor de posicoes, p.ex., [3, 1, 4, 2] -> [2, 4,
     1, 3]
o numero de rounds eh o numero de quedas do vetor + 1 (
     bijecao)
agora queremos adicionar o elemento 'n' no vetor.
Ha 'k' posicoes que nao alteram (finais das rampas)
Ha 'n - k + 1' que alteram (demais posicoes)
*/
int dp[MAX][MAX];

void solve(){
    int n; cin >> n;

    for(int i = 1; i <= n; i++) {
        dp[i][1] = 1;
        for(int j = 2; j <= i; j++) {
            dp[i][j] = ((ll)j * dp[i - 1][j] + ((ll)i - j +
                1) * dp[i - 1][j - 1]) % MOD;
        }
    }

    for(int i = 1; i <= n; i++) {
        cout << dp[n][i] << ' ';
    }
    cout << '\n';

}
```

13.6 Counting Bishops

Your task is to count the number of ways k bishops can be placed on an $n \times n$ chessboard so that no two bishops attack each other. Two bishops attack each other if they are on the same diagonal.

Input
The only input line has two integers n and k: the board size and the number of bishops.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 500$ • $1 \leq k \leq n^2$

```
const int MAX = 508;
const int MOD = 1e9 + 7;

/*
dp[p][i][j] eh o numero de formas de colocar
j bispos nas diagonais de paridade p com indice ate i
(os indices das diagonais sao definidos da menor para a maior
    )
*/
int dp[2][MAX][2 * MAX];

int num_squares(bool p, int d) {
    if (p)
        return ((d - 1) / 2) * 2 + 1;
    else
        return ((d + 1) / 2) * 2;
}

void solve(){
    int n, k; cin >> n >> k;

    if (k >= 2 * n) {
        cout << "0\n";
        return;
    }

    for (int i = 0; i <= n; i++) {
        dp[0][i][0] = 1;
        dp[1][i][0] = 1;
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            dp[1][i][j] = (dp[1][i-1][j] + dp[1][i-1][j-1] *
                (num_squares(1, i) - j + 1) % MOD) % MOD;
            if(i < n)
            dp[0][i][j] = (dp[0][i-1][j] + dp[0][i-1][j-1] *
                (num_squares(0, i) - j + 1) % MOD) % MOD;
        }
    }

    int ans = 0;
    for (int i = 0; i <= k; i++) {
        ans = (ans + dp[0][n - 1][i] * dp[1][n][k-i]) % MOD;
    }

    cout << ans << '\n';
}
```

13.7 Counting Permutations

A permutation of integers $1, 2, \dots, n$ is called beautiful if there are no adjacent elements whose difference is 1. Given n, your task is to count the number of beautiful permutations.

Input
The only input line contains an integer n.

Output
Print the number of beautiful permutations of $1, 2, \dots, n$ modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 1000$

```
const int maxn = 1e3 + 10;
const int mod = 1e9 + 7;

int sum( int a, int b ){
    return (a + b)%mod;
}

int prod( int a, int b ){
    return 1LL * a * b % mod;
}

int dp[2][maxn][3][3];

void solve(){
    int n; cin >> n;

    auto push = []( int dp, int &next ){ next = sum( next, dp
        ); };

    int cur = 0, next = 1;
    dp[cur][1][2][2] = 1;

    for( int i = 1; i < n; i++, swap( cur, next ) )
        for( int k = 1; k <= n; k++ )
            for( int b = 0; b < 3; b++ )
```

```
                for( int e = 0; e <= b; e++ ) {
                    int &res = dp[cur][k][b][e];

                    // Adicionar componente nova

                    push( prod( k - 1, res ), dp[next][k +
                        1][2][0] ); // No meio
                    push( prod( 2    , res ), dp[next][k +
                        1][2][1] ); // Nas pontas

                    // Colocar na ponta de uma componente

                    push( prod( 2*(k - 1) - b + e, res ), dp[
                        next][k][1][0] ); // No meio
                    push( prod( 2              - e, res ), dp[
                        next][k][1][1] ); // Nas pontas

                    // Mergear componentes

                    push( prod( k - 1 - b + e, res ), dp[next
                        ][k - 1][0][0] );

                    res = 0;
                }
    int ans = 0;
    for( int b = 0; b < 3; b++ ) for( int e = 0; e <= b; e++
        ) ans = sum( ans, dp[cur][1][b][e] );
    cout << ans << '\n';
}
```

13.8 Counting Sequences

Your task is to count the number of sequences of length n where each element is an integer between $1 \dots k$ and each integer between $1 \dots k$ appears at least once in the sequence. For example, when $n = 6$ and $k = 4$, some valid sequences are $[1, 3, 1, 4, 3, 2]$ and $[2, 2, 1, 3, 4, 2]$.

Input
The only input line has two integers n and k.

Output
Print one integer: the number of sequences modulo $10^9 + 7$.

Constraints
- $1 \le k \le n \le 10^6$

```
const int M = 1e9 + 7;
const int MAX = 1e6 + 10;
int fat[MAX], invfat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}

int calc(int a, int b) {
    if(b < 0 || b > a) return 0;
    return fat[a] * (invfat[b] * invfat[a - b] % M) % M;
}

// inclusao - exclusao em Ai = {seq's tais que i nao aparece}
void solve() {
    int n, k; cin >> n >> k;

    int sgn = 1, tot = 0;

    for(int i = 1; i < k; i++) {
        int pot = (fexp(k - i, n) * calc(k, i)) % M;
        pot = (pot * sgn + M) % M;

        tot = (tot + pot) % M;
        sgn = -sgn;
    }

    cout << (fexp(k, n) - tot + M) % M << '\n';
}

signed main() {

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) %
        M;
    invfat[MAX - 1] = inv(fat[MAX - 1]);
    for(int i = MAX - 2; i >= 0; i--) invfat[i] = (invfat[i +
        1] * (i + 1)) % M;

    solve();
    return 0;
}
```

13.9 Empty String

You are given a string consisting of n characters between a and z. On each turn, you may remove any two adjacent characters that are equal. Your goal is to construct an empty string by removing all the characters. In how many ways can you do this?

Input
The only input line has a string of length n.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
- $1 \le n \le 500$

```
const int M = 1e9 + 7;
const int MAX = 505;
int dp[MAX][MAX]; //dp[a][b] = formas de deletar os
    caracteres de a ate b
int fat[MAX], fatinv[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

void calcfat() {
    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (i * fat[i - 1]) %
        M;
    fatinv[MAX - 1] = fexp(fat[MAX - 1], M - 2);
    for(int i = MAX - 2; i >= 0; i--) fatinv[i] = ((i + 1) *
        fatinv[i + 1]) % M;
}

int choose(int a, int b) {
    return ((fat[a] * fatinv[b] % M) * fatinv[a - b]) % M;
}

void calc(const string& s) {
    for(int i = 0; i + 1 < s.size(); i++) if(s[i] == s[i+1])
        dp[i][i+1] = 1;

    for(int tam = 4; tam <= s.size(); tam+=2)
    for(int i = 0, j = i + tam - 1; j < s.size(); i++, j++) {
        // s[i] opera com s[k]

        if(s[i] == s[i + 1])
            dp[i][j] = (dp[i][j] + dp[i + 2][j] * (j - i + 1)
                / 2) % M;

        for(int k = i + 3; k < j; k += 2) {
            if(s[i] == s[k]) dp[i][j] = (dp[i][j] + (dp[i +
                1][k - 1] * dp[k + 1][j] % M) * choose((j -
                i + 1) / 2, (j - k) / 2)) % M;
        }

        if(s[i] == s[j]) dp[i][j] = (dp[i][j] + dp[i + 1][j -
            1]) % M;
    }
}

void solve() {
    string s; cin >> s;
    calc(s);
    cout << dp[0][(int)s.size() - 1] << '\n';
}

signed main() {
    calcfat();
    solve();
    return 0;
}
```

13.10 Filled Subgrid Count I

You are given a grid of letters. Your task is to calculate, for each letter, the number of square subgrids whose each letter is the same.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print k lines: for each letter, the number of subgrids.

Constraints
- $1 \le n \le 3000$ • $1 \le k \le 26$

```cpp
using ll = long long;

const int maxn = 3e3 + 10;
const int alphabet = 26;

char mat[maxn][maxn];
int square[maxn][maxn]; // maior quadrado com canto inf-dir em ( i, j );
ll resp[alphabet];

int main(){
  int n, k; cin >> n >> k;

  for( int i = 0; i < n; i++ ){
    for( int j = 0; j < n; j++ ){
      cin >> mat[i][j];

      if( !i || !j ) square[i][j] = 1;
      else if( mat[i][j] != mat[i - 1][j] || mat[i][j] != mat[i][j - 1] ) square[i][j] = 1;
      else{
        int x = min( square[i - 1][j], square[i][j - 1] );
        if( mat[i][j] != mat[i - x][j - x] ) square[i][j] = x;
        else square[i][j] = x + 1;
      }

      resp[mat[i][j] - 'A'] += square[i][j];
    }
  }

  for( int i = 0; i < k; i++ ) cout << resp[i] << endl;
}
```

13.11 Filled Subgrid Count II

You are given a grid of letters. Your task is to calculate, for each letter, the number of rectangular subgrids whose each letter is the same.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print k lines: for each letter, the number of subgrids.

Constraints
• $1 \le n \le 3000$ • $1 \le k \le 26$

```cpp
using ll = long long;

int main(){
  int n, k; cin >> n >> k;
  vector<int> h(n);
  vector<string> mat(n);
  vector<ll> resp(k);

  for( int i = 0; i < n; i++ ){
    cin >> mat[i];

    stack<pair<int, int>> pilha; // stack monotonica crescente -> ( altura, comprimento )
    int area = 0;
    for( int j = 0; j < n; j++ ){
      if( i && mat[i - 1][j] == mat[i][j] ) h[j]++;
      else h[j] = 1;

      if( j && mat[i][j] != mat[i][j - 1] ){
        area = 0;
        while( !pilha.empty() ) pilha.pop();
      }

      int lenght = 1;
      area += h[j];

      while( !pilha.empty() && pilha.top().first >= h[j] ){
        auto [H, L] = pilha.top();
        pilha.pop();

        area -= (H - h[j])*L;
        lenght += L;
      }

      pilha.push({ h[j], lenght });

      resp[mat[i][j] - 'A'] += area;
    }
  }

  for( int i = 0; i < k; i++ ) cout << resp[i] << endl;
}
```

13.12 Functional Graph Distribution

A functional graph is a directed graph where each node has outdegree 1. For example, here is a functional graph that has 9 nodes and 2 components:
Given n, your task is to calculate for each $k = 1 \dots n$ the number of functional graphs that have n nodes and k components.

Input
The only input line has an integer n: the number of nodes.

Output
Print n lines: for each $k = 1 \dots n$ the number of graphs modulo $10^9 + 7$.

Constraints
• $1 \le n \le 5000$

```cpp
const int M = 1e9 + 7;
const int MAX = 5008;
int fat[MAX], invfat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}

int choose(int a, int b) {
    if(b < 0 || b > a) return 0;
    return fat[a] * (invfat[b] * invfat[a - b] % M) % M;
}

// dp[n][k] = numero de formas de fazer k ciclos com n elementos (Numeros de Stirling)
// dp[n][k] = dp[n - 1][k - 1] + (n - 1)dp[n - 1][k]
// n faz o proprio ciclo ou adiciona em um existente
int dp[MAX][MAX];

// T(n, k) = sum dp[j][k] * (n choose j) * j * n^(n-j-1)
// numero de ciclos * permutacoes * numero de arvores enraizadas em j caras
void solve() {
    int n; cin >> n;

    dp[1][1] = 1;
    for(int i = 2; i <= n; i++) {
        for(int k = 1; k <= i; k++) {
            dp[i][k] = (dp[i - 1][k - 1] + (i - 1) * dp[i - 1][k]) % M;
        }
    }

    for(int k = 1; k <= n; k++) {
        int ans = 0;
        int pot = 1;
        for(int j = n; j >= k; j--) {
            ans = (ans + dp[j][k] * (choose(n - 1, j - 1) * pot % M)) % M;
            pot = (pot * n) % M;
        }

        cout << ans << '\n';
    }

}

signed main() {

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) % M;
    invfat[MAX - 1] = inv(fat[MAX - 1]);
    for(int i = MAX - 2; i >= 0; i--) invfat[i] = (invfat[i + 1] * (i + 1)) % M;

    solve();
    return 0;
}
```

13.13 Grid Completion

Your task is to create an $n \times n$ grid whose each row and column has exactly one A and B. Some of the characters have already been placed. In how many ways can you complete the grid?

Input
The first input line has an integer n: the size of the grid. After this, there are n lines that describe the grid. Each line has n characters: . means an empty square, and A and B show the characters already placed. You can assume that every row and column has at most one A and B.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $2 \leq n \leq 500$

```cpp
const int M = 1e9 + 7;
const int MAX = 508;
int fat[MAX], invfat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}

int choose(int a, int b) {
    if(b < 0 || b > a) return 0;
    return fat[a] * (invfat[b] * invfat[a - b] % M) % M;
}

/*
 * Solucao: Principio da Inclusao-Exclusao.
 * Universo: Distribuir os As e Bs restantes de forma independente (n - N_A)! * (n - N_B)!
 * PIE: Subtrair as configuracoes invalidas ('A' e 'B' na mesma célula)
 * esse cenario so eh possivel quando:
 *      k1: um novo 'B' cai sobre um 'A' que ja estava fixo no grid (total de c_A opcoes)
 *      k2: um novo 'A' cai sobre um 'B' que ja estava fixo no grid (total de c_B opcoes)
 *      k3: um novo 'A' e um novo 'B' caem juntos em uma celula inicialmente vazia
 * Formula:
 *      F(k3) = sum_{k1=0}^{c_A} (-1)^{k1} * C(c_A, k1) * (n - N_B - k1 - k3)!
 *      G(k3) = sum_{k2=0}^{c_B} (-1)^{k2} * C(c_B, k2) * (n - N_A - k2 - k3)!
 *      Ans   = sum_{k3=0}^{min(r_E, c_E)} (-1)^{k3} * C(r_E, k3) * C(c_E, k3) * k3! * F(k3) * G(k3)
 */
void solve() {
    int n; cin >> n;

    vector<string> grid(n);
    vector<bool> A_row(n, false), A_col(n, false);
    vector<bool> B_row(n, false), B_col(n, false);
    int N_A = 0, N_B = 0;

    for (int i = 0; i < n; i++) {
        cin >> grid[i];
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 'A') {
                A_row[i] = A_col[j] = true;
                N_A++;
            } else if (grid[i][j] == 'B') {
                B_row[i] = B_col[j] = true;
                N_B++;
            }
        }
    }

    int c_A = 0, c_B = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 'A' && !B_row[i] && !B_col[j])
                c_A++;
            if (grid[i][j] == 'B' && !A_row[i] && !A_col[j])
                c_B++;
        }
    }

    int r_E = 0, c_E = 0;
    for (int i = 0; i < n; i++) if (!A_row[i] && !B_row[i])
        r_E++;
    for (int j = 0; j < n; j++) if (!A_col[j] && !B_col[j])
        c_E++;

    int teto = min(r_E, c_E);
    vector<int> F(teto + 1, 0), G(teto + 1, 0);

    for (int k3 = 0; k3 <= teto; k3++) {
        for (int k1 = 0; k1 <= c_A; k1++) {
            int term = choose(c_A, k1) * fat[n - N_B - k1 - k3] % M;
            if (k1 % 2 == 1) F[k3] = (F[k3] - term + M) % M;
            else F[k3] = (F[k3] + term) % M;
        }
        for (int k2 = 0; k2 <= c_B; k2++) {
            int term = choose(c_B, k2) * fat[n - N_A - k2 - k3] % M;
            if (k2 % 2 == 1) G[k3] = (G[k3] - term + M) % M;
            else G[k3] = (G[k3] + term) % M;
        }
    }

    int ans = 0;
    for (int k3 = 0; k3 <= teto; k3++) {
        int term = choose(r_E, k3) * choose(c_E, k3) % M;
        term = term * fat[k3] % M;
        term = term * F[k3] % M;
        term = term * G[k3] % M;

        if (k3 % 2 == 1) ans = (ans - term + M) % M;
        else ans = (ans + term) % M;
    }

    cout << ans << "\n";
}

signed main() {

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) % M;
    invfat[MAX - 1] = inv(fat[MAX - 1]);
    for(int i = MAX - 2; i >= 0; i--) invfat[i] = (invfat[i + 1] * (i + 1)) % M;

    solve();
    return 0;
}
```

13.14 Grid Paths II

Consider an $n \times n$ grid whose top-left square is $(1,1)$ and bottom-right square is (n,n). Your task is to move from the top-left square to the bottom-right square. On each step you may move one square right or down. In addition, there are m traps in the grid. You cannot move to a square with a trap. What is the total number of possible paths?

Input
The first input line contains two integers n and m: the size of the grid and the number of traps. After this, there are m lines describing the traps. Each such line contains two integers y and x: the location of a trap. You can assume that there are no traps in the top-left and bottom-right square. You can also assume that each square has at most one trap.

Output
Print the number of paths modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$ • $1 \leq m \leq 1000$ • $1 \leq y, x \leq n$

```cpp
const int M = 1e9 + 7;
const int MAX = 2e6 + 10;
int fat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}

int calc(int a, int b) {
    return fat[a + b] * (inv(fat[a]) * inv(fat[b]) % M) % M;
}

void solve() {
    int n; cin >> n;
    int m; cin >> m;

    vector<int> dp(m + 1);
    vector<pair<int, int>> trap(m + 1);

    for(int i = 0; i < m; i++) {
        int x, y; cin >> x >> y;
        trap[i] = {x - 1, y - 1};
        if(x == n && y == n) {
            cout << 0 << '\n';
            return;
        }
    }
    trap[m] = {n - 1, n - 1};

    sort(trap.begin(), trap.end());

    for(int i = 0; i <= m; i++) {
        dp[i] = calc(trap[i].first, trap[i].second);
        for(int j = 0; j < i; j++) {
            if(trap[i].second < trap[j].second) continue;
            dp[i] = (dp[i] - dp[j] * calc(trap[i].first - trap[j].first, trap[i].second - trap[j].second)) % M;
            dp[i] = (M + dp[i]) % M;
        }
    }

    cout << dp[m] << '\n';
}

signed main() {

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) % M;
```

```
55
56      solve();
57      return 0;
58  }
```

13.15 Permutation Inversions

Your task is to count the number of permutations of $1, 2, \ldots, n$ that have exactly k inversions (i.e., pairs of elements in the wrong order). For example, when $n = 4$ and $k = 3$, there are 6 such permutations:
• $[1, 4, 3, 2]$ • $[2, 3, 4, 1]$ • $[2, 4, 1, 3]$ • $[3, 1, 4, 2]$ • $[3, 2, 1, 4]$ • $[4, 1, 2, 3]$

Input
The only input line has two integers n and k.

Output
Print the answer modulo $10^9 + 7$.

Constraints
• $1 \le n \le 500$ • $0 \le k \le \frac{n(n-1)}{2}$

```
1  const int MAX = 501;
2  const int MOD = 1e9 + 7;
3
4  // dp[t] = quantidade de perm com t invercoes
5  int dp[MAX * MAX / 2];
6
7  void solve() {
8      int n, k; cin >> n >> k;
9
10     dp[0] = 1;
11
12     for(int i = 2; i <= n; i++) {
13         int maxj = i * (i - 1) / 2;
14         int window = 0;
15
16         for(int j = maxj - (i - 1); j <= maxj; j++) {
17             window = (window + dp[j]) % MOD;
18         }
19
20         for(int j = maxj; j >= 0; j--) {
21             int ant = dp[j];
22             dp[j] = window;
23             if(j >= i) window = ((ll)window - ant + dp[j - i]
                       + MOD) % MOD;
24             else window = (window - ant + MOD) % MOD;
25         }
26     }
27
28     cout << dp[k] << '\n';
29 }
```

13.16 Raab Game II

Consider a two player game where each player has n cards numbered $1, 2, \ldots, n$. On each turn both players place one of their cards on the table. The player who placed the higher card gets one point. If the cards are equal, neither player gets a point. The game continues until all cards have been played. You are given the number of cards n and the players' scores at the end of the game, a and b. Your task is to count the number of possible games with that outcome.

Input
The first line contains one integer t: the number of tests. Then there are t lines, each with three integers n, a and b.

Output
For each test case print the number of possible games modulo $10^9 + 7$.

Constraints
• $1 \le t \le 1000$ • $1 \le n \le 5000$ • $0 \le a, b \le n$

```
1  const int MOD = 1e9 + 7, MAX = 5005;
2
3  /*
4  se k = a + b, e A = 1 2 3 ... n
5  fixamos n - k posições iguais (empates) e permutamos A
6  ans = n! * binom(n, n - k) * ans(k, a)
7  onde ans(n, a) é a resposta quando n = a + b e A = 1 2 3 ...
       n
8  ou seja, número de perm. caóticas de tam. 'k' com 'a'
       elementos maiores q os originais
9  ans(n, a) = a * ans(n-1, a) + (n-a) * ans(n-1, a-1) + (n-1) *
        ans(n-2, a-1)
10 ans(n-1, a) -> pega um elemento dos 'a' e troca por 'n'
11 ans(n-1, a-1) -> pega um elemento dos 'b' e troca por 'n' (
       adicionando 1 no a)
12 ans(n-2, a-1) -> considera que um dentre os n-1 ta "empatado"
       (add dois elementos, o do empate e o n)
13 */
14
15 vector<vector<int>> dp(MAX, vector<int>(MAX));
16 vector<int> fac(MAX);
17
18 int fexp(int x, int y) {
19     int ret = 1;
20     while (y) {
21         if (y & 1) ret = (ret * x) % MOD;
22         y >>= 1;
23         x = (x * x) % MOD;
24     }
25     return ret;
26 }
27
28 void calc() {
29     fac[0] = 1;
30     for(int i = 1; i < MAX; i++) fac[i] = (fac[i - 1] * i) %
           MOD;
31
32     dp[2][1] = 1;
33     dp[3][1] = 1;
34     dp[3][2] = 1;
35
36     for(int i = 4; i < MAX; i++)
37         for(int j = 1; j < i; j++)
38             dp[i][j] = (j * dp[i-1][j] + (i - j) * dp[i-1][j
                   -1] + (i-1) * dp[i-2][j-1]) % MOD;
39 }
40
41 int binom(int a, int b) {
42     if(a < b) return 0;
43     int ans = fac[a];
44     ans = ( ans * fexp(fac[b], MOD - 2) ) % MOD;
45     ans = ( ans * fexp(fac[a - b], MOD - 2) ) % MOD;
46     return ans;
47 }
48
49 void solve() {
50     int n; cin >> n;
51     int a, b; cin >> a >> b;
52     if(a == 0 && b == 0) {
53         cout << fac[n] << '\n';
54         return;
55     }
56     if(a + b > n) {
57         cout << 0 << '\n';
58         return;
59     }
60     int k = a + b;
61     int ans = fac[n];
62     ans = (ans * binom(n, k)) % MOD;
63     ans = (ans * dp[k][a]) % MOD;
64     cout << ans << '\n';
65
66 }
67
68 signed main() { _
69     calc();
70     int t; cin >> t; while(t--)
71     solve();
72 }
```

13.17 Tournament Graph Distribution

A tournament graph is a directed graph where a single directed edge exists between every pair of nodes. Given n, your task is to calculate for each $k = 1 \ldots n$ the number of tournament graphs that have n nodes and k strongly connected components.

Input
The only line has an integer n: the number of nodes.

Output
Print n lines: for each $k = 1 \ldots n$ the number of graphs modulo $10^9 + 7$.

Constraints
• $1 \le n \le 500$

```
1  const int MAX = 505;
2  const int MOD = 1e9 + 7;
3
4  int ans[MAX][MAX];
5  int bin[MAX][MAX];
6
7  /*
8  existe exatamente uma ordenação topológica que representa o
       torneio
9  (considerando apenas as k componentes)
10
11 k > 1
12 ans[n][k] = sum ans[n - i][k - 1] * ans[i][1] * (n choose i)
13
14 i é a quantidade de elementos no primeiro grupo da ordenação
15
16 (a1 a2 ... ai) | (resto)
```

```

ans[n][1] = 2^{n(n-1)/2} - sum ans[n][i]
*/

int fexp(int x, int y) {
    int ret = 1;
    while (y) {
        if (y & 1) ret = (ret * x) % MOD;
        y >>= 1;
        x = (x * x) % MOD;
    }
    return ret;
}

void calc() {
    ans[1][1] = 1;
    bin[0][0] = 1;

    for(int n = 1; n < MAX; n++) {
        bin[n][0] = 1;
        for(int k = 1; k <= n; k++) {
            bin[n][k] = (bin[n - 1][k] + bin[n - 1][k - 1]) %
                MOD;
        }
    }

    for(int n = 2; n < MAX; n++) {
        int tot = 0;
        for(int k = 2; k <= n; k++) {
            for(int i = 1; n - i >= k - 1; i++) {
                int aux = (ans[n - i][k - 1] * ans[i][1]) %
                    MOD;
                aux = (aux * bin[n][i]) % MOD;
                ans[n][k] = (ans[n][k] + aux) % MOD;
            }
            tot = (tot + ans[n][k]) % MOD;
        }
        ans[n][1] = fexp(2, n * (n - 1) / 2);
        ans[n][1] = ((ans[n][1] - tot) % MOD + MOD) % MOD;
    }
}

void solve() {
    int n; cin >> n;

    for(int i = 1; i <= n; i++) {
        cout << ans[n][i] << '\n';
    }
}

signed main() { _
    calc();
    solve();
}
```

14 Additional Problems I

14.1 Advertisement

A fence consists of n vertical boards. The width of each board is 1 and their heights may vary. You want to attach a rectangular advertisement to the fence. What is the maximum area of such an advertisement?

Input
The first input line contains an integer n: the width of the fence. After this, there are n integers $k_1, k_2, \ldots, k_n$: the height of each board.

Output
Print one integer: the maximum area of an advertisement.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le k_i \le 10^9$

```cpp
const int maxn = 200010;

vector< pair< int, int > > v;

long long int l[maxn], r[maxn], h[maxn];

int main(){

    int n; scanf("%d", &n);

    v.push_back({ 0, 0 });

    for( int i = 0; i < maxn; i++){

        r[i] = n + 1;

        l[i] = 0;
    }

    for( int i = 1; i <= n; i++){

        int x; scanf("%d", &x);

        h[i] = x;

        pair< int, int > p = { x, i };

        while( v.back().first >= p.first ){

            r[v.back().second] = p.second;

            v.pop_back();

        }

        l[i] = v.back().second;

        v.push_back(p);
    }

    r[n] = n + 1;

    long long int resp = 0;

    for( int i = 1; i <= n; i++) resp = max( resp, (r[i] - l[
        i] - 1)*h[i]);

    printf("%lld", resp);
}
```

14.2 Beautiful Permutation II

A permutation of integers $1, 2, \ldots, n$ is called beautiful if there are no adjacent elements whose difference is 1. Given n, construct the lexicographically minimal beautiful permutation if such a permutation exists.

Input
The only line contains an integer n.

Output
Print the lexicographically minimal beautiful permutation of integers $1, 2, \ldots, n$. If there is no such permutation, print "NO SOLUTION".

Constraints
• $1 \le n \le 10^6$

```cpp
void solve() {
    int n; cin >> n;
    vector<int> v(n);

    if(n == 1) cout << "1\n";
    else if(n <= 3) cout << "NO SOLUTION\n";
    else {
        int i = 0;
        for(i = 0; 5 * i + 8 < n; i++) {
            v[5 * i + 0] = 1 + 5 * i;
            v[5 * i + 1] = 3 + 5 * i;
            v[5 * i + 2] = 5 + 5 * i;
            v[5 * i + 3] = 2 + 5 * i;
            v[5 * i + 4] = 4 + 5 * i;
        }

        if(n % 5 == 4) {
            v[5 * i + 0] = 5 * i + 2;
            v[5 * i + 1] = 5 * i + 4;
            v[5 * i + 2] = 5 * i + 1;
            v[5 * i + 3] = 5 * i + 3;
        }

        if(n % 5 == 0) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 4;
        }

        if(n % 5 == 1) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 4;
            v[5 * i + 5] = 5 * i + 6;
        }

        if(n % 5 == 2) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 4;
            v[5 * i + 5] = 5 * i + 6;
            v[5 * i + 6] = 5 * i + 7;
        }

        if(n % 5 == 3) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 6;
            v[5 * i + 5] = 5 * i + 8;
            v[5 * i + 6] = 5 * i + 4;
            v[5 * i + 7] = 5 * i + 7;
        }

        for(auto x : v) cout << x << ' ';
        cout << '\n';
    }
}
```

14.3 Bit Inversions

There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same.

Input
The first input line has a bit string consisting of n bits. The bits are numbered $1, 2, \ldots, n$. The next line contains an integer m: the number of changes. The last line contains m integers $x_1, x_2, \ldots, x_m$ describing the changes.

Output
After each change, print the length of the longest substring whose each bit is the same.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le x_i \le n$

```cpp
using vi = vector<int>;

void solve(){
    string s; cin >> s;
    int n = s.size();

    set<int> pos[2];
    multiset<int> len;

    pos[0].insert(-1); pos[1].insert(-1);

    for( int i = 0; i < n; i++ ){
        len.insert(i - *pos[s[i] - '0'].rbegin());
        pos[s[i] - '0'].insert(i);
    }

    len.insert(n - *pos[0].rbegin());
    pos[0].insert(n);
    len.insert(n - *pos[1].rbegin());
    pos[1].insert(n);
```

```
21
22      int q; cin >> q;
23      while( q-- ){
24          int p; cin >> p; p--;
25          int x = s[p] - '0';
26
27          // Remover do set
28
29          auto it1 = pos[x].erase(pos[x].find(p));
30          len.erase(len.find(*it1 - p));
31          len.erase(len.find(p - *prev(it1)));
32          len.insert(*it1 - *prev(it1));
33
34          // Inserir no outro
35
36          x = 1 - x;
37          s[p] = '0' + x;
38          auto it2 = pos[x].insert(p).first;
39          len.erase(len.find(*next(it2) - *prev(it2)));
40          len.insert(*next(it2) - *it2);
41          len.insert(*it2 - *prev(it2));
42
43          cout << *len.rbegin() - 1 << "␣";
44      }
45      cout << endl;
46  }
```

14.4 Bubble Sort Rounds I

Bubble sort is a sorting algorithm that consists of a number of rounds. On each round the algorithm scans the array from left to right and swaps any adjacent elements that are in the wrong order. Given an array of n integers, calculate the number of bubble sort rounds needed to sort the array.

Input
The first line has an integer n: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print one integer: the number of rounds.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```
1  using pii = pair<int, int>;
2  using vi = vector<int>;
3
4  void solve(){
5      int n; cin >> n;
6      vector<pii> v(n);
7      for( int i = 0; i < n; i++ ){ cin >> v[i].first; v[i].
           second = i; }
8      sort( v.rbegin(), v.rend() );
9
10     int ans = 0, invs = 0, p = n - 1;
11
12     vi marc(n);
13     for( auto [_, i] : v ){
14         marc[i] = true;
15         if( i <= p ) invs++;
16         while( p >= 0 && marc[p] ){ invs--; p--; }
17         ans = max( ans, invs );
18     }
19
20     cout << ans << '\n';
21 }
```

14.5 Bubble Sort Rounds II

Bubble sort is a sorting algorithm that consists of a number of rounds. On each round the algorithm scans the array from left to right and swaps any adjacent elements that are in the wrong order. Given an array of n integers, find out the contents of the array after k bubble sort rounds.

Input
The first line has two integers n and k: the array size and the number of rounds. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print n integers: the contents of the array after k rounds.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le k \le 10^9$ • $1 \le x_i \le 10^9$

```
1  using pii = pair<int, int>;
2  using vi = vector<int>;
3  using ll = long long;
4
5  class PersistentSegtree{
6      private:
7
8      struct Node{
9          ll sum;
10         int l, r;
11         Node( ll sum = 0 ) : sum(sum), l(0), r(0) {}
12         void merge( Node L, Node R ){
13             sum = L.sum + R.sum;
14         }
15     };
16
17     Node *seg;
18     vi rt;
19     int MIN, MAX, id;
20
21     int create(){
22         return id++;
23     }
24
25     int copy( int node ){
26         int cp = create();
27         return seg[cp] = seg[node], cp;
28     }
29
30     int update( int node, int ti, int tf, int id, int x ){
31         int cp = copy(node);
32         if( ti == tf ){
33             seg[cp].merge(seg[node], Node(x) );
34             return cp;
35         }
36
37         int tm = (ti + tf)>>1, &l = seg[cp].l, &r = seg[cp].r
               ;
38         if( id <= tm ) l = update( seg[node].l, ti, tm, id, x
               );
39         else r = update( seg[node].r, tm + 1, tf, id, x );
40
41         seg[cp].merge( seg[l], seg[r] );
42         return cp;
43     }
44
45     ll query( int n1, int n2, int ti, int tf, ll qi, ll qf ){
46         if( qi > tf || ti > qf || n1 == n2 ) return 0;
47         if( qi <= ti && tf <= qf ) return seg[n2].sum - seg[
               n1].sum;
48         int tm = (ti + tf)>>1;
49         return query( seg[n1].l, seg[n2].l, ti, tm, qi, qf )
               + query( seg[n1].r, seg[n2].r, tm + 1, tf, qi,
               qf );
50     }
51
52     public:
53
54     PersistentSegtree( int n, int MIN, int MAX ) : MIN(MIN),
           MAX(MAX), rt(1), id(0) {
55         seg = new Node[n*__lg(MAX - MIN + 1) + 2*n];
56         create();
57     }
58
59     void update( int id, int x ){
60         rt.push_back(update(rt.back(), MIN, MAX, id, x ) );
61     }
62
63     ll query( int L, int R, ll D, ll U ){
64         return query( rt[L], rt[R + 1], MIN, MAX, D, U );
65     }
66 };
67
68 void solve(){
69     int n, k; cin >> n >> k;
70     vector<pii> xs(n);
71     vi v(n);
72     FOR(i, 0, n){ cin >> v[i]; xs[i] = pii( v[i], i ); }
73
74     PersistentSegtree seg(n, 0, n - 1);
75
76     sort(all(xs));
77     FOR(i, 0, n){
78         v[i] = lower_bound(all(xs), pii(v[i], i)) - xs.begin
               ();
79         seg.update(v[i], 1);
80     }
81
82     vi p(n); iota(all(p), 0);
83     stable_sort(all(p), [&]( int i, int j ){
84         bool inv = (i > j);
85         if( i > j ) swap( i, j );
86
87         if( v[i] < v[j] ) inv ^= true;
88         else inv ^= (seg.query( 0, j - 1, v[i] + 1, n - 1 )
               >= k);
89         return inv;
90     });
91
92     FOR(i, 0, n) cout << xs[v[p[i]]].first << "␣"; cout << '\
           n';
93 }
```

14.6 Counting Lcm Arrays

Given two integers n and k, your task is to count the number of arrays $a_1, a_2, \ldots, a_n$ of positive integers where $\text{lcm}(a_i, a_{i+1}) = k$ for all $1 \leq i < n$.

Input
The first line has an integer t: the number of test cases. The next t lines have two integers n and k: the length of the array and the value of lcm.

Output
Print t integers: the answer to each test case modulo $10^9 + 7$.

Constraints
• $1 \leq t \leq 1000$ • $2 \leq n \leq 10^9$ • $1 \leq k \leq 10^9$

```
/*
    k = prod p[i]**q[i];

    dp[n] = dp[n - 1] + q*dp[n - 2];
    dp[0] = 1;
    dp[1] = q + 1
*/
using ll = long long;

const int mod = 1e9 + 7;

int sum( int a, int b ){
    return ((a + b)%mod + mod)%mod;
}

int mult( int a, int b ){
    return (1LL*a*b)%mod;
}

template<class T, int N> struct Matrix{
    array<array<T, N>, N> d{};
    M operator * ( const M& m ) const {
        M a;
        for( int i = 0; i < N; i++ ) for( int j = 0; j < N; j
             ++ )
            for( int k = 0; k < N; k++ ) a.d[i][j] = sum( a.d
                 [i][j], mult( d[i][k], m.d[k][j] ) );
        return a;
    }
    array<T, N> operator*( const array<T, N>& vec ) const{
        array<T, N> ret{};
        for( int i = 0; i < N; i++ ) for( int j = 0; j < N; j
             ++ ) ret[i] = sum( ret[i], mult( d[i][j], vec[j
             ] ) );
        return ret;
    }
    M operator ^ ( ll p ) const {
        assert(p >= 0);
        M a, b(*this);
        for( int i = 0; i < N; i++ ) a.d[i][i] = 1;
        for(;p; p >>= 1){
            if( p&1 ) a = a*b;
            b = b*b;
        }
        return a;
    }
};

int solve( int n, int q ){
    Matrix<int, 2> base{ array<int, 2>{0, 1}, array<int, 2>{q
        , 1} };
    array<int, 2> dp{ 1, q + 1 };

    dp = (base^(n - 1))*dp;
    return dp[1];
}

void solve(){
    int n, k; cin >> n >> k;

    int ans = 1;
    for( int p = 2; p*p <= k; p++ ){
        if( k%p != 0 ) continue;
        int q = 0;
        while( k%p == 0 ){
            q++;
            k /= p;
        }

        ans = mult( ans, solve(n, q) );
    }
    if( k > 1 ) ans = mult( ans, solve(n, 1));

    cout << ans << "\n";
}
```

14.7 Cyclic Array

You are given a cyclic array consisting of n values. Each element has two neighbors; the elements at positions n and 1 are also considered neighbors. Your task is to divide the array into subarrays so that the sum of each subarray is at most k. What is the minimum number of subarrays?

Input
The first input line contains integers n and k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array. There is always at least one division (i.e., no value in the array is larger than k).

Output
Print one integer: the minimum number of subarrays.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq k \leq 10^{18}$

```
using ll = long long;

const int maxn = 4e5 + 10;
const int logn = 20;

int v[maxn], jmp[logn][maxn];

int query( int l, int r ){
    int ans = 0;
    for( int i = logn - 1; i >= 0; i-- ) if( jmp[i][l] < r )
        ans += (1<<i), l = jmp[i][l];
    return ans + 1;
}

void solve(){
    int n; ll k; cin >> n >> k;
    for( int i = 0; i < n; i++ ){ cin >> v[i]; v[i + n] = v[i
        ]; }

    for( ll i = 0, j = 0, sum = 0; i < 2*n; sum -= v[i++] ){
        while( j < 2*n && sum + v[j] <= k ) sum += v[j++];
        jmp[0][i] = j;
    }

    jmp[0][2*n] = 2*n;

    for( int i = 1; i < logn; i++ )
        for( int j = 0; j <= 2*n; j++ ) jmp[i][j] = jmp[i -
            1][jmp[i - 1][j]];

    int ans = n;
    for( int i = 0; i < n; i++ ) ans = min( ans, query( i, i
        + n ) );
    cout << ans << '\n';
}
```

14.8 Distinct Values Splits

You are given an array of n integers. Your task is to count the number of ways to split the array into continuous segments such that all segments consists of distinct values.

Input
The first line has an integers n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the answer to the problem modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
const int mod = 1e9 + 7;

int main(){
  int n; cin >> n;
  vector<int> v(n);
  for( int &x : v ) cin >> x;

  vector<int> dp(n + 2);
  dp[0] = 1;
  dp[1] = -1;

  map<int, bool> marc;
  for( int i = 0, j = -1; i < n; i++ ){

    while( j + 1 < n && !marc[v[j + 1]] )
      marc[v[++j]] = true;

    dp[i + 1] += dp[i];
    dp[i + 1] %= mod;

    dp[j + 2] -= dp[i];
    dp[j + 2] %= mod;
    marc[v[i]] = false;

    dp[i + 1] = ((dp[i + 1] + dp[i])%mod + mod)%mod;
  }

  cout << dp[n] << endl;
}
```

14.9 Distinct Values Sum

You are given an array $x_1, x_2, \ldots, x_n$. Let $d(a, b)$ denote the number of distinct values in the subarray $x_a, x_{a+1}, \ldots, x_b$. Your task is to calculate the sum $\sum_{a=1}^{n} \sum_{b=a}^{n} d(a, b)$, i.e., the sum of $d(a, b)$ for all subarrays.

Input
The first line has an integer n: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print one integer: the required sum.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```cpp
void solve(){
    int n; cin >> n;
    vector<int> v(n);
    for(auto &x : v) cin >> x;
    auto aux = v;

    sort(all(aux));
    aux.erase(unique(all(aux)), aux.end());

    int max_el = 0;

    for(int i = 0; i < n; i++) {
        v[i] = lower_bound(all(aux), v[i]) - aux.begin();
        max_el = max(max_el, v[i]);
    }

    vector<vi> pos(max_el + 1, vi(1, -1));

    for(int i = 0; i < n; i++) {
        int aux = i - pos[v[i]].back(); pos[v[i]].pop_back();
        pos[v[i]].push_back(aux);
        pos[v[i]].push_back(i);
    }

    for(int i = 0; i <= max_el; i++) {
        if(pos[i].size() == 1) pos[i].pop_back();
        else {
            int aux = n - pos[i].back(); pos[i].pop_back();
            pos[i].push_back(aux);
        }
    }

    ll tot = 0;

    for(int i = 0; i <= max_el; i++) {
        if(pos[i].size() == 0) continue;
        int m = pos[i].size();
        vector<ll> pref_sum(m, 0), pref_sq(m, 0);

        pref_sum[0] = pos[i][0];
        pref_sq[0] = (ll)pos[i][0] * pos[i][0];

        for(int j = 1; j < m; j++) {
            pref_sum[j] = pref_sum[j - 1] + pos[i][j];
            pref_sq[j] = pref_sq[j - 1] + pos[i][j] * pos[i][
                 j];
        }

        ll aux = 0;

        for(int j = 0; j < m; j++) {
            int at = pos[i][j];
            // a_j * a_k * (j + 1), j < k
            aux -= at * (j + 1) * (pref_sum[m - 1] - pref_sum
                 [j]);
            // a_j * a_k * (j), j > k
            if(j > 0) aux += at * (j) * (pref_sum[j - 1]);
        }

        tot += aux;
    }

    cout << ((ll) n ) * (n + 1) * (n + 2) / 6 - tot << endl;

}
```

14.10 List Of Sums

List A consists of n positive integers, and list B contains the sum of each element pair of list A. For example, if $A = [1, 2, 3]$, then $B = [3, 4, 5]$, and if $A = [1, 3, 3, 3]$, then $B = [4, 4, 4, 6, 6, 6]$. Given list B, your task is to reconstruct list A.

Input
The first input line has an integer n: the size of list A. The next line has $\frac{n(n-1)}{2}$ integers: the contents of list B. You can assume that there is a list A that corresponds to the input, and each value in A is between $1 \ldots k$.

Output
Print n integers: the contents of list A. You can print the values in any order. If there are more than one solution, you can print any of them.

Constraints
• $3 \le n \le 100$ • $1 \le k \le 10^9$

```cpp
vi v;

pair<int, vi> check(int a1) {
    multiset<int> ms;
    for(auto x : v) ms.insert(x);
    vi ans = {a1};

    while(ms.size() > 0) {
        int nxt = *ms.begin() - a1;
        for(auto x : ans) {
            auto it = ms.find(x + nxt);
            if(it == ms.end()) return {0, {}};
            ms.erase(it);
        }
        ans.push_back(nxt);
    }

    return {1, ans};
}

// os elementos vao ser a1 + a2, a1 + a3, ..., a1 + ak, a2 +
     a3, ...
// basta achar esse k
void solve(){
    int n; cin >> n;
    v.resize(n * (n - 1) / 2);
    for(auto &x : v) cin >> x;
    sort(all(v));

    for(int k = 2; k <= n; k++) {
        int a1 = (v[0] + v[1] - v[k - 1]) / 2;
        auto [flag, ans] = check(a1);
        if(flag) {
            for(auto x : ans) cout << x << ' ';
            cout << '\n';
            break;
        }
    };
}
```

14.11 Maximum Average Subarrays

You are given an array of n integers. For each $i = 1, 2, \ldots, n$, your task is to find the subarray ending at index i with the largest average. If there are multiple subarrays with the largest average, you should find the longest one.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print n integers: the length of the subarray ending at index i with the largest average for each $i = 1, 2, \ldots, n$.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^6$

```cpp
struct Line {
    mutable ll k, m;
    mutable ld p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ld x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    static constexpr ld inf = 1e18;
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = ((ld)y->m - x->m) / ((ld)x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase
             (y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
};

void solve(){
    int n; cin >> n;
    LineContainer cht;
    cht.add(0, 0);
    ll pref = 0;
    for(int i = 1; i <= n; i++) {
```

```
        int v; cin >> v; pref += v;
        cht.add(i, -pref);
        int bst = next(cht.rbegin()) -> k;
        cout << i - bst << ' ';
    }
    cout << '\n';
}
```

14.12 Maximum Building I

You are given a map of a forest where some squares are empty and some squares have trees. What is the maximum area of a rectangular building that can be placed in the forest so that no trees must be cut down?

Input
The first input line contains integers n and m: the size of the forest. After this, the forest is described. Each square is empty (.) or has trees (*).

Input
Print the maximum area of a rectangular building.

Constraints
- $1 \leq n, m \leq 1000$

```
const int MAX = 1e3 + 5;

bool M[MAX][MAX];
int v[MAX];

vector<int> next_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = 0; i <= n; i++) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

vector<int> prev_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = n; i >= 0; i--) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i <= n + 1; i++) {
        M[i][0] = 1;
        M[i][m + 1] = 1;
    }
    for(int j = 0; j <= m + 1; j++) {
        M[0][j] = 1;
        M[n + 1][j] = 1;
    }
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            char c; cin >> c;
            if(c == '*') M[i][j] = 1;
        }
    }

    int ans = 0;

    for(int i = 1; i <= n; i++) {
        v[0] = -1; v[m + 1] = -1;
        for(int j = 1; j <= m; j++) {
            if(M[i][j]) v[j] = 0;
            else v[j]++;
        }

        auto nxt = next_smaller(m + 1);
        auto prev = prev_smaller(m + 1);

        for(int j = 1; j <= m; j++) {
            int h = v[j], w = nxt[j] - prev[j] - 1;
            ans = max(ans, h * w);
        }
    }

    cout << ans << '\n';
}
```

14.13 Multiplication Table

Find the middle element when the numbers in an $n \times n$ multiplication table are sorted in increasing order. It is assumed that n is odd. For example, the 3×3 multiplication table

1&2&3
2&4&6
3&6&9

is as follows: The numbers in increasing order are $[1, 2, 2, 3, 3, 4, 6, 6, 9]$, so the answer is 3.

Input
The only input line has an integer n.

Output
Print one integer: the answer to the task.

Constraints
- $1 \leq n < 10^6$

```
using ll = long long;

const ll inf = 1e12;

bool check( ll n, ll k ){
  ll cont = 0;
  for( int i = 1; i <= n; i++ ) cont += min( n, k/i );
  return cont >= (n*n + 1)/2;
}

ll bs( int n ){
  ll l = 0, r = 1LL*n*n;
  while( l < r ){
    ll mid = ( l + r )/2;
    if( check( n, mid ) ) r = mid;
    else l = mid + 1;
  }
  return r;
}

int main(){
  int n; cin >> n;
  cout << bs(n) << endl;
}
```

14.14 Nearest Campsites I

A campground is represented as a grid where each square can contain a campsite that is either reserved or free. The distance between two squares (x_1, y_1) and (x_2, y_2) is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Your task is to find the maximum distance from a free campsite to the nearest reserved campsite.

Input
The first line has two integers n and m: the number of reserved and free campsites. The next n lines describe the locations of the reserved campsites. Each line has two integers x and y. The next m lines describe the locations of the free campsites. Each line has two integers x and y. You can assume that each square contains at most one campsite.

Output
Print one integer: the longest distance to the nearest reserved campsite.

Constraints
- $1 \leq n, m \leq 10^5$
- $1 \leq x, y \leq 10^6$

```
const int MAX = 2e6+10;
const int INF = 1e9;

struct FT {
    vector<ll> pref;
    vector<ll> suff;

    FT(int n) : pref(n, INF), suff(n, INF) {}

    void update(int pos, ll dif) {
        for (int i = pos; i < sz(pref); i |= i + 1) {
            pref[i] = min(pref[i], dif);
        }
        for (int i = pos; i >= 0; i = (i & (i + 1)) - 1) {
            suff[i] = min(suff[i], dif);
        }
    }

    ll query_pref(int pos) { // [0, pos - 1]
        ll res = INF;
        for (int i = pos; i > 0; i &= i - 1) {
            res = min(res, pref[i-1]);
        }
        return res;
    }

    ll query_suff(int pos) { // [pos, MAX - 1]
        ll res = INF;
```

```
        for (int i = pos; i < sz(suff); i |= i + 1) {
            res = min(res, suff[i]);
        }
        return res;
    }
};

void solve() {
    int n, m; cin >> n >> m;
    vector<tuple<int, int, int>> p1(n), p2(m);
    vector<int> ans(m);

    for(int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        p1[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        p2[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    sort(all(p1)); sort(all(p2));
    auto aux1 = p1, aux2 = p2;

    reverse(all(p1));

    FT nw(MAX), sw(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a <= x) {
                nw.update(b, -a + b);
                sw.update(b, -a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_nw = nw.query_suff(y) + x - y;
        int ans_sw = sw.query_pref(y) + x + y;

        ans[ind] = min(ans_sw, ans_nw);
    }

    p1 = aux1; p2 = aux2;
    reverse(all(p2));
    FT ne(MAX), se(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a >= x) {
                ne.update(b, a + b);
                se.update(b, a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_ne = ne.query_suff(y) - x - y;
        int ans_se = se.query_pref(y) - x + y;

        ans[ind] = min({ans[ind], ans_se, ans_ne});
    }

    int res = -1;
    for(auto x : ans) res = max(res, x);
    cout << res << '\n';
}
```

14.15 Nearest Campsites II

A campground is represented as a grid where each square can contain a campsite that is either reserved or free. The distance between two squares (x_1, y_1) and (x_2, y_2) is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Your task is to find the distance from each free campsite to the nearest reserved campsite.

Input
The first line has two integers n and m: the number of reserved and free campsites. The next n lines describe the locations of the reserved campsites. Each line has two integers x and y. The next m lines describe the locations of the free campsites. Each line has two integers x and y. You can assume that each square contains at most one campsite.

Output
Print m integers: the distances from each free campsite to the nearest reserved campsite, in the input order.

Constraints
• $1 \le n, m \le 10^5$ • $1 \le x, y \le 10^6$

```
const int MAX = 2e6+10;
const int INF = 1e9;

struct FT {
    vector<ll> pref;
    vector<ll> suff;

    FT(int n) : pref(n, INF), suff(n, INF) {}

    void update(int pos, ll dif) {
        for (int i = pos; i < sz(pref); i |= i + 1) {
            pref[i] = min(pref[i], dif);
        }
        for (int i = pos; i >= 0; i = (i & (i + 1)) - 1) {
            suff[i] = min(suff[i], dif);
        }
    }

    ll query_pref(int pos) { // [0, pos - 1]
        ll res = INF;
        for (int i = pos; i > 0; i &= i - 1) {
            res = min(res, pref[i-1]);
        }
        return res;
    }

    ll query_suff(int pos) { // [pos, MAX - 1]
        ll res = INF;
        for (int i = pos; i < sz(suff); i |= i + 1) {
            res = min(res, suff[i]);
        }
        return res;
    }
};

void solve() {
    int n, m; cin >> n >> m;
    vector<tuple<int, int, int>> p1(n), p2(m);
    vector<int> ans(m);

    for(int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        p1[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        p2[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    sort(all(p1)); sort(all(p2));
    auto aux1 = p1, aux2 = p2;

    reverse(all(p1));

    FT nw(MAX), sw(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a <= x) {
                nw.update(b, -a + b);
                sw.update(b, -a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_nw = nw.query_suff(y) + x - y;
        int ans_sw = sw.query_pref(y) + x + y;

        ans[ind] = min(ans_sw, ans_nw);
    }

    p1 = aux1; p2 = aux2;
    reverse(all(p2));
    FT ne(MAX), se(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a >= x) {
                ne.update(b, a + b);
                se.update(b, a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_ne = ne.query_suff(y) - x - y;
        int ans_se = se.query_pref(y) - x + y;

        ans[ind] = min({ans[ind], ans_se, ans_ne});
    }

    for(auto res : ans) cout << res << ' ';
    cout << '\n';
}
```

14.16 Permutation Subsequence

Given two arrays which are permutations, find their longest common subsequence. A subsequence is a sequence of array elements from left to right that can contain gaps. A common subsequence is a subsequence that appears in both arrays.

Input

The first line has two integers n and m: the sizes of the arrays. The second line has n integers $a_1, a_2, \ldots, a_n$: the contents of the first array. The third line has m integers $b_1, b_2, \ldots, b_m$: the contents of the second array.

Output
First print the length of the longest common subsequence. After that, an example of such a sequence. If there are several solutions, you can print any of them.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $1 \leq a_i \leq n$ • $1 \leq b_i \leq m$

```cpp
const int INF = 2000000000;

template<typename T> vector<T> lis(vector<T>& v) {
    int n = v.size(), m = -1;
    vector<T> d(n+1, INF);
    vector<int> l(n);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        // Para non-decreasing use upper_bound()
        int t = lower_bound(d.begin(), d.end(), v[i]) - d.
            begin();
        d[t] = v[i], l[i] = t, m = max(m, t);
    }

    int p = n;
    vector<T> ret;
    while (p--) if (l[p] == m) {
        ret.push_back(v[p]);
        m--;
    }
    reverse(ret.begin(),ret.end());

    return ret;
}

void solve(){
    int n, m; cin >> n >> m;
    vector<int> a(n), b(m);
    vector<int> dec(n + 1);
    for(int i = 0; i < n; i++) {
        cin >> a[i];
        dec[a[i]] = i;
    }
    for(int j = 0; j < m; j++) cin >> b[j];
    if (n < m) {
        swap(n, m);
        swap(a, b);
    }
    vector<int> c(m + 1);
    for(int j = 0; j < m; j++) c[j] = dec[b[j]];

    vector<int> ans = lis<int>(c);
    cout << ans.size() << '\n';
    for(auto x : ans) {
        cout << a[x] << '␣';
    }
    cout << '\n';
}
```

14.17 Pyramid Array

You are given an array consisting of n distinct integers. On each move, you can swap any two adjacent values. You want to transform the array into a pyramid array. This means that the final array has to be first increasing and then decreasing. It is also allowed that the final array is only increasing or decreasing. What is the minimum number of moves needed?

Input
The first input line has an integer n: the size of the array. The next line has n distinct integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the minimum number of moves.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```cpp
using ll = long long;
using vll = vector<ll>;
using vi = vector<int>;

struct BIT{
    int *bit;
    int n;
    BIT( int n ) : n(n), bit(new int[n]) {}
    void update( int i, int x ){
        for(i++; i - 1 < n; i += i&-i ) bit[i - 1] += x;
    }

    int query( int i ){
        int ans = 0;
        for(i++; i - 1 >= 0; i -= i&-i ) ans += bit[i - 1];
        return ans;
    }

    int query( int l, int r ){
        return query(r) - query(l - 1);
    }

    void reset(){
        fill( bit, bit + n, 0 );
    }
};

void solve(){
    int n; cin >> n;
    vi v(n);
    for( int &x : v ) cin >> x;

    vi o = v;
    sort( all(o) );

    BIT bit(n);

    vll pref(n), suf(n);
    for( int i = 0; i < n; i++ ){
        v[i] = lower_bound( all(o), v[i] ) - o.begin();

        pref[i] += bit.query( v[i], n - 1 );
        bit.update( v[i], 1 );
    }

    ll ans = 0;

    bit.reset();
```

```
    for( int i = n - 1; i >= 0; i-- ){
        suf[i] += bit.query( v[i], n - 1 );
        bit.update( v[i], 1 );

        ans += min( pref[i], suf[i] );
    }

    cout << ans << "\n";

}
```

14.18 Shortest Subsequence

You are given a DNA sequence consisting of characters A, C, G, and T. Your task is to find the shortest DNA sequence that is not a subsequence of the original sequence.

Input
The only input line contains a DNA sequence with n characters.

Output
Print the shortest DNA sequence that is not a subsequence of the original sequence. If there are several solutions, you may print any of them.

Constraints
- $1 \le n \le 10^6$

```
int main(){
  string s; cin >> s;

  auto get_id = [&]( char c ){
    if( c == 'A' ) return 0;
    if( c == 'T' ) return 1;
    if( c == 'C' ) return 2;
    return 3;
  };

  vector<bool> ok(4);
  int cont = 0;
  string resp;
  string letras = "ATCG";

  for( char c : s ){
    if( !ok[get_id(c)] ) cont++;
    ok[get_id(c)] = true;

    if( cont == 4 ){
      resp.push_back(c);
      fill( ok.begin(), ok.end(), false );
      cont = 0;
    }
  }

  for( int i = 0; i < 4; i++ ) if( !ok[i] ){
    resp.push_back(letras[i]);
    break;
  }

  cout << resp << endl;

}
```

14.19 Sorting Methods

Here are some possible methods using which we can sort the elements of an array in increasing order:
At each step, choose two adjacent elements and swap them. At each step, choose any two elements and swap them. At each step, choose any element and move it to another position. At each step, choose any element and move it to the front of the array.
Given a permutation of numbers $1, 2, \ldots, n$, calculate the minimum number of steps to sort the array using the above methods.

Input
The first input line contains an integer n. The second line contains n integers describing the permutation.

Output
Print four numbers: the minimum number of steps using each method.

Constraints
- $1 \le n \le 2 \cdot 10^5$

```
// inversion count
int sort_swap_adj(int n, vector<int> l) {
    vector<int> r = l; sort(r.begin(), r.end());
    vector<int> v(n), bit(n);

    vector<pair<int, int>> w;
    for (int i = 0; i < n; i++) w.push_back({r[i], i+1});
    sort(w.begin(), w.end());

    for (int i = 0; i < n; i++) {
        auto it = lower_bound(w.begin(), w.end(), make_pair(l
            [i], 0LL));
        if (it == w.end() or it->first != l[i]) return -1;
        v[i] = it->second;
        it->second = -1;
    }

    ll ans = 0;
    for (int i = n-1; i >= 0; i--) {
        for (int j = v[i]-1; j; j -= j&-j) ans += bit[j];
        for (int j = v[i]; j < n; j += j&-j) bit[j]++;
    }
    return ans;
}

int sort_swap(int n, vector<int> v) {
    vector<int> find(n);
    int tot = 0;
    for(int i = 0; i < n; i++) find[v[i]] = i;
    for(int i = 0; i < n; i++) {
        if(v[i] != i) {
            int x = v[i];
            swap(v[i], v[find[i]]);
            swap(find[i], find[x]);
            tot++;
        }
    }
    return tot;
}

```

```
// n - lis
int sort_move(int n, vector<int> v) {
    vector<int> ans;
    for (auto t : v){
        auto it = lower_bound(ans.begin(), ans.end(), t);
        if (it == ans.end()) ans.push_back(t);
        else *it = t;
    }
    return n - ans.size();
}

int sort_front(int n, vector<int> v) {
    int at = n - 1;
    for(int i = n - 1; i >= 0; i--) {
        if(v[i] == at) at--;
    }
    return at + 1;
}

void solve() {
    int n; cin >> n;
    vector<int> v(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i]; v[i]--;
    }

    cout << sort_swap_adj(n, v) << ' ' << sort_swap(n, v) <<
        ' ' << sort_move(n, v) << ' ' << sort_front(n, v)
        << '\n';
}
```

14.20 Special Substrings

A substring is called special if every character that appears in the string appears the same number of times in the substring. Your task is to count the number of special substrings in a given string.

Input
The only input line has a string of length n. Every character is between a...z.

Output
Print one integer: the number of special substrings.

Constraints
- $1 \le n \le 2 \cdot 10^5$

```
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;

const int mod = 1e9 + 7;
const int b1 = 2e6 + 39;
const int b2 = 2e6 + 3;

int sum( int a, int b ){
    return ((a + b)%mod + mod)%mod;
}

int prod( int a, int b ){
    return (1LL*a*b)%mod;
}

void solve(){
```

```
    string s; cin >> s;

    string letras = s;
    sort( all(letras) );
    letras.erase( unique(all(letras)), letras.end() );

    vi p1(sz(letras), 1), p2(sz(letras), 1);

    int h1 = 0, h2 = 0;
    for( int i = 0; i + 1 < sz(letras); i++ ){
        p1[i + 1] = prod( p1[i], b1 );
        p2[i + 1] = prod( p2[i], b2 );
        h1 = sum( h1, prod( sz(s), p1[i] ) );
        h2 = sum( h2, prod( sz(s), p2[i] ) );
    }

    map<pii, int> cnt;
    cnt[pii(h1, h2)]++;

    ll ans = 0;
    for( int i = 0; i < sz(s); i++ ){
        char c = s[i];
        int x = lower_bound( all(letras), c ) - letras.begin
            ();

        if( x > 0 ){
            h1 = sum( h1, -p1[x - 1] );
            h2 = sum( h2, -p2[x - 1] );
        }
        if( x + 1 < sz(letras) ){
            h1 = sum( h1, p1[x] );
            h2 = sum( h2, p2[x] );
        }

        ans += cnt[pii( h1, h2 )];
        cnt[pii( h1, h2 )]++;
    }

    cout << ans << "\n";

}
```

14.21 Square Subsets

Given an array of n integers, count the number of subsets whose elements' product is a square number. Also count the empty subset with product equal to one.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the answer to the problem modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 5000$ • $1 \leq x_i \leq 5000$

```
using vi = vector<int>;

const int maxx = 5e5 + 10;
const int mod = 1e9 + 7;
const int logx = 20;

int fator[maxx], id[maxx];

int prod( int a, int b ){
    return a * 1LL * b % mod;
}

struct XorBasis{
    vi b;
    int LD;
    XorBasis( int n ) : b(n, -1), LD(1) {}

    void insert( int mask, int msb ){
        if( msb == -1 ){ LD = prod( 2, LD ); return; }
        if( b[msb] == -1 ){ b[msb] = mask; return; }
        else mask ^= b[msb];

        for( int i = logx - 1; i >= 0; i-- ) if( (mask>>i)&1
            ){
            if( b[i] == -1 ){ b[i] = mask; return; }
            mask ^= b[i];
        }
        LD = prod( 2, LD );
    }
};

int crivo(){
    int tot = 0;
    FOR(i, 2, maxx ) if( !fator[i] ){
        id[i] = tot++;
        for( int j = i; j < maxx; j += i ) fator[j] = i;
    }
    return tot;
}

void solve( int tot_primos ){
    int n; cin >> n;

    XorBasis basis(tot_primos);

    FOR(i, 0, n){
        int x; cin >> x;
        vi v;
        for(; x > 1; x /= fator[x] ){
            if( !v.empty() && v.back() == fator[x] ) v.
                pop_back();
            else v.push_back(fator[x]);
        }

        int mask = 0, msb = (v.empty() ? -1 : id[v[0]]);
        FOR(j, 1, sz(v)) mask |= (1<<id[v[j]]);

        basis.insert( mask, msb );
    }

    cout << basis.LD << '\n';
}

int main(){
    solve(crivo());
}
```

14.22 Stack Weights

You have n coins, each of which has a distinct weight. There are two stacks which are initially empty. On each step you move one coin to a stack. You never remove a coin from a stack. After each move, your task is to determine which stack is heavier (if we can be sure that either stack is heavier).

Input
The first input line has an integer n: the number of coins. The coins are numbered $1, 2, \ldots, n$. You know that coin i is always heavier than coin $i-1$, but you don't know their exact weights. After this, there are n lines that describe the moves. Each line has two integers c and s: move coin c to stack s ($1 =$ left, $2 =$ right).

Output
After each move, print $<$ if the right stack is heavier, $>$ if the left stack is heavier, and ? if we can't know which stack is heavier.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$

```
class Segtree{
    private:

    struct Node{
        int tot, suf;
        Node( int tot = 0, int suf = 0 ) : tot(tot), suf(suf)
            {}
        Node operator + ( Node o ){ return Node( tot + o.tot,
            max( o.suf, suf + o.tot ) ); }
    };

    Node *seg;
    int n;

    public:

    void update( int node, int ti, int tf, int id, int x ){
        if( ti == tf ){ seg[node] = Node( x, max( x, 0 ) );
            return; }
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        if( id <= tm ) update( l, ti, tm, id, x );
        else update( r, tm + 1, tf, id, x );
        seg[node] = seg[l] + seg[r];
    }

    Segtree( int n ) : n(n), seg(new Node[4*n]){}

    void update( int id, int x ){
        update( 1, 0, n - 1, id, x );
    }

    int query(){ return seg[1].suf; }
};

void solve(){
    int n; cin >> n;

    Segtree left(n), right(n);
    for( int i = 0; i < n; i++ ){
        int x, t; cin >> x >> t; x--;

        if( t == 2 ) t = -1;

        left.update( x, t );
        right.update( x, -t );

        if( left.query() > 0 && right.query() > 0 ) cout << "
            ?\n";
        else if( left.query() > 0 ) cout << ">\n";
        else cout << "<\n";
    }
```

```
48 }
```

14.23 Subarray Sum Constraints

Your task is to construct an array $x_1, x_2, \ldots, x_n$ consisting of n integers. The array must satisfy m constraints of the form (l, r, s): the sum $x_l + x_{l+1} + \cdots + x_r$ must equal s.

Input
The first line has two integers n and m: the length of the array and the number of constraints. The next m lines each have three integers l, r and s: the description of the constraints.

Output
If a solution exists, print YES on the first line. On the second line, print n integers $x_1, x_2, \ldots, x_n$: the contents of the array. All elements of the array must satisfy $-10^{15} \le x_i \le 10^{15}$ and the array must satisfy all given constraints. You can print any valid solution. If no solution exists, just print NO.

Constraints
• $1 \le n \le 5000$ • $0 \le m \le 2 \cdot 10^5$ • $1 \le l \le r \le n$ • $-10^9 \le s \le 10^9$

```
using ll = long long;
using vi = vector<int>;
using pii = pair<ll, ll>;

const ll inf = 1e15;

struct UnionFind{
    int *p, *sz;
    ll *w;

    UnionFind( int n ) : p(new int[n]), sz(new int[n]), w(new
          ll[n]){
        iota( p, p + n, 0 ); fill( sz, sz + n, 1 );
    }

    int find( int u ){
        while( u != p[u] ) u = p[u];
        return u;
    }

    ll cost( int u ){
        ll ans = 0;
        while( u != p[u] ){ ans += w[u]; u = p[u]; }
        return ans;
    }

    bool join( int u, int v, ll val ){
        ll wu = cost(u);
        ll wv = cost(v);

        u = find(u); v = find(v);
        if( u == v ) return wu - wv == val;

        if( sz[u] < sz[v] ) val = -val, swap( u, v ), swap(
             wu, wv );

        p[v] = u;
        w[v] = wu - wv - val;
        sz[u] += sz[v];
        return true;
    }
};

struct InequationSystem{
    vector<vector<pii>> adj;
    vector<ll> ans;
    InequationSystem( int n ) : adj(n), ans(n, LLONG_MAX) {}
    void add( int x1, int x2, ll y ){ // Adicionar inequacao
         x1 <= x2 + y
        adj[x2].push_back({ x1, y });
    }

    void solve( int x0, ll v0 ){
        ans[x0] = v0;
        bool done = false;
        while( !done ){
            done = true;
            for( int u = 0; u < adj.size(); u++ )
                for( auto [v, w] : adj[u] ) if( ans[v] > ans[
                     u] + w ) ans[v] = ans[u] + w, done =
                     false;
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;
    UnionFind dsu(n + 1);

    while( m-- ){
        int l, r, x; cin >> l >> r >> x;
        if( !dsu.join( l - 1, r, x ) ){ cout << "NO\n";
             return; }
    }

    InequationSystem system(n + 1);
    for( int i = 0; i <= n; i++ ){
        if( dsu.p[i] != i ){
            system.add( i, dsu.p[i], -dsu.w[i] );
            system.add( dsu.p[i], i, dsu.w[i] );
        }
        if( i + 1 <= n ){
            system.add( i + 1, i, inf );
            system.add( i, i + 1, inf );
        }
    }

    system.solve( 0, 0 );
    cout << "YES\n";
    for( int i = 1; i <= n; i++ ) cout << system.ans[i] -
         system.ans[i - 1] << "␣"; cout << "\n";
}
```

14.24 Subsets With Fixed Average

You are given an array of n integers. Your task is to count the number of non-empty subsets of the given array with average equal to a.

Input
The first line has two integers n and a: the size of the array and the target average. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of non-empty subsets with average equal to a, modulo $10^9 + 7$.

Constraints
• $1 \le n \le 500$ • $1 \le a \le 500$ • $1 \le x_i \le 500$

```
const int maxx = 500*500 + 10;
const int mod = 1e9 + 7;

int dp[2*maxx];

int sum( int a, int b ){
    return ((a + b)%mod + mod)%mod;
}

void solve(){
    int n, k; cin >> n >> k;
    dp[maxx] = 1;
    for( int i = 0; i < n; i++ ){
        int x; cin >> x; x -= k;
        if( x >= 0 ) for( int j = 2*maxx - 1; j >= x; j-- )
             dp[j] = sum( dp[j], dp[j - x] );
        else for( int j = 0; j - x < 2*maxx; j++ ) dp[j] =
             sum( dp[j], dp[j - x] );
    }

    cout << sum( dp[maxx], -1 ) << "\n";
}
```

14.25 Swap Game

You are given a 3×3 grid containing the numbers $1, 2, \ldots, 9$. Your task is to perform a sequence of moves so that the grid will look like this:

```
1&2&3
4&5&6
7&8&9
```

On each move, you can swap the numbers in any two adjacent squares (horizontally or vertically). What is the minimum number of moves required?

Input
The input has three lines, and each of them has three integers.

Output
Print one integer: the minimum number of moves.

```
vector<int> adj[9];
unordered_map<int, int> dp;
vector<pair<int, int>> macaco = {{1, 0}, {0, 1}};

int p[9] = {100000000, 10000000, 1000000, 100000, 10000,
     1000, 100, 10, 1};

int troca(int at, int i, int j) {
    int d1 = (at / p[i]) % 10;
    int d2 = (at / p[j]) % 10;
    return at + (d2 - d1) * p[i] + (d1 - d2) * p[j];
}

void solve() {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) {
```

```
            for(auto a : macaco) {
                if(a.first + i < 0 || a.first + i > 2 || a.
                      second + j < 0 || a.second + j > 2)
                      continue;
                adj[i * 3 + j].emplace_back((i + a.first) * 3
                      + (j + a.second));
            }
        }
    }

    int b = 123'456'789; dp[b] = 0;
    queue<int> q; q.push(b);

    int target = 0;
    for(int i = 0; i < 9; i++) {
        int a; cin >> a;
        target = 10 * target + a;
    }

    if(target == b) {
        cout << "0\n";
        return;
    }

    while(!q.empty()) {
        int at = q.front(); q.pop();
        int ans = dp[at];
        for(int k = 0; k < 9; k++) {
            for(auto x : adj[k]) {
                int u = troca(at, k, x);
                if (!dp.count(u)) {
                    dp[u] = ans + 1;
                    if(u == target) {
                        cout << ans + 1 << '\n';
                        return;
                    }
                    q.push(u);
                }
            }
        }
    }
}
```

14.26 Two Array Average

You are given two arrays of n integers. Your task is to select a nonempty prefix from both arrays such that the average of all selected numbers is as large as possible.

Input
The first line has an integer n. The second line has n integers $a_1, a_2, \ldots, a_n$: the numbers in the first array. The third line has n integers $b_1, b_2, \ldots, b_n$: the numbers in the second array.

Output
Print two numbers: the prefix sizes. Your answer is considered correct if the absolute or relative difference to the maximum average is at most 10^{-6}.

Constraints
• $1 \le n \le 10^5$ • $1 \le a_i, b_i \le 10^9$

```
using ld = long double;
using vi = vector<int>;

const ld inf = 1e9;
const int loginf = 60;

pair<ld, int> solve( vector<ld> &v, ld x ){
    int n = v.size();
    ld dp = -inf;
    int len = 0;
    for( auto val : v ){
        if( dp < 0 ) dp = 0, len = 0;
        dp += val - x;
        len++;
    }

    return make_pair( dp, len );
}

ld bs( vector<ld> &a, vector<ld> &b ){
    ld l = 0, r = inf;
    for( int i = 0; i < loginf; i++ ){
        ld m = (l + r)/2;
        auto [dpA, lenA] = solve( a, m );
        auto [dpB, lenB] = solve( b, m );
        if( dpA + dpB >= 0 ) l = m;
        else r = m;
    }
    return r;
}

void solve(){
    int n; cin >> n;
    vector<ld> a(n), b(n);
    for( auto &x : a ) cin >> x;
    for( auto &x : b ) cin >> x;

    reverse( all(a) );
    reverse( all(b) );

    ld ans = bs( a, b );

    auto [dpA, lenA] = solve( a, ans );
    auto [dpB, lenB] = solve( b, ans );

    cout << lenA << " " << lenB << "\n";
}
```

14.27 Water Containers Moves

There are two water containers: container A has volume a and container B has volume b. Your want to measure x units of water using the containers. Initially both containers are empty. On each move, you can fill a container, empty a container or move water from a container to another container. When you move water, you must always fill or empty at least one container. After the moves, container A must have x units of water. Find a sequence of moves where the total amount of moved water is minimum or state that it is impossible to measure the water.

Input
The only line has three integers a, b and x.

Output
First print two integers n and m: the number of moves and the total amount of water moved. After that print a sequence of n moves. Each move must move at least one unit of water and be one of the following:
• FILL A: fill container A • FILL B: fill container B • EMPTY A: empty container A • EMPTY B: empty container B • MOVE A B: move water from container A to container B • MOVE B A: move water from container B to container A
If it is not possible to measure the water, only print -1.

Constraints
• $1 \le a, b, x \le 1000$

```
const int MAX = 1005;
const int INF = 1e18;
// A_cheio = (0, 1); B_vazio = (1, 0), etc
vector<vector<vector<int>>> dp(2, vector<vector<int>>(2,
      vector<int>(MAX, INF)));

void solve() {
    int a, b, x; cin >> a >> b >> x;
    dp[0][0][0] = 0; dp[0][0][b] = b;
    dp[1][0][0] = 0; dp[1][0][a] = a;
    dp[0][1][0] = a; dp[0][1][b] = a + b;
    dp[1][1][0] = b; dp[1][1][a] = a + b;

    for(int j = 0; j < 4 * MAX; j++)
    for(int i = 1; i < MAX; i++) {
        // FILL A
        dp[0][1][i] = min(dp[0][1][i], dp[0][0][i] + a);
        // FILL B
        dp[1][1][i] = min(dp[1][1][i], dp[1][0][i] + b);

        // EMPTY A
        dp[0][0][i] = min(dp[0][0][i], dp[0][1][i] + a);
        // EMPTY B
        dp[1][0][i] = min(dp[1][0][i], dp[1][1][i] + b);

        // MOVE A B
        if(i <= b) dp[0][0][i] = min(dp[1][0][i] + i, dp
              [0][0][i]);
        if(i >= a && i <= b) dp[0][0][i] = min(dp[0][1][i - a
              ] + a, dp[0][0][i]);

        if(b + i <= a) dp[1][1][i] = min(dp[1][0][b + i] + b,
               dp[1][1][i]);
        if(a - i >= 0 && b >= a - i) dp[1][1][i] = min(dp
              [1][1][i], dp[0][1][b - (a - i)] + (a - i));

        // MOVE B A
        if(i <= a) dp[1][0][i] = min(dp[0][0][i] + i, dp
              [1][0][i]);
        if(i >= b && i <= a) dp[1][0][i] = min(dp[1][1][i - b
              ] + b, dp[1][0][i]);

        if(a + i <= b) dp[0][1][i] = min(dp[0][0][a + i] + a,
               dp[0][1][i]);
        if(b - i >= 0 && a >= b - i) dp[0][1][i] = min(dp
              [0][1][i], dp[1][1][a - (b - i)] + (b - i));
    }

    vector<string> ans;
    int min_cost = min(dp[1][0][x], dp[1][1][x]);

    if(min_cost >= INF) {
        cout << "-1\n";
        return;
    }

    bool A = 1, c = 1;
```

```
    if(dp[1][0][x] < dp[1][1][x]) c = 0;

    while(true) {

        if((A == 0 || A == 1) && c == 0 && x == 0) break;
        if(A == 0 && c == 0 && x == b) {
            ans.push_back("FILL B");
            break;
        }
        if(A == 1 && c == 0 && x == a) {
            ans.push_back("FILL A");
            break;
        }
        if(A == 0 && c == 1 && x == 0) {
            ans.push_back("FILL A");
            break;
        }
        if(A == 1 && c == 1 && x == 0) {
            ans.push_back("FILL B");
            break;
        }
        if(A == 0 && c == 1 && x == b) {
            ans.push_back("FILL A");
            ans.push_back("FILL B");
            break;
        }
        if(A == 1 && c == 1 && x == a) {
            ans.push_back("FILL A");
            ans.push_back("FILL B");
            break;
        }

        if(A == 0 && c == 1) {
            // FILL A
            if(dp[0][1][x] == dp[0][0][x] + a) {
                ans.push_back("FILL A");
                c = 0;
            }
            // MOVE B A
            else if(b >= x && a >= b - x && dp[0][1][x] == dp
                  [1][1][a - (b - x)] + (b - x)) {
                ans.push_back("MOVE B A");
                A = 1;
                x = a - (b - x);
            }
            else if(a + x <= b && dp[0][1][x] == dp[0][0][a +
                   x] + a) {
                ans.push_back("MOVE B A");
                c = 0;
                x = a + x;
            }
        }
        else if(A == 1 && c == 1) {
            // FILL B
            if(dp[1][1][x] == dp[1][0][x] + b) {
                ans.push_back("FILL B");
                c = 0;
            }
            // MOVE A B
            else if(a >= x && b >= a - x && dp[1][1][x] == dp
                  [0][1][b - (a - x)] + (a - x)) {
                ans.push_back("MOVE A B");
                A = 0;
                x = b - (a - x);
            }
            else if(b + x <= a && dp[1][1][x] == dp[1][0][b +
                   x] + b) {
                ans.push_back("MOVE A B");
                c = 0;
                x = b + x;
            }
        }
        else if(A == 0 && c == 0) {
            // EMPTY A
            if(dp[0][0][x] == dp[0][1][x] + a) {
                ans.push_back("EMPTY A");
                c = 1;
            }
            else if(x <= b && dp[0][0][x] == dp[1][0][x] + x)
                  {
                ans.push_back("MOVE A B");
                A = 1;
            }
            else if(x >= a && x <= b && dp[0][0][x] == dp
                  [0][1][x - a] + a){
                ans.push_back("MOVE A B");
                c = 1;
                x = x - a;
            }
        }
        else if(A == 1 && c == 0) {
            // EMPTY B
            if(dp[1][0][x] == dp[1][1][x] + b) {
                ans.push_back("EMPTY B");
                c = 1;
            }
            else if(x <= a && dp[1][0][x] == dp[0][0][x] + x)
                  {
                ans.push_back("MOVE B A");
                A = 0;
            }
            else if(x >= b && x <= a && dp[1][0][x] == dp
                  [1][1][x - b] + b){
                ans.push_back("MOVE B A");
                c = 1;
                x = x - b;
            }
        }
    }

    cout << ans.size() << ' ';
    cout << min_cost << '\n';

    reverse(ans.begin(), ans.end());

    for(auto x : ans) cout << x << '\n';
}
```

14.28 Water Containers Queries

There are two water containers: container A has volume a and container B has volume b. You want to measure x units of water using the containers. Initially both containers are empty. On each move, you can fill a container, empty a container or move water from a container to another container. When you move water, you must always fill or empty at least one container. After the moves, container A must have x units of water. Your task is to efficiently check if it is possible to measure the water in several cases.

Input
The first line has integer n: the number of tests. After this, there are n lines. Each line has three integers a, b and x.

Output
For each test, print YES if it is possible to measure the water and NO otherwise.

Constraints

- $1 \le n \le 1000$
- $1 \le a, b, x \le 10^9$

```
void solve(){
    int a, b, x; cin >> a >> b >> x;
    cout << ((x <= a && x%__gcd(a, b) == 0) ? "YES" : "NO" )
         << '\n';
}
```

14.29 Writing Numbers

You would like to write a list of positive integers $1, 2, 3, \ldots$ using your computer. However, you can press each key 0–9 at most n times during the process. What is the last number you can write?

Input
The only input line contains the value of n.

Output
Print the last number you can write.

Constraints

- $1 \le n \le 10^{18}$

```
ll pot[19], memo[19];
const ll INF = 1e18;

ll ans(ll n) {
    if(n == 0) return 0;
    if(n < 10) return 1;
    if(n == 10) return 2;
    ll a = n, k = 0;
    while(a > 9) {
        a /= 10;
        k++;
    }

    if(a > 1) {
        return ans(n - pot[k] * a) + memo[k] * a + pot[k];
    }
    else {
        return ans(n - pot[k] * a) + (n - pot[k] * a + 1) +
               memo[k];
    }
}

void solve(){
    ll n; cin >> n;
    ll l = 0, r = INF;

    while(r - l > 1) {
        ll m = (l + r) / 2;
        if(ans(m) <= n) l = m;
        else r = m;
    }

    cout << l << '\n';
}

signed main() {
    pot[0] = 1;
```

```
37      for(int i = 1; i <= 18; i++) {
38          pot[i] = 10 * pot[i - 1];
39      }
40      for(int i = 0; i <= 18; i++) {
41          memo[i] = i * pot[i - 1];
42      }
43      solve();
44      return 0;
45  }
```

15 Additional Problems II

15.1 Binary Subsequences

Your task is to find a minimum length bit string that has exactly n distinct subsequences. For example, a correct solution for $n = 6$ is 101 whose distinct subsequences are 0, 1, 01, 10, 11 and 101.

Input
The only input line has an integer n.

Output
Print one bit string: a solution to the task. You can print any valid solution.

Constraints
• $1 \leq n \leq 10^6$

```
int ans_tam(int a, int b) {
    if (a == b || a == 0 || b == 0) return 0;
    else if (a > b) return a / b + ans_tam(a % b, b);
    else return b / a + ans_tam(a, b % a);
}

string ans(int a, int b) {
    if (a == b) return "";
    else if (a > b)
        return '1' + ans(a - b, b);
    else
        return '0' + ans(a, b - a);
}

void solve() {
    int n; cin >> n; n++;
    int a, b, mini = -1;
    int bsta, bstb;

    for (a = 1; a <= n + 1; a++) {
        if (__gcd(n + 1, a) != 1) continue;
        b = n + 1 - a;
        int tam = ans_tam(a, b);
        if (mini == -1 || tam < mini) {
            mini = tam;
            bsta = a;
            bstb = b;
        }
    }

    cout << ans(bsta, bstb) << '\n';
}
```

15.2 Bit Substrings

You are given a bit string of length n. Your task is to calculate for each k between $0 \ldots n$ the number of non-empty substrings that contain exactly k ones. For example, if the string is 101, there are:
• 1 substring that contains 0 ones: 0 • 4 substrings that contain 1 one: 01, 1, 1, 10 • 1 substring that contains 2 ones: 101 • 0 substrings that contain 3 ones

Input
The only input line contains a binary string of length n.

Output
Print $n + 1$ values as specified above.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$

```
using ll = long long;
using cd = complex<long double>;

const double PI = acos(-1);

void FFT( vector<cd> &a, bool invert ){
  int n = a.size();
  for( int i = 1, j = 0; i < n; i++ ){
    int bit = (n>>1);
    for(; j&bit; bit >>= 1 ) j ^= bit;
    j ^= bit;

    if( i < j ) swap( a[i], a[j] );
  }

  for( int len = 2; len <= n; len <<= 1 ){
    double ang = ((invert) ? -1.0 : 1.0 )*2*PI/len;
    cd wlen( cos(ang), sin(ang) );
    for( int i = 0; i < n; i += len ){
      cd w(1);
      for( int j = 0; j < len/2; j++ ){
        cd u = a[i + j], v = a[i + j + len/2]*w;
        a[i + j] = u + v;
        a[i + j + len/2] = u - v;
        w *= wlen;
      }
    }
  }

  if( invert ) for( auto &x : a ) x /= n;
}

vector<ll> mult( vector<int> &a, vector<int> &b ){
  vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
  int n = 1;
  while( n < a.size() + b.size() ) n <<= 1;
  fa.resize(n); fb.resize(n);

  FFT( fa, false ); FFT( fb, false );
  for( int i = 0; i < n; i++ ) fa[i] *= fb[i];
  FFT( fa, true );

  vector<ll> resp(n);
  for( int i = 0; i < n; i++ ) resp[i] = round(fa[i].real());
  return resp;
}

int main(){

  string s; cin >> s;
  int n = s.size();

  vector<int> a(n + 1), b(n + 1);

  ll resp0 = 0;

  a[0] = 1;
  for( int i = 0, sum = 0, cont0 = 0; i < n; i++ ){
    sum += s[i] - '0';
    a[sum]++;

    if( s[i] == '0' ) cont0++;
    else cont0 = 0;

    resp0 += cont0;
  }

  for( int i = 0; i <= n; i++ ) b[i] = a[n - i];

  vector<ll> v = mult( a, b );

  cout << resp0 << " ";
  for( int i = n + 1; i <= 2*n; i++ ) cout << v[i] << " ";
        cout << endl;
}
```

15.3 Book Shop II

You are in a book shop which sells n different books. You know the price, the number of pages and the number of copies of each book. You have decided that the total price of your purchases will be at most x. What is the maximum number of pages you can buy? You can buy several copies of the same book.

Input
The first input line contains two integers n and x: the number of book and the maximum total price. The next line contains n integers $h_1, h_2, \ldots, h_n$: the price of each book. The next line contains n integers $s_1, s_2, \ldots, s_n$: the number of pages of each book. The last line contains n integers $k_1, k_2, \ldots, k_n$: the number of copies of each book.

Output
Print one integer: the maximum number of pages.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^5$ • $1 \leq h_i, s_i, k_i \leq 1000$

```
void solve(){
    int n, x; cin >> n >> x;
    vector<pll> list;

    vector<ll> p(n), h(n), k(n);
    for(int i = 0; i < n; i++) cin >> p[i];
    for(int i = 0; i < n; i++) cin >> h[i];
    for(int i = 0; i < n; i++) cin >> k[i];

    for (int i = 0; i < n; i++) {
        ll c = 1;
        while (k[i] > c) {
            k[i] -= c;
            list.push_back({c * p[i], c * h[i]});
            c *= 2;
        }
        list.push_back({k[i] * p[i], k[i] * h[i]});
    }

    vector<ll> dp(x + 1, 0);

```

```
    for(auto item : list) {
        ll peso = item.first, valor = item.second;

        for(int j = x; j >= peso; j--) {
            dp[j] = max(dp[j], dp[j - peso] + valor);
        }
    }

    cout << dp[x] << '\n';
}
```

15.4 Bouncing Ball Steps

There is a ball at the top-left corner of an $n \times m$ grid. The rows of the grid are numbered $1, 2, \ldots, n$, and the columns are numbered $1, 2, \ldots, m$. The ball is initially moving diagonally away from the top-left corner. At every step, it moves one cell. Whenever the ball hits the border of the grid, it changes its direction. What is the location of the ball after k steps and how many times has it changed direction?

Input
The first line has an integer t: the number of tests. After this, there are t lines. Each line has three integers n, m and k: the size of the grid and the number of steps.

Output
For each test, print three integers: the location of the ball and the number of direction changes.

Constraints
• $1 \le t \le 1000$ • $2 \le n, m \le 10^9$ • $0 \le k \le 10^{18}$

```
using ll = long long;

void solve(){
    ll n, m, k; cin >> n >> m >> k;

    auto calc = [&]( ll len, ll dist ){
        ll flips = dist/(len - 1);
        ll pos = (flips%2) ? len - dist%(len - 1) : dist%(len
              - 1) + 1;
        return make_pair( pos, flips );
    };

    auto [i, qi] = calc( n, k );
    auto [j, qj] = calc( m, k );

    ll lcm = (n - 1)*(m - 1)/__gcd(n - 1, m - 1);

    cout << i << "␣" << j << "␣" << qi + qj - k/lcm << "\n";
}
```

15.5 Coding Company

Your company has n coders, and each of them has a skill level between 0 and 100. Your task is to divide the coders into teams that work together. Based on your experience, you know that teams work well when the skill levels of the coders are about the same. For this reason, the penalty for creating a team is the skill level difference between the best and the worst coder. In how many ways can you divide the coders into teams such that the sum of the penalties is at most x?

Input
The first input line has two integers n and x: the number of coders and the maximum allowed penalty sum. The next line has n integers $t_1, t_2, \ldots, t_n$: the skill level of each coder.

Output
Print one integer: the number of valid divisions modulo 10^9+7.

Constraints
• $1 \le n \le 100$ • $0 \le x \le 5000$ • $0 \le t_i \le 100$

```
using vi = vector<int>;

const int maxn = 1e2 + 5;
const int maxx = 5e3 + 5;
const int mod = 1e9 + 7;

int sum( int a, int b ){
    return (a + b)%mod;
}

int prod( int a, int b ){
    return 1LL*a*b % mod;
}

int dp[2][maxn][maxx];

void solve(){
    int n, x; cin >> n >> x;
    vi v(n);
    for( int &x : v ) cin >> x;

    sort(all(v));

    int cur = 0, prev = 1;
    for( int j = 0; j <= x; j++ ) dp[cur][0][j] = 1;

    for( int t : v ){
        swap( cur, prev );
        for( int j = 0; j <= n/2; j++ )
            for( int k = 0; k < maxx; k++ ){
                // Colocar em um grupo existente ou em um
                    grupo proprio
                dp[cur][j][k] = prod( j + 1, dp[prev][j][k] )
                    ;

                // Criar novo grupo
                if( k + t < maxx ) dp[cur][j][k] = sum( dp[
                    cur][j][k], dp[prev][j + 1][k + t] );

                // Terminar um grupo
                if( k - t >= 0 && j > 0  ) dp[cur][j][k] =
                    sum( dp[cur][j][k], prod( j, dp[prev][j
                     - 1][k - t] ) );
            }
    }

    cout << dp[cur][0][0] << '\n';
}
```

15.6 Coin Grid

There is an $n \times n$ grid whose each square is empty or has a coin. On each move, you can remove all coins in a row or column. What is the minimum number of moves after which the grid is empty?

Input
The first input line has an integer n: the size of the grid. The rows and columns are numbered $1, 2, \ldots, n$. After this, there are n lines describing the grid. Each line has n characters: each character is either . (empty) or o (coin).

Output
First print an integer k: the minimum number of moves. After this, print k lines describing the moves. On each line, first print 1 (row) or 2 (column), and then the number of a row or column. You can print any valid solution.

Constraints
• $1 \le n \le 100$

```
using vi = vector<int>;
const int maxn = 100;

vi adj[2*maxn];
int match[2*maxn];
bool marc[2*maxn];

bool dfs( int u ){
    for( int v : adj[u] ) if( !marc[v] ){
        marc[v] = true;
        if( match[v] == -1 || dfs(match[v]) ){
            match[u] = v; match[v] = u;
            return true;
        }
    }
    return false;
}

int matching( int n ){
    bool ok = true;
    int ans = 0;

    fill( match, match + 2*n, -1 );
    while( ok ){
        ok = false;
        fill( marc + n, marc + 2*n, false );
        for( int i = 0; i < n; i++ ) if( match[i] == -1 &&
             dfs(i) ){
            ans++; ok = true;
        }
    }
    return ans;
}
```

```
33
34 vi get_cover( int n ){
35     fill( marc, marc + 2*n, false );
36     queue<int> q;
37     for( int i = 0; i < 2*n; i++ ) if( match[i] == -1 ){
38         q.push(i);
39     }
40
41     vi cover;
42     for(; !q.empty(); q.pop() ){
43         int u = q.front();
44         for( int v : adj[u] ) if( !marc[v] ){
45             cover.push_back(v); marc[v] = true;
46             q.push(match[v]);
47         }
48     }
49
50     for( int i = 0; i < 2*n; i++ ) if( match[i] != -1 ){
51         if( !marc[i] && !marc[match[i]] ){
52             marc[i] = true;
53             cover.push_back(i);
54         }
55     }
56     return cover;
57 }
58
59 void solve(){
60     int n; cin >> n;
61     for( int i = 0; i < n; i++ )
62         for( int j = 0; j < n; j++ ){
63             char c; cin >> c;
64             if( c == 'o' ){
65                 adj[i].push_back(j + n);
66                 adj[j + n].push_back(i);
67             }
68         }
69
70     cout << matching(n) << endl;
71
72     vi cover = get_cover(n);
73     for( int x : cover ) cout << x/n + 1 << " " << x%n + 1 <<
           "\n";
74 }
```

15.7 Food Division

There are n children around a round table. For each child, you know the amount of food they currently have and the amount of food they want. The total amount of food in the table is correct. At each step, a child can give one unit of food to his or her neighbour. What is the minimum number of steps needed?

Input
The first input line contains an integer n: the number of children. The next line has n integers $a_1, a_2, \ldots, a_n$: the current amount of food for each child. The last line has n integers $b_1, b_2, \ldots, b_n$: the required amount of food for each child.

Output
Print one integer: the minimum number of steps.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le a_i, b_i \le 10^6$

```
1 void solve(){
2     int n; cin >> n;
3     vector<int> x(n), y(n), a(n), S(n);
4     for(int i = 0; i < n; i++) cin >> x[i];
5     for(int i = 0; i < n; i++) cin >> y[i];
6
7     for(int i = 1; i < n; i++)
8     {
9         S[i] = S[i - 1] + y[i] - x[i];
10    }
11
12    int ans = 0;
13    sort(S.begin(), S.end());
14    int median = S[n/2];
15
16    for(int i = 0; i < n; i++)
17    {
18        ans += abs(median - S[i]);
19    }
20
21    cout << ans << '\n';
22 }
```

15.8 Gcd Subsets

You are given an array of n integers. Your task is to calculate the number of non-empty subsets whose elements' greatest common divisor is equal to k for each $k = 1, \ldots, n$.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print n integers as specified above modulo $10^9 + 7$.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le n$

```
1 using ll = long long;
2
3 const ll mod = 1e9 + 7;
4
5 ll pot( ll b, ll e, ll mod ){
6   ll resp = 1;
7   for(; e > 0; e >>= 1 ){
8     if( e&1 ) resp = (resp*b)%mod;
9     b = (b*b)%mod;
10  }
11  return resp;
12 }
13
14 int main(){
15   int n; cin >> n;
16   vector<ll> cont(n + 1);
17   for( int i = 0; i < n; i++ ){
18     int x; cin >> x;
19     for( int j = 1; j*j <= x; j++ ) if( x%j == 0 ){
20       cont[j]++;
21       if( j*j != x ) cont[x/j]++;
22     }
23   }
24
25   for( int i = n; i >= 1; i-- ){
26     cont[i] = (pot( 2, cont[i], mod ) - 1 + mod)%mod;
27     for( int j = 2*i; j <= n; j += i ) cont[i] = (cont[i] -
           cont[j] + mod)%mod;
28   }
29
30   for( int i = 1; i <= n; i++ ) cout << cont[i] << " "; cout
         << endl;
31 }
```

15.9 Grid Coloring II

You are given an $n \times m$ grid where each cell contains one character A, B or C. For each cell, you must change the character to A, B or C. The new character must be different from the old one. Your task is to change the characters in every cell such that no two adjacent cells have the same character.

Input
The first line has two integers n and m: the number of rows and columns. The next n lines each have m characters: the description of the grid.

Output
Print n lines each with m characters: the description of the final grid. You may print any valid solution. If no solution exists, just print IMPOSSIBLE.

Constraints
• $1 \le n, m \le 500$

```
1 const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
2
3 struct TwoSat {
4     int N;
5     vector<vi> gr;
6     vi values; // 0 = false, 1 = true
7
8     TwoSat(int n = 0) : N(n), gr(2*n) {}
9
10    int addVar() { // (optional)
11        gr.emplace_back();
12        gr.emplace_back();
13        return N++;
14    }
15
16    void either(int f, int j) {
17        f = max(2*f, -1-2*f);
18        j = max(2*j, -1-2*j);
19        gr[f].push_back(j^1);
20        gr[j].push_back(f^1);
21    }
22    void setValue(int x) { either(x, x); }
23
24    void atMostOne(const vi& li) { // (optional)
25        if (li.size() <= 1) return;
26        int cur = ~li[0];
27        for(int i = 2; i < li.size(); i++) {
28            int next = addVar();
29            either(cur, ~li[i]);
30            either(cur, next);
31            either(~li[i], next);
32            cur = ~next;
33        }
34        either(cur, ~li[1]);
```

```
    }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ?: dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        for(int i = 0; i < 2 * N; i++) if (!comp[i]) dfs(i);
        for(int i = 0; i < 1 * N; i++) if (comp[2*i] == comp
             [2*i+1]) return 0;
        return 1;
    }
};

void solve(){
    int n, m; cin >> n >> m;
    char M[n][m];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            cin >> M[i][j];

    TwoSat ts(n * m);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(i + 1 < n)
            {
                if(M[i][j] == M[i + 1][j])
                {
                    ts.either((i * m + j), ((i + 1) * m + j))
                        ;
                    ts.either(~(i * m + j), ~((i + 1) * m + j
                        ));
                }
                else if(M[i][j] + M[i + 1][j] == 'B' + 'C')
                {
                    ts.either((i * m + j), ((i + 1) * m + j))
                        ;
                }
                else if(M[i][j] + M[i + 1][j] == 'A' + 'B')
                {
                    ts.either(~(i * m + j), ~((i + 1) * m + j
                        ));
                }
                else if(M[i][j] == 'A' && M[i + 1][j] == 'C')
                {
                    ts.either((i * m + j), ~((i + 1) * m + j)
                        );
                }
                else
                {
                    ts.either(~(i * m + j), ((i + 1) * m + j)
                        );
                }
            }

            if(j + 1 < m)
            {
                if(M[i][j] == M[i][j + 1])
                {
                    ts.either((i * m + j), (i * m + (j + 1)))
                        ;
                    ts.either(~(i * m + j), ~(i * m + (j + 1)
                        ));
                }
                else if(M[i][j] + M[i][j + 1] == 'B' + 'C')
                {
                    ts.either((i * m + j), (i * m + (j + 1)))
                        ;
                }
                else if(M[i][j] + M[i][j + 1] == 'A' + 'B')
                {
                    ts.either(~(i * m + j), ~(i * m + (j + 1)
                        ));
                }
                else if(M[i][j] == 'A' && M[i][j + 1] == 'C')
                {
                    ts.either((i * m + j), ~(i * m + (j + 1))
                        );
                }
                else
                {
                    ts.either(~(i * m + j), (i * m + (j + 1))
                        );
                }
            }
        }
    }

    if(!ts.solve())
    {
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            int ans = ts.values[i * m + j];
            if(M[i][j] == 'A') cout << "BC"[ans];
            if(M[i][j] == 'B') cout << "AC"[ans];
            if(M[i][j] == 'C') cout << "AB"[ans];
        }
        cout << '\n';
    }
}
```

15.10 Grid Puzzle I

There is an $n \times n$ grid, and your task is to choose from each row and column some number of squares. How can you do that?

Input
The first input line has an integer n: the size of the grid. The rows and columns are numbered $1, 2, \ldots, n$. The next line has n integers $a_1, a_2, \ldots, a_n$: You must choose exactly a_i squares from the ith row. The las line has n integers $b_1, b_2, \ldots, b_n$: You must choose exactly b_j squares from the jth column.

Output
Print n lines describing which squares you choose (X means that you choose a square, . means that you don't choose it). You can print any valid solution. If it is not possible to satisfy the conditions print only -1.

Constraints

- $1 \leq n \leq 50$
- $0 \leq a_i \leq n$
- $0 \leq b_j \leq n$

```
int main(){
  int n; cin >> n;
  int source = 2*n, sink = 2*n + 1;

  int N = 2*n + 2;

  int total_rows = 0, total_cols = 0;

  vector<vector<int>> capacity( N, vector<int>(N)), adj(N);
  for( int i = 0; i < n; i++ ){
    int x; cin >> x;
    adj[i].push_back(source);
    adj[source].push_back(i);

    capacity[source][i] = x;

    total_rows += x;
  }

  for( int i = n; i < 2*n; i++ ){
    int x; cin >> x;
    adj[i].push_back(sink);
    adj[sink].push_back(i);

    capacity[i][sink] = x;

    total_cols += x;
  }

  for( int i = 0; i < n; i++ )
    for( int j = n; j < 2*n; j++ ){
      adj[i].push_back(j); adj[j].push_back(i);
      capacity[i][j] = 1;
    }

  vector<int> pai(N);

  auto bfs = [&](){
    queue<pair<int, int>> fila;

    fill( pai.begin(), pai.end(), -1 );

    fila.push({ N, source });
    while( !fila.empty() ){
      auto [flow, cur] = fila.front(); fila.pop();
```

```
46        if( cur == sink ) return flow;
47
48        for( int viz : adj[cur] ) if( capacity[cur][viz] && pai
              [viz] == -1 ){
49          fila.push({ min( flow, capacity[cur][viz]), viz });
50          pai[viz] = cur;
51        }
52      }
53      return 0;
54    };
55
56    int flow = 0;
57    for( int f = bfs(); f > 0; f = bfs() ){
58      flow += f;
59      for( int cur = sink; cur != source; cur = pai[cur] ){
60        capacity[pai[cur]][cur] -= f;
61        capacity[cur][pai[cur]] += f;
62      }
63    }
64
65    if( flow != total_cols || total_cols != total_rows ){ cout
          << -1 << endl; return 0; }
66
67    vector<string> resp(n, string(n, '.'));
68    for( int i = 0; i < n; i++ ) for( int j = n; j < 2*n; j++ )
          if( capacity[i][j] == 0 ) resp[i][j - n] = 'X';
69
70    for( string &s : resp ) cout << s << endl;
71
72  }
```

15.11 Grid Puzzle II

There is an $n \times n$ grid whose each square has some number of coins in it. You know for each row and column how many squares you must choose from that row or column. You get all coins from every square you choose. What is the maximum number of coins you can collect and how could you choose the squares so that the given conditions are satisfied?

Input
The first input line has an integer n: the size of the grid. The rows and columns are numbered $1, 2, \ldots, n$. The next line has n integers $a_1, a_2, \ldots, a_n$: You must choose exactly a_i squares from the ith row. The next line has n integers $b_1, b_2, \ldots, b_n$: You must choose exactly b_j squares from the jth column. Finally, there are n lines describing the grid. You can assume that the sums of $a_1, a_2, \ldots, a_n$ and $b_1, b_2, \ldots, b_n$ are equal.

Output
First print an integer k: the maximum number of coins you can collect. After this print n lines describing which squares you choose (X means that you choose a square, . means that you don't choose it). If it is not possible to satisfy the conditions print only -1.

Constraints
• $1 \le n \le 50$ • $0 \le a_i \le n$ • $0 \le b_j \le n$ • $0 \le c_{ij} \le 1000$

```
1  const int inf = 2e9;
2
```

```
3  int main(){
4    int n; cin >> n;
5    int tot_rows = 0, tot_cols = 0;
6    int source = 2*n, sink = 2*n + 1;
7
8    int N = 2*n + 2;
9    vector<vector<int>> capacity( N, vector<int>(N)), adj(N),
         cost(N, vector<int>(N));
10   for( int i = 0; i < n; i++ ){
11     int x; cin >> x;
12     tot_rows += x;
13
14     adj[i].push_back(source);
15     adj[source].push_back(i);
16
17     capacity[source][i] = x;
18   }
19
20   for( int i = n; i < 2*n; i++ ){
21     int x; cin >> x;
22     tot_cols += x;
23
24     adj[i].push_back(sink);
25     adj[sink].push_back(i);
26
27     capacity[i][sink] = x;
28   }
29
30   for( int i = 0; i < n; i++ ) for( int j = n; j < 2*n; j++ )
         {
31     adj[i].push_back(j);
32     adj[j].push_back(i);
33     capacity[i][j] = 1;
34
35     cin >> cost[j][i];
36     cost[i][j] = -cost[j][i];
37   }
38
39   vector<int> dist(N), pai(N), inq(N);
40
41   auto bellman_ford = [&](){
42     fill( dist.begin(), dist.end(), inf );
43     fill( pai.begin(), pai.end(), -1 );
44     fill( inq.begin(), inq.end(), false );
45     dist[source] = 0;
46
47     queue<int> fila;
48     fila.push(source);
49
50     while( !fila.empty() ){
51       int u = fila.front();
52       fila.pop();
53       inq[u] = false;
54
55       for( int v : adj[u] ) if( capacity[u][v] > 0 && dist[v]
             > dist[u] + cost[u][v] ){
56         dist[v] = dist[u] + cost[u][v];
57         pai[v] = u;
58
59         if( !inq[v] ) fila.push(v);
60       }
61     }
62   };
63
64   auto min_cost_flow = [&](){
65     int flow = 0;
66     int cost = 0;
67     while(true){
68       bellman_ford();
69       if( dist[sink] == inf ) break;
70
71       int f = inf;
72       for( int cur = sink; cur != source; cur = pai[cur] )
73         f = min( f, capacity[pai[cur]][cur] );
74
75       flow += f;
76       cost += f*dist[sink];
77
78       for( int cur = sink; cur != source; cur = pai[cur] ){
79         capacity[pai[cur]][cur] -= f;
80         capacity[cur][pai[cur]] += f;
81       }
82     }
83     return make_pair( flow, cost );
84   };
85
86   auto [flow, val] = min_cost_flow();
87
88   if( flow != tot_cols || flow != tot_rows ){ cout << -1 <<
         endl; return 0; }
89   cout << -val << endl;
90   vector<string> resp(n, string(n, '.'));
91   for( int i = 0; i < n; i++ ) for( int j = n; j < 2*n; j++ )
         if( capacity[i][j] == 0 ) resp[i][j - n] = 'X';
92   for( auto &s : resp ) cout << s << endl;
93 }
```

15.12 Increasing Array II

You are given an array of n integers. You want to modify the array so that it is increasing, i.e., every element is at least as large as the previous element. On each move, you can increase or decrease the value of any element by one. What is the minimum number of moves required?

Input
The first input line contains an integer n: the size of the array. Then, the second line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the minimum number of moves.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```
1  void solve(){
2      int n; cin >> n;
3      priority_queue<int> pq;
4      int tot = 0;
5      for(int i = 0; i < n; i++) {
6          int x; cin >> x;
7          // slope aumenta em 2
8          pq.push(x); pq.push(x);
9          tot += pq.top() - x;
10         pq.pop();
11     }
12     cout << tot << '\n';
13 }
```

15.13 K Subset Sums I

You are given an array of n integers. Consider the sums of all 2^n subsets of the given array (including the empty subset with sum equal to zero). Your task is to find the k smallest subset sums.

Input
The first line has two integers n and k: the size of the array and the number of subset sums k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset sums in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min(2^n, 2 \cdot 10^5)$ • $-10^9 \leq x_i \leq 10^9$

```cpp
void solve(){
    int n, k; cin >> n >> k;
    vector<ll> v(n);
    ll base = 0;

    for(auto &x : v) {
        cin >> x;
        if(x < 0) base += x;
        x = abs(x);
    }
    sort(all(v));

    cout << base << ' ␣'; k--;
    // (minimo, posicao minima)
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    pq.push({base + v[0], 0});

    while(!pq.empty() && k--) {
        auto [at, pos] = pq.top(); pq.pop();
        cout << at << ' ␣';
        if(pos == n - 1) continue;
        pq.push({at - v[pos] + v[pos + 1], pos + 1});
        pq.push({at + v[pos + 1], pos + 1});
    }
    cout << '\n';

}
```

15.14 K Subset Sums II

You are given an array of n integers. Consider the sums of all $\binom{n}{m}$ subsets of the given array with exactly m elements. Your task is to find the k smallest subset sums.

Input
The first line has three integers n, m and k: the size of the array, the size of the subsets and the number of subset sums k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset sums in increasing order.

Constraints
• $1 \leq m < n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min\left(\binom{n}{m}, 2 \cdot 10^5\right)$ • $-10^9 \leq x_i \leq 10^9$

```cpp
void solve(){
    int n, m, k; cin >> n >> m >> k;
    vector<ll> v(n);
    ll base = 0;

    for(auto &x : v) {
        cin >> x;
    }
    sort(all(v));

    for(int i = 0; i < m; i++) base += v[i];

    // (minimo possivel, ficha que ta mexendo, pos_atual,
       pos_final)
    priority_queue<tll, vector<tll>, greater<tll>> pq;

    pq.push({base, m - 1, m - 1, n - 1});
    vector<ll> ans;

    while(!pq.empty() && ans.size() < k) {
        auto [mini, at, pos, fim] = pq.top(); pq.pop();
        ans.push_back(mini);

        if(pos + 1 <= fim)
            pq.push({mini - v[pos] + v[pos + 1], at, pos + 1,
                fim});

        if(at >= 1 && at <= pos - 1)
            pq.push({mini - v[at - 1] + v[at], at - 1, at,
                pos - 1});
    }

    for(auto x : ans) cout << x << ' ␣';
    cout << '\n';

}
```

15.15 Knight Moves Queries

There is a knight on an infinite chessboard. The rows and columns are 1-indexed. Your task is to efficiently process queries of the form: when the knight starts at position (x, y), what is the minimum number of moves the knight needs to do to reach the top-left corner.

Input
The first line has an integer n: the number of queries. After this, there are n lines. Each line has two integers x and y: the knight position.

Output
For each query, print the minimum number of moves.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq x, y \leq 10^9$

```cpp
void solve(){
    int i, j; cin >> i >> j;
    i--; j--;

    int ans = 0;
    if(j <= i / 2)
    {
        ans = (i / 2);
        if(i == 1 && j == 0)
            ans = (3);

        else if((i % 4 == 1 || i % 4 == 2) && j % 2 == 0)
            ans = (i / 2 + 1);

        else if((i % 4 == 3 || i % 4 == 0) && j % 2 == 1)
            ans = (i / 2 + 1);

        else if(i % 4 == 3 && j % 2 == 0)
            ans = (i / 2 + 2);

        else if(i % 4 == 1 && j % 2 == 1)
            ans = (i / 2 + 2);
    }

    else if(i <= j / 2)
    {
        ans = (j / 2);
        if(j == 1 && i == 0)
            ans = (3);

        else if((j % 4 == 1 || j % 4 == 2) && i % 2 == 0)
            ans = (j / 2 + 1);

        else if((j % 4 == 3 || j % 4 == 0) && i % 2 == 1)
            ans = (j / 2 + 1);

        else if(j % 4 == 3 && i % 2 == 0)
            ans = (j / 2 + 2);

        else if(j % 4 == 1 && i % 2 == 1)
            ans = (j / 2 + 2);
    }

    else
    {
        ans = ((i + j) / 3);
        if(j == 1 && i == 1)
            ans = (4);

        else if(j == 2 && i == 2)
            ans = (4);

        else if((i + j) % 3 == 1)
            ans = ((i + j) / 3 + 1);

        else if((i + j) % 3 == 2)
            ans = ((i + j) / 3 + 2);
    }

    cout << ans << '\n';
}
```

15.16 Maximum Building II

You are given a map of a forest where some squares are empty and some squares have trees. You want to place a rectangular building in the forest so that no trees need to be cut down. For each building size, your task is to calculate the number of ways you can do this.

Input
The first input line contains integers n and m: the size of the forest. After this, the forest is described. Each square is empty (.) or has trees (*).

Output
Print n lines each containing m integers.

Constraints
- $1 \leq n, m \leq 1000$

```cpp
const int MAX = 1e3 + 5;

bool M[MAX][MAX];
bool mark[MAX][MAX];
int pref[MAX][MAX];
int v[MAX];

vector<int> next_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = 0; i <= n; i++) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

vector<int> prev_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = n; i >= 0; i--) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i <= n + 1; i++) {
        M[i][0] = 1;
        M[i][m + 1] = 1;
    }
    for(int j = 0; j <= m + 1; j++) {
        M[0][j] = 1;
        M[n + 1][j] = 1;
    }
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            char c; cin >> c;
            if(c == '*') M[i][j] = 1;
        }
    }

    int ans = 0;

    for(int i = 1; i <= n; i++) {
        v[0] = -1; v[m + 1] = -1;
        for(int j = 1; j <= m; j++) {
            if(M[i][j]) v[j] = 0;
            else v[j]++;
        }

        auto nxt = next_smaller(m + 1);
        auto prev = prev_smaller(m + 1);

        vector<pair<int, int>> unmark;

        for(int j = 1; j <= m; j++) {
            if(mark[nxt[j]][prev[j]]) continue;
            mark[nxt[j]][prev[j]] = 1; unmark.push_back({nxt[
                j], prev[j]});

            int h1 = max(1, max(v[nxt[j]], v[prev[j]]) + 1);
            int h = v[j], w = nxt[j] - prev[j] - 1;

            if (h1 <= h) {
                pref[h1][w]++;
                pref[h + 1][w]--;
            }
        }

        for(auto [x, y] : unmark) mark[x][y] = 0;
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            pref[i][j] += pref[i - 1][j];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = m - 1; j >= 1; j--) {
            pref[i][j] += pref[i][j + 1];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = m - 1; j >= 1; j--) {
            pref[i][j] += pref[i][j + 1];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            cout << pref[i][j] << ' ';
        }
        cout << '\n';
    }

}
```

15.17 Mex Grid Queries

Consider a two-dimensional grid whose rows and columns are 1-indexed. Each square contains the smallest nonnegative integer that does not appear to the left on the same row or above on the same column. Your task is to calculate the value at square (y, x).

Input
The only input line contains two integers y and x.

Output
Print one integer: the value at square (y, x).

Constraints
• $1 \le y, x \le 10^9$

```
void solve(){
    int x, y; cin >> x >> y;
    cout << ((x - 1) ^ (y - 1)) << '\n';
}
```

15.18 Minimum Cost Pairs

Given an array of n integers, consider pairings of k pairs. Each number can appear in at most one pair, and the cost of a pair (a, b) is $|a - b|$. The cost of a pairing is the sum of all costs of the pairs. Calculate the minimum costs of pairings for $k = 1, 2, \ldots, \lfloor n/2 \rfloor$.

Input
The first line has an integer: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print $\lfloor n/2 \rfloor$ integers: the minimum costs of pairings.

Constraints
• $2 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```
const int INF = 10000000000000000;

void solve(){
    int n; cin >> n;
    vector<int> x(n);

    // diferenca, L, R
    set<tuple<int, int, int>> d;
    // elementos com L = i, R = i
    vector<tuple<int, int, int>> L(n - 1), R(n - 1);

    for(int i = 0; i < n; i++) cin >> x[i];
    sort(all(x));

    for(int i = 0; i < n - 1; i++) {
        d.insert({x[i + 1] - x[i], i, i});
        L[i] = {x[i + 1] - x[i], i, i};
        R[i] = {x[i + 1] - x[i], i, i};
    }

    int tot = 0;

    for(int k = 1; k <= n / 2; k++) {
        auto top = *d.begin(); d.erase(d.begin());
        tot += get<0>(top);

        tuple<int, int, int> a, b;
        // elemento tq R = top.L - 1
        if(get<1>(top) == 0) a = {INF, 0, 2 * n}; // INF pq trocar as pontas so piora
        else a = R[get<1>(top) - 1];

        // elemento tq L = top.R + 1
        if(get<2>(top) == n - 2) b = {INF, 2 * n, n - 2};
        else b = L[get<2>(top) + 1];

        d.insert({get<0>(a) + get<0>(b) - get<0>(top), get<1>(a), get<2>(b)});
        L[get<1>(a)] = {get<0>(a) + get<0>(b) - get<0>(top), get<1>(a), get<2>(b)};
        R[get<2>(b)] = {get<0>(a) + get<0>(b) - get<0>(top), get<1>(a), get<2>(b)};
        d.erase(a); d.erase(b);

        cout << tot << ' ';
    }

    cout << '\n';
}
```

15.19 Programmers And Artists

A company wants to hire a programmers and b artists. There are a total of n applicants, and each applicant can become either a programmer or an artist. You know each applicant's programming and artistic skills. Your task is to select the new employees so that the sum of their skills is maximum.

Input
The first input line has three integers a, b and n: the required number of programmers and artists, and the total number of applicants. After this, there are n lines that describe the applicants. Each line has two integers x and y: the applicant's programming and artistic skills.

Output
Print one integer: the maximum sum of skills.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le a, b \le n$ • $a + b \le n$ • $1 \le x, y \le 10^9$

```
const ll LINF = 4e18;

vector<pair<int, int>> v;
set<pair<int, int>> A, B, difA, difB;

int rem(char x, int pos){
    int s = 0;
    if(x == 'A') {
        A.erase({v[pos].first, pos});
        difA.erase({v[pos].first - v[pos].second, pos});
        s -= v[pos].first;
    }
    if(x == 'B') {
        B.erase({v[pos].second, pos});
        difB.erase({v[pos].second - v[pos].first, pos});
        s -= v[pos].second;
    }
    return s;
}

int add(char x, int pos){
    int s = 0;
    if(x == 'A') {
        A.insert({v[pos].first, pos});
        difA.insert({v[pos].first - v[pos].second, pos});
        s += v[pos].first;
    }
    if(x == 'B') {
        B.insert({v[pos].second, pos});
        difB.insert({v[pos].second - v[pos].first, pos});
        s += v[pos].second;
    }
    return s;
}

void solve(){
    int n, a, b; cin >> a >> b >> n;
    v.resize(n);
    ll s = 0;

    for(int i = 0; i < n; i++) cin >> v[i].first >> v[i].second;

    for(int i = 0; i < a + b; i++) {
        s += add('A', i);
    }

    for(int i = 0; i < b; i++) {
        auto it = difA.begin();
        int pos = it -> second;
        s += rem('A', pos);
        s += add('B', pos);
    }

    for(int i = a + b; i < n; i++) {
        ll maxs = 0, s1 = -LINF, s2 = -LINF, s3 = -LINF, s4 = -LINF;
        int posA_rem = -1, posB_rem = -1, posA_swap = -1, posB_swap = -1;

        // add i as a and remove other from a
        if (!A.empty()) {
            posA_rem = A.begin()->second;
            s1 = (ll)v[i].first - v[posA_rem].first;
        }

        // add i as b and remove other from b
        if (!B.empty()) {
            posB_rem = B.begin()->second;
            s2 = (ll)v[i].second - v[posB_rem].second;
        }

        // add i as a , swap some a to b and remove other from b
        if (!A.empty() && !B.empty()) {
            posA_swap = difA.begin()->second;
            posB_rem = B.begin()->second;
            s3 = (ll)v[i].first - v[posA_swap].first + v[posA_swap].second - v[posB_rem].second;
        }

        // add i as b , swap some b to a and remove other from a
        if (!A.empty() && !B.empty()) {
```

```
            posB_swap = difB.begin()->second;
            posA_rem = A.begin()->second;
            s4 = (ll)v[i].second - v[posB_swap].second + v[
                 posB_swap].first - v[posA_rem].first;
        }

        maxs = max({0LL, s1, s2, s3, s4});
        s += maxs;

        if(maxs == 0) continue;
        else if(maxs == s1) {
            rem('A', posA_rem);
            add('A', i);
        }else if(maxs == s2) {
            rem('B', posB_rem);
            add('B', i);
        }else if(maxs == s3) {
            rem('A', posA_swap);
            rem('B', posB_rem);
            add('A', i);
            add('B', posA_swap);
        }else if(maxs == s4) {
            rem('B', posB_swap);
            rem('A', posA_rem);
            add('B', i);
            add('A', posB_swap);
        }
    }

    cout << s << '\n';
}
```

15.20 Removing Digits II

You are given an integer n. On each step, you may subtract from it any one-digit number that appears in it. How many steps are required to make the number equal to 0?

Input
The only input line has an integer n.

Output
Print one integer: the minimum number of steps.

Constraints
- $1 \le n \le 10^{18}$

```
const int logn = 20;

ll dp[logn][10][10];
int val[logn][10][10];

void init(){
  for( int m = 0; m < 10; m++ ) for( int x = 0; x < 10; x++ )
       {
    dp[1][m][x] = 1;
    val[1][m][x] = (10 + x - max(m, x))%10;
    if( !val[1][m][x] && m > 0 ){
      dp[1][m][x]++;
      val[1][m][x] = (10 - m)%10;
    }
  }
  for( int k = 2; k < logn; k++ ){
    for( int m = 0; m < 10; m++ ){
      dp[k][m][0] = dp[k - 1][m][0];
      val[k][m][0] = val[k - 1][m][0];
      for( int x = 1; x < 10; x++ ){
        val[k][m][x] = x;
        for( int i = 9; i >= 0; i-- ){
          dp[k][m][x] += dp[k - 1][max(m, i)][val[k][m][x]];
          val[k][m][x] = val[k - 1][max(m, i)][val[k][m][x]];
        }
      }
    }
  }

  // 10^k - 10 + x
}

int main(){
  init();
  ll x; cin >> x;
  ll resp = 0;
  if( x == 1e18 ){ resp++; x--; }
  int unit = x%10;
  for( ll i = 2, pot = 10; resp == 0 || unit != 0; i++, pot
       *= 10 ){

    for( int j = (x%(pot*10) - x%pot)/pot; j >= 0; j--, x -=
         pot ){

      ll maxi = 0;
      for( ll p = pot; p <= 1e17; p *= 10 ) maxi = max( maxi,
           (x%(p*10) - x%p)/p);

      resp += dp[i - 1][maxi][unit];
      unit = val[i - 1][maxi][unit];
    }
  }
  if( unit ) resp++;
  cout << resp << endl;
}
```

15.21 Replace With Difference

You are given an array of n integers. You will perform $n-1$ operations on the array. In one operation, you will choose two numbers a and b from the array, delete both of them from the array and add $|a-b|$ into the array. Your task is to find a sequence of operations such that the last number remaining in the array is 0.

Input
The first line has an integer n: the length of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n-1$ lines each containing two integers a and b: the numbers chosen in the operations. You can print any valid solution. If no solution exists, print only -1.

Constraints
- $2 \le n \le 1000$
- $1 \le x_i \le 1000$

```
using vi = vector<int>;
using ll = long long;

const int maxn = 1e3 + 10;
const int sqrtn = 40;
const int maxx = sqrtn*maxn;

bool dp[maxn][2*maxx];

void solve(){
    mt19937 rng(chrono::steady_clock::now().time_since_epoch
        ().count());
    int n; cin >> n;
    vi v(n);
    for( int &x : v ) cin >> x;
    shuffle(all(v), rng);

    dp[0][maxx] = true;
    FOR(i, 1, n + 1){
        FOR(j, 0, 2*maxx){
            if( j - v[i - 1] >= 0 ) dp[i][j] |= dp[i - 1][j -
                v[i - 1]];
            if( j + v[i - 1] < 2*maxx ) dp[i][j] |= dp[i -
                1][j + v[i - 1]];
        }
    }

    if( !dp[n][maxx] ){ cout << -1 << '\n'; return; }

    vi a, b;

    for( int i = n, j = maxx; i >= 1; i-- ){
        if( j - v[i - 1] >= 0 && dp[i - 1][j - v[i - 1]] ){
            a.push_back(v[i - 1]);
            j -= v[i - 1];
        }
        else{
            b.push_back(v[i - 1]);
            j += v[i - 1];
        }
    }

    while( !a.empty() && !b.empty() ){
        cout << a.back() << " " << b.back() << '\n' ;
        int x = min(a.back(), b.back());
        a.back() -= x; b.back() -= x;
        if( !a.back() ) a.pop_back();
        else b.pop_back();
    }
}
```

15.22 Reversal Sorting

You have an array that contains a permutation of integers $1, 2, \ldots, n$. Your task is to sort the array in increasing order by reversing subarrays. You can construct any solution that has at most n reversals.

Input
The first input line has an integer n: the size of the array. The array elements are numbered $1, 2, \ldots, n$. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
First print an integer k: the number of reversals. After that, print k lines that describe the reversals. Each line has two integers a and b: you reverse a subarray from position a to position b.

Constraints

- $1 \le n \le 2 \cdot 10^5$

```cpp
using vi = vector<int>;
using ll = long long;
using pii = pair<int, int>;

const int inf = 2e9;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().
      count());

struct Node{
    Node *l = 0, *r = 0;
    int val, mini, y, c = 1, flip = 0;
    Node( int val = 0 ) : val(val), mini(val), y(rng()) {}
};

struct Treap{
    int cnt( Node *n ){ return n ? n->c : 0; }
    int val( Node *n ){ return n ? n->val : INT_MAX; }
    int mini( Node *n ){ return n ? n->mini : INT_MAX; }
    void recalc( Node *n ){
        if( n->flip ){
            swap( n->l, n->r );
            if(n->l) n->l->flip ^= 1;
            if(n->r) n->r->flip ^= 1;
            n->flip = 0;
        }
        n->c = cnt(n->l) + cnt(n->r) + 1;
        n->mini = min({ val(n), mini(n->l), mini(n->r) });
    }

    Node *rt = 0;

    template<class F>
    void each( Node *n, F f ){
        if(n){ each(n->l, f); f(n->val); each(n->r, f); }
    }

    pair<Node*, Node*> split( Node* n, int k ){
        if(!n) return {};
        recalc(n);
        if(cnt(n->l) >= k){
            auto [L, R] = split(n->l, k);
            n->l = R;
            recalc(n);
            return {L, n};
        }
        auto [L, R] = split(n->r, k - cnt(n->l) - 1);
        n->r = L;
        recalc(n);
        return {n, R};
    }

    Node *merge( Node *l, Node *r ){
        if( !l ) return r;
        if( !r ) return l;

        recalc(l);
        recalc(r);

        if( l->y > r->y ){
            l->r = merge(l->r, r);
            return recalc(l), l;
        }
        r->l = merge(l, r->l);
        return recalc(r), r;
    }

    Node* ins( Node *t, Node *n, int pos ){
        auto [l, r] = split(t, pos);
        return merge(merge(l, n), r);
    }

    int query( Node *n ){
        if(!n) return 0;
        recalc(n);

        if( mini(n->l) < val(n) && mini(n->l) < mini(n->r) )
            return query(n->l);
        if( val(n) < mini(n->r) ) return cnt(n->l) + 1;
        return cnt(n->l) + 1 + query(n->r);
    }

    void update( Node *n ){
        if(n){ n->flip ^= 1; recalc(n); }
    }

    void move( Node *& t, int l, int r, int k ){
        Node *a, *b, *c;
        tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
        if( k <= l ) t = merge(ins(a, b, k), c);
        else t = merge(a, ins(c, b, k - r));
    }
};

void solve(){
    int n; cin >> n;
    Treap t;

    FOR(i, 0, n){
        int x; cin >> x;
        Node *n = new Node(x);
        t.rt = t.merge(t.rt, n);
    }

    cout << n << '\n';
    FOR(i, 0, n){
        int pos = t.query(t.rt);
        cout << i + 1 << " " << i + pos << '\n';

        auto [a, b] = t.split(t.rt, pos);
        t.update(a);
        t.rt = t.merge(a, b);
        t.rt = t.split(t.rt, 1).second;
    }
}
```

15.23 Same Sum Subsets

Given a set of n positive integers, your task is to choose two disjoint subsets of the elements that have the same sum.

Input
The first line has an integer n: the set size. The second line has n integers $x_1, x_2, \ldots, x_n$: the set elements.

Output
For both subsets, first print the size of the subset and then its contents. You can print any valid solution. If there is no solution, print IMPOSSIBLE.

Constraints
• $3 \le n \le 40$ • $\sum_{i=1}^{n} x_i \le 2^n - 2$

```cpp
mt19937_64 rng((ll) chrono::steady_clock::now().
      time_since_epoch().count());

const int t = 1000000;
ll v[40], n, full_mask;
pair<ll, ll> aux[t];

void solve() {
    cin >> n;
    for(int i = 0; i < n; i++) cin >> v[i];
    sort(v, v + n);

    vector<tuple<ll, ll, ll>> prec;
    for(int i = 0; i < n; i++) {
        for(int j = i; j < n; j++) {
            ll s = 0;
            for(int k = i; k <= j; k++) s += v[k];
            prec.push_back({s, i, j});
        }
    }
    sort(prec.begin(), prec.end());
    for(int i = 0; i < sz(prec) - 1; i++) {
        if(get<0>(prec[i]) == get<0>(prec[i + 1])) {
            cout << get<2>(prec[i]) - get<1>(prec[i]) + 1 <<
                 '\n';
            for(int j = get<1>(prec[i]); j <= get<2>(prec[i])
                 ; j++) cout << v[j] << ' ';
            cout << '\n';

            cout << get<2>(prec[i + 1]) - get<1>(prec[i + 1])
                  + 1 << '\n';
            for(int j = get<1>(prec[i + 1]); j <= get<2>(prec
                 [i + 1]); j++) cout << v[j] << ' ';
            cout << '\n';
            return;
        }
    }

    full_mask = (1LL << n) - 1LL;
    uniform_int_distribution<ll> uid(1LL, full_mask);

    while(true) {
        for(int i = 0; i < t; i++) {
            ll mask = (ll)(uid(rng));
            aux[i] = {0, mask};

            while(mask) {
                aux[i].first += v[ctz(mask)];
                mask &= mask - 1;
            }
        }

        sort(aux, aux + t);

        for(int i = 0; i < t - 1; i++) {
            if(aux[i].first == aux[i + 1].first && aux[i].
                 second != aux[i + 1].second) {
                ll a = aux[i].second;
                ll b = aux[i + 1].second;

                ll nd = a & b;
                a ^= nd; b ^= nd;

                cout << pc(a) << '\n';
                for(int i = 0; i < n; i++) if(a & (1LL << i))
                       cout << v[i] << ' ';
                cout << '\n';

                cout << pc(b) << '\n';
                for(int i = 0; i < n; i++) if(b & (1LL << i))
                       cout << v[i] << ' ';
                cout << '\n';

                return;
            }
        }
    }
}
```

15.24 School Excursion

A group of n children are coming to Helsinki. There are two possible attractions: a child can visit either Korkeasaari (zoo) or Linnanmäki (amusement park). There are m pairs of children who want to visit the same attraction. Your task is to find all possible alternatives for the number of children that will visit Korkeasaari. The children's wishes have to be taken into account.

Input
The first input line has two integers n and m: the number of children and their wishes. The children are numbered $1, 2, \ldots, n$. After this, there are m lines describing the children's wishes. Each line has two integers a and b: children a and b want to visit the same attraction.

Output
Print a bit string of length n where a one-bit at index i indicates that it is possible that exactly i children visit Korkeasaari (the bit string is to be considered one-indexed).

Constraints
• $1 \le n \le 10^5$ • $0 \le m \le 10^5$ • $1 \le a, b \le n$

```cpp
const int MAX = 1e5 + 5;
vi g[MAX];
int vis[MAX];

int dfs(int v) {
    vis[v] = 1;
    int at = 1;

    for(auto x : g[v]) {
        if(vis[x]) continue;
        at += dfs(x);
    }

    return at;
}

void solve() {
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].pb(b);
        g[b].pb(a);
    }

    vector<int> sizes;
    for(int i = 0; i < n; i++) {
        if(!vis[i]) sizes.push_back(dfs(i));
    }
    sort(all(sizes));

    bitset<MAX> dp = 1;

    for(auto x : sizes) {
        dp |= (dp << x);
    }

    for(int i = n - 1; i >= 0; i--) cout << dp[i];
    cout << '\n';
}
```

15.25 Stick Divisions

You have a stick of length x and you want to divide it into n sticks, with given lengths, whose total length is x. On each move you can take any stick and divide it into two sticks. The cost of such an operation is the length of the original stick. What is the minimum cost needed to create the sticks?

Input
The first input line has two integers x and n: the length of the stick and the number of sticks in the division. The second line has n integers $d_1, d_2, \ldots, d_n$: the length of each stick in the division.

Output
Print one integer: the minimum cost of the division.

Constraints
• $1 \le x \le 10^9$ • $1 \le n \le 2 \cdot 10^5$ • $\sum d_i = x$

```cpp
int main(){
  ll x, n; cin >> x >> n;
  multiset<ll> s;
  while( n-- ){ int v; cin >> v; s.insert(v); }
  ll resp = 0;
  while( s.size() > 1 ){
    ll a = *s.begin(); s.erase(s.begin());
    ll b = *s.begin(); s.erase(s.begin());
    resp += a + b;
    s.insert(a + b);
  }
  cout << resp << endl;
}
```

15.26 Swap Round Sorting

You are given an array containing a permutation of numbers $1, 2, \ldots, n$, and your task is to sort the array using swap rounds. On each swap round, you can choose any number of distinct pairs of elements and swap each pair. Your task is to find the minimum number of rounds and show how you can choose the pairs in each round.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the initial permutation.

Output
First print an integer k: the minimum number of rounds. Then, for each round, print the number of swaps and the indices of each swap. You can print any valid solution.

Constraints
- $1 \le n \le 2 \cdot 10^5$

```cpp
const int maxn = 2e5 + 10;
bool marc[maxn];
int p[maxn]; int n;
deque<int> deq;

int count(){
    int cont = 0;
    for( int i = 1; i <= n; i++ ){
        if( p[i] == i ) cont = max( cont, 0 );
        else if( p[p[i]] == i ) cont = max( cont, 1 );
        else cont = 2;
    }
    return cont;
}

void marca( int cur ){
    deq.clear();
    while( true ){ marc[cur] = true; deq.push_back(cur); cur = p[cur]; if( marc[cur] ) return; }
}

void quebra(){
    vector< pair< int, int > > v;
    for( int i = 1; i <= n; i++ ){
        if( p[i] == i || p[p[i]] == i ) continue;
        if( marc[i] ) continue; marca(i);
        deq.pop_back();
        while( deq.size() > 1 ){
            int ini = deq.front(), fim = deq.back();
            v.push_back({ ini, fim });
            swap( p[ini], p[fim] );
            deq.pop_front(); deq.pop_back();
        }
    }
    cout << v.size() << endl;
    for( auto x : v ) cout << x.first << " " << x.second << endl;
}

void troca(){
    vector< pair< int, int > > v;
    for( int i = 1; i <= n; i++ ) if( p[p[i]] == i && p[i] > i ) v.push_back({ i, p[i] });
    cout << v.size() << endl;
    for( auto x : v ) cout << x.first << " " << x.second << endl;
}

int main(){
    cin >> n;
    for( int i = 1; i <= n; i++ ) cin >> p[i];
    int qtd = count();
    cout << qtd << endl;
    if( qtd == 2 ) quebra();
    if( qtd >= 1 ) troca();
}
```

15.27 Two Stacks Sorting

You are given an input list that consists of n numbers. Each integer between 1 and n appears exactly once in the list. Your task is to create a sorted output list using two stacks. On each move you can do one of the following:
• Move the first number from the input list to a stack • Move a number from a stack to the end of the output list

Input
The first input line has an integer n. The second line has n integers: the contents of the input list.

Output
Print n integers: for each number the stack where it is moved (1 or 2). You can print any valid solution. If there are no solutions, print IMPOSSIBLE.

Constraints
- $1 \le n \le 2 \cdot 10^5$

```cpp
using vi = vector<int>;
using pii = pair<int, int>;
using ll = long long;

const int maxn = 2e5;
const int inf = 2e9;

struct Segtree{
    pii *seg;
    int n;
    Segtree( int n ) : n(n), seg(new pii[4*n]){
        fill(seg, seg + 4*n, pii(inf, -1));
    }

    void update( int node, int ti, int tf, int id, pii p ){
        if( ti == tf ){ seg[node] = p; return; }
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        if( id <= tm )  update( l, ti, tm, id, p );
        else            update( r, tm + 1, tf, id, p );
        seg[node] = min( seg[l], seg[r] );
    }

    void update( int id, pii p ){
        update( 1, 0, n - 1, id, p );
    }

    pii query( int node, int ti, int tf, int qi, int qf ){
        if( qi > tf || ti > qf ) return pii( inf, -1 );
        if( qi <= ti && tf <= qf ) return seg[node];
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        return min(query( l, ti, tm, qi, qf ), query( r, tm + 1, tf, qi, qf ));
    }

    pii query( int l, int r ){
        return query( 1, 0, n - 1, l, r );
    }
};

bool check( vector<pii> &v ){
    reverse(all(v));
    stack<pii> s;

    for( auto [L, R] : v ){
        while( !s.empty() && s.top().second < R ) s.pop();
        if( !s.empty() && s.top().first < R ) return false;
        s.push({ L, R });
    }
    return true;
}

void solve(){
    int n; cin >> n;
    vi v(n), in(n), out(n), marc(n, -1);
    int cur = 0, t = 0;
    FOR(i, 0, n){
        cin >> v[i];
        marc[--v[i]] = i;

        in[i] = t++;
        while( cur < n && marc[cur] != -1 ) out[marc[cur++]] = t++;
    }

    Segtree ins(t), outs(t);
    FOR(i, 0, n){
        ins.update( out[i], pii(in[i], i) );
        outs.update( in[i], pii(-out[i], i) );
    }

    fill(all(marc), -1);
    vector<pii> choices[2];
    vi ans(n);
    FOR(i, 0, n){
        if( marc[i] == -1 ){
            queue<int> q;
            marc[i] = 0;
            for(q.push(i); !q.empty(); q.pop()){
                int j = q.front();

                // Olhar seg dos ins
                for( auto [IN, k] = ins.query(in[j] + 1, out[j] - 1);
                    IN < in[j];
                    tie(IN, k) = ins.query(in[j] + 1, out[j] - 1) ){

                    q.push(k);
                    marc[k] = !marc[j];

                    ins.update( out[k], pii(inf, -1) );
                    outs.update( in[k], pii(inf, -1) );
                }

                // Olhar seg dos outs
                for( auto [OUT, k] = outs.query(in[j] + 1, out[j] - 1);
                    out[j] < -OUT;
                    tie(OUT, k) = outs.query(in[j] + 1, out[j] - 1) ){

                    q.push(k);
```

```
                    marc[k] = !marc[j];

                    ins.update( out[k], pii(inf, -1) );
                    outs.update( in[k], pii(inf, -1) );
                }
            }
        }
        choices[marc[i]].push_back({ in[i], out[i] });
        ans[i] = marc[i] + 1;
    }

    if( !check( choices[0] ) || !check( choices[1] ) ){ cout
         << "IMPOSSIBLE\n"; return; }
    for( int x : ans ) cout << x << "␣"; cout << '\n';
}
```

16 Bitwise Operations

16.1 All Subarray Xors

Given an array of n integers, your task is to find all integers that are the xor sum in some subarray.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
First print an integer k: the number of distinct integers that are the xor sum in some subarray. After this print k integers: the xor sums in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^6$

```
const int maxn = 1048576;
const int mod = 998244353;

int inverse(int x, int mod) {
    return x == 1 ? 1 : mod - mod / x * inverse(mod % x, mod)
          % mod;
}

void xormul(vector<int> a, vector<int> b, vector<int> &c) {
    int m = (int) a.size();
    c.resize(m);
    for (int n = m / 2; n > 0; n /= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = a[i + j], y = a[i + j + n];
                a[i + j] = (x + y) % mod;
                a[i + j + n] = (x - y + mod) % mod;
            }
    for (int n = m / 2; n > 0; n /= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = b[i + j], y = b[i + j + n];
                b[i + j] = (x + y) % mod;
                b[i + j + n] = (x - y + mod) % mod;
            }
    for (int i = 0; i < m; i++)
        c[i] = a[i] * b[i] % mod;
    for (int n = 1; n < m; n *= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = c[i + j], y = c[i + j + n];
                c[i + j] = (x + y) % mod;
                c[i + j + n] = (x - y + mod) % mod;
            }
    int mrev = inverse(m, mod);
    for (int i = 0; i < m; i++)
        c[i] = c[i] * mrev % mod;
}

void solve(){
    int n; cin >> n;

    vector<int> A(maxn, 0);
    vector<int> pref(n + 1, 0);
    A[0]++;
    for(int i = 1; i <= n; i++)
    {
        int a; cin >> a;
        pref[i] = a ^ pref[i - 1];
        A[pref[i]]++;
    }

    vector<int> B = A, C;
    xormul(A, B, C);

    vector<int> ans;

    if(C[0] > n + 1) ans.push_back(0);
    for(int i = 1; i < maxn; i++)
    {
        if(C[i] > 0) ans.push_back(i);
    }

    cout << ans.size() << '\n';
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}
```

16.2 And Subset Count

You are given an array of n integers. Your task is to calculate the number of non-empty subsets whose elements' bitwise and is equal to k for each $k = 0, 1, \ldots, n$.

Input
The first line has an integer n: the size of the array. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print $n + 1$ integers as specified above modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq a_i \leq n$

```
const int LMAX = 18, MOD = 1e9 + 7;
vector<int> sub(1 << LMAX);

void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step)
            {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
            inv ? pii((v - u + MOD) % MOD, u) : pii(v, (u + v
                ) % MOD); // AND
        }
    }
}

ll pow(ll x, ll y, ll m) {
    ll ret = 1;
    while (y) {
        if (y & 1) ret = (ret * x) % m;
        y >>= 1;
        x = (x * x) % m;
    }
    return ret;
}

/*
    a ideia pro FST eh pensar no polinomio prod(1 + x^a_i),
        mas com AND
    fica muito lento fazer os FST's, entao tem que ajeitar
        isso
    testando, da pra perceber que os FST's ficam iguais aos
        andares do triangulo de pascal em mod 2 (quem eh 0
        fica 1, quem eh 1 fica 2)
    ai eh so calcular o produto com SOS dp e destransformar o
        FST
*/

void solve(){
    int n; cin >> n;
    vi a(n), sub(1 << LMAX);

    for(int i = 0; i < n; i++) {
        cin >> a[i]; sub[a[i]]++;
    }

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask <
        (1 << LMAX); mask++) {
        if (~mask>>i&1) sub[mask] += sub[mask^(1<<i)];
    }

    vector<int> ans(1 << LMAX);
    for(int i = 0; i < sz(ans); i++) {
        ans[i] = pow(2, sub[i], MOD);
    }
    reverse(all(ans));
    FST(ans, 1);

    for(int i = 0; i <= n; i++) cout << ans[i] << ' ';
    cout << '\n';
}
```

16.3 Counting Bits

Your task is to count the number of one bits in the binary representations of integers between 1 and n.

Input
The only input line has an integer n.

Output
Print the number of one bits in the binary representations of integers between 1 and n.

Constraints
• $1 \leq n \leq 10^{15}$

```
const ll INF = 1e15, logINF = 50;

vector<ll> a(50);

void solve(){
    ll n; cin >> n;
    ll tot = 0;

    a[0] = 1;
    for(int i = 1; i < logINF; i++)
        a[i] = a[i - 1] * 2 + (1LL << i);

    ll aux = 0;

    for(int bit = 0; bit <= logINF; bit++) {
        if((n & (1LL << bit)) == 0) continue;

```

```
        if(bit == 0) {
            aux += (1LL << bit);
            tot = 1;
            continue;
        }

        tot += (aux + 1) + a[bit - 1];
        aux += (1LL << bit);
    }

    cout << tot << '\n';
}
```

16.4 K Subset Xors

You are given an array of n integers. Consider the xors of all 2^n subsets of the array (including the empty subset with xor equal to zero). Your task is to find the k smallest subset xors.

Input
The first line has two integers n and k: the size of the array and the number of subset xors k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset xors in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min(2^n, 2 \cdot 10^5)$ • $0 \leq x_i \leq 10^9$

```
const int INF = 1e9 + 10, logINF = 30, logmax = 18;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual ( basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n, k; cin >> n >> k;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    int tot = 0;

    vector<int> ans = {0};

    for (int i = 0; i < logINF; i++) {
        if (Gauss.basis[i].any()) {
            tot++;

            if(tot <= logmax)
            {
                int B = Gauss.basis[i].to_ullong();
                int sz = ans.size();
                for(int i = 0; i < sz; i++)
                    ans.push_back(ans[i] ^ B);
            }
        }
    }

    sort(ans.begin(), ans.end());
    int nul = n - tot;
    if(nul > 25) nul = 25;
    int mul = 1 << nul;
    for(auto x : ans)
    {
        for(int i = 0; i < mul; i++)
        {
            cout << x << ' ';
            k--;
            if(k == 0) {
                cout << '\n';
                return;
            }
        }
    }
}
```

16.5 Maximum Xor Subarray

Given an array of n integers, your task is to find the maximum xor sum of a subarray.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum xor sum in a subarray.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

```cpp
int find_ans(vector<int>&v0, vector<int>&v1, int mxbit) {
    if(v0.size() == 0 || v1.size() == 0) return 0;
    if(mxbit == 0) {
        return (v0.back() ^ v1.back());
    }

    int mxbit_new = 0;
    for(int i = 0; i < mxbit; i++) {
        for(auto x : v0) if(x & (1 << i)) mxbit_new = i;
        for(auto x : v1) if(x & (1 << i)) mxbit_new = i;
    }

    vector<int> v00, v01, v10, v11;

    for(auto x : v0) {
        if(x & (1 << mxbit_new)) v01.push_back(x);
        else v00.push_back(x);
    }
    for(auto x : v1) {
        if(x & (1 << mxbit_new)) v11.push_back(x);
        else v10.push_back(x);
    }
    if((v00.size() == 0 || v11.size() == 0) && (v01.size() == 0 || v10.size() == 0)) {
        return find_ans(v0, v1, mxbit_new);
    }

    return max(find_ans(v00, v11, mxbit_new), find_ans(v01, v10, mxbit_new));
}

void solve(){
    int n; cin >> n;
    vector<int> v(n + 1);
    for(int i = 1; i <= n; i++) {
        cin >> v[i];
        v[i] ^= v[i - 1];
    }

    sort(v.begin(), v.end());

    int lst = v.back(), mxbit = 0;
    for(int i = 0; i <= 30; i++) {
        if(lst & (1 << i)) mxbit = i;
    }

    vector<int> v0, v1;

    for(auto x : v) {
        if(x & (1 << mxbit)) v1.push_back(x);
        else v0.push_back(x);
    }

    int ans = find_ans(v0, v1, mxbit);

    cout << ans << '\n';
}
```

16.6 Maximum Xor Subset

Given an array of n integers, your task is to find the maximum xor sum of a subset.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum xor sum of a subset.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

```cpp
const int INF = 1e9 + 10, logINF = 30;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual ( basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n; cin >> n;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    bitset<logINF> res;
    for (int i = logINF - 1; i >= 0; i--) {
        if (!res[i] && Gauss.basis[i][i]) {
            res ^= Gauss.basis[i];
        }
    }

    int ans = (int)(res.to_ulong());
    cout << ans << '\n';
}
```

16.7 Numberof Subset Xors

Given an array of n integers, your task is to find the number of different subset xors.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of different subset xors.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

```cpp
const int INF = 1e9 + 10, logINF = 30;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual ( basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
```

```
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^=
             keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.
             push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n; cin >> n;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    int ans = 1;
    for (int i = logINF - 1; i >= 0; i--) {
        if (Gauss.basis[i].any()) {
            ans *= 2;
        }
    }

    cout << ans << '\n';
}
```

16.8 SOS Bit Problem

Given a list of n integers, your task is to calculate for each element x:
the number of elements y such that $x \mid y = x$ the number of elements y such that $x \,\&\, y = x$ the number of elements y such that $x \,\&\, y \neq 0$

Input
The first line has an integer n: the size of the list. The next line has n integers $x_1, x_2, \ldots, x_n$: the elements of the list.

Output
Print n lines: for each element the required values.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^6$

```
const int LMAX = 20;
vector<int> sub(1 << LMAX);
vector<int> sup(1 << LMAX);

void solve(){
    int n; cin >> n; vector<int> v(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
        sub[v[i]]++; sup[v[i]]++;
    }

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask <
         (1 << LMAX); mask++) {
        if (mask>>i&1)  sub[mask] += sub[mask^(1<<i)];
        if (~mask>>i&1) sup[mask] += sup[mask^(1<<i)];
    }

    int msk = (1 << LMAX) - 1;

    for(int i = 0; i < n; i++) {
        cout << sub[v[i]] << ' ' << sup[v[i]] << ' ' << n -
             sub[msk - v[i]] << '\n';
    }
}
```

16.9 Xor Pyramid Diagonal

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid:
Given the bottom row of the pyramid, your task is to find the leftmost number of each row.

Input
The first line has an integer n: the size of the pyramid. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print n integers: the leftmost numbers of the rows from bottom to top.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq 10^9$

```
const int LMAX = 18;
vector<int> v(1 << LMAX);

void solve(){
    int n; cin >> n;
    for(int i = 0; i < n; i++) cin >> v[i];

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask <
         (1 << LMAX); mask++)
        if (mask>>i&1) v[mask] ^= v[mask^(1<<i)];

    for(int i = 0; i < n; i++) cout << v[i] << ' '; cout << '
        \n';
}
```

16.10 Xor Pyramid Peak

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid:
Given the bottom row of the pyramid, your task is to find the topmost number.

Input
The first line has an integer n: the size of the pyramid. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print one integer: the topmost number.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq 10^9$

```
int bin_v2(int n, int k) {
    return pc(k) + pc(n - k) - pc(n);
}

void solve(){
    int n; cin >> n;
    int ans = 0;
    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        if(bin_v2(n - 1, i) == 0) ans ^= a;
    }

    cout << ans << '\n';
}
```

16.11 Xor Pyramid Row

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid: Given the bottom row of the pyramid, your task is to find the numbers on the k-th row from the top.

Input
The first line has two integers n and k: the size of the pyramid and the given row. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print k integers: the numbers on the k-th row from the top.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq 10^9$

```cpp
int bin_v2(int n, int k) {
    return pc(k) + pc(n - k) - pc(n);
}

void solve(){
    int n, k; cin >> n >> k;
    int shift = n - k;
    vector<int> v(n);
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = 1 << 30; i > 0; i >>= 1) {
        if((i & shift) == 0) continue;

        vector<int> aux(n - i);
        for(int j = 0; j < n - i; j++) {
            aux[j] = v[j] ^ v[j + i];
        }

        v = aux;
        n = aux.size();
    }

    for(auto x : v) cout << x << ' '; cout << '\n';
}
```

17 Interactive Problems

17.1 Colored Chairs

There are n chairs arranged in a circle. Each chair is either red or blue. The chairs are numbered $1, 2, \ldots, n$; chairs i and $i+1$ are next to each other for all $1 \leq i \leq n$. Here chair $n+1$ refers to chair 1. Your task is to find two chairs that have the same color and are next to each other. To do this, you can ask questions: you can choose a chair and you will be told the color of that chair.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You should start by reading a single integer n: the number of chairs. On your turn, you can print one of the following:
• "? i", where $1 \leq i \leq n$: ask the color of chair i. The grader will return R or B for red or blue. • "! i": report that chairs i and $i+1$ have the same color. Your program must terminate after this.
Each line should be followed by a line break. You must make sure the output gets flushed after printing each line.

Constraints
• $3 \leq n \leq 2 \cdot 10^5$, n is odd • you can ask at most 20 questions of type ?

```
char c;

// se a gnt tem uma reta: R ? ? ... ? R
// e a qntt de numeros ? é par, sempre há 2 vizinhos de mesma cor
// já se: R ? ? ... ? B
// e a qntt de numeros ? é ímpar, sempre há 2 vizinhos de mesma cor

void solveline(int bgn, char cb, int end, char ce) {
    while(true) {
        if(end - bgn <= 1) {
            cout << "! " << bgn << endl;
            return;
        }

        int k = (bgn + end) / 2;

        cout << "? " << k << endl;
        cin >> c;

        if(c == cb) {
            if((k - bgn) % 2 == 1) {
                end = k;
                ce = c;
            } else {
                bgn = k;
                cb = c;
            }
        }
        else {
            if((k - bgn) % 2 == 0) {
                end = k;
                ce = c;
            } else {
                bgn = k;
                cb = c;
            }
        }
    }
}

void solvecirc() {
    int n; cin >> n;
    cout << "? " << 1 << endl;
    cin >> c;

    solveline(1, c, n + 1, c);
}

signed main() { _
    solvecirc();
}
```

17.2 Hidden Integer

There is a hidden integer x. Your task is to find the value of x. To do this, you can ask questions: you can choose an integer y and you will be told if $y<x$.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You can start asking questions right away. On your turn, you can print one of the following:
• "? y", where $1 \leq y \leq 10^9$: ask if $y<x$. The grader will return YES if $y<x$ and NO otherwise. • "! x": report that the hidden integer is x. Your program must terminate after this.
Each line should be followed by a line break. You must make sure the output gets flushed after printing each line.

Constraints
• $1 \leq x \leq 10^9$ • you can ask at most 30 questions of type ?

```
const int MAXN = 1e9;

void solve(){
  int l = 0, r = MAXN;

  while(r - l > 1)
  {
    int m = (l + r) / 2;
    cout << "? " << m << endl;
    string ans; cin >> ans;
    if(ans == "YES")
      l = m;
    else
      r = m;
  }
  cout << "! " << r << '\n';
}
```

17.3 Hidden Permutation

There is a hidden permutation $a_1, a_2, \ldots, a_n$ of integers $1, 2, \ldots, n$. Your task is to find this permutation. To do this, you can ask questions: you can choose two indices i and j and you will be told if $a_i < a_j$.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You should start by reading a single integer n: the length of the permutation. On your turn, you can print one of the following:
• "? i j", where $1 \le i, j \le n$: ask if $a_i < a_j$. The grader will return YES if $a_i < a_j$ and NO otherwise. • "! a_1 a_2 ... a_n": report that the hidden permutation is $a_1, a_2, \ldots, a_n$. Your program must terminate after this.
Each line should be followed by a line break. You must make sure the output gets flushed after printing each line.

Constraints
• $1 \le n \le 1000$ • you can ask at most 10^4 questions of type ?

```cpp
bool comparator(int a, int b)
{
    cout << "?␣" << a << '␣' << b << endl;
    string ans;
    cin >> ans;
    if(ans == "YES")
        return true;
    else
        return false;
}

void solve() {
    int n; cin >> n;
    vector<int> lista(n);
    for(int i = 1; i <= n; i++) lista[i - 1] = i;
    stable_sort(lista.begin(), lista.end(), comparator);

    vector<int> p(n+1);
    for(int i = 0; i < n; i++){
        int pos = lista[i];
        p[pos] = i+1;
    }

    cout << "!␣";
    for(int i = 1; i <= n; i++){
        cout << p[i] << '␣';
    }
    cout << '\n';
}
```

17.4 Inversion Sorting

There is a hidden permutation $a_1, a_2, \ldots, a_n$ of integers $1, 2, \ldots, n$. Your task is to sort the permutation by reversing subarrays. On each turn, you can reverse a subarray of the permutation. After that, you will be reported the number of inversions in the permutation. If the number of inversions is 0 (i.e., the permutation is sorted), you win.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You should start by reading a single integer n: the length of the permutation. On your turn, print two integers i and j: reverse the subarray between indices i and j. After this, the next input line has a single integer: the number of inversions after the operation. If the number is 0, you win and your program must terminate after this.

Constraints
• $1 \le n \le 1000$ • you can make at most $4n$ operations

```cpp
/*
    [1, k] ordenado -> inserir k + 1

    flippar [1, k + 1] -> ans1
    flippar [1, k + 1] -> ans2

    ans1 = X + k*(k - 1)/2 + menores
    ans2 = X + k - menores

    ans1 - ans2 = 2*menores + k*(k - 1)/2 - k
    menores = (ans1 - ans2 - k*(k - 3)/2)/2
*/

int query( int l, int r ){
    cout << l << "␣" << r << '\n'; cout.flush();
    int ans; cin >> ans;
    if( ans == 0 ) exit(0);
    return ans;
}

void solve(){
    int n; cin >> n;
    if( n == 1 ) query( 1, 1 );
    for( int i = 1; i < n; i++ ){
        int x1 = query(1, i + 1);
        int x2 = query(1, i + 1);

        int smaller = (x1 - x2 - i*(i - 3)/2)/2;

        if( smaller < i ){
            query( smaller + 1, i );
            query( smaller + 1, i + 1 );
        }
    }
}
```

17.5 K-th Highest Score

There were n coders from Finland and n coders from Sweden in a programming contest. It turned out that after the contest, each coder had a distinct score. Your task is to find the k-th highest score in the contest. To do this, you can ask questions: you can choose a country (Finland or Sweden) and an integer i and you will be told the i-th highest score for the chosen country.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You should start by reading two integers n and k. On your turn, you can print one of the following:
• "F i", where $1 \le i \le n$: ask the i-th highest score for Finland. • "S i", where $1 \le i \le n$: ask the i-th highest score for Sweden. • "! s": report that the k-th highest score is s. Your program must terminate after this.
Each line should be followed by a line break. You must make sure the output gets flushed after printing each line.

Constraints
• $1 \le n \le 10^5$ • $1 \le k \le 2n$ • each score is between 1 and 10^9 • you can ask at most 100 queries of the first two types in total

```cpp
// {1, 2, ..., k} -> F contribui com i, S com k - i
// k-th = min(f_i, s_{k - i})
// se f_i > s_{k - i}
// i > j : f_j > f_i > s_{k - i} > s_{k - j}
// i < j : s_{k - j}, f_i > s_{k - i} > f_j
// logo, i é o maior valor tq f_{i} > s_{k - i}

// se s_{k - i} > f_{i}
// i > j : f_j, s_{k - i} > f_i > s{k - j}
// i < j : s_{k - j} > s_{k - i} > f_i > f_j
// logo, i é o menor valor tq s_{k - i} > f_{i}

void solve(){
    int n, k; cin >> n >> k;
    char c[2] = {'F', 'S'};

    // testa k = 1 ou 2
    cout << "F␣1" << endl;
    int a; cin >> a;
    cout << "S␣1" << endl;
    int b; cin >> b;

    if (k == 1) {
        cout << "!␣" << max(a, b) << endl;
        return;
    }

    int r = min(k - 1, n), l = k - r;

    // edge cases garantem f_l > s_{k-l}  e  f_r < s_{k-r}
    for(int i = 0; i <= 1; i++)
    {
        cout << c[i] << "␣" << r << endl;
        cin >> a;
        cout << c[1 - i] << "␣" << l << endl;
        cin >> b;
        if (a > b) {
            a = -1;
            if(r + 1 <= n)
            {
                cout << c[i] << "␣" << r + 1 << endl;
                cin >> a;
            }
            cout << "!␣" << max(a, b) << endl;
            return;
        }
    }

    // f_l > s_{k-l}  e  f_r < s_{k-r}
    while (abs(r - l) > 1) {
        int mid = (l + r) / 2;
        cout << "F␣" << mid << endl;
        cin >> a;
```

```
        cout << "S " << k - mid << endl;
        cin >> b;
        if (a > b) {
            l = mid;
        } else {
            r = mid;
        }
    }

    // testa o melhor entre l e r
    cout << "S " << k - l << endl;
    cin >> b;
    cout << "F " << r << endl;
    cin >> a;

    cout << "! " << max(a, b) << endl;
}
```

17.6 Permuted Binary Strings

There is a hidden permutation $a_1, a_2, \ldots, a_n$ of integers $1, 2, \ldots, n$. Your task is to find this permutation. To do this, you can ask questions: you can choose a binary string $b_1 b_2 \ldots b_n$ and you will receive the binary string $b_{a_1} b_{a_2} \ldots b_{a_n}$.

Interaction
This is an interactive problem. Your code will interact with the grader using standard input and output. You should start by reading a single integer n: the length of the permutation. On your turn, you can print one of the following:
• "? $b_1 b_2 \ldots b_n$", where $b_i \in \{0, 1\}$: The grader will return the binary string $b_{a_1} b_{a_2} \ldots b_{a_n}$. • "! a_1 a_2 $\ldots$ a_n": report that the hidden permutation is $a_1, a_2, \ldots, a_n$. Your program must terminate after this.
Each line should be followed by a line break. You must make sure the output gets flushed after printing each line.

Constraints
• $1 \leq n \leq 1000$ • you can ask at most 10 questions of type ?

```
using i128 = __int128;
using ll = long long;

// 00001111
// 00110011
// 01010101
// dependendo da seq do dígito, descobrimos quem é quem
// se b_{a_i} teve 011 -> a_i = 011 + 1 = 4

void solve(){
    int n; cin >> n;
    vector<string> b(10, string(n, '0'));
    for(int i = 0; i < n; i++)
    {
        for(int j = 9; j >= 0; j--)
        {
            if(1 << j & i)
                b[9 - j][i] = '1';
            else
                b[9 - j][i] = '0';
        }
    }

    vector<string> a(10);

    for(int i = 0; i < 10; i++)
    {
        cout << "? ";
        cout << b[i] << endl;
        cin >> a[i];
    }

    cout << "! ";
    for(int i = 0; i < n; i++)
    {
        int ans = 0;
        for(int j = 0; j < 10; j++)
        {
            if(a[j][i] == '1')
                ans += (1 << (9 - j));
        }
        cout << ans + 1 << " ";
    }
    cout << endl;
}
```

18 Sliding Window Problems

18.1 Sliding Window Advertisement

A fence consists of n vertical boards. The width of each board is 1 and their heights may vary. You want to attach a rectangular advertisement to the fence. Your task is to calculate the maximum area of such an advertisement in each window of k vertical boards, from left to right.

Input
The first line contains two integers n and k: the width of the fence and the size of the window. After this, there are n integers $x_1, x_2, \ldots, x_n$: the height of each board.

Output
Print $n - k + 1$ integers: the maximum areas of the advertisements.

Constraints
• $1 \le k \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```cpp
using vi = vector<int>;
using ll = long long;
using pii = pair<int, int>;

struct LichaoTree{
    struct Line{
        ll a, b; Line( ll a = 0, ll b = 0 ) : a(a), b(b) {}
        ll f(ll x){ return a*x + b; }
    };

    Line *t;
    int n;

    LichaoTree( int n ) : n(n), t(new Line[4*n]){}

    void update( int node, int ti, int tf, Line line ){
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        if( t[node].f(tm) < line.f(tm) ) swap( line, t[node] );
        if( ti == tf ) return;

        if( t[node].f(ti) < line.f(ti) ) update( l, ti, tm, line );
        else update( r, tm + 1, tf, line );
    }

    void update( int node, int ti, int tf, int qi, int qf, Line line ){
        if( qi > tf || ti > qf ) return;
        if( qi <= ti && tf <= qf ){ update( node, ti, tf, line ); return; }
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        update( l, ti, tm, qi, qf, line ); update( r, tm + 1, tf, qi, qf, line );
    }

    void insert( int l, int r, ll a, ll b ){
        update( 1, 0, n - 1, l, r, Line( a, b ) );
    }

    ll query( int node, int ti, int tf, int id ){
        ll ans = t[node].f(id);
        if( ti == tf ) return ans;
        int l = 2*node, r = 2*node + 1, tm = (ti + tf)/2;
        if( id <= tm ) return max( query( l, ti, tm, id ), ans );
        return max( query( r, tm + 1, tf, id ), ans );
    }

    ll query( int x ){
        return query( 1, 0, n - 1, x );
    }
};

/*
    Considere a window [i, i + k - 1] e o indice j

    Caso 1: i <= l[j] && r[j] <= i + k - 1

        Ou seja, r[j] - k + 1 <= i <= l[j]
        Adicionar reta 0*i + v[j]*(r[j] - l[j] - 1)

    Caso 2: l[j] < i && i + k - 1 < r[j]

        Ou seja, l[j] + 1 <= i <= r[j] - k
        Adicionar reta 0*i + v[j]*k

    Caso 3: l[j] < i <= j && r[j] <= i + k - 1

        Ou seja, max( l[j] + 1, r[j] - k + 1 ) <= i <= j
        Adicionar a reta -v[j]*i + v[j]*r[j]

    Caso 4: i <= l[j] && j <= i + k - 1 < r[j]

        Ou seja, j - k + 1 <= i <= min( l[j], r[j] - k )
        Adicionar reta v[j]*i + v[j]*(k - 1 - l[j])
*/

void solve(){
    int n, k; cin >> n >> k;
    vi l(n, -1), r(n, n), v(n);
    stack<pii> s; s.push({ -1, -1 });
    FOR(i, 0, n){
        cin >> v[i];
        for(; s.top().first >= v[i]; s.pop() ) r[s.top().second] = i;
        l[i] = s.top().second;
        s.push({ v[i], i });
    }

    LichaoTree t(n);
    FOR(j, 0, n){
        t.insert(r[j] - k + 1, l[j], 0, 1LL*v[j]*(r[j] - l[j] - 1) );
        t.insert(l[j] + 1, r[j] - k, 0, 1LL*v[j]*k );
        t.insert(max(l[j] + 1, r[j] - k + 1), j, -v[j], 1LL*v[j]*r[j] );
        t.insert(j - k + 1, min(l[j], r[j] - k), v[j], 1LL*v[j]*(k - 1 - l[j]) );
    }

    FOR(i, 0, n - k + 1) cout << t.query(i) << "␣"; cout << '\n';
}
```

18.2 Sliding Window Cost

You are given an array of n integers. Your task is to calculate for each window of k elements, from left to right, the minimum total cost of making all elements equal. You can increase or decrease each element with cost x where x is the difference between the new and the original value. The total cost is the sum of such costs.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Output $n - k + 1$ values: the costs.

Constraints
• $1 \le k \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

```cpp
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void fix(multiset<int>* m, multiset<int>* M, ll* sm, ll* sM){
    if((*m).size() < (*M).size()){
        int x = *(*M).begin();
        (*m).insert(x);
        (*M).erase((*M).begin());
        *sm += x;
        *sM -= x;
    }
    else if((*m).size() >= (*M).size() + 2){
        int x = *(*m).rbegin();
        (*M).insert(x);
        (*m).erase((*m).find(x));
        *sM += x;
        *sm -= x;
    }
}

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    multiset<int> m, M;
    ll sm = 0, sM = 0;

    vector<int> aux(k);
    for(int i = 0; i < k; i++) aux[i] = v[i];
    sort(all(aux));

    for(int i = 0; i < (k + 1) / 2; i++) {
        m.insert(aux[i]);
        sm += aux[i];
    }
    for(int i = (k + 1) / 2; i < k; i++) {
        M.insert(aux[i]);
        sM += aux[i];
    }

    for(int i = 0; i < n - k + 1; i++){
        int rem = v[i];
        int add = v[i + k];
        int a = *m.rbegin();
        if(k % 2 == 1)
            cout << sM  - sm + a << '␣';
        else
            cout << sM  - sm     << '␣';
        if(rem <= a) {
            m.erase(m.find(rem));
            sm -= rem;
        }
        else {
```

```
            M.erase(M.find(rem));
            sM -= rem;
        }
        fix(&m, &M, &sm, &sM);

        if(M.empty()) {
            m.insert(add);
            sm += add;
        }
        else {
            a = *M.begin();
            if(add >= a) {
                M.insert(add);
                sM += add;
            }
            else {
                m.insert(add);
                sm += add;
            }
        }
        fix(&m, &M, &sm, &sM);

    }
    cout << '\n';

}
```

18.3 Sliding Window Distinct Values

You are given an array of n integers. Your task is to calculate the number of distinct values in each window of k elements, from left to right.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the numbers of distinct values.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
void solve(){
    int n, k; cin >> n >> k;
    vector<int> a(n);

    for(int i = 0; i < n; i++) {
        cin >> a[i];
    }

    int ans = 0;
    map<int, int> freq;
    for(int i = 0; i < k; i++) {
        freq[a[i]]++;
        if(freq[a[i]] == 1) {
            ans++;
        }
    }

    cout << ans << "␣";

    for(int i = k; i < n; i++) {
        freq[a[i - k]]--;
        if(freq[a[i - k]] == 0) {
            ans--;
        }
        freq[a[i]]++;
        if(freq[a[i]] == 1) {
            ans++;
        }
        cout << ans << "␣";
    }

    cout << '\n';
}
```

18.4 Sliding Window Inversions

You are given an array of n integers. Your task is to calculate the number of inversions in each window of k elements, from left to right. An inversion is a pair of elements where the left element is larger than the right element.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the numbers of inversions.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
using ll = long long;

int main(){
  int n, k; cin >> n >> k;
  vector<int> v(n);
  map<int, int> mp;
  for( int &x : v ){
    cin >> x;
    mp[x] = 1;
  }

  int cont = 1;
  for( auto &[key, val] : mp ) val = cont++;

  vector<int> bit(cont);

  auto update = [&]( int id, int val ){
    for( int i = id; i < (int)bit.size(); i += i&-i) bit[i]
        += val;
  };

  auto query = [&]( int id ){
    int sum = 0;
    for( int i = id; i > 0; i -= i&-i ) sum += bit[i];
    return sum;
  };

  ll inversions = 0;
  for( int i = 0; i < n; i++ ){
    v[i] = mp[v[i]];
    inversions += query(cont - 1) - query(v[i]);
    update(v[i], 1);
    if( i + 1 < k ) continue;
    cout << inversions << "␣";
    inversions -= query(v[i - k + 1] - 1);
    update(v[i - k + 1], -1);
  }
  cout << endl;
}
```

18.5 Sliding Window Median

You are given an array of n integers. Your task is to calculate the median of each window of k elements, from left to right. The median is the middle element when the elements are sorted. If the number of elements is even, there are two possible medians and we assume that the median is the smaller of them.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the medians.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void fix(multiset<int>* m, multiset<int>* M){
    if((*m).size() < (*M).size()){
        (*m).insert(*(*M).begin());
        (*M).erase((*M).begin());
    }
    else if((*m).size() >= (*M).size() + 2){
        (*M).insert(*(*m).rbegin());
        (*m).erase((*m).find(*(*m).rbegin()));
    }
}

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    multiset<int> m, M;

    vector<int> aux(k);
    for(int i = 0; i < k; i++) aux[i] = v[i];
    sort(all(aux));

    for(int i = 0; i < (k + 1) / 2; i++) m.insert(aux[i]);
    for(int i = (k + 1) / 2; i < k; i++) M.insert(aux[i]);

    for(int i = 0; i < n - k + 1; i++){
        int rem = v[i];
        int add = v[i + k];
        int a = *m.rbegin();
        cout << a << '␣';

        if(rem <= a) m.erase(m.find(rem));
```

```
        else M.erase(M.find(rem));
        fix(&m, &M);

        if(M.empty()) m.insert(add);
        else{
            a = *M.begin();
            if(add >= a) M.insert(add);
            else m.insert(add);
        }
        fix(&m, &M);

    }
    cout << '\n';

}
```

18.6 Sliding Window Mex

You are given an array of n integers. Your task is to calculate the mex of each window of k elements, from left to right. The mex is the smallest nonnegative integer that does not appear in the array. For example, the mex for $[3, 1, 4, 3, 0, 5]$ is 2.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the mex values.

Constraints
• $1 \le k \le n \le 2 \cdot 10^5$ • $0 \le x_i \le 10^9$

```
void solve() {
    int n, k; cin >> n >> k;
    vector<int> x(n);
    vector<int> v(n, 0);
    set<int> ans;
    int rem = 0;
    for(int i = 0; i < n; i++) {
        cin >> x[i];
        if(x[i] < n && i < k) v[x[i]]++;
    }
    ans.insert(n);
    for(int i = 0; i < n; i++) {
        if(v[i] == 0) {
            ans.insert(i);
        }
    }
    cout << *ans.begin() << ' ';
    for(int i = k; i < n; i++) {
        if(x[i] < n) {
            v[x[i]]++;
            ans.erase(x[i]);
        }
        if(x[i - k] < n) {
            v[x[i - k]]--;
            if(v[x[i - k]] == 0) ans.insert(x[i - k]);
        }
        cout << *ans.begin() << ' ';
    }
    cout << '\n';
}
```

18.7 Sliding Window Minimum

You are given an array of n integers. Your task is to calculate the minimum of each window of k elements, from left to right. In this problem the input data is large and it is created using a generator.

Input
The first line contains two integers n and k: the number of elements and the size of the window. The next line contains four integers x, a, b and c: the input generator parameters. The input is generated as follows:
• $x_1 = x$ • $x_i = (ax_{i-1} + b) \bmod c$ for $i = 2, 3, \ldots, n$

Output
Print the xor of all window minimums.

Constraints
• $1 \le k \le n \le 10^7$ • $0 \le x, a, b \le 10^9$ • $1 \le c \le 10^9$

```
deque<int> minq;

void addmin(int a) {
    while(!minq.empty() && minq.back() > a)
        minq.pop_back();
    minq.push_back(a);
}

void remmin(int a) {
    if(!minq.empty() && minq.front() == a)
        minq.pop_front();
}

void solve(){
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    int first = x;
    int sum = 0, at = x;
    int ans = 0;
    for(int i = 0; i < k; i++) {
        addmin(at);
        at = (at * a + b) % c;
    }
    sum = minq.front();
    ans = ans ^ sum;
    for(int i = k; i < n; i++) {
        addmin(at);
        remmin(first);
        sum = minq.front();
        at = (at * a + b) % c;
        first = (first * a + b) % c;
        ans ^= sum;
    }
    cout << ans << '\n';
}
```

18.8 Sliding Window Mode

You are given an array of n integers. Your task is to calculate the mode each window of k elements, from left to right. The mode is the most frequent element in an array. If there are several possible modes, choose the smallest of them.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the modes.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

```
unordered_map<int, int> qntt;

void solve() {
    int n, k; cin >> n >> k;
    vector<int> x(n);
    vector<set<int>> lvl(n + 1);
    int bst = 0;
    for(int i = 0; i < n; i++) {
        cin >> x[i];
        if(i < k) {
            qntt[x[i]]++;
            bst = max(bst, qntt[x[i]]);
        }
    }
    for(int i = 0; i < k; i++) {
        lvl[qntt[x[i]]].insert(x[i]);
    }

    cout << *lvl[bst].begin() << ' ';
    for(int i = k; i < n; i++) {
        lvl[qntt[x[i]]].erase(x[i]);
        qntt[x[i]]++;
        lvl[qntt[x[i]]].insert(x[i]);
        bst = max(bst, qntt[x[i]]);

        lvl[qntt[x[i - k]]].erase(x[i - k]);
        qntt[x[i - k]]--;
        if(qntt[x[i - k]] != 0)
            lvl[qntt[x[i - k]]].insert(x[i - k]);
        if(lvl[bst].empty())
            bst--;

        cout << *lvl[bst].begin() << ' ';
    }
    cout << '\n';
}
```

18.9 Sliding Window Or

You are given an array of n integers. Your task is to calculate the bitwise or of each window of k elements, from left to right. In this problem the input data is large and it is created using a generator.

Input
The first line contains two integers n and k: the number of elements and the size of the window. The next line contains four integers x, a, b and c: the input generator parameters. The input is generated as follows:
• $x_1 = x$ • $x_i = (ax_{i-1} + b) \bmod c$ for $i = 2, 3, \ldots, n$

Output
Print the xor of all window ors.

Constraints
• $1 \leq k \leq n \leq 10^7$ • $0 \leq x, a, b \leq 10^9$ • $1 \leq c \leq 10^9$

```
void solve(){
    int n, k; cin >> n >> k;
    vector<int> v(n);
    int a, b, c; cin >> v[0] >> a >> b >> c;

    for(int i = 1; i < n; i++)
        v[i] = ((long long)v[i - 1] * a + b) % c;

    int buckets = (n + k - 1) / k;
    vector<int> pref(n + 1, 0);
    vector<int> suf(n + 1, 0);

    for(int i = 0; i < buckets; i++)
    {
        for (int j = min(k, n - i * k) - 1; j >= 0; --j)
            suf[i * k + j] = suf[i * k + j + 1] | v[i * k + j
                ];
    }

    for(int i = buckets - 1; i >= 0; i--)
    {
        for (int j = 0; j < k && i * k + j < n; j++)
            pref[i * k + j + 1] = pref[i * k + j] | v[i * k +
                j];
    }

    int ans = 0;

    for(int i = 0; i <= n - k; i++)
    {
        if(i % k == 0)
            ans ^= suf[i];
        else
            ans ^= suf[i] | pref[i + k];
    }

    cout << ans << '\n';
}
```

18.10 Sliding Window Sum

You are given an array of n integers. Your task is to calculate the sum of each window of k elements, from left to right. In this problem the input data is large and it is created using a generator.

Input
The first line contains two integers n and k: the number of elements and the size of the window. The next line contains four integers x, a, b and c: the input generator parameters. The input is generated as follows:
• $x_1 = x$ • $x_i = (ax_{i-1} + b) \bmod c$ for $i = 2, 3, \ldots, n$

Output
Print the xor of all window sums.

Constraints
• $1 \leq k \leq n \leq 10^7$ • $0 \leq x, a, b \leq 10^9$ • $1 \leq c \leq 10^9$

```
void solve(){
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    int first = x;
    int sum = 0, at = x;
    int ans = 0;
    for(int i = 0; i < k; i++)
    {
        sum += at;
        at = (at * a + b) % c;
    }
    ans ^= sum;
    for(int i = k; i < n; i++)
    {
        sum += at - first;
        at = (at * a + b) % c;
        first = (first * a + b) % c;
        ans ^= sum;
    }
    cout << ans << '\n';
}
```

18.11 Sliding Window Xor

You are given an array of n integers. Your task is to calculate the bitwise xor of each window of k elements, from left to right. In this problem the input data is large and it is created using a generator.

Input
The first line contains two integers n and k: the number of elements and the size of the window. The next line contains four integers x, a, b and c: the input generator parameters. The input is generated as follows:
• $x_1 = x$ • $x_i = (ax_{i-1} + b) \bmod c$ for $i = 2, 3, \ldots, n$

Output
Print the xor of all window xors.

Constraints
• $1 \leq k \leq n \leq 10^7$ • $0 \leq x, a, b \leq 10^9$ • $1 \leq c \leq 10^9$

```
void solve(){
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;

    int sum = 0;
```

```
6       int at = x;
7       for(int i = 0; i < k - 1; i++){
8           if((i + 1) % 2 == 1)
9               sum ^= at;
10          at = (a * at + b) % c;
11      }
12      for(int i = k - 1; i <= n - k; i++){
13          if(k % 2 == 1)
14              sum ^= at;
15          at = (a * at + b) % c;
16      }
17      for(int i = n - k + 1; i < n; i++){
18          if((n - i) % 2 == 1)
19              sum ^= at;
20          at = (a * at + b) % c;
21      }
22
23      cout << sum << '\n';
24  }
```